

前言

自从本人 2019 年年初开始正式进行群星 stellaris 的 MOD 的时候，就一直到处寻找各种国内中文的编辑教程，可惜的是，这些教程要不就是版本过时严重，要么就是只言片语，很难有一个完整的叙述，迫使我不得不去查看英文 wiki 和原版，在没有其他人的帮助下自学了一段时间，期中由于没有指导的困难使我早期走了不少弯路，虽然到目前为止我依旧有很多不足的地方，但想到自己曾经的经历，故决定写下此教程。不求有多少作用，但求能让新人有一个可以参考的地方。

由于本人并非计算机相关专业出身，亦不从事相关职业，故本教程各种用语可能并非专业，也可能有存在概念有所不符的地方，还希望看到此教程各位能帮助斧正！

2020.12.11

Cmdim

参与 Common 教程编辑

当初我学习 Mod 的过程关于该内容可以说是少之又少，资料几乎无处查找，仅仅有一篇官方 Wiki 提供的帮助，经过不断的摸索之后总算有了一些头绪，到现在便想着能否写下一些东西给后来者提供一些帮助，少走一些弯路。

本部分教程内容大部分来自自己的经验以及官方 Wiki，其中也包括其他各位 Modder 的经验集合。在我编写该教程之时群星正式版本为 2.8.1，在此介绍的 Event 内容不能够保证版本不同造成的差异所带来的问题，语句可能在不同版本会有相应调整，不过大体框架理论上不会有变化，在此需要各位读者自行斟酌内容的正确性。

毕竟是经验总结，难免会有错误，如果有错误或者疑问，亦或是有更多的要求，可以考虑加入我们的 Mod 交流群一同探讨。

无言 Infinite

参与 Events 教程编辑

第 1 页 共 311

目录

前言.....	1
内容更替情况.....	7
一、建立你的第一个 MOD.....	9
1.1 通过启动器的方式.....	9
1.2 通过直接修改（创建）一个新的文件夹和.mod 文件.....	13
二、MOD 基础知识与准备.....	15
2.1 MOD 的工作原理.....	15
2.2 Stellaris 的具体文件组成.....	16
2.2.1 Common 中的主要文件.....	16
2.2.2 其他一些比较常用和重要的文件.....	24
2.3 工具准备.....	24
2.3.1 编辑器的选择.....	24
2.3.2 CWTools 插件安装.....	25
2.3.3 Paradox Language Support.....	26
2.4 粒子编辑器.....	29
三、Common Modding.....	31
3.1 .议程(agenda)s修改.....	31
3.1.1 认识议程的组成.....	31
3.1.2 添加自己的第一个议程.....	33
3.2 本地化 (localisation) 修改.....	36
3.2.1 认识本地化文件的组成.....	36
3.2.2 为第一个议程写一个本地化文件.....	38
3.3 科技 (technology) 的修改.....	40
3.3.1 认识科技的组成.....	40
3.3.2 修改（创造）新科技.....	43
3.3.3 为你的 MOD 添加第一个图标.....	45
3.4 法令 (edicts) 修改.....	47
3.4.1 法令 (edicts) 的组成与修改.....	47
3.5 军队 (Armies) 修改.....	49
3.5.1 军队 (Armies) 的组成与修改.....	49
3.5.2 军队 (Armies) 图标的修改.....	51
3.6 种族和领袖特质 (traits).....	53
3.6.1 种族特质修改.....	53
3.6.2 领袖特质修改.....	53
3.7 建筑 (buildings) 与职业 (pop_jobs) 修改.....	55
3.7.1 建筑 (buildings) 修改.....	55
3.7.2 职业 (pop_jobs) 修改.....	58
3.8 政府 (governments) 修改.....	61
3.8.1 政体 (authorities).....	61
3.8.2 公民性 (civics).....	62
3.8.3 起源(origin)修改.....	63

3.8.4 政府形式 (governments)	64
3.9 战争借口 (casus_belli) 与战争目标 (war_goals)	66
3.9.1 战争借口 (casus_belli)	66
3.9.2 战争目标 (war_goals)	66
3.10 飞升 (ascension_perks)	68
3.11 政策(policies).....	71
3.12 舰船相关.....	73
3.12.1 舰船部件 (component_template)	73
3.12.2 组件集合 (component_sets)	75
3.12.3 舰船类型 (ship_sizes)	76
3.12.4 舰船区段 (section_templates)	77
3.13 遗迹与遗珍相关	78
3.13.1 小型文物的在遗珍界面的消耗选项(artifact_actions).....	78
3.13.2 考古的坟 (archaeological_site_types)	79
3.13.3 遗珍 (relic)	79
3.14 星球相关.....	81
3.14.1 星球类型 (Planet Class)	81
3.14.2 区划 (Districts)	83
3.15 巨像相关.....	86
3.15.1 舰船组件部分	86
3.15.2 on_actions	87
3.15.3 scripted_triggers	89
3.15.4 events.....	92
3.15.5 本地化.....	93
3.16 评价 (opinion_modifiers)	93
3.16 局势 (Situation)	93
3.16.1 局势系统	94
3.16.2 属性解析	95
四、Event Modding.....	100
4.1 Event 简介	100
4.1.1 Event 功能.....	100
4.1.2Event 本质.....	100
4.2 工具准备.....	100
4.2.1 编辑器选择	100
4.2.2 CWTools 安装	101
4.2.3 CWTools 使用	103
<1> 自动补齐	103
<2> 查看引用	104
<3> 代码风格	104
<4> 批量注释	106
<5> 额外功能	107
4.2.4 Paradox Language Support	112
<1> 实时预览本地化以及 Scope 解析.....	113

<2> 条目图标预览	115
<3> 智能完善本地化	116
4.3 Event 基础	116
4.3.1 触发器(Trigger)	118
<1> Mean Time To Happen	119
<2> Is Triggered Only	120
4.3.2 条件(Condition)	122
4.3.3 影响(Effect)	124
<1> Immediate	125
<2> Option	126
<3> After	128
4.3.4 附加属性	131
4.3.5 其他属性	132
4.4 作用域(Scope)	136
4.4.1 系统作用域(System Scope)	136
4.4.2 链式作用域(Chaining Scopes)	140
4.4.3 触发器作用域(Triggers and Scopes)、效果作用域(Effect and Scopes)	140
4.4.4 作用域种类(Scope Types)、作用域列表(List of Types)	141
作用域类型表	141
作用域表	142
4.5 Event 调试	147
4.5.1 正常触发	148
4.5.2 无限弹窗	148
4.5.3 游戏崩溃	150
4.5.4 未寻找到 Event	151
4.6 Event 进阶	154
4.6.1 标识(Flag)	154
4.6.2 事件对象(Event Target)	166
4.6.3 脚本效果(Scripted Effect)	185
4.6.4 脚本条件(Scripted Trigger)	187
4.6.5 脚本变量(Scripted Variables)	189
4.6.6 脚本值(Script Values)	190
4.6.6.1 Export 语句介绍	191
4.6.6.2 脚本值 (Script Values) 的简单应用	198
4.6.6.3 脚本值 (Script Values) 属性解释	200
五、Event Modding 进阶	202
5.1 绪论	202
5.2 认识你与 event 的极限	206
5.2.1 我是谁，我在哪 —— scope	207
5.2.2 老三样——effect、condition 以及 modifier	209
5.2.3 如何讲蠢驴语——事件代码构思思路	213
范例 1-2 烂大街的自动人口迁移（其一）	214
5.2.4 你与事件的极限——mod 做不到的事	217

1. 事件能不能实现这个?	217
2. 我能不能做到这个?	218
5.2.5 小结.....	218
习题一	219
5.3 如何找到目标 Scope.....	222
5.3.1 对象转换——scope 跳跃与调用	222
5.3.2 遍历，还是遍历——遍历型 scope 跳跃语句	223
5.3.3 PREVPREVPREV.FROMFROMFROMFROM ——流程 scope 跳跃语句	228
5.3.4 从前有艘船，船所有者……—属性 scope 跳跃	242
六、scripted effect 与 trigger 的函数应用	247
6.1 前言	247
6.2 scripted effect 函数化	247
6.2.1 参数传递	248
6.2.3 Fallback value 参数预设值	251
6.2.4 Inline Math 参数计算.....	253
6.2.5 recursion 递归	255
七、GFX 文件与 sprite type.....	257
7.1 认识 sprite 与创建 sprite type	257
7.2 sprite type 动画.....	261
7.3 sprite type 按钮.....	266
7.4 其他 sprite type.....	267
7.4.1 frameAnimatedSpriteType.....	268
7.4.2 corneredTileSpriteType	268
八、Custom GUI 入门	269
8.1 GUI 文件的构成	270
8.2 GUI 模板.....	272
九、代码覆盖规则	282
9.1 基础知识.....	282
9.2 排序规则.....	283
9.3 覆盖规则.....	284
9.4 各文件的覆盖规则列表.....	285
9.4.1 common 文件夹	285
9.4.2 Localisation 本地化文件夹	288
9.4.3 Events 文件夹	288
9.4.2 Interface 文件夹	288
实例展示	289
一、新建一个物种并向其添加专属特质	289
1. 素材制作	289
2. 素材导入	294
3. 应用	295
4. 新建物种大类(Species Archetypes)	299
5. 新建物种类型 (Species Classes)	300
6. 专属特质	302

二、向游戏导入字库并应用在 Custom GUI	309
--------------------------------	-----

内容更替情况

-2020.08

完成第一版教程内容——Cmdim

-2020.12.22

修正 Common Modding 内容，现已适用于 2.8——无言 Infinite

-2020.02.21

增加 scripted effect 与 trigger 的函数应用章节——VVlolicon

-2021.11.07

增加 gfx 文件与 sprite type 章节——VVlolicon

-2023.2.25

1.修正 Common Modding 表单内容，现已适用 3.6 版本

2.新增插件推荐：IDEA 的 Paradox Language Support

3.添加了一个新的实例章节与几个例子以供参考学习

4.整合_OAO_所编写的粒子编辑器开启教程

——无言 Infintie

-2023.2.27

新增 Script Values 的应用章节

——无言 Infinite

-2023.3.1

补充 Common Modding 条目：

1.区划 (District)

2.局势 (Situation)

——无言 Infinite

一、建立你的第一个 MOD

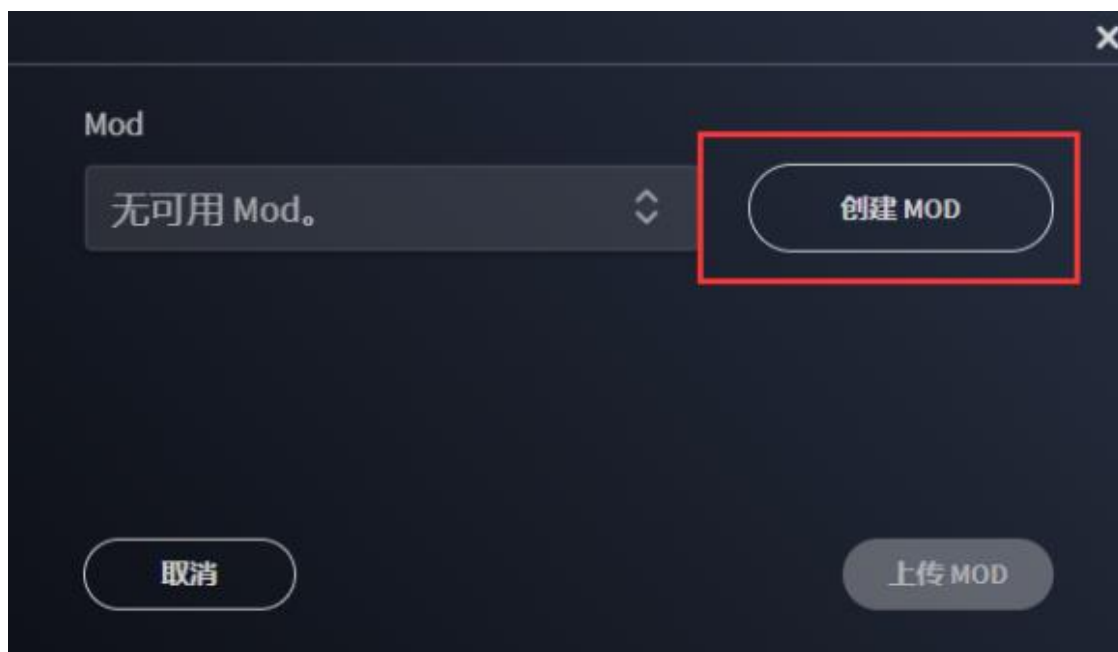
1.1 通过启动器的方式

(1)此方法不适用 2.3 和之前的版本!!!

首先，需要在你的 steam 库里面打开 stellaris 的启动器（废话），操作如下图



漫长的加载之后，你应该会进入这个界面



名称 版本

就是你这个mod叫什么名称 就是这个mod支持的版本，例如2.8.1

目录

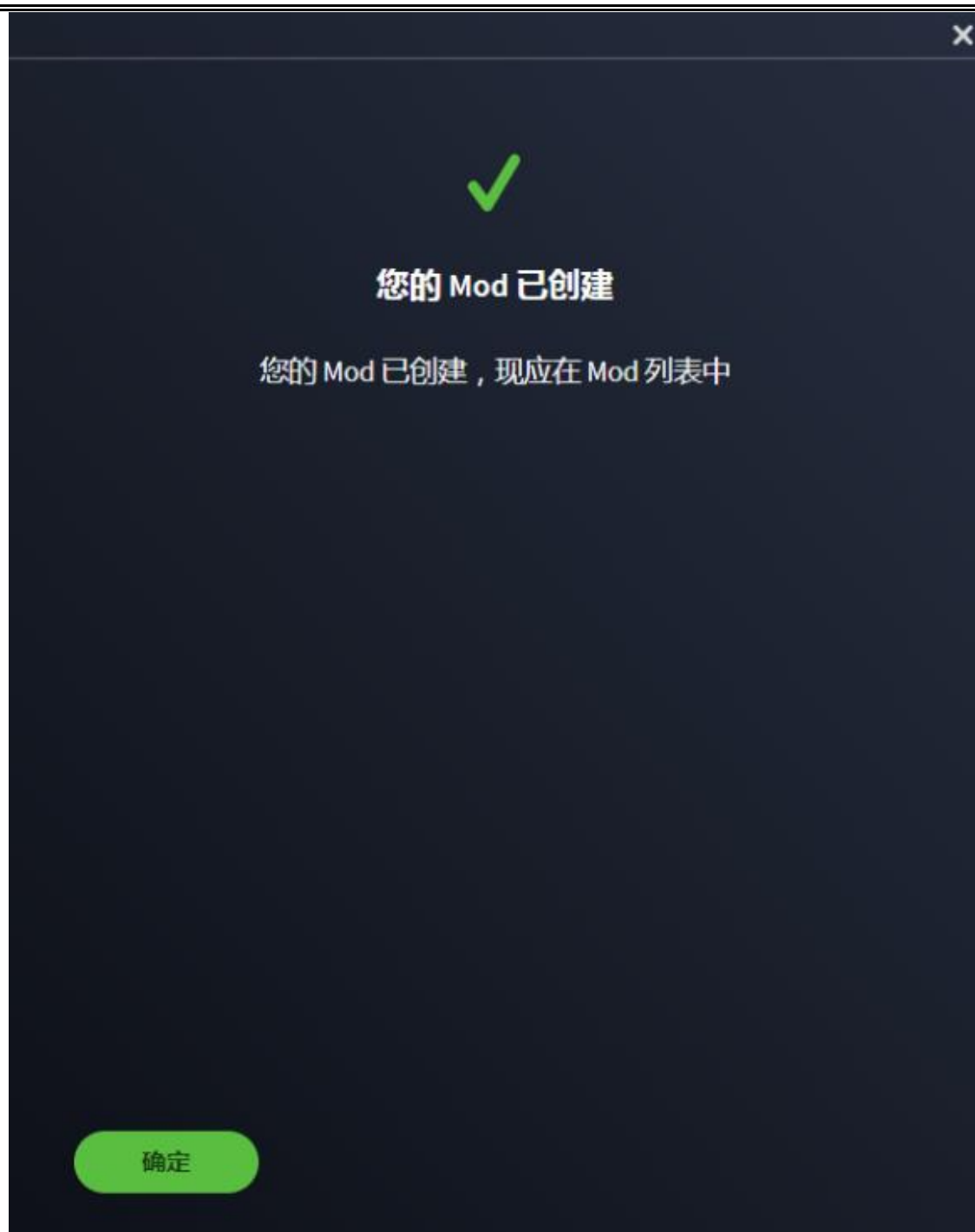
mod/ 在MOD文件夹下的目录名称

标签 你觉得这个玩意是用来修饰什么的？比如加了新的科技，政体什么

☐ Balance	☐ Loading Screen
☐ Buildings	☐ Military
☐ Diplomacy	☐ Overhaul
☐ Economy	☐ Sound
☐ Events	☐ Spaceships
☐ Fixes	☐ Species
☐ Font	☐ Technologies
☐ Galaxy Generation	☐ Total Conversion
☐ Gameplay	☐ Translation

返回 都写完后点击 创建 MOD

当填写好这些之后，点击“创建 MOD”，之后会



如果不出现这样的信息，那么很有可能是因为文件名重复或者名字不规范，根据提示信息修改。当你做好这一步之后，那么进入该路径进行确认，对于大部分人而言，这个路径应该在“我的文档\Paradox Interactive\Stellaris\mod 目录下”，比如我的路径是 D:\Documents\Paradox Interactive\Stellaris\mod，可在下图见到确实创立了这个文件夹，以及相应名称的.mod 文件。



好了，现在你就成功的创建了一个 mod 了....虽然目前还没有实际上的内容

(2)此方法不适用 2.4 和之后的版本!!!

首先开启你的 stellaris 启动器，（由于现阶段的启动器中文显示有误，故截图使用英文版启动器）之后在界面内找到“**Mods**”>>>“**Mod tools**”，在之后的弹窗之中，选择“**Create Mod**”。

在第一项写上你自己 mod 的名称（例如我写的“test”），第二项写上装载这个 mod 的文件夹名称，一般建议和你的 mod 名称一致，方便之后修改或者其他操作。第三项是支持的版本，没有什么特殊需求一般是保持与本版本一致。**Mod tags** 是这个 mod 的标签，有很多可以选择的项目。

当填写好这些之后，点击“**Create Mod**”，之后会在左下方显示这个 mod 被创建的路径（见图 3），如果不出现这样的信息，那么很有可能是因为文件名重复或者名字不规范，根据提示信息修改。当你做好这一步之后，那么进入该路径进行确认，比如我的路径是 [D:\Documents\Paradox Interactive\Stellaris\mod](#)，可在图 4 见到确实创立了这个文件夹，以及相应名称的.mod 文件。



图 1

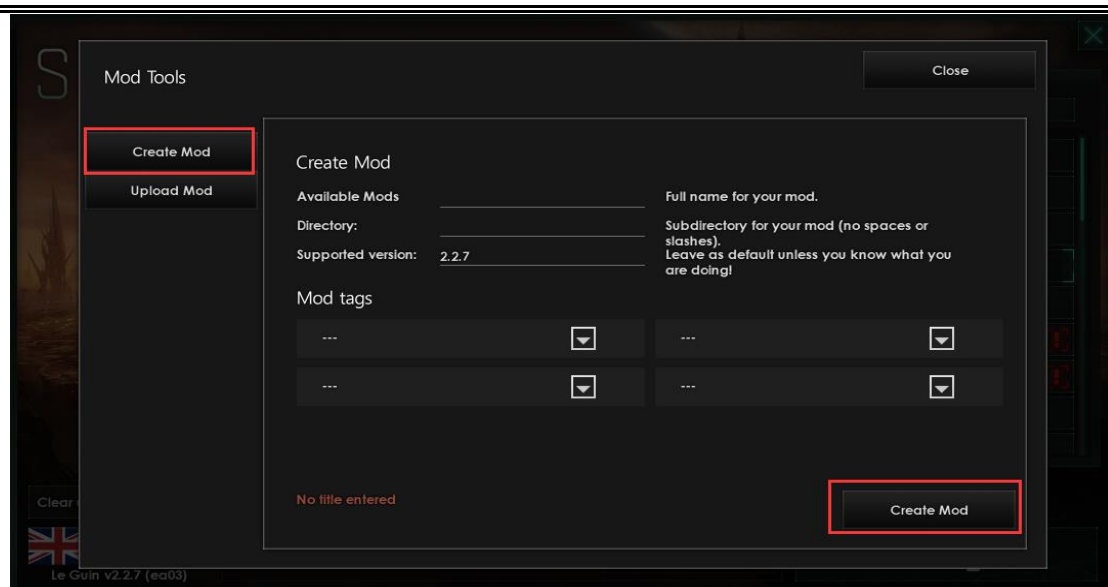


图 2

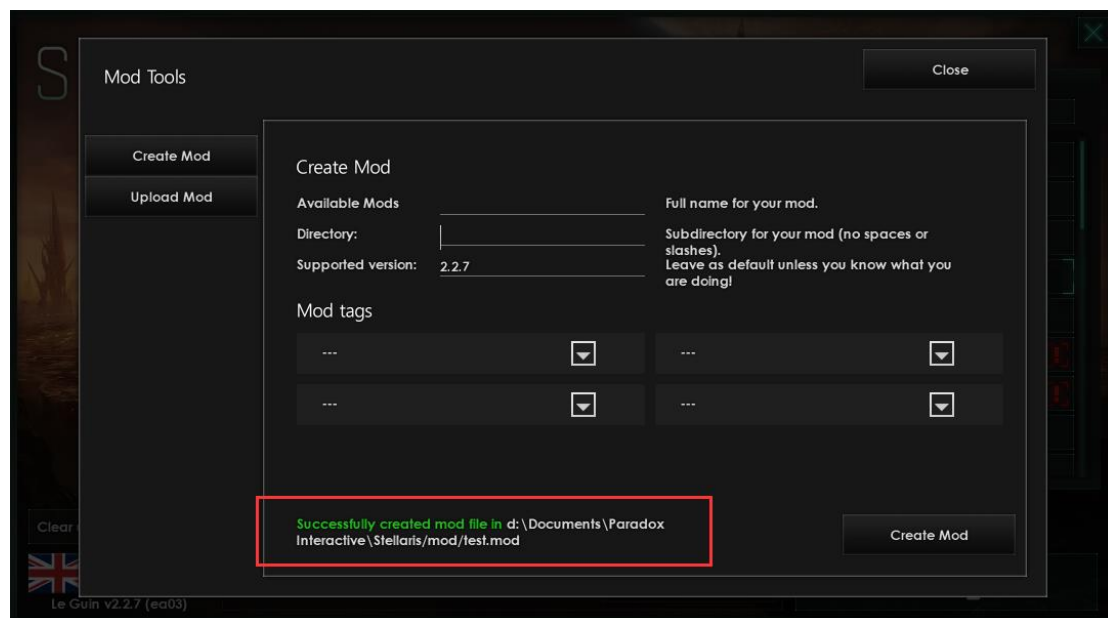


图 3

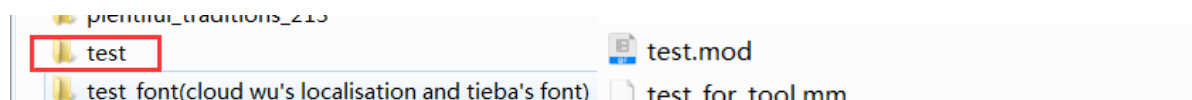


图 4

1.2 通过直接修改（创建）一个新的文件夹和.mod 文件

通过对上述第一种方法的观察，我们可以发现，启动器本质上是创建了一个文件夹和.mod 文件，那么相应的，我们也可以通过直接修改他们达到一样的效果。新建文件夹不是我们的重点，重点是.mod 文件的填写。

用软件打开.mod 文件，可见以下的代码

```
name="test"                #你所创建的 mod 的名称
path="mod/test"            #你所创建的 mod 对应的相对路径
tags={                     #这个 mod 的标签/分类
    "Buildings"
}
picture=""                 #这个 mod 的封面图片（放在文件夹里面）
remote_file_id="xxxxxxx"   #这个 mod 在创意工坊的 ID，一般不建议修改
supported_version="2.2.7"  #如果是“2.2.*”则代表全部的 2.2 版本适用
```

通过修改这几项代码，从而实现对于 mod 文件的直接创造新的 mod 或者继续之前的工作（比如更改版本适应或者加入别的 mod 的内容等）

二、MOD 基础知识与准备

2.1 MOD 的工作原理

众所周知，MOD 是游戏的一个非官方扩展，而 P 社的 MOD 的工作原理基于相同文件的映射和覆盖，比如 MOD 里面存在一个和原版 stellaris **相同文件夹的相同名称的文件**，那么 MOD 将会替代它，如果不是相同文件则会加入到游戏之中成为额外的部分。

如图 5（stellaris 原版文件夹）图 6（某个 mod 的文件夹），当 mod 被运行的时候，图 6 的内容会优先覆盖图 5 文件夹里面的内容，当然也添加了图 6 没有重复名称的文件。通过这样的一种机制，便实现了对于原版的修改或者添加新的内容。如果想要探讨不同 MOD 之间的覆盖规则详见第九章。

名称	修改日期	类型
common	2019/4/4 18:43	文件夹
dlc	2019/3/3 12:06	文件夹
events	2019/4/4 18:43	文件夹
flags	2019/5/6 14:05	文件夹
fonts	2019/3/3 12:04	文件夹
gfx	2019/3/3 12:05	文件夹
interface	2019/4/4 18:43	文件夹
licenses	2019/3/3 12:05	文件夹
locales	2019/3/3 12:05	文件夹
localisation	2019/3/3 12:05	文件夹
localisation_synced	2019/3/7 21:56	文件夹
map	2019/3/3 12:05	文件夹
music	2019/3/3 12:05	文件夹
pdx_launcher	2019/3/3 12:05	文件夹
pdx_online_assets	2019/3/3 12:06	文件夹
prescripted_countries	2019/3/3 12:06	文件夹
previewer_assets	2019/3/3 12:06	文件夹
sound	2019/3/3 12:06	文件夹
tweakergui_assets	2019/3/3 12:06	文件夹

图 5

名称	修改日期	类型
common	2019/5/1 10:56	文件夹
events	2019/4/28 20:19	文件夹
gfx	2019/4/29 18:17	文件夹
interface	2019/4/29 14:15	文件夹
localisation	2019/4/30 21:47	文件夹
localisation_synced	2019/5/1 19:27	文件夹

图 6

2.2 Stellaris 的具体文件组成

2.2.1 Common 中的主要文件

[Stellaris/common/](#)

下方表格蓝色部分可以按住 Ctrl 左键点击即可跳转至相应条目，橙色部分代表该条目属性解析暂空缺。

重要提醒：尽量不要从议程开始学习，由于版本更替，议程已并不适合作为初学 mod 的基础条目，建议从法令（edicts）开始学习较为简单！之后可以自行接触其他条目，觉得简单，亦或是感兴趣的皆可，不必拘泥于教程所展示的顺序。

教程当中暂缺的条目大部分可以在蠢驴的原文档下方找到说明文件，能够看懂的可以参照说明进行编写。

注：由于教程编写时间以及蠢驴更新条目的原因，目录所示条目与下表介绍顺序并不相符，因此从下表或目录索引到你需要的条目解释皆可。

agendas	议程，所有的可以使用的议程的集合
agreement_presets	外交附庸条例条目之一
agreement_resources	外交附庸条例条目之一
agreement_term_values	外交附庸条例条目之一
agreement_term_values	外交附庸条例条目之一
ai_budget	定义 AI 的行为相关，主要是用于 AI 的计划经济
ai_espionage	AI 的间谍行动相关
ai_force_species	定义 AI 优先选择哪类物种增长的逻辑
artifact_actions	小型文物的使用选项
archaeological_site_types	各种坟头
ambient_objects	用于定义出现在异常之中的模型
anomalies	异常，所有的可使用异常
armies	陆军，不论是事件生成的还是可以建造的（灵能部队/基因部队等）
ascension_perks	飞升，一切飞升的集合（三大飞升等）
asteroid_belts	小行星带，用于定义小行星带的模型
attitudes	用于定义 AI 的态度类型（谨慎/中立/敌对等）
bombardment_stances	用于定义轨道轰炸的类型（末日轰炸/有限轰炸等）
buildings	建筑，各类建筑的集合（合金厂/剧院等）
button_effects	用于自定义 UI 界面的按钮效果
bypass	用于定义星门/虫洞类设施相关（就是可以从一个星系传送到

	另一个星系的玩意)
<u>casus_belli</u>	战争借口，用于定义战争的借口（全面战争/侵略等）
<u>colony_automation</u>	定义殖民地自动化建造模板内容
<u>colony_automation_exceptions</u>	定义一般殖民地以外的自动化建造模板（犯罪星球之类的）
<u>colony_automation_categoress</u>	定义殖民地自动化建造模板类别
<u>colony_types</u>	殖民地类别，定义殖民地的类别（精炼星球/科研星球等）
<u>colors</u>	颜色，用于国家颜色选择的定义
<u>component_sets</u>	舰船部件的定义文件
<u>component_tags</u>	用于将修正器和舰船的组件关联在一起（能量/动能等）
<u>component_templates</u>	舰船部件，每种武器的属性设置（伤害/槽位/冷却等）
<u>country_container</u>	用于定义国家组成部分所属于的经济分类，例如收支类型的自定义修正本地化，需要同时修改 economic_categories 才有效
<u>country_customization</u>	定义给予各种帝国定制化的操作，给予自动生成帝国的旗帜与物种肖像定制（说白了就是让某一类国家旗帜趋向于同质化，不同国家体现差异化，蠢驴只给了机械，蜂巢和普通帝国的定制）
<u>country_limits</u>	蠢驴抽风的表现条目之一，定义各类船只的容量以及使用限制
<u>country_types</u>	定义国家类型，设置他们的规则（堕落/普通/天灾/各种野怪/中立势力等）

crisis_levels	定义化身天灾的天灾等级（可修改内容极少）
decisions	星球法令，生效在星球上的各种法令（促进人口增长/宣布禁止人口增长等）
defines	设置游戏的基本参数
deposits	地块，设置星球上的行星地块（高山/富矿/海岛/城市废墟/工业废土等），包括非殖民星球的资源产出。
deposit_categories	定义地块的经济类别以及一些其他属性（稀有，障碍等性质）
diplo_phrases	外交语言触发，设置国家间使用的外交用语的逻辑与触发（敌对语句触发/交战语句触发等）
diplomacy_economy	定义外交行动的经济条件类型以及所需条件
diplomatic_actions	定义外交行动，目前只能修改现有的，无法新增
districts	区划，设置各种类型的行星区划（城市/发电/矿产/农业等）
dynamic_text	目前作用未知
<u>edicts</u>	法令，设置各国所可用法令（封圣/科研补贴/圣地/外交补贴等）
ethics	思潮，设置各种思潮，以及对应的国家修正（军国/极端和平等），若需要设置开局可选，必须修改 UI 文件
ethic_categories	定义思潮类别
espionage_assets	定义间谍行动的线人
espionage_operation_categories	定义间谍行动的类别
espionage_operation_types	定义间谍行动的具体内容

event_chains	事件链，设置可用的事件链
economic_categories	经济分类文件。用于定义整个游戏内所有的经济收益（区分是什么部分，整个部分是否有加成，有什么加成等）*注意，有些内容和硬代码相关，不要更改！
economic_plans	AI 的经济计划，设定 AI 的经济收入目标以驱动 AI 运营
fallen_empires	堕落帝国，设置可被生成的堕落帝国
federation_law_categories	联邦法律类型
federation_laws	联邦法律，具体的内容
federation_perks	联邦加成，用于提升联邦等级后的加成
federation_types	联邦类型，如霸权联邦，贸易联盟等
first_contact	首次接触，用于首次接触的特殊项目（其实就是坟头），由于涉及到了一定的硬代码，不建议修改。
galactic_community_actions	定义星海共同体主席国/皇帝的行动花销（目前只能修改经济类别与操作成本，用处不大）
galactic_focuses	定义星海共同体的银河焦点内容
game_rules	用于各种游戏操作的逻辑，即是否可以奴役 pop。这里的逻辑不会覆盖硬编码规则
global_ship_designs	设置全局具体舰船配置，供以各类刷船使用
<u>governments</u>	设置公民性（国策）、政体、政府形式（矿业工会/寡头制/同甘共苦等）以及起源
graphical_culture	定义图形文化以供其他地方使用（开局让你选母星背景的地

	方)
leader_classes	领袖等级，定义领袖等级的相关设定
inline_scripts	内联脚本，将会单独开一个章节介绍
map_modes	地图模式，可以允许一定条件下用不同颜色显示银河系，不需要改动 UI
megastructures	巨构建筑相关
message_types	用于 UI 上方提示信息内容，原版就是被宣战/科技研发完成/探索完成等
name_lists	名称列表，定义不同种族的不同随机名称
observation_station_missions	土著观察任务，定义观察任务的类型（被动观察/主动观察等）
on_actions	触发器，设置各类需要特定触发的事件列表
opinion_modifiers	关系修正，一个国家对于另一个国家的关系的修正（有利的贸易协定+100 等）
personalities	AI 性格，由于 AI 的角色扮演设置（民主斗士/奴隶暴君等）
planet_classes	星球类别，所有星球，包括恒星的设置（G 级恒星/大陆星球等）
planet_modifiers	星球修正器，用于设置星系生成时自然添加的星球 buff 的概率设定
policies	国家政策，所有的国家政策的集合（奴隶制/战争哲学/轰炸政策/经济政策等）
pop_categories	人口阶级（统治者/专家/劳工/复杂子人口等）

pop_faction_types	党派，设置不同方面的党派生成和需求（军国主义派系/孤立主义派系等）
pop_jobs	人口职业，所有工作的集合（矿工/农夫/科研人员/工匠等）
relics	遗珍，比如银河之星
random_names	随机名称，定义随机生成的名称（帝国/党派等）
scripted_effects	封装触发的效果，一般用于事件效果（帝国生成/小灰转化等）
scripted_loc	封装触发的文本，用于经常被引用的文本生成
scripted_triggers	封装条件判断，用于经常被引用的事件判断
scripted_variables	常量，用于经常被引用的数值（科技数值等）
script_values	脚本值，用于复杂变量操作
section_templates	船只的分段，例如战列舰有三个分段，这里就是定义某一个分段的文件
ship_behaviors	船只行为，用于设置舰船以及舰载机的船只行为
ship_sizes	舰船类型，例如普通帝国的战列舰，不管用什么分段，基础情况下这个战列舰就算是一个“类型”
solar_system_initializers	恒星系生成器，用于一切恒星系的生成
special_projects	特殊项目，所有特殊项目的集合
species_archetypes	效果是设定一个种族类别最大的修饰特征数目（肉体/机械等）
species_classes	种族类别，例如所有的哺乳类算是一个类别
species_names	种族随机名称，设置一个种族类别可以使用的随机名称

species_rights	种族权利，设置种族可以被赋予的权利（公民性/居留权等）
star_classes	恒星系类别，不同于星球等级，这个是设置整个恒星系的类别（G 恒星系/双恒星系等）
starbase_buildings	太空基地建筑，比如舰队学院等
starbase_levels	太空基地等级
starbase_modules	太空基地模块，比如炮台或者船坞等
starbase_types	太空基地类别，主要用于解锁相对应的太空基地建筑的建造
start_screen_messages	开始窗口的文本信息，主要是用于在开局描述这个国家的概括，或者设置自动生成的文本
static_modifiers	固定修正器，几乎所有的修正的都在这里，不管是星球还是帝国
strategic_resources	资源，各种资源，包括能源，矿物，合金等（甚至还有时间）
technology	科技，所有的科技的集合
terraform	地形改造，所有的可改造地形的方案
trade_conversions	贸易转化，定义贸易转化相关（1 贸易值换 1 电/1 贸易值换 0.5 点和 0.15 凝聚力等）
tradition_categories	传统树，定义某个类别的传统
traditions	传统，具体定义在某个传统类别的具体传统
traits	特质，不论是种族特质还是领袖特质
war_goals	具体的可选择战争项目（羞辱/巨像等）

2.2.2 其他一些比较常用和重要的文件

Stellaris/events 事件相关，所有的事件的集合，不论是隐藏的还是非隐藏的

Stellaris/flags 帝国旗帜，包括背景

Stellaris/fonts 字体，定义不同情况下的使用字体情况

Stellaris/gfx 图像，最重要的图像文件夹，包括建模，粒子效果，贴图等[待补充]

Stellaris/interface 这里面定义了 gui 和一些贴图的路径等

Stellaris/localisation 本地化文件，不论是汉语还是英语亦或者是其他奇奇怪怪的语言

Stellaris/localisation_synced 一些特有名称的本地化

Stellaris/music 音乐，注意这不是音效

Stellaris/map 银河系地形地图，但修改幅度很有限

Stellaris/sound 音效，注意这不是音乐

Stellaris/prescribed_countries 预设国家设定，在这里你可以看到弟联和人联等

2.3 工具准备

2.3.1 编辑器的选择

工欲善其事，必先利其器。由于蠢驴的大部分 Mod 文件都是.txt 文件，首选建议采用

VSC(Visual Studio Code)进行 Mod 编写，当然你也可以选择 N++,UE 等。



VSC 官网: <https://code.visualstudio.com/>

当然，其他文本编辑器也是可以的，如下是我推荐的其他几个文本编辑器，不过没有对编写蠢驴语言有帮助的插件。

Sublime Text3: <http://www.sublimetext.com/3>

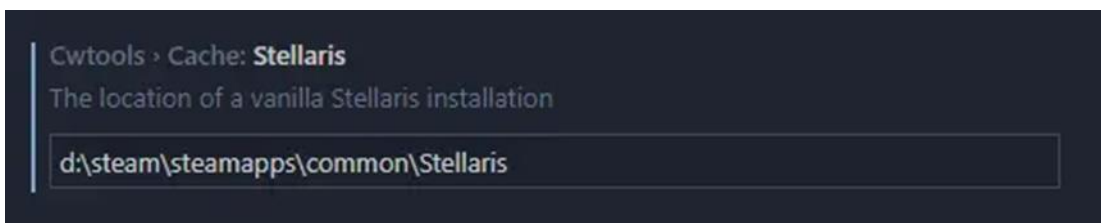
NotePad++: <https://notepad-plus-plus.org/>

2.3.2 CWTools 插件安装

在 VSC 中有一个蠢驴全家桶插件，功能较为完善。该插件支持错误提示，自动补齐，生成科技树、事件树，一键生成本地化 Key 等等一系列功能。在进行编写的时候还能够查看一堆东西，以及提示和补全代码，非常方便。该插件还自带了一个语法高亮，不过配色有点花哨，建议配合其他主题组合使用。

如何安装插件：

- a. 拓展中搜索 CWTools，点击安装
- b. 右键安装的 CWTolls，点击拓展设置
- c. 将群星(Stellaris)的根目录路径复制到 Stellaris 一栏中



- d. 重启 VSC，通过 VSC 打开文件夹，插件将会自动加载。

注意，此 mod 必须在默认文件夹当中，也就是路径为 Paradox Interactive\Stellaris\mod 下才能生效，并且请用文件夹形式打开你 mod 的根目录：



这样就将你所有 mod 内容全部加载入 CWT，这样做的好处就是避免做一个 mod 就要到处翻查，以充分节省时间与精力。

2.3.3 Paradox Language Support

[Paradox Language Support](#) (以下简称 PLS) 是由 @微风的龙骑士 开发的一个 [IntelliJ IDEA](#) 插件，对蠢驴的语言提供了较为完善的支持。有了它，你就可以使用强大的 IDEA IDE 开发 Stellaris (以及其他 P 社游戏) 的 Mod，使用并享受其所提供的一系列强大、便捷而实用的功能。例如：代码高亮、引用解析、快速文档、代码导航、代码检查、代码补全、内嵌提示、动态模版，差异比较，等等。这个插件也支持在快速文档和内嵌提示中渲染本地化文本和 DDS 图片，也能用来直接在 IDE 中打开以及导航到 DDS 图片和 CWT 规则文件。

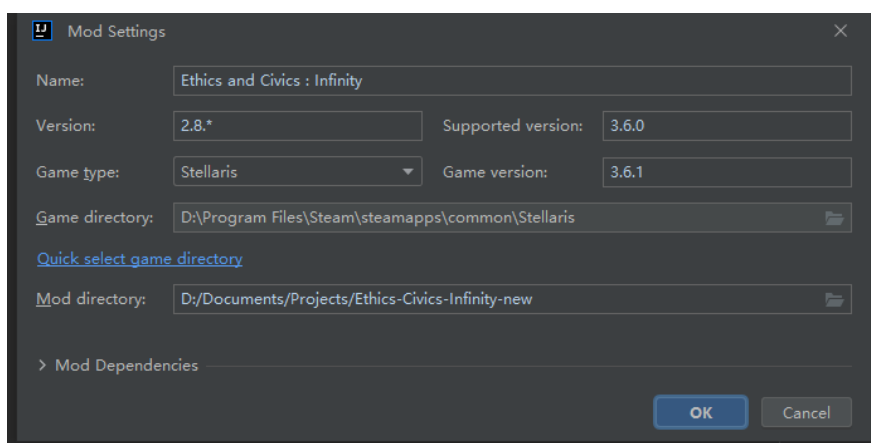
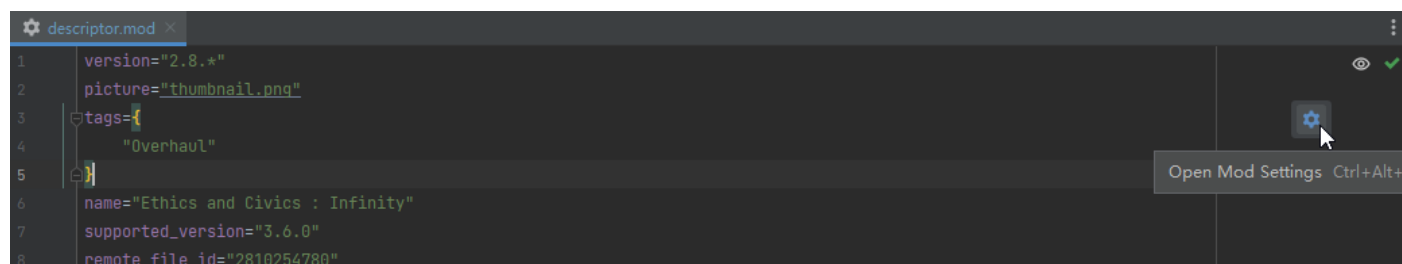
快速上手：

1. 下载最新版本(编写教程时为 2022.2) 的 IntelliJ IDEA。(直接使用社区版即可，也可以使用 JB 全家桶中的其他 IDE，例如 PyCharm)
2. 通过 IDE 打开你的模组根目录。(目前要求直接包含模组描述符文件 descriptor.mod，也可以用来打开其他游戏的模组根目录以及游戏目录，或者打开的项目目录中包含这类目录)
3. 打开模组描述符文件。(如果你没有安装此插件，此时 IDEA 应当会提示你安装，安装完成后需

要重启 IDE)



4. 点击位于编辑器右上角的悬浮工具栏（或者编辑器右键菜单）中的模组配置按钮，配置模组的游戏类型、游戏目录和模组依赖。（现已支持从启动器的播放集 json 文件导入导出模组依赖！）



5. 点击确定按钮完成配置，然后等待 IDE 索引完成。（很快就好）

6. 开始你的模组编程之旅吧！

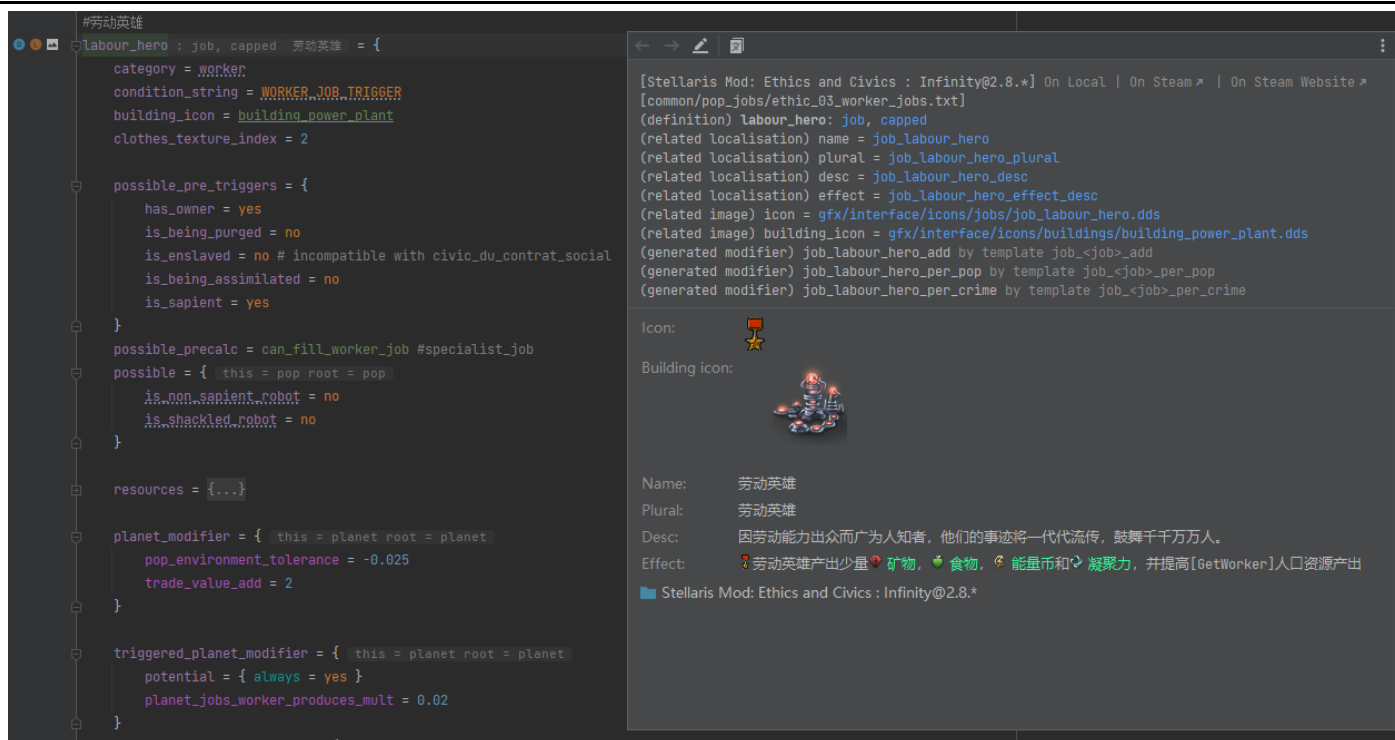
FAQ:

Q: 我为什么要放弃小巧、流行的 VSC，转而使用臃肿、难用的 IDEA？

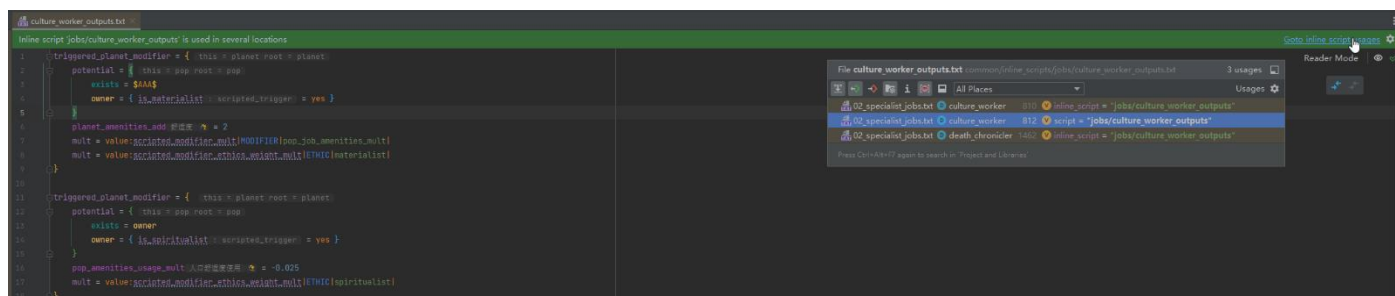
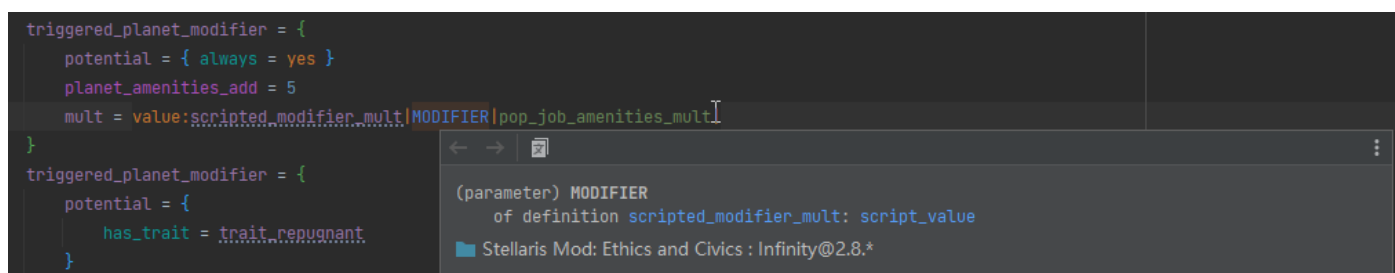
A: 因为 IDEA 相比 VSC 更加强大，这意味着 PLS 相比 CWTools 也可以更加强大。

Q: 那么请告诉我 PLS 相比 CWTools 有哪些独到的功能？

A: 要说目前最亮眼的功能，大概就是通过快速文档或者直接通过内嵌提示，提示包括本地化文本、DDS 图片、作用域上下文在内的各种信息了。



除此之外 PLS 对 SV (script_value) 和内联脚本 (inline_script) 也提供了相当完善的支持。



Q: 那请你也说说它目前的不足之处。

A: 相比 CWTools 以及个人预期，插件目前存在以下几点缺陷。

- 无法生成本地化。
- 无法生成事件树、科技树等图表。
- 尚不支持基于具体的使用处推断 scripted_trigger、scripted_effect 等的作用域上下文。

- 尚不支持 [Irony Mod Manager](#) 提供的一些高级功能。例如模组合并、冲突检查。
- 对于其他 P 社游戏和游戏模组的支持可能并不理想。

Q: 我想提出需求或者反馈 BUG。

A: 如果有任何问题, 欢迎通过 [Github 的 Issue 页面](#) 反馈。当然, 如果你能直接联系到作者本人, 你可以直接私聊……

2.4 粒子编辑器

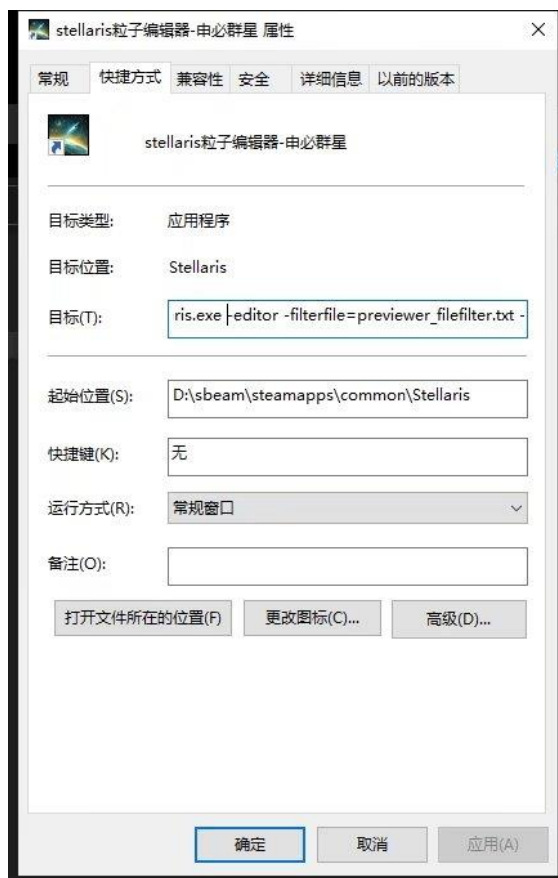
如果想要预览蠢驴的各种模型以及粒子, 就可以启用粒子编辑器进行查看。首先, 粒子编辑器并非一个独立的应用程序, 而是包含在游戏主程序中的一个分支程序(这么说应该不严谨, 不过明白意思就行)。

为了使用粒子编辑器, 首先需要找到 stellaris.exe 程序位置(不是 STEAM 创建的桌面图标)右键复制, 并于任意位置[粘贴快捷方式], 然后右键此快捷方式, 选择属性, 在[目标]处的路径后面添加:

```
-editor -filterfile=previewer_filefilter.txt -mount="C:\Users\admin\Documents\Paradox Interactive\Stellaris\mod\[你 MOD 的根文件夹]"
```

注意每个” -” 之前都有且只有一个空格, 其余位置没有空格。

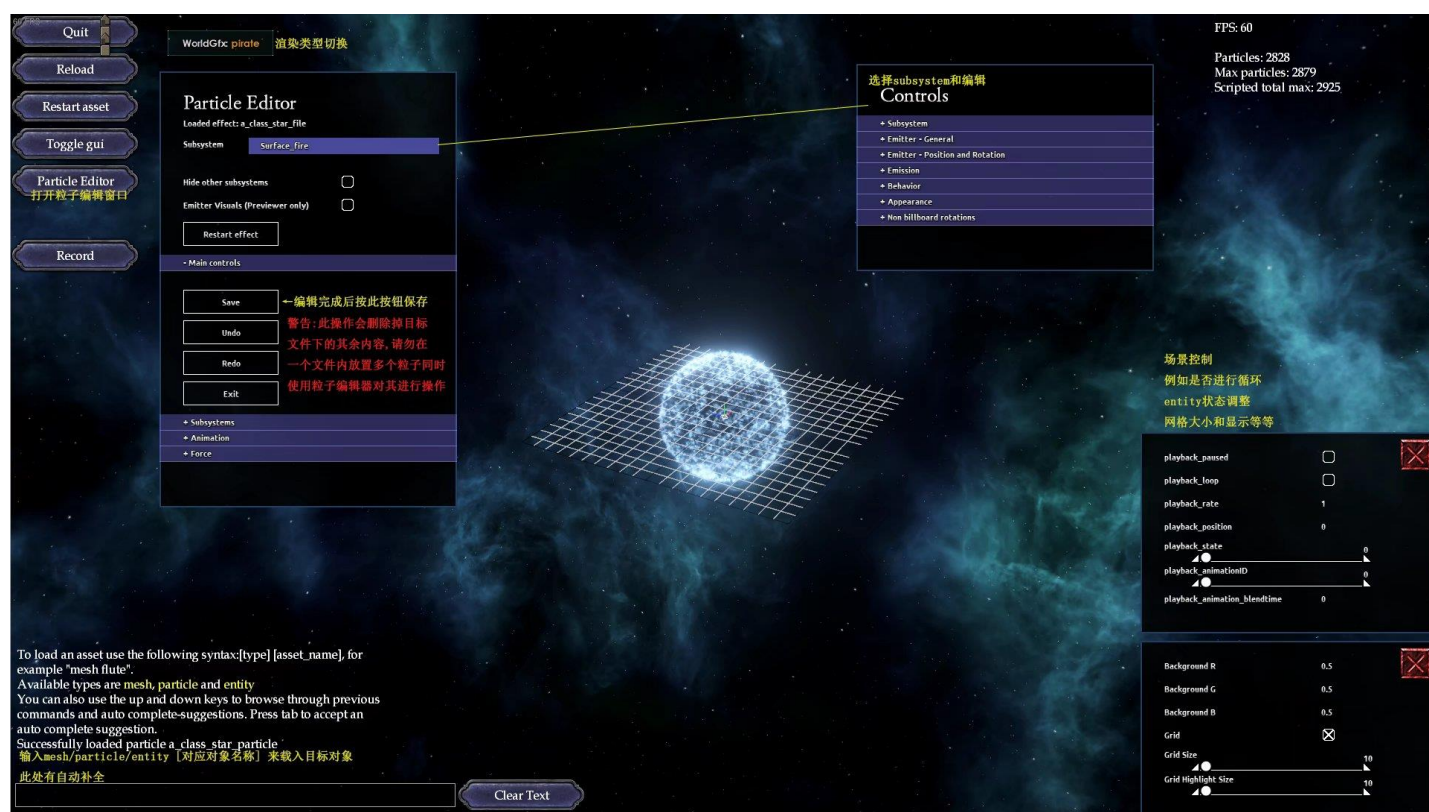
如果你的 MOD 文件夹位于其他位置, Mount 处的路径也需要相应修改来指向正确位置。



随后,前往 Stellaris\previewer_assets 文件夹内,打开 previewer_filefilter.txt,将文件内的几个注释符”#”删除(P社默认注释了它们,不知为何)。

```
D: > sbeam > steamapps > common > Stellaris > previewer_assets > previewer_filefilter.txt
1  assetviewer_interface/*
2  interface/tweaker*
3
4  interface/core.gfx
5  interface/general_stuff.gfx
6  interface/console.*
7  interface/fonts.gfx
8
9  gfx/worldgfx/*
10 gfx/lights/*
11 gfx/FX/*
12 gfx/models/*
13 gfx/particles/*
14 #gfx/map/*
15
16 fonts/newsfeed_*
17 FX/newsfeed_*
18 interface/online_*
19 interface/test_asset*
20 interface/editor_view*
21 interface/planet_view.gfx
```


然后就可以打开粒子编辑器并使用了,部分重要按钮参考下图



注意:有时候加载 MOD 粒子后试图打开编辑器窗口会直接崩溃,此时可以先打开一个原版粒子,打开窗口,再加载 MOD 粒子。

或是打开 MOD 粒子,再打开原版粒子,打开编辑器,再通过左上角 PREV 切换上一个粒子。

三、Common Modding

3.1 .议程(agenda)s修改

3.1.1 认识议程的组成

上面讲了那么多,不如实际自己操作一把,我们先仿照 stellaris 文件夹的内容,创建一个 common 文件夹,之后里面再复制 stellaris 原文件的 agendas.txt, 然后改名为 test_agendas.txt 之后删除里面的内容。(不必拘泥于该条目,由于蠢驴的问题该条目显示并不清晰,建议移步[法令部](#)

分开始学习)

让我们先学习一下原版是怎么添加议程的：

```
#范围是引用具有此议程的领导者
agenda_naval_focus = {      #agenda_naval_focus 指的是这个议程在游戏里面的 ID
    weight_modifier = { #权重修正器，这个指这个议程能被刷出来的权重（概率）的修改
        weight = 100      #基础值
        modifier = {
            factor = 1.5    # from = 国家，所以这里指的是国家拥有 ethic_militarist 时的修正
            from = {        #factor 为乘法，即拥有 ethic_militarist 权重乘以 1.5 倍
                has_ethic = ethic_militarist
            }
        }
    }
    modifier = {            #同理，即拥有 ethic_fanatic_militarist 权重乘以 2 倍
        factor = 2
        from = {
            has_ethic = ethic_fanatic_militarist
        }
    }
    modifier = {          #leader_class 这里指的是担任帝国元首的领袖曾经的类别（将军什么的）
        factor = 1.5
        leader_class = admiral
    }
}

modifier = {      #修正器，指对于这个帝国的修正（buff 或者 debuff）
    starbase_shipyard_build_cost_mult = -0.10    #太空基地船厂的造船建造花费-10%
    ships_upkeep_mult = -0.05    #舰队维护费-5%
    starbase_shipyard_build_speed_mult = 0.20    #太空基地船厂的造船建造速度+20%
}
}
```

从上面的例子，我们可以看得出，议程包括以下几个模块

1.议程的名称

2.刷出的权重（包括对于权重的修改）

3.修正器（国家的 buff 或者 debuff）

3.12 添加自己的第一个议程

在学习到了一个议程有怎么样的组成之后，现在让我们开始真正的动工，写下你的 mod 里面第一个内容——你的第一个议程 9

值得注意的是，Stellaris 里面不论是什么项目的 ID，都应该是**唯一且确定**的，除非你是修改原版的议程，不然一定要确认原版是否已经使用了这个 ID，比如这次我们起一个“agenda_test”名称的，那么现在应该看起来是这样的(“#”后面的内容并不会被游戏所读取，所以一般用于添加注释)

```
#我的第一个议程
agenda_test = {
}
}
```

现在看起来空空如也，一点也不好看，那么我们仿照上面的例子，添加一个**权重**

```
#我的第一个议程
agenda_test = {
    weight_modifier = {
        weight = 10000 #一般情况下 100 为比较常见的数值，但是这里为了能我们的议程效果被很容易的看见
                        #所以写成了 10000，即正常值的 100 倍
    }
}
```

在添加了权重之后，自然是各位第四天灾喜闻乐见的 buff 环节，众所周知科技是第一生产力，那么这次我们就写一个**科学研究速度+50%**的议程吧

```
#我的第一个议程
agenda_test = {
    weight_modifier = {
        weight = 10000 #一般情况下 100 为比较常见的数值，但是这里为了能我们的议程效果被很容易的看见
                        #所以写成了 10000，即正常值的 100 倍
    }
    modifier = {
        all_technology_research_speed = 0.50 #帝国科研速度+50%
    }
}
```

好，到此我们保存我们的 mod 文件，效果应该如上图所示。之后我们需要打开一次 stellaris 来

验证是否真正的在游戏里面被添加了这个“agenda_test”议程。启动游戏，加载我们的“test” MOD！之后随意开一局是寡头政体的国家 0

当你发现你的议程是像图 8 那样显示的时候，那么恭喜你，你的 mod 被成功读取且还能正常显示了，不过没有显示的也不要灰心，开启控制台，刷影响力后刷选举，如果几次之后能看得见自己的议程，那么证明也是成功的。不过如果确实找不到，那么还请检查是否按照了上面的操作，或者是检查括号的位置，相信一定可以成功的。



图 7

不过对于这样强力的议程来说，我们一般不想要任何帝国都能刷到这个议程，这个时候就要对刷出这个议程的权重做出调整和某些限制，比如就拿这个议程而言，我们想要唯物（物质）主义才能获得，而其他思潮无法刷到，，那么应该这样：如图 9（在这里 NOT 是取反语句，即当目标不符合括号里面的条件的时候成立）

```
#我的第一个议程
agenda_test = {
  weight_modifier = {
    weight = 10000 #一般情况下 100 为比较常见的数值，但是这里为了能我们的议程效果被很容易的看见
                  #所以写成了 10000，即正常值的 100 倍
    modifier = {
      factor = 0
      from = { #factor 为乘法，即当没有唯物主义或者极端唯物主义时，权重乘以 0 倍
        NOT = { has_ethic = ethic_materialist } #ethic_materialist 唯物主义思潮
      }
      NOT = { has_ethic = ethic_fanatic_materialist } #ethic_materialist 极端唯物主义思潮
    }
  }
}
modifier = {
  all_technology_research_speed = 0.50 #帝国科研速度+50%
}
}
```

图 8

当做好这些修改再进入游戏里面，你就会发现如果不选择唯物或者极端唯物，那么永远刷不出来这个议程——而带上了唯物之后，就会刷的出来（效果和之前一样）。这样便实现了我们所需要的判断，即拥有唯物主义或者极端唯物主义可以刷出这个议程，没有则无法刷出。同样的，如果没有实现这样的效果，请确保括号以及其他操作的正确。至于为什么显示的是乱码，那是因为我们还没有填写汉化文本，所以自然也就没有它的文本，而是显示代码。

3.2 本地化 (localisation) 修改

3.2.1 认识本地化文件的组成

以原版的 simp_chinese 文件夹之中的 l_simp_chinese.yml 为例，一个标准的本地化文件的名称应该是：**[项目名称]_l_[语言代码].yml**，同样的，相同名字的本地化文件会在 mod 加载后被 mod 覆盖。任何的本地化文件编码格式都应该是 **UTF-8-BOM**，任何不是这样编码的文件都无法被保证能否正常被游戏读取。当打开 l_simp_chinese.yml 之后，你应该会看见如下的内容：

l_simp_chinese: #这是相应本地化文件的**必要开头**，如果是英文版就是“l_english:”其他语言也都差不多

…… #前面略过

#大多数的本地化项目的格式应该为 **[需要本地化的项目]:O "[相对应的内容]"**

#注意，能被正确引用的都是**英文状态下的双引号**

“§ [色彩代码]”是指从这里开始转化颜色

§! 是上一个颜色转换的结束符号。一般而言，一个“§ [色彩代码]”要搭配一个“§!”

#例如“§ R 这是红色!”效果就是：**这是红色!**

#[色彩代码] B **蓝** E **青** G **绿** H **橙** L **棕** M **紫** P **亮红** R **红** S **深橙** T **灰** W **白** Y **黄** !和上一个色彩代码匹配以结束颜色效果

RELATIVE_POWER:O "相对实力"

RELATIVE_POWER_1:O "§ G 不值一提 §!"

RELATIVE_POWER_1_DESC:O "他们远比我们弱得多。我们随时都可以干掉他们。"

RELATIVE_POWER_2:O "§ G 略逊一筹 §!"

RELATIVE_POWER_2_DESC:0 "他们比我们弱，不需要太过在意。"

RELATIVE_POWER_3:0 " § Y 难分伯仲 § !"

RELATIVE_POWER_3_DESC:0 "他们的实力与我们不分伯仲。"

RELATIVE_POWER_4:0 " § H 更胜一筹 § !"

RELATIVE_POWER_4_DESC:0 "他们比我们厉害。我们可能需要寻找盟友才能抵消双方之间的差距。"

RELATIVE_POWER_5:0 " § R 无与伦比 § !"

RELATIVE_POWER_5_DESC:0 "他们远比我们强大。除非我方有强力的盟友，否则请避免与对方发生冲突。"

#效果分别是不值一提 略逊一筹 难分伯仲 更胜一筹 无与伦比 (底色是为了看清楚显示色, 底色非实际显示效果)

#上面是大家都很熟悉的实力对比, 相信这样应该能对色彩代码有一个初步的了解

下面我们打开 megacorp_1_simp_chinese.yml

```
#"$"代码是引用已经本地化的内容，像这次举出的实例"$job_researcher_plural$"就是会显示“研究人员”
#注意：一定要确认引用内容的准确性，每一个字母都不能错，不然就会显示空格
#"£"代码是引用图标，像这个"£consumer_goods£"就是指“ ”这个图标
job_researcher:0 "研究人员"
job_researcher_plural:0 "研究人员"
job_researcher_desc:0 "探索神秘世界的道路永无止境。"
job_researcher_effect_desc:0 "£job_researcher $job_researcher_plural$将
£consumer_goods£ $Y$consumer_goods$$!变为£physics£society£ £engineering£ $Y 研究点数$!，然后
将其用于新科技"
mod_job_researcher_add:0 "$job_researcher$职业"
mod_job_researcher_per_pop:0 "每$VALUE$个人口有$G+1$!个$job_researcher$职业"
mod_job_researcher_per_pop_short:0 "\n 每$VALUE$个£pop£有$G+1$!个£mod_job_researcher_add"
..... #略过
#"\\n"代码的效果就是按了一次回车（分行）
d_betharian_deposit:0 "发现贝塞里安石\\n£minerals£ +2"
```

至于更多的本地化内容，我将会在后面的章节系统性阐述，这里仅仅是为了简单的本地化我们之前做的议程。

3.2.2 为第一个议程写一个本地化文件

之前说了如此多的理论内容，接下来便是实际的操作。类似的，复制 localisation 文件夹，删除除了 simp_chinese 之外的文件夹，之后复制任意一个 .yaml 文件之后，将其改名为 test_1_simp_chinese.yaml，打开，除了第一行的 "1_simp_chinese:" 其他都删了。随后删除除了 test_1_simp_chinese.yaml 之外的全部 .yaml 文件。之后输入

```
1_simp_chinese:
agenda_test:0 "我的议程" #由于之前的议程名称为 agenda_test，所以应该这样写
```

保存，进入游戏。记得带唯物，不然不会刷出来（详见前面的第三章内容），如果操作无误，应该是图 10 的效果。我们可以看见蠢驴很贴心的给我们的名称上了橙色，但是堂堂第四天灾的议程怎么能和普通的一样？所以再次打开刚刚的本地化文件改成：

```
1_simp_chinese:
agenda_test:0 "$M 我的议程$!" #由于之前的议程名称为 agenda_test，所以应该这样写
```

这样一来，打开游戏，就会显示出图 11 的效果，这样就显得与众不同了。当然根据上述所学的代码，还可以搞出一些很生动的本地化效果，不过现阶段就先讲述到此。



图 9

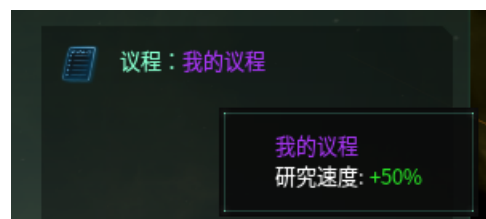


图 10

3.3 科技（technology）的修改

3.3.1 认识科技的组成

以原版的 technology 文件夹下的 00_eng_tech.txt 为例

```
tech_corvettes = { #同样的，tech_corvettes 为 ID，这个科技是初始的护卫舰科技
    cost = 0 #这项科技的花费
    area = engineering #分类，指的是工程学（物理学/社会学/工程学）
    start_tech = yes #为初始科技，= no 就不是（不过一般非初始科技都不写这行）
    category = { voidcraft } #具体的分类，这里指的是太空飞船（灵能学/政治学/军事学等）
    prerequisites = { "tech_starbase_2" } #前置科技，当没有研究前置科技之前，无法刷到本科技
    tier = 0 #是多少级的科技？这里写的是 0 级科技
    #一个 n 级科技需要最少 6 项(n-1)级科技的研究完成，才会被刷出来
    prereqfor_desc = { #提示文本，例如“这项科技可以带给我们 xxx 领域的进步”等
        ship = {
            title = "TECH_UNLOCK_CORVETTE_CONSTRUCTION_TITLE" #文本标题
            desc = "TECH_UNLOCK_CORVETTE_CONSTRUCTION_DESC" #内容
        }
    }
}
```



```

tech_destroyers = {      #同样的, tech_destroyers 为 ID, 这个科技是驱逐舰科技
    cost = @tier2cost1    #@指引用数值, common\scripted_variables 目录下里面有详细数值, 本项为 2000
    area = engineering    #分类: 工程学
    tier = 2              #是 2 级科技
    category = { voidcraft }      #具体分类: 太空飞船
    prerequisites = { "tech_corvettes" }    #前置科技: 护卫舰
    weight = @tier2weight1      #引用数值, 为 85

    gateway = ship #指通向舰船科技的大门 (据说可以提升权重, 但目前而言看起来似乎没什么效果, 待补充)
    ## unlock destroyer
    prereqfor_desc = {      #文本
        ship = {
            title = "TECH_UNLOCK_DESTROYER_CONSTRUCTION_TITLE"
            desc = "TECH_UNLOCK_DESTROYER_CONSTRUCTION_DESC"
        }
    }
    modifier = {      #修正器, 这里的修正范围是整个帝国
        country_command_limit_add = 10      #舰队控制上限+10
    }
    weight_modifier = { #权重修改器, 在符合条件的时候, 改变权重

```

```

    modifier = {
        factor = 0.1
        NOT = { years_passed > 10 }      #当开局没超过 10 年，权重乘以 0.1
    }
    modifier = {
        factor = 10
        any_neighbor_country = {
            has_technology = tech_destroyers #拥有护卫舰科技权重乘以 10
        }
    }
    modifier = {
        factor = 2
        years_passed > 15      #开局超过 15 年，权重乘以 2
    }
    modifier = {
        factor = 3
        years_passed > 20      #开局超过 15 年，权重乘以 3
    }
    modifier = {
        factor = 4
        years_passed > 30      #开局超过 15 年，权重乘以 4
    }
    modifier = {
        factor = 1.25
        has_tradition = tr_supremacy_adopt      #树立霸权传统，则权重乘以 1.25
    }
    modifier = {
        factor = 1.25
        research_leader = {      #当工程学科研究的领导科学家拥有太空飞船专长特质时，权重乘 1.25
            area = engineering
            has_trait = "leader_trait_expertise_voidcraft"
        }
    }
}

ai_weight = {      #AI 的刷出权重，如果设置为 0 则 AI 永远不会研究它
    factor = 100 #important tech
    modifier = {
        factor = 1.25
        research_leader = {      #当工程学科研究的领导科学家拥有太空飞船专长特质时，权重乘 1.25
            area = engineering
            has_trait = "leader_trait_expertise_voidcraft"
        }
    }
}
}

```

从上面的例子，我们可以看得出，科技应该包括以下几个模块

- 1.科技的名称，花费，等级，类别，种类等
- 2.刷出的权重以及其修改
- 3.修正器(modifier)（国家的 buff 或者 debuff）
- 4.AI 的使用权重以及其修改
- 5.其他的杂项

3.3.2 修改（创造）新科技

同样的，将 technology 文件夹下的全部内容删除，保留一个 txt 并删除其之中内容，重命名为“00_test_tech.txt”这次演示做一个科技，效果是“来自工作产生的能量+50%和舰队容量+100”，类似第三章的操作，完成如下的操作：

```

tech_test = {
    cost = @tier2cost3      #@指引用数值
    area = society          #社会学领域
    tier = 1
    category = { military_theory } #军事科技

    weight = 10000 #一般情况下 100 为比较常见的数值，但是这里为了能我们的议程效果被很容易的看见
                  #所以写成了 10000，即正常值的 100 倍

    prereqfor_desc = {      #文本
        ship = {
            title = "TECH_MY_TEST_TITLE"
            desc = "TECH_MY_TEST_DESC"
        }
    }

    weight_modifier = {
        modifier = {        #这里的默认范围是帝国，所以不需要表示范围的语句（例如 from/owner 等）
            factor = 0       #factor 为乘法，即当没有唯物主义或者极端唯物主义时，权重乘以 0 倍
            NOR = {
                has_ethic = ethic_materialist #ethic_materialist 唯物主义思潮
                has_ethic = ethic_fanatic_materialist #ethic_materialist 极端唯物主义思潮
            }
        }
    }
}

modifier = {
    planet_jobs_energy_produces_mult = 0.50 #来自职业的能源产出+50%
    country_naval_cap_add = 100 #舰队容量+100
}

ai_weight = {
    factor = 0
}
}

```

和议程一样，没有本地化文本一样无法显示我们想要的内容，仿照第四章的做法，我们在我们的本地化文件之中加上以下内容：

```

tech_test:0 "唯物革命！"
tech_test_desc:0 "呵呵哈哈哈哈哈！我们是睿智的唯物主义！" #_desc 代表这个科技的说明和介绍
TECH_MY_TEST_TITLE:0 "这是唯物革命！"
TECH_MY_TEST_DESC:0 "唯物革命能极大的提升我国实力！！!"

```

进入游戏，惯例是唯物，应该会看见以下内容（图 12-15），不过我们应该发现了，我们的科技并

没有图标，那么接下来就是添加科技的图标。



图 11



图 12



图 13



图 14

3.3.3 为你的 MOD 添加第一个图标

在添加自己的图标之前，先来看看蠢驴的图标都放到哪里：

[Stellaris\gfx\interface\icons\technologies](#) 这里面都是原版 stellaris 的科技图标，如果你只是

想简单的套用蠢驴的图标，那么很简单，只要在刚刚的写的科技里面加上这样一句“icon = [科技图标名称]”例如 icon = tech_manifest_destiny，那么就完成了。

不过如果你想要自己添加一个独一无二的图标，那么就得编辑.dds 文件了。Photoshop 有能修改.dds 文件的插件，本教程不提供 PS 极其插件的下载链接。在此我先默认大家都有这样的条件。同样的，仿照 stellaris 的科技图标目录建立文件夹，之后只留下一个 tech_manifest_destiny.dds 文件，用 PS 打开之后根据自己的喜好修改（本教程不提供 PS 方面的教程，还请见谅）比如我的就像是图 16 那样，保存时应该按照图 17 那样设置，这样可以使得你的贴图拥有最高的质量，不易发生失真，保存名字应该与科技同名，这样方便之后查询以及调用。

进入查看效果，我的效果是这样，红框应该显示了你修改之后的图标，倘若存在错误，那么请检查是不是正确的引用了这个图标。如果你想要科技看起来是稀有科技或者危险科技，只需要在刚刚的科技里面打上“is_rare = yes”（是**稀有科技**）或者“is_dangerous = yes”（是**危险科技**）那么对于科技的简易修改，就先到这里了。



图 15

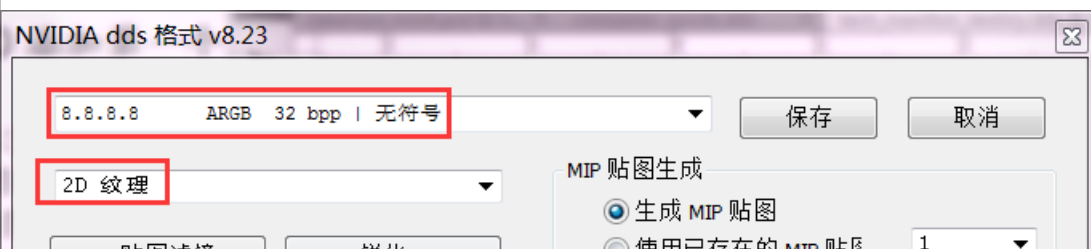


图 16



图 17

3.4 法令 (edicts) 修改

3.4.1 法令 (edicts) 的组成与修改

法令(edicts)存在于 `Stellaris\common\edicts` 里，主要生效于整个国家，多余的话不多说，下面是一个实例，用于了解法令的基础结构。

```
farming_subsidies = {   #ID
    length = @EdictDuration #时长，单位为天@指引用数值
    resources = {       #消耗的资源
        category = edicts #消耗的类型为法令
        cost = {
            influence = @Edict2Cost #具体消耗的资源 @指引用数值
        }
    }

    #is_ambition = yes 是否是凝聚力雄心？如果为真则消耗为解锁下一个传统所需要的凝聚力值，且不必
    #填写 resources 语句（如果你这两个都填写了，会出现很奇怪的效果）

    show_tech_unlock_if = { NOT = { has_authority = auth_machine_intelligence } }
    #当什么情况下显示科技解锁它
    modifier = {           #修正器，为国家范围
        country_food_produces_mult = 0.20
    }

    prerequisites = {"tech_eco_simulation"}#一般填写需要的前置科技
    #allow = {} #allow 语句决定这个法令是不是可用，如果为否则显示不可点击
    #effect = {} #该语句为启用这个法令后的效果。几乎可以添加所有的效果（如果你保证这不会炸了 CPU
    #的话）
    potential = {#限制，满足这里的条件则可以被使用，如果为否则无法被看见，哪怕 allow 语句满足也不行。
        NOT = { has_authority = auth_machine_intelligence }
    }

    ai_weight = {          #AI 使用这个法令的权重
        weight = 1
    }
}
```

法令相对简单，**不需要图标，只需要本地化文本**。不过最关键的是设置**解锁其的科技或者其他条件**或者判断**这个法令是否可用的逻辑语句**，保持基础的逻辑自治。除非你想让唯物主义可以设置圣地

朝圣。

注意: 在 2.7 版本, 法令内容有了稍许改变, 下图展示了 2.7 版本之后的一个凝聚力法令的写法:

```
test_edict = {  
    length = 100  
    icon = GFX_edict_type_policy #指定图标, 未指定则默认为该图标  
    edict_cap_usage = 1 #法令点消耗数  
    is_ambition = yes #凝聚力法令  
    unity_cost_mult = 0.1 #凝聚力消耗, 修正传统点数花费  
    modifier = { #国家修正  
  
    }  
}
```

一般来说 length 为 0 的法令, edict_cap_usage 应该为 0(当然, 这取决于你)。

3.5 军队 (Armies) 修改

3.5.1 军队 (Armies) 的组成与修改

军队 (Armies) 文件存在于 [Stellaris\common\armies](#) 目录下，下面以一个实例用于了解军队的基本结构与内容，以及它的修改

```

# Clone Armies
clone_army = { #ID
    #defensive = yes    是否是防御军队?
    #is_pop_spawned = yes    是否是由人口自动生成，一般用于防御部队（如果没有 defensive = yes 则会
    产生很奇怪的效果）
    damage = 1.00        #伤害，一倍标准值是 1.50-3.00，这里的数值都是其倍数，下同
    health = 1.00        #生命值，一倍标准为 200
    morale = 1.00        #士气，一倍标准为 200
    #morale_damage = 0.75    士气伤害，标准值为一倍
    collateral_damage = 1.25    #附带伤害
    war_exhaustion = 0.50    #厌战产生，一般防御部队为 0
    time = 30            #生产的基础天数
    icon_frame = 7    #图标，具体图标在 Stellaris\gfx\interface\planetview\army_icon.dds
    pop_limited = no    #是否有人口限制？一般都有人口限制（没有人口限制的都是异端!!!）
    resources = {        #消耗的资源
        category = armies    #类型：军队
        cost = {
            minerals = 75    #具体资源以及其数目
        }
        upkeep = {
            energy = 0.75    #每月的维护费
        }
    }
    prerequisites = { "tech_gene_banks" }    #需要的前置科技

    show_tech_unlock_if = { #当满足什么情况时，显示科技可以解锁它
        OR = {
            NOT = { has_authority = auth_machine_intelligence }
            has_civic = civic_machine_assimilator
        }
    }
    potential = {        #限制，不满足无法被看见 “always = no”代表永远无法建造
        from = {

```

```

        NOR = {
            has_trait = "trait_mechanical"
            has_trait = "trait_machine_unit"
            is_sapient = no
        }
    }
    owner = {
        OR = {
            NOT = { has_authority = auth_machine_intelligence }
            has_civic = civic_machine_assimilator
        }
    }
}
}
}

```

军队相对简单，一般不需要图标，除非你想改 `Stellaris\gfx\interface\planetview\army_icon.dds` 的文件，需要本地化文本。不过最关键的是设置解锁其的科技或者其他条件或者判断这个军队是否可用的逻辑语句，保持基础的逻辑自治。我相信你~~应该不会想让圣卫爆出来一堆机器人军队的...吧？（圣卫：MMP!!GUNA!!!）~~。

3.5.2 军队 (Armies) 图标的修改

不习惯原版的图标？想要自行发挥？当然可以，下面我们来自定义军队图标

`Stellaris\gfx\interface\planetview\army_icon.dds` 是图标的目录，你应该会看见如下的图

(图 19)



图 18

打开它，编辑，注意：每一个小图标都是 **34*34** 的像素，也就是说你的长度**必须是 34 的整数倍**，倘若不为 34 的整数倍，可能会造成无法成功被读取的结果。当你改动之后，记得写入阿尔法图层以实现透明效果。例如我改动之后：



图 19

这样一共有 13 个图标了（原版的 12+1 个我的），不过现在由于没有 `Stellaris\interface\icons.gfx` 里面注册，新的图标是会被读取的，这样我们需要打开 `icons.gfx` 里面，找到下面的语句

```
spriteType = {
    name = "GFX_army_icons"
    textureFile = "gfx/interface/planetview/army_icon.dds"
    noOfFrames = 12
}
```

将其改成为（**这里所述的全部改动都应该在你的 mod 里面复制一份然后修改**）：

```
spriteType = {
    name = "GFX_army_icons"
    textureFile = "gfx/interface/planetview/army_icon.dds"
    noOfFrames = 13 #现在是 13 个图标，自然填写 13
}
```

这样进入游戏，应该会有图 21 的效果：



图 20

这样，一个新增加的图标的完成了。更多的图标也类似于上述的操作，不过一定要注意**长宽必须是 34 的整数倍!!!**

如果你想问为什么，详见本教程第七章 **GFX 文件与 sprite type**。

3.6 种族和领袖特质 (traits)

3.6.1 种族特质修改

种族特质文件在 [Stellaris\common\traits](#) 文件夹下

```
trait_enigmatic_intelligence_failed = { #ID 应该以“trait”开头
    cost = 0 #花费
    initial = no #能否在初始界面被选择
    randomized = no #是否可以随机生成该特质
    modification = no #是否能够被修改
    improves_leaders = yes #是否影响到领袖特质生成
    advanced_trait = yes #是否是高级特质
    opposites = { "trait_intelligent" } #与什么特质互斥
    allowed_archetypes = { BIOLOGICAL } #允许的种族类别（有机，机械等）
    #prerequisites = { tech_xxx } #需要的前置科技
    #icon = "<文件相对路径>" #用于定义图标，默认情况下是 gfx/interface/icons/traits/< ID >.dds
    #custom_tooltip = TRAIT_WOMEN_WEAR_EFFECT #显示的文本，注意：将会覆盖自动生成的修正器文本
    #slave_cost = {energy = 100} #进入奴隶市场被售卖后所得
    #potential_crossbreeding_chance = 1.0#生育优先度，越高的数值代表越有可能是有该特质物种增长
    modifier = { #人口的修正
        planet_jobs_engineering_research_produces_mult = -0.6
        planet_jobs_physics_research_produces_mult = -0.6
        planet_jobs_society_research_produces_mult = -0.6
    }
}
```

种族特质（领袖特质亦同）所需要的图标应该是 **29*29 像素的 8 8 8 8 ARGB 的 dds 文件**。错误的深度会导致贴图色差。同上，种族特质需要本地化和在本地化文件里面的后缀为_desc 项目的说明文本，而“§ L”的颜色效果是原版游戏之中最常用的，其次，对于一些特殊的特质，应该也需要对于 custom_tooltip 的本地化文本，注意如果手动加入 custom_tooltip，**则会覆盖自动生成的文本效果（主要是修正的外部表现）**。

3.6.2 领袖特质修改

类似于种族特质修改，原版文件在 [Stellaris\common\traits](#) 文件夹下

```

leader_trait_unyielding = {#ID 应该以“leader”开头
    cost = 1      #花费，领袖特质里面默认为 1
    modification = no    #是否可修改，领袖特质里面默认不可
    icon = "gfx/interface/icons/traits/leader_traits/leader_trait_unyielding.dds"#图标同上
    modifier = {      #修正器，对于这个领袖所在的区域生效（国家/舰队/陆军/星球等）
        ship_hull_mult = 0.1
        ship_weapon_damage = 0.05
        ship_disengage_chance_mult = -0.33
    }
    #self_modifier = {} #对于自身的修正，比如自身经验获取和招募花费等
    #prerequisites = { tech_xxx } #需要的前置科技
    #custom_tooltip = TRAIT_WOMEN_WEAR_EFFECT #显示的文本，注意：将会覆盖自动生成的修正器文本
    #initial = no    #能否在初始界面被选择
    #randomized = no    #是否可以随机生成该特质
    #requires_traits = {trait_psionic} #拥有哪个特质的人口可以生成带有这个特质的领袖
    leader_trait = { admiral } #属于哪个范围的特质，一般和 leader_class 一样
    leader_class = { admiral } #属于哪个类型的特质，一般和 leader_trait 一样
    opposites = { "leader_trait_nervous" "leader_trait_trickster" } #和什么是互斥的
}

```

需要的事项基本与种族特质一样（本地化，贴图等）

3.7 建筑（buildings）与职业（pop_jobs）修改

原版的建筑文件在 [Stellaris\common\buildings](#) 目录下，职业则在 [Stellaris\common\pop_jobs](#) 目录下，一般而言，建筑不直接生成资源，而是通过提供职业来完成其功能（例如水培农场提供 2 个农夫职业等）。

3.7.1 建筑（buildings）修改

建筑的一般范围是某个星球，提供的任何职业都属于该星球的星球修正，而稳定度、幸福度、贸易值、舒适度、犯罪值等都属于星球修正，而能在 [Stellaris\common\strategic_resources](#) 里面所找到的，都属于资源类型，更容易支持各种修正的添加。上述这些对于职业（pop_jobs）也一样。

官方的例子文件在：[Stellaris\common\buildings\00_example.txt](#)

```
building_research_lab_1 = { #ID 应该以“building”开头
    base_buildtime = 360    #基础建造时间，单位为天
    # capital = yes        #是否是首都建筑
    # can_build = no       #是否可修建
    # can_demolish = no    #是否可被拆除
    # can_be_ruined = no   #能否被废弃
    # can_be_disabled = no #能否被停止运作
    #icon = xxx #图标文件名称，默认情况下和 ID 一样。贴图目录在 gfx\interface\icons\buildings
    category = research    #分类，可以让玩家更好的找到该建筑
    potential = {          #限制，没有满足这些条件则该建筑在可建造列表之中不可见，优先度高于 allow 语句
        NOT = { has_modifier = resort_colony }
        NOT = { has_modifier = slave_colony }
        NOT = { is_planet_class = pc_habitat }
    }
    allow = {              #允许，满足下列条件，则该建筑可以被允许建造
        has_upgraded_capital = yes
    }
    #abort_trigger = {}    #当该触发器满足条件时，建造将被取消
    #ruined_trigger = {}  #当该触发器满足条件时，建筑将被废弃
    #destroy_trigger = {} #当该触发器满足条件时，建筑将被摧毁
}
```

```

# convert_to = {}可被转化的目标[建筑 ID](如果判断为不可用),例如智械帝国征服了蜂群帝国之后, 改变星球首府建筑
#planet_modifier = { } #星球范围内的修正
# country_modifier = {} #国家范围内的修正。
triggered_planet_modifier = {    #条件触发的修正器, 不满足条件则无法获得修正
    potential = {                #条件, 需要全部满足
        exists = owner          #是否存在建筑所有者
        owner = { is_regular_empire = yes } #建筑所有者是常规帝国
    }
    modifier = {                #修正
        job_researcher_add = 2   #job_<职业 ID>_per_pop = xxx 每 1/xxx 人口提供一个职业
    }
}
triggered_planet_modifier = {    #条件触发的修正器
    potential = {
        exists = owner
        owner = { is_hive_empire = yes }    #是蜂群帝国
    }
    modifier = {
        job_brain_drone_add = 2
    }
}
triggered_planet_modifier = {
    potential = {
        exists = owner
        owner = { is_machine_empire = yes } #是智械帝国
    }
    modifier = {
        job_calculator_add = 2
    }
}
resources = {    #资源相关
    category = planet_buildings #类型
    cost = {      #一次性花费
        minerals = 400
    }
    upkeep = {    #维持, 为每月多少
        energy = 4
    }
}
upgrades = {      #升级到什么建筑
    "building_research_lab_2"
}
prerequisites = {    #需要的前置科技

```



```

    "tech_basic_science_lab_1"
  }
  triggered_desc = { #条件触发的文本，满足条件才对玩家显示建筑的文本
    trigger = {
      exists = owner
      owner = { is_regular_empire = yes }
    }
    text = job_researcher_effect_desc #具体的文本
  }
  triggered_desc = { #条件触发的文本
    trigger = {
      exists = owner
      owner = { is_hive_empire = yes }
    }
    text = job_brain_drone_effect_desc
  }
  triggered_desc = { #条件触发的文本
    trigger = {
      exists = owner
      owner = { is_machine_empire = yes }
    }
    text = job_calculator_effect_desc
  }
  ai_weight = { #AI 的建造权重
    weight = 2
    modifier = {
      factor = 0
      num_buildings = { type = building_research_lab_1 value >= 2 }
    }
    # There are already unworked jobs
    # Blocks buildings unworkable by remaining pops (slaves/robots etc)
    # Will be improved... at some point
    modifier = {
      factor = 0
      free_jobs > 0
    }
  }
}

```

建筑需要本地化 (包括有触发器的文本)，贴图 (一般是 98*98 的 8.8.8.8.ARGB 的 dds 文件，

目录在 [Stellaris\gfx\interface\icons\buildings](#))

3.7.2 职业 (pop_jobs) 修改

直接看结构

```
clerk = {  #ID
    category = worker  #阶级
    condition_string = WORKER_JOB_TRIGGER  #用于检测这个工作是否可用的一系列逻辑
    #在 Stellaris\common\scripted_triggers 里面
    possible_precalc = can_fill_worker_job  #能胜任什么工作, game_rule 里面改
    building_icon = building_residence  #这个工作的背景建筑图标, 图标位置同建筑
    clothes_texture_index = 2  #外表, 仅影响外观

    planet_modifier = { #星球修正, 同建筑
        planet_amenities_add = 2
        trade_value_add = 2
    }
    #resources = { } 同建筑, 可以添加生产/消耗等
    # country_modifier = {} 同建筑, 国家范围内的修正。
    #pop_modifier = {} 对于承担该职业的 pop 的修正
    triggered_planet_modifier = {  #条件触发的星球修正, 同建筑
        potential = {
            OR = {
                has_trait = trait_robot_domestic_protocols
                has_trait = trait_charismatic
            }
        }
        modifier = {
            planet_amenities_add = 0.4
        }
    }
    triggered_planet_modifier = {  #条件触发的星球修正
        potential = {
            has_trait = trait_repugnant
        }
        modifier = {
            planet_amenities_add = -0.4
        }
    }
    triggered_planet_modifier = {  #条件触发的星球修正
        potential = {
            OR = {
                has_trait = trait_thrifty
                has_trait = trait_nuismatic_administration
            }
        }
    }
}
```

```

    }
  }
  modifier = {
    trade_value_add = 0.5
  }
}
weight = { #pop 意图工作的权重, 拥有高权重的 pop 会导致低权重的 pop 被替换
  weight = @clerk_job_weight #@指引用数值
  modifier = {
    factor = 0.25
    is_enslaved = yes
    can_take_servant_job = no
  }
  modifier = {
    factor = 2
    OR = {
      is_non_sapient_robot = yes
      is_shackled_robot = yes
    }
    can_take_servant_job = no
  }
  modifier = {
    factor = 1.5
    OR = {
      has_trait = trait_thrifty
      has_trait = trait_nuumismatic_administration
    }
  }
  modifier = {
    factor = 0.1
    can_take_servant_job = yes
  }
}
}

```

工作需要本地化，一般包括：

job_<职业 ID>:0 "占位符"

job_<职业 ID>_plural:0 "表达占位符的复数形式"

job_<职业 ID>_desc:0 "表达占位符的描述"

job_<职业 ID>_effect_desc:0 "显示在建筑里面的占位符的效果, 例如消耗什么生成什么"

mod_job_<职业 ID>_add:0 "\$job_<职业 ID>\$职业"

mod_job_<职业 ID>_per_pop:0 "每\$VALUE\$个人口有 § G+1 § !个\$job_<职业 ID>\$职业"

mod_job_<职业 ID>_per_pop_short:0 "每\$VALUE\$个£pop£有 § G+1 § !个£mod_job_<职业 ID>_add"

也需要图标, 一般包括:

job_<职业 ID>的图标, 原版在 [Stellaris\gfx\interface\icons\jobs](#) 里面

mod_job_<职业 ID>_add 的图标, 原版在 [Stellaris\gfx\interface\icons\modifiers](#) 里面

3.8 政府 (governments) 修改

原版的文件在 [Stellaris\common\governments](#) 目录下，包含政体（例如寡头制/帝制等）（[authorities](#)），公民性/国策/民政（奴隶行会/封建社会等）（不同主流汉化不一样，本文一律以“公民性”代称）（[civics](#)），政府形式（选举君主制/直接民主制等）（[00_governments.txt](#)），与政体和公民性不一样的是，政府形式是根据逻辑语句自动判断生成的，而非自由选择。

3.8.1 政体 (authorities)

从一个例子来看

```
auth_machine_intelligence = {    #ID, 一般以“auth”开头
    machine_empire = yes        #是否是机械帝国
    has_heir = no              #是否有继承人, 默认无
    #uses_mandates = yes 是否使用承诺, 默认无, 在 Stellaris\common\mandates 目录
    #has_agendas = yes 是否使用议程, 默认无, 在 Stellaris\common\agendas 目录
    #max_election_candidates = 4 最大候选人槽位, 默认无限
    #election_term_years = 40    选举年限, 多少年举行一次, 默认 0
    #election_term_variance = 10    选举年限的上下浮动区间, 如本例, 实际应该为 40±10 年, 默认
0
    #can_have_emergency_elections = yes 能否重新选举
    #emergency_election_cost = 250 重新选举的影响力花费
    election_type = none        #选举类型, 目前有“无”、“民主”、“寡头”三个类型
    has_factions = no          #是否有派系
    can_reform = no            #是否可以改革政府, 例如改变公民性或者政体等, 默认允许
    localization_postfix = machine_intelligence    #用于本地化的前缀

    valid_for_released_vassal = no #能否放出这样政体的附庸

    playable = {                #什么条件下可以被玩家使用, 一般用于 dlc 的判断
        host_has_dlc = "Synthetic Dawn Story Pack"
    }
    possible = {                #在什么条件下可用
        ethics = {
            value = ethic_gestalt_consciousness
        }
    }
    random_weight = {            #随机刷出权重, 一般仅用 dlc 的判断
        modifier = {
```

```

        factor = 0
        NOT = { host_has_dlc = "Synthetic Dawn Story Pack" }
    }
}

traits = { #必须拥有什么特质
    trait = trait_machine_unit
}

country_modifier = { #这个政体对整个帝国的修正
    colony_start_num_pops_add = 1
    station_gatherers_produces_mult = 0.1
}

tags = { #本地化 key
    AUTH_MACHINE_INTELLIGENCE_IMMORTAL_LEADERS
    AUTH_MACHINE_NATIVE_POPS
    AUTH_MACHINE_FOREIGN_POP_SURVIVAL
    AUTH_MACHINE_TECH_ADDITIONS
    AUTH_MACHINE_TECH_RESTRICTIONS
    AUTH_MACHINE_ASCENSION_RESTRICTIONS
}
}

```

和大多数项目一样，政体需要本地化和贴图。需要注意的是贴图的格式，原版贴图在 [Stellaris\gfx\interface\icons\governments\authorities](#) 目录下，需要两个版本，一个是 44*44，一个是 35*35 大小。

3.8.2 公民性 (civics)

也是从一个例子来看

```

civic_beacon_of_liberty = { #ID,应该以“civic”开头
    #pickable_at_start = yes 是否可以在开局被选择，默认是
    #playable = {} 满足什么条件才可以被玩家使用（通过 UI）
    #can_build_ruler_ship = no 意义不明，原版没见使用，默认否
    #modification = yes 能否被替换，默认否
    #traits = { trait = xxx } 需要什么种族特质，当被选择之后自动添加该特质
    #description = xxx 描述，会覆盖自动生成的文本
    #has_secondary_species = { #是否有第二种族
    #    title = civic_machine_assimilator_secondary_species #对于第二种族的描述
    #    traits = {

```

```

#          trait = trait_cybernetic          #第二种族需要的种族特质
#      }
#  }
potential = {    #对于“限制”的逻辑判断
    ethics = { NOT = { value = ethic_gestalt_consciousness } }
    authority = { NOT = { value = auth_corporate } }
}
possible = {    #对于“可用”的逻辑判断
    authority = {
        value = auth_democratic
    }
    ethics = {
        OR = {
            text = civic_tooltip_egalitarian
            value = ethic_egalitarian
            value = ethic_fanatic_egalitarian
        }
        NOR = {
            text = civic_tooltip_not_xenophobe
            value = ethic_xenophobe
            value = ethic_fanatic_xenophobe
        }
    }
}
random_weight = { base = 5 }#AI 随机刷出率，支持逻辑判断
modifier = {    #国家修正器
    country_unity_produces_mult = 0.15
}
}

```

与政体类似，也需要本地化和贴图文件（原版贴图目录

[Stellaris\gfx\interface\icons\governments\civics](#)）

3.8.3 起源(origin)修改

在 2.6 新加入的起源系统，使得公民性更加倾向于加 buff 而非增加功能，将更多的功能应用于起源，*由于起源是由公民性派生而来，因此与公民性处于同一目录下*，不过两者属性有些许不同，我们通过解析文件来看看起源结构：

```

origin_life_seeded = {
    is_origin = yes #声明是否为起源
    icon = "gfx/interface/icons/origins/origins_life_seeded.dds" #起源图标
}

```

```

picture = GFX_origin_life_seeded #起源大图，需要注册后使用

starting_colony = pc_gaia #设定开局殖民地类型
habitability_preference = pc_gaia #设置偏好星球（优先度高于 civic）
preferred_planet_class_neighbor = no

# has_secondary_species = { #通公民性，是否拥有第二物种
#   title = x
# }

playable = { #同 civic
  host_has_dlc = "Apocalypse"
}

possible = { #同 civic
  authority = { NOT = { value = auth_machine_intelligence } }
}

description = "civic_tooltip_life_seeded_effects" #起源效果描述本地化 key

random_weight = { #ai 随机使用该起源权重
  base = 5
}

#advanced_start = no #是否为高级开局
# traits = { #使用该起源所需的物种特质

# }
#non_colonizable_planet_class_neighbor = yes #是否保证周围有最小宜居星球
# initializers = { #初始化星系列表，其中填写的均是预设星系
#在 common/solar_system_initializers 文件夹中声明，涉及预设星系的编写
# }
# flags = { #设定游戏开始的 country_flag（在 event 中使用）

# }
}

```

3.8.4 政府形式 (governments)

其结构如下：

```

# Theocratic Oligarchy
gov_theocratic_oligarchy = {   #ID 应该以“gov”开头
  ruler_title = RT_CHIEF_PRECENTOR      #统治者的名称（默认、男性）
  ruler_title_female = RT_CHIEF_PRECENTOR_FEMALE #统治者的名称（女性）
  #heir_title = HT_CROWN_PRINCE      #继承人的名称（默认、男性）
}

```



```
#heir_title_female = HT_CROWN_PRINCESS      #继承人的名称（女性）
#use_regnal_names = yes / no    #是否使用区域名称，应该与帝制政府有关，意义不明
#dynastic_last_names = yes / no    #是否使用王朝的姓，应该与帝制政府有关，具体意义不明
#should_force_rename = yes / no#是否强制命名，应该用于强制生成本地化帝国名称，但具体意义不明
election_candidates = { #领袖参与选举的权重的逻辑判断
    modifier = {
        add = 100
        leader_class = ruler
    }
    modifier = {
        add = 10
        leader_class = governor
    }
}

possible = {    #对于“可用”的逻辑判断
    has_authority = auth_oligarchic
    OR = {
        has_ethic = ethic_spiritualist
        has_ethic = ethic_fanatic_spiritualist
    }
}

weight = {    #这个政府形式被赋予这个国家的权重（满足条件的情况下）
    base = 5
}
}
```

政府形式一般不需要图标，仅需要本地化文本

3.9 战争借口 (casus_belli) 与战争目标 (war_goals)

3.9.1 战争借口 (casus_belli)

原版文件在 [Stellaris\common\casus_belli](#) 里面，下面举一个例子

比如我们找到蠢驴的官方注释文件（写在最前面的）

```
my_casus_belli = { #借口的 ID，一般不限定特殊格式
    potential = { ... }      #是否可见，满足下列条件时可见，范围默认为“国家 (country)”
    #为拥有这个借口的进攻者，不填写默认为不需要条件即可满足
    #常用语句: 1.is_country_type = <country_type.name> 这语句决定什么国家类型下可见
    #2.has_valid_civic=<civic.name> 这语句决定拥有什么样可用的国策下可见
    #3.has_technology = <technology.name> 这语句决定拥有什么科技下可见

    is_valid = { ... }      #是否可用，在可见的基础上，再次判定是否可用（游戏里面表现为是否是灰色）
    #默认的范围是“国家 (country)”，为拥有这个借口所有者，“from”的范围为防方。
    #常用语句: 1.has_total_war_cb = no/yes，这个语句决定他是不是全面战争借口
    #2.has_policy_flag=<policy_flag.name>，这个语句决定在什么政策下可用
    #不填写默认为不需要条件即可满足

    destroy_if = { ... } #满足下列条件则摧毁它，原版貌似没见过，其用途待补充，范围与“is_valid”相同
    show_in_diplomacy = no/yes #是否在外交界面显示？满足的话可用看见“xxx 对 yyy 有 zzz 的宣战借口”
    #默认是
    show_notification = no/yes #是否在这个借口可用情况发生改变（从可用变成不可，反之亦然）时
    #默认是
}
```

此外，本条目需要本地化：本条目与本条目的 desc

3.9.2 战争目标 (war_goals)

原版文件在 [Stellaris\common\war_goals](#) 目录下，下面举一个例子（如果看得懂英文且基础不错，可直接看蠢驴的示范文件 [wg_example.txt](#)）

```
#wg_example = { #战争目标的 ID
#    casus_belli = cb_example #这个战争目标所需要的战争借口
#
#    set_defender_wargoal = wg_example #此语句用来强制设定防御方的战争目标，例如被洁癖宣战时的防御战争借口一定为“消除威胁”，倘若没有此语句，那么防御方可以选择防御目标（前提是满足条件）
}
```

```

#
#
#   defender_default = yes #此语句为“yes”的效果：防御方超过选择宣战借口时限时强制选择的战争目标，默认“no”
#       #如果多个战争目标都被满足，那么就会按照文本顺序选择第一个满足的
#
#   #threat_multiplier = 1.0 #当占领星系或者星球时产生的威胁度，默认为 1.0，防御目标此值应设置的很小。
#       #除非你想变成战狂或者你是抖 M
#
#
#   hide_if_no_cb = no      #当战争借口不满足时，是否隐藏？默认为“no”
#
#   cede_claims = yes/occupied_only/no #直译是“放弃对和平的要求？”效果不明，原版似乎存在感不高
#
#   ##   surrender_acceptance = 0 #此语句会影响到 AI 有多大可能由于此战争目标而投降#
#       #当数值越大则越有可能，一般设置为负数，通常战争目标多为-25，全面战争多为-1000
#
#
#   war_exhaustion = 1.0 #此语句决定了默认战争损耗程度（即厌战度增长），默认 1.0
#
#   #show_claims_in_description = yes #原版没被使用过，效果不明。默认“no”
#
#
#   potential = {          #设置可见的条件，满足其才可见。“from”范围为被指定为战争目标的国家（即你的敌人）
#       #默认范围（this）则为拥有此战争目标的国家。默认无需条件即可见
#   }
#   possible = {           #设置可用的条件，满足其才可见。“from”范围为被指定为战争目标的国家（即你的敌人）
#       #默认范围（this）则为拥有此战争目标的国家。默认无需条件即可用
#
#
#   }
#   on_status_quo = { #如果战争以“维持现状”结束后，生效的语句
#
#   }
#   #   on_accept = { #如果战争以“胜利”结束后，生效的语句
#
#   }
#}

```

此外，战争目标不仅需要本地化，还需要在 [Stellaris\interface](#) 注册贴图文件，此贴图文件需要

ARGB 8 8 8 8, 例如原版很多地方都通用的图标为：（目录在 [gfx/interface/icons/diplomacy/diplomacy_war.dds](#)）。更多详见本教程第七章 [GFX 文件与 sprite type](#)。

。

3.10 飞升 (ascension_perks)

飞升的文件目录在 [Stellaris/common/ascension_perks](#)，而在原版之中，[00_ascension_perks.txt](#) 则是原版的飞升文件。而飞升不仅需要本地化文件，还需要在 [gfx/interface](#) 定义贴图文件目录，一般为名称为 GFX_<ID>，其图片文件应该在 [gfx\interface\icons\ascension_perks](#)，而 dds 文件则应该为 52*60 的图片，其阿尔法通道的图片样式应该为正六边形状（因为飞升图标为正六边形）。

下面我们以原版的巨像飞升作为例子

```
ap_colossus = { #ID,你的飞升在程序里面的名称
    potential = { #限制语句，没有符合此条件，则无法在 UI 内被显示
        host_has_dlc = "Apocalypse" #对于启用的 dlc 判断，一般我们不需要做这个判断。
        NOT = {
            has_ascension_perk = ap_colossus #一般对于一个飞升而言，我们都希望不要点击之后还能看见它
        }
    }

    on_enabled = { #指在启动时触发的语句，可以是效果 effect 也可以是事件 event
        custom_tooltip = enable_colossus_project #所显示的启动效果文本，需要本地化文件
        custom_tooltip = describe_colossus_project
        hidden_effect = {
```

```

        country_event = { id = apoc.100 }
    }
}

possible = {
    #允许的判断，比 potential 的优先级要低。当不满足时显示为黑白图
    has_technology = tech_titans          #需要科技---泰坦
    custom_tooltip = {
        #不满足条件所显示的语句
        fail_text = "requires_ascension_perks_3"      #语句文本
        num_ascension_perks > 2      #实际的判断语句
    }
}

#modifier = {
    #所提供的修正,@为引用数值
    #    description = ap_engineered_evolution_modifier_desc      #此修正效果显示的语句
    #    description_parameters = { #用于实现自动生成修正效果文本的语句，不常见。
    #        POINTS = @ap_engineered_evolution_POINTS
    #        COST_MULT = @ap_engineered_evolution_COST_MULT
    #    }
    #    BIOLOGICAL_species_trait_points_add = @ap_engineered_evolution_POINTS
    #    modify_species_cost_mult = @ap_engineered_evolution_COST_MULT
#}

ai_weight = {
    #对于 AI 的点击的权重判定
    factor = 10
    modifier = {
        factor = 10
        OR = {
            has_valid_civic = civic_fanatic_purifiers
            has_valid_civic = civic_machine_terminator
        }
    }
}

modifier = {
    factor = 0.1
    is_xenophile = yes
}

modifier = {
    factor = 0.25
    is_ethics_aligned = { ARG1 = pacifist }
}

modifier = {
    factor = 2
    is_xenophobe = yes
}

modifier = {
    factor = 2
    is_militarist = yes
}

```

```
    }  
  }  
}
```

3.11 政策(policies)

政策文件目录在 [Stellaris/common/policies](#) 里面，而原版文件则为 00_policies.txt 。政策一般用于设置一个政策标识（即 **policy_flags**），用于各种地方的判定（事件触发，事件选项，科技，战争等）亦或者提供加成。

其只需要本地化，汉化条目应该为 **policy_<ID>**

以原版的战争哲学（无限制战争/解放战争/防御战争）作为例子

重复内容我将删除，只研究一个选项的代码（政策可以写任意数目的选项，但应该至少多于一个）

```
war_philosophy = {      #ID, 你的政策的名称

    potential = {      #限制语句，没有它则不会出现在 UI
        OR = {
            is_country_type = default
            is_country_type = fallen_empire
            is_country_type = awakened_fallen_empire
        }
    }

    allow = {  #什么情况下允许修改
        is_at_war = no # Not allowed to change policies in this group while at war
    }

    option = {  #第一个选项
        name = "unrestricted_wars"  #名称，用于本地化。选项的本地化项目不需要加"policy_"的前缀

        policy_flags = {  #所设置的政策标识。一般使用 has_policy_flags = <flag_ID> 进行判断
            unrestricted_wars
        }

        valid = {      #可用，不符合这些条件则不能选择
            NOR = {
                has_ethic = "ethic_pacifist"
                has_ethic = "ethic_fanatic_pacifist"
                has_valid_civic = "civic_inwards_perfection"
            }
        }
    }
}
```

```

    }

    #modifier = {} #修正语句，在此写上修正

    AI_weight = {          #对于 AI 的权重判定
        modifier = {
            factor = 10
            OR = {
                has_valid_ai_personality = no
                is_ai = no
            }
            NOT = {
                has_ethic = "ethic_pacifist"
                has_ethic = "ethic_fanatic_pacifist"
            }
        }
        modifier = {
            factor = 10
            has_valid_ai_personality = yes
            is_ai = yes
            OR = {
                has_ai_personality_behaviour = conqueror
                has_ai_personality_behaviour = subjugator
                has_ai_personality_behaviour = purger
            }
        }
        modifier = {
            factor = 0
            OR = {
                has_ethic = "ethic_pacifist"
                has_ethic = "ethic_fanatic_pacifist"
            }
        }
    }
}

#..... 多余内容已经删除
}

```

注意：政策的 option 至少要为 2

3.12 舰船相关

舰船与武器装备往往相互依存并相互作用，`component_sets` 为武器装备组的定义文件，作为一个可被玩家使用的武器，这是必须的；`component_slot_templates` 是定义一个槽位，比如原版的 X,H 槽等，一般情况下不做改动；`component_tags` 指拥有的属性，例如能量，动能等，一般情况下不做改动；`component_templates` 是武器的配置，具体的定义了一个武器的各种数据。`ship_behaviors` 是指舰船在交战的时候的动作，比如线列，蜂群，炮击电脑等；`ship_sizes` 是定义一个船的所属类型；`section_templates` 舰船区段，这个是为了确定一个某种类型的船的区段能装有什么样的槽位，比如在一个战列舰上装 3L 槽；`global_ship_designs` 是指一个特定的舰船装配，一般用于事件船只

3.12.1 舰船部件 (component_template)

#教程

```

weapon_component_template = { #声明这是什么配件，目前这个是武器部件，
utility_component_template 是防御组件（盾等）， strike_craft_component_template 是舰载机类
    key = "LARGE_RED_LASER_IMBA" #ID,这个武器装备在游戏内的独一无二的识别 key
    size = large #属于哪一个槽位，比如 S,M,L,P,H 等
    type = instant #属于什么类型的武器，instant 为直射武器，point_defence 是点防御，missile 是导弹，planet_killer 是巨像等
    icon = "GFX_ship_part_laser_1" #图标，原版的在 Stellaris\gfx\interface\icons\ship_parts 里面，需要用 interface 进行注册
    icon_frame = 1
    power = 250000000 #正数指生成多少部件能源（不是能量币！）负数指消耗
    prerequisites = { #所属科技
        "tech_lasers_1_imba"
    }
    component_set = "RED_LASER_IMAB" #所属的组件集
    projectile_gfx = "infrared_laser_1" #发射时候的武器效果
    tags = { #属于什么类型的武器，比如能量系，动能系等，主要影响加成科技
        weapon_type_energy
        weapon_type_kinetic
        weapon_type_explosive
        weapon_type_strike_craft
    }
}

```

```

}
ai_tags = { #这个武器的 AI
    weapon_role_anti_armor
}
upgrades_to = "LARGE_BLUE_LASER" #下一个升级
damage = { #伤害浮动
    min = 0
    max = -1474836480
}
windup = { #注意, (windup_min+windup_max) / 20 + total_fire_time / 10 = 实际开火时间, 单位为天
    min = 0
    max = 0
}
total_fire_time = 0.1
range = 400 #范围
accuracy = 1 #命中, 1=100%
tracking = 1 #追踪, 1=100%
hull_damage = 0.5 #船体伤害系数 1=正常, 2 = +100%, 0.5 = -50%, 下面的护盾与护甲都一样
shield_damage = 0.5 #护盾伤害系数
armor_damage = 0.5 #护甲伤害系数
shield_penetration = 0.75 #护盾穿透系数, 1 = +100%, 下面的护甲穿透也一样适用
armor_penetration = 0.75 #护甲穿透系数
resources = { #资源, cost 花费, produces 生产, upkeep 维护费
    category = ship_components #所属的经济分类
    cost = {
        alloys = @l_t1_cost
    }
    produces = {
        energy = 0
        minerals = 0
        food = 0
    }
    upkeep = {
        energy = @l_t1_upkeep_energy
        alloys = @l_t1_upkeep_alloys
    }
}
hostile_aura = { #此为敌对光环, friendly_aura 为友军光环
    name = "aura_shield_dampener_ev"
    radius = 2000 #半径
    apply_on = ships
    stack_info = {
        id = hostile_shield_dampener_ev #ID 应该唯一
        priority = 1
    }
}

```

```

    }
    modifier = { #所受到的修正
        ship_shield_reduction = 10
        ship_armor_reduction = 10
        ship_hull_mult = -10
    }
    damage_per_day = { #每日在这个光环内受到的伤害
        shield_damage = 100 #造成的是 100 点的护盾伤害
        accuracy = 0.5 #指的是准确度 50%
    }
    graphics = {
        area_effect = {
            entity = "circle_area_entity"
            dynamic_scale = yes
        }
        ship_effect = {
            entity = "ship_aura_negative_entity"
            dynamic_scale = no
        }
    }
}
ai_weight = {
    weight = 0 #AI 权重相关
}
}

```

3.12.2 组件集合 (component_sets)

```

#以下为组件集
component_set = { #通用开头，不要更改
    key = "RED_LASER_IMAB" #组件集的 ID
    icon = "GFX_ship_part_laser_1" #图标
    icon_frame = 1
}

#以下为槽位定义
point_defence_turret = { #槽位 ID，不能重复
    size = point_defence #所属的大类
    component = weapon #这是武器还是防御部件？
    entities = { #实体
        weapon_type_kinetic = "small_kinetic_gun_entity"
        weapon_type_energy = "turret_point_defence_entity"
    }
}
}

```

3.12.3 舰船类型 (ship_sizes)

```
#舰船类型
corvette = { #舰船类型 ID
    formation_priority = @corvette_formation_priority #间距，没什么卵用
    max_speed = @speed_very_fast #最大速度，为舰船本身基础速度
    acceleration = 0.35 #加速度
    rotation_speed = 0.1 #转动速度
    collision_radius = @corvette_collision_radius
    max_hitpoints = @corvette_hp #生命值，为舰船本身生命值（结构值）
    modifier = { #自带修正
        ship_evasion_add = @corvette_evasion
        ship_piracy_suppression_add = 10
    }
    size_multiplier = 1 #所占用舰队容量，等于 1 就是指占据 1 舰队容量
    fleet_slot_size = 1 #所占舰队的舰队组成比的大小
    section_slots = { "mid" = { locator = "part1" } } #拥有的区段，名称随意
    num_target_locators = 2 #最多的点位
    is_space_station = no #是否是空间站
    icon_frame = 2 #图标
    base_buildtime = @corvette_build_time #建造的基础时间
    can_have_federation_design = yes #是否能成为联邦舰船
    enable_default_design = yes #是否是生成一个默认设计
    default_behavior = swarm #初始作战 AI 的类型
    prerequisites = { "tech_corvettes" } #所需科技
    combat_disengage_chance = 1.00 #紧急跃迁能跳出的概率修正
    class = shipclass_military #属于哪个大类的船只
    construction_type = starbase_shipyard #建造方式，是通过工程船还是船坞等方式建造
    required_component_set = "power_core" #右侧的部件设定，下同（有些船只可能没有这些）
    required_component_set = "ftl_components"
    required_component_set = "thruster_components"
    required_component_set = "sensor_components"
    required_component_set = "combat_computers"
    resources = { #资源
        category = ships #资源类型
        upkeep = {
            energy = @corvette_upkeep_energy
            alloys = @corvette_upkeep_alloys
        }
    }
    min_upgrade_cost = { #最小升级花费，貌似没啥用
        alloys = 1
    }
}
```

3.12.4 舰船区段 (section_templates)

```
#舰船区段
ship_section_template = {
    key = "BATTLESHIP_BOW_L1M1S2" #ID,不能重复
    ship_size = battleship #所属类别
    fits_on_slot = bow #是哪个区段?
    should_draw_components = yes
    entity = "battleship_bow_L1M1S2_entity" #区段模型
    icon = "GFX_ship_part_core_bow" #图标
    icon_frame = 1
    component_slot = { #武器位
        name = "LARGE_GUN_01" #名称,单个区段内不能相同,否则只有一个能生效
        template = "large_turret" #槽位所述
        locatorname = "large_gun_01" #位点名称,就是模型的武器发射位置
    }
    component_slot = {
        name = "MEDIUM_GUN_01"
        template = "medium_turret"
        locatorname = "medium_gun_01"
    }
    component_slot = {
        name = "SMALL_GUN_01"
        template = "small_turret"
        locatorname = "small_gun_01"
    }
    component_slot = {
        name = "SMALL_GUN_02"
        template = "small_turret"
        locatorname = "small_gun_02"
    }
    large_utility_slots = 3 #防御槽位的简略写法,这里指的是3个L槽防御位
    resources = {
        category = ship_sections
        cost = {
            alloys = @section_cost
        }
    }
}
```

3.13 遗迹与遗珍相关

3.13.1 小型文物的在遗珍界面的消耗选项(artifact_actions)

目录在 [Stellaris\common\artifact_actions](#) ，主要是一些小型文物的消耗选项，比如卖掉换能量币，或者加一些 buff 什么的，老规矩，用原版做例子。~~其实本质上就是一个触发效果的工具人~~

```
artifact_arcane_deciphering = { #ID, 不应该重复
    resources = { #经济系统
        category = artifact_actions
        cost = {
            minor_artifacts = 5 #消耗 5 个小型文物
        }
    }

    ai_weight = { #AI 权重相关
        weight = 100
        modifier = {
            factor = 0
            has_resource = { type = minor_artifacts amount < 105 }
        }
    }

    potential = { #限制语句，如果不满足条件则无法出现在 UI 内
        has_technology = tech_arcane_deciphering
    }

    allow = { #允许，不满足就是灰色
        custom_tooltip = { #当不满足这个语句内的条件时，将会显示 fail_text 的内的本地化内容
            fail_text = "requires_arcane_deciphering_no_cooldown" #文本
            NOT = { has_modifier = arcane_deciphering_cooldown }
        }
    }

    effect = { #效果，可以是执行事件，也可以是直接执行某个效果
        custom_tooltip = artifact_arcane_deciphering_effect
        hidden_effect = {
            if = {
                limit = {
                    is_ai = no
                    NOT = { has_country_flag = arcana_achievement }
                }
            }
        }
    }
}
```

```

        set_country_flag = arcana_achievement
    }
    add_modifier = {
        modifier = "arcane_deciphering_cooldown"
        days = 720
    }
    owner = {
        country_event = { id = ancrel.10000 }
    }
}
}
}

```

3.13.2 考古的坟 (archaeological_site_types)

目录在 [Stellaris\common\archaeological_site_types](#)，里面都是一些考古的坟。注意，坟头跟事件绑定！

```

site_type_name = {
    #坟头的 ID
    #picture = <sprite key>      #图片，跟事件类似，需要注册路径
    #desc = <triggered event desc> #这个坟头的描述
    max_instances = <int>        #决定最大生成数目。只有在使用
    “ create_archeological_site = random”时，才可以检查该类型星系的实例数
    #weight = <scriptable value>    #权重为随机权重，仅在使用
    “ create_archeological_site = random”时使用。脚本值类型由“ <int>”或“ <mean_time_to_happen>”定
    义。
    stages = <int>                #有多少个阶段，一般最大为 6
    potential = <trigger>          #潜在的，不满足条件不出现在 UI. 范围为: this=舰队, from=坟头
    allow = <trigger>              #是否允许，范围同上
    visible = <trigger>            #检查具有 this =国家的范围是否可以看到 from =坟头
    stage = {
        # difficulty = <interval int>    #难度，应该为正整数
        icon = <string>                 # 此阶段的小图标
        event = <string>                # 会触发什么事件
    }
    on_roll_failed = <effect>         #掷骰失败时的随机效果，范围 this =舰队, from =坟头，这
    些效果一般写在 scripted_effects
    on_create = <effect>              #当这个坟头被创造时，触发的效果，范围同上
    on_visible = <effect>             #当做个坟头被看见，触发的效果，范围同上
}

```

3.13.3 遗珍 (relic)

目录在 [Stellaris\common\relics](#)

```

r_ev_fleet_maintenance = { #ID,这个遗珍的唯一 ID
    activation_duration = @triumph_duration #冷却时间
    portrait = "GFX_relic_ev_fleet_maintenance" #图标, 动态, 需要定义
    sound = "relic_activation_the_rubricator" #点击的声音
    resources = { #经济系统
        category = relics
        # Activation cost
        cost = {
            influence = @activation_cost
        }
    }
    ai_weight = { #AI 权重
        weight = 10
    }
    triggered_country_modifier = { #选择性的国家修正, 满足条件的时候才生效
        potential = {
            NOT = {
                is_country_type = default_ev
            }
        }
        country_base_engineering_research_produces_add = 500
    }
    country_modifier = { #国家修正
        ship_homeless_ev_influence_upkeep_mult = -0.25
    }
    score = 4000 #获得这个遗珍计算多少分, 用于游戏内比分
    active_effect = { #启动的时候的效果
        if = {
            limit = {
                is_country_type = default_ev
            }
            change_variable = {
                which = ev_maintenance_value
                value = 10
            }
            add_modifier = {
                modifier = "relic_ev_fleet_maintenance_1"
                days = @triumph_duration_short
            }
        }
        else_if = {
            limit = {
                NOT = {
                    is_country_type = default_ev

```



```

    }
  }
  add_modifier = {
    modifier = "relic_ev_fleet_maintenance_2"
    days = 2400
  }
}
custom_tooltip = relic_triumph_cooldown
hidden_effect = {
  add_modifier = {
    modifier = "relic_activation_cooldown"
    days = @triumph_duration
  }
}
}
possible = { #检测是否能被启动
  custom_tooltip = {
    fail_text = "requires_relic_no_cooldown"
    NOT = {
      has_modifier = relic_activation_cooldown
    }
  }
}
}
}

```

至于遗珍的图像内容，详见本教程第七章 [GFX 文件与 sprite type](#)。

3.14 星球相关

3.14.1 星球类型 (Planet Class)

目录在 [Stellairs/common/planet_classes](#), 里面是原版的一些星球类型

```

pc_gaia = { #星球类型的 id

  entity = "gaia_planet" #星球使用的实体
  entity_scale = @planet_standard_scale #与其他星球模型相比，该星球模型的大小
                                     #通常使用@planet_standard_scale
  icon = GFX_planet_type_gaia #星球的图标
  atmosphere_color = hsv { 0.58 0.3 0.7 } #星球大气颜色，hsv 色调
  atmosphere_intensity = 1.0 #星球大气颜色的饱和度
}

```

```

atmosphere_width = 0.5      #星球大气厚度
city_color_lut = "gfx/portraits/misc/colorcorrection_tropical.dds"
#如果可居住，将应用于城市的透明度过滤器，使其更好地与背景融合
modifier = { #该星球的修正
    planet_jobs_produces_mult = 0.10
    biological_pop_happiness = 0.10
    lithoid_pop_happiness = 0.10
}
min_distance_from_sun = 60
max_distance_from_sun = 95  #该星球与星系中央的最小与最大距离
#可使用@habitable_min_distance 与@habitable_max_distance 这两个变量默认值为 60 与 100
spawn_odds = 0.05          #该星球类型随机生成的概率
extra_orbit_size = 0        #星球的轨道大小，0 为默认，高于 0 则会增加轨道大小
extra_planet_count = 0      #星球被生成时会被计算成多少颗星球，0 并非 0 颗，而是默认数量
#以上两项 extra_orbit_size 及 extra_planet_count 只有气态巨行星默认高于 0
chance_of_ring = 0.2        #生成一个环的概率，这几乎只有视觉效果无实际意义
planet_size = { min = @habitable_planet_min_size max =
@habitable_planet_max_size }
#若该星球是行星，使用的最小与最大尺寸，上方变量的默认值为 12 与 25
moon_size = { min = @habitable_moon_min_size max = @habitable_moon_max_size }
#若该星球是卫星，使用的最小与最大尺寸，上方变量的默认值为 10 与 15
colonizable = yes           #此星球类型是否允许玩家或 AI 殖民
district_set = standard     #区划设置类型，standard 为默认
ideal = yes                 #是否为理想的，是则永久对所有物种 100%宜居
starting_planet = no        #是否能作为玩家或 AI 的起始行星，并不影响起源(origin)
uses_alternative_skies_for_moons = no
#如果为 yes，当星球以卫星被生成时，将会有有一个替代的星球界面背景
carry_cap_per_free_district = @carry_cap_high
#以上为盖亚星球类型默认设置，以下为补充
#can_be_moon = yes 默认为 yes，决定该星球类型是否会作为环绕其他行星的卫星
#climate = "dry" 星球的气候类型，可以为 dry、wet 和 cold 即干燥、潮湿和寒冷，也可以不指定
#enable_tilt = yes 默认为 yes，决定星球是否会倾斜
#fixed_entity_scale = no 默认为 no，如果为 yes 则无视星球尺寸使使用该星球类型的所有星球可见大小相同
#fixed_city_level = <>修正星球背景上可见的城市贴图，如 fixed_city_level = 6，原版见于城市球
#picture = <>使用已有的星球界面背景而不是自定义背景，示例:picture = pc_gaia，原版见于灵能耗子事件的特殊死寂球
#star = no 默认为 no，决定这个星球是否为恒星
#star_gfx = yes 当星球为恒星时，该项默认为 yes，决定是否发光
}

```

实体（entity），其实就是模型，可自定义，具体可参照

[gfx/models/planet/_planetary_entities.asset](#)

3.14.2 区划 (Districts)

目录在 [Stellairs/common/districts](#)，定义各个星球上的区划。

```
district_azur_ocean_city = {      #图标在 gfx/interface/icons/districts 下方塞入同名
dds 文件即可
    #区划的格子颜色也可以进行修改，在 gfx/interface/icons/districts/grid_box 下塞入
同名 dds 文件即可
    #区划的格子图标格式参考原版的即可，由于是蠢驴内部自动切图生成显示，因此不太建议做
过多修改。
    base_buildtime = 240          #建造时间
    is_capped_by_modifier = no    #是否需要对应修正增加区划最大建造数量
    #如果为 yes 则初始可建造数量为 0，需要通过修正“最大 xxx 区划数量”来增加其最大建造数
量
    #原版普通星球的矿，电，食物区划均为 yes，城市以及工业区划为 no
    show_on_uncolonized = {      #需要满足何种条件才会显示在未殖民的星球上
        #this = planet
        uses_district_set = azur_ocean
    }
    potential = {                #需要满足何种条件才会显示在已殖民的星球上
        #this = planet owner = country
        uses_district_set = azur_ocean
    }
    resources = {                #建造所需资源消耗以及维护费定义
        #亦可声明产出，不过依照蠢驴的传统，一般是区划->岗位->资源的产出模式
        category = planet_districts
        cost = {
            minerals = 300
            energy = 200
        }
        upkeep = {
            energy = 3
        }
    }
    convert_to = {               #定义将可以转换为其他哪些区划
        #如果星球易主亦或是因为其他原因导致不再满足区划的 potential，将会按照一定比例
        转换成其他区划
        #比如该示例下，如果该星球不再满足区划的 potential，那么将会转换为原版城市区划
        (district_city)
```

#被转换的区划也需要遵循其自身的 `potential` 显示规律，可以写多个转换区划以适应不同情况

#不进行声明并没有任何问题，只是不再满足区划 `potential` 之时将会损失所有已建造的该区划

```
district_city
}
conversion_ratio = 1 #定义转换比例，需要声明 convert_to 属性，与其配合使用
#假定该区划已建造 10 个，转换比为 0.8，则转换将转换为 8 个其他已声明的转换区划。
```

```
planet_modifier = { #星球修正，一般是提供岗位，建筑槽和住房
    #亦可添加其他修正（注意 scope，在这里添加国家类的修正是无效的）
    planet_housing_add = 10
    job_clerk_add = 6
    planet_max_buildings_add = 2
}
```

```
triggered_planet_modifier = { #满足条件进行的星球修正
    potential = {
        exists = owner
        owner = {
            is_gestalt = no
            is_fallen_empire_spiritualist = no
        }
        has_active_building = azur_building_hall_judgment
    }
    modifier = {
        job_enforcer_add = 0.25
    }
}
```

```
triggered_planet_modifier = {
    potential = {
        exists = owner
        owner = {
            is_gestalt = no
            is_fallen_empire_spiritualist = no
        }
        has_active_building = azur_building_military_academy
    }
    modifier = {
        job_soldier_add = 1
    }
}
```

```

triggered_planet_modifier = {
    potential = {
        exists = owner
        owner = {
            is_gestalt = no
            is_fallen_empire_spiritualist = no
        }
        has_active_building = azur_building_cathedral
    }
    modifier = {
        job_azur_nun_add = 1
    }
}

triggered_planet_modifier = {
    potential = {
        exists = owner
        owner = {
            is_gestalt = no
            is_fallen_empire_spiritualist = no
        }
        has_active_building = azur_building_mind_square
    }
    modifier = {
        job_culture_worker_add = 1
    }
}

triggered_desc = { #提供不同条件下不同描述的文本（将会替代自动生成的区划描述本地化）
    #为了适应不同区划在不同条件下的职能进行选择性的文本显示
    #如果不考虑一个区划多个情况下使用完全可以不用写
    text = job_clerk_effect_desc
}

# triggered_desc = {    #带有一定条件的写法
#     trigger = {
#         exists = owner
#         owner = { is_regular_empire = yes }
#     }
#     text = job_clerk_effect_desc
# }
}

```

3.15 巨像相关

巨像武器是群星中最为典型的 event 武器之一，通过开火时触发事件以达到某些目的，不是理论上，而是任何 event 能做到的东西，巨像都能做到。因为其中用到了 event 的相关知识，所以如果你还没有学习 event 的内容，建议先去后面几章学习一下再翻回头看本节内容。

想要写出一个能用的巨像，你需要写以下几个内容：technology、component_templates，component_sets，on_actions，scripted_triggers，events，如果有独特 ai 态度变化需求的话，还需要写 opinion_modifiers，最后是必不可少的本地化(localisation)。

*下面内容将以地爆天星(World Cracker)为例。

3.15.1 舰船组件部分

原版中，这部分写在 [common/component_templates/00_weapons_planet_killer.txt](#) 里。

```
weapon_component_template = {
    key = "PLANET_KILLER_CRACKER"    #武器的 KEY，这个 KEY 后面还会用到
    size = planet_killer
    type = planet_killer
    use_ship_kill_target = no        #这个需要写，但暂时不清楚作用
    prerequisites = { "tech_pk_cracker" }

    icon = GFX_ship_part_world_cracker
    icon_frame = 1

    size_restriction = { colossus }
    component_set = "PLANET_KILLER_CRACKER"

    planet_destruction_gfx = "shatter_planet_gfx" #毁灭星球时的特效

    ai_weight = {
        factor = 2
    }
}
```

一个武器的 type 也可以同时是 instant 和 planet_killer，以实现作为巨像武器的同时作为常规武器进行输出。

如你所见，巨像所需的特殊语句并没有多少，同时地爆天星的代码中并没有写 windup、cost、energy 等数据，因为所有原版武器的数据都在 weapon_component.csv 中写着，我们也并不需要去管这个，下面是来自舟 mod 的“冬痕”代码，这是 mod 中更为常见的组件写法。

```

weapon_component_template = { # FROSTNOVA
    key = "PLANET_KILLER_ARK_FROSTNOVA"
    size = planet_killer
    type = planet_killer
    use_ship_kill_target = no

    power = -7500
    windup = {min = 90 max = 90 } #在巨像类武器中，这个数值被定义为开火准备时间，单位天
    total_fire_time = 30 #在巨像类武器中，这个数值被定义为开火时间，单位为天
    accuracy = 1 #
    damage = {min = 0 max = 0 }
    range = 0
    resources = {
        category = ship_components
        cost = {
            originiums = 50
        }
    }

    prerequisites = { "tech_pk_ark_frostnova" }

    icon = GFX_ship_part_ark_pk_frostnova
    icon_frame = 1

    size_restriction = { colossus }
    component_set = "PLANET_KILLER_ARK_FROSTNOVA"

    planet_destruction_gfx = "neutron_planet_gfx"
}

```

其他语句已经在之前的舰船组件章节详细说明，组件集合（component_sets）和解锁所需的科技（technology）也不再赘述。

3.15.2 on_actions

原版的代码在 [common/on_actions/01_planet_destruction.txt](#) 中

语句要求为 on_destroy_planet_with_KEY，这里的 KEY 即为之前在组件中定义的 KEY。

```
# Scope:
# this/root = planet that have been fired upon
# from = fleet that fired
#on_destroy_planet_with_PLANET_DESTRUCTION_COMPONENT = {
#  events = {
#    planet_destruction.1
#  }
#}

# Note: The _queued on_actions also fire when the command is made even if there is nothing
else in the queue
# Similarly, the _unqueued on_actions fire even if you cancel the order while the weapon
is on the wind-up to fire
# They use the same scopes as the standard events

on_destroy_planet_with_PLANET_KILLER_CRACKER = {
  events = {
    crisis.5015 # Menace Objective - Destroy World/Empire
    planet_destruction.110
    planet_destruction.600 # frag holy world
    planet_destruction.100 # goes last because it can kill a country, making some
trigger checks impossible
  }
}

on_destroy_planet_with_PLANET_KILLER_CRACKER_queued = {
}

on_destroy_planet_with_PLANET_KILLER_CRACKER_unqueued = {
}
```

在此解释一下最上面的一坨英文是什么意思。最上面的是事件发生的 scope，THIS 或 ROOT 是 planet，FROM 是执行开火的舰队，可以用于 scope 转换；下面是一个小例子，再下面的两个 note 解释了后面蠢驴空着的两个 on_action 是什么。XXX_queued 没有看懂。。。希望补充；XXX_unqueued 指的是即使在巨像 windup 时你取消了行动，里面的 events 也会被触发，因为最主要的那个 on_action 只有在巨像充能并开火后才会触发。

接着说主要的 on_actions 中的内容。crisis.5015 是灾飞后炸球得分的事件；planet_destruction.110 是最重要的 event，也是后面需要我们自己写的事件；planet_destruction.600 是当你炸了圣地时圣卫想要扬了你的事件；planet_destruction.100 似乎是一个检测事件，防止炸球毁灭国家而导致的一系列 bug 而存在。第一、三、四个事件在写新的巨像时建议一并带上。

3.15.3 scripted_triggers

原版内容位于 [common/scripted_triggers/01_scripted_triggers_planet_killers.txt](#) 中。

这部分的作用是限制巨像的开火条件，即“什么可以打”，“什么不能打”，这个文件是整个巨像系统中最麻烦的，没有之一，很容易写得心态爆炸，也很容易写成逻辑屎山。

所以建议不熟悉这些时不要管什么意思，抄就完了，能跑起来就是胜利

Scope: 【THIS:planet】 【FROM:fleet】

```
can_destroy_planet_with_PLANET_KILLER_CRACKER = {#同样需要用巨像武器的 KEY 命名

    custom_tooltip = {          #限制条件 1，满足下列条件，可以开火
        fail_text = is_not_a_standard_planet_or_megastructure #不满足条件时，显示什么
        is_a_planet_megastructure = yes #条件 1，是星球或巨构
    } #只有星球和巨构可以被当作目标

    custom_tooltip = {          #限制条件 2，满足下列条件，可以开火
        fail_text = is_shielded #如果不满足条件，显示什么信息
        NOR = {                #或非门，满足其中条件之一时输出“假”
            is_planet_class = pc_shielded          #被屏蔽的星球
            is_planet_class = pc_ringworld_shielded #被屏蔽的环世界
            is_planet_class = pc_habitat_shielded   #被屏蔽的居住站
        }
    } #双重否定表肯定，即满足以上三个条件之一时不能开火

    #custom_tooltip = {        #这里蠢驴注释掉了！！！！因为没什么用
        #fail_text = is_not_hostile
        OR = {                #或门，下列条件其中之一为真时输出“真”
```

```

custom_tooltip = { #条件 3，满足下列条件，可以开火
    text = is_not_hostile_cracker #如果不满足条件，显示什么信息

```

!!!! 第一次来这里不要看下面这一坨，先抄得能跑起来再说!!!!

#关于下面的 space_owner 和 owner:

#我们都知道对于一个星系有这几种情况：**【1】**无主，没有空间站**【2】**属于自己，有自己的空间站**【3】**属于和你不在战争中的 ai，有 ai 的空间站**【4】**属于和你在战争中的 ai，有敌对 ai 的空间站。在我们游玩中可以知道，**【1】【2】【4】**星系中的球我们可以随便爆破的，而**【3】**星系中的球我们不能随意爆破，于是就有了 space_owner 和 owner。

#space_owner 的作用是检测星系的所有情况（即哨站的所有者），如果巨像的目标星球为**【1】**星系中的星球，这个检测并不生效，因为没人管你。如果这个星系有主人，开始判断 exists = space_owner，如果该星系中的球属于你指定的 space_owner，可以开火，如果不属于（即**【3】**星系），不能开火。

#而 owner 用来检测星球的所有情况，道理同上，因此殖民地、星系、土著需要分开判断。

```

OR = { #或门
    AND = { #与门，其中所有条件为真时，输出“真”

        exists = space_owner #检测该 scope 的目标是否存在
        space_owner = { #该 scope (country) 都是谁
            OR = { #或门
                is_at_war_with = from.owner #和你处于战争状态
                is_country_type = swarm #虫灾
                is_country_type = extradimensional #恶魔 1
                is_country_type = extradimensional_2 #恶魔 2
                is_country_type = extradimensional_3 #恶魔 3
                is_country_type = ai_empire #肃正
            }
        }
    }#当星球所有者是上面那些时，可以开火

    AND = { # colony 关于殖民地的条件
        exists = owner #检测该 scope 的目标是否存在
        owner = { #该 scope (country) 都是谁
            OR = { #或门
                is_at_war_with = from.owner
                is_country_type = swarm
                is_country_type = extradimensional
                is_country_type = extradimensional_2
                is_country_type = extradimensional_3
                is_country_type = ai_empire
            }
        }
    }

    AND = { # primitive 土著

```



```

        is_planet_class = pc_relic          #遗落星球
        is_planet_class = pc_nuked         #死寂星球
        is_planet_class = pc_ringworld_habitable #环世界
        is_planet_class = pc_habitat       #居住站
    }
}
}
#}
}

```

3.15.4 events

原版内容在 [events/planet_destruction_events.txt](#) 中。

这是巨像的核心，在你开火后会发生什么事件，由于篇幅过长，而且和普通 events 写法并没有什么差别（也就是说直接抄就行），因此我不在这里放代码了，只说几个需要注意的点。

1. 这就是个无比纯正的 event，今天把球炸成理想城明天全银河暴毙也可以。
2. 最好加一个第一次炸球的弹窗(不加也无所谓)，详见 planet_destruction.101
3. 40-48 行为触发摧毁观测站的事件，当你爆了土著的球，也没必要接着观测了。
4. 注意适配天灾的一些计数和事件（炸掉肃正的四个常规节点之一、炸掉肃正的核心、炸掉虫群入侵的星球需要计数等），比如 50-107 行都是这些内容。
5. 110-113 行是获得成就，显然一个 modder 是不需要关注成就的。
6. 当你需要拥有独特的 ai 态度变化（爆破了我们的星球-1000）时，相关事件代码在 110-254 行，相应的态度变化去 common/opinion_modifiers 看，抄就对了！
7. 256-272 行是给敌对国家增加厌战度
8. 星球类型转换一定要考虑到所有情况，星球会变成什么？环世界会变成什么？居住站会变成什么？否则就会出现更多巨构中炸掉奥尔德森盘后只留下一个很小的碎球这种哭笑不得的事

情（虽然这也没办法，毕竟想要变成破碎盘的话需要改掉香草代码，没有必要的话这是万万不可的）。

9. 更详细的内容参考第四章 event 编辑教程。

3.15.5 本地化

这里没什么好说的，vsc 哪里出黄线写哪里的本地化就行，另外 scripted_triggers 中的 fail_text 也是可以自定义文本的，直接改掉后面的文字就可，有需求可以去创建，同样需要对应的本地化。

3.16 评价 (opinion_modifiers)

蠢驴对各种行为影响帝国彼此评价的方式是调用 opinion_modifiers，原版文件中普遍使用在 event 中直接给对应帝国添加 modifier，部分 mod 则直接创建有 trigger 的 opinion modifier。

一个 opinion_modifier 应该由以下部分组成：

trigger（可选）：内部应为 condition，用于控制该 modifier 的自动触发

opinion：该 modifier 的具体评价影响，正值为加，负值为减。内部代码模块必须有 base，以控制其基础的评价影响。FROM 指向评价会发生改变的帝国

accumulative：只能赋予 yes 或 no，用于控制该 modifier 可否累加

unique：只能赋予 yes 或 no，用于控制该 modifier 可否重新施加

monthly：只能赋予 yes 或 no，用于控制该 modifier 的衰减是否为每月

months：只能赋予整数值。wiki 没有写明用途，原版文件中亦仅在“与盟友交战中”的 modifier 中使用，赋值为 12

min/max：只能赋予整数值，用于控制该 modifier 能增加/减少的评价的极限

decay：用于控制 modifier 的衰减速度。FROM 指向评价会发生改变的帝国

3.16 局势 (Situation)

局势为蠢驴在最近版本才进行更新的系统（3.6），主要用于设计需要追踪和影响正在进行的故事的系统，其功能较之前所列举的 Common 条目更为复杂，但不同于联邦相关内容（Federation）的复杂，该条目仅仅只涉及两个主要文件夹，即 common/situations 以及 events。并且其复杂性带来了极强的可编辑属性，是自考古遗址（Archaeological Site）条目之后一个新的叙事方式，并且个人认为其应该是现在 Stellaris 当中最具有开发潜力的 Common 条目，也是最为适合讲述长线故事的条目，那么接下来让我们一起对其进行较为细致的解析（示例条目将会十分繁杂，希望各位读者做好心理准备）。

3.16.1 局势系统

- 局势系统的编写需要有一定的事件（Events）基础！其功能与事件息息相关，缺一不可。
- 局势需要一个能够承载功能实现的一个目标（Target），较为电信的是一个星球（Planet），一个国家（Country）。
- 局势需要用来控制故事进程的进度条（Progress Bar），从 0 到 100 被分割成不同的阶段（Stages）。
- 局势需要用来控制故事走向的措施（Approaches）
- 局势需要一个或两个结束局势的触发器（Triggers）
- 局势需要月度触发的随机事件与固定事件（Events）
- 局势系统的复杂些与多样属性可以使得你有较大的空间进行编写，其可以从 0 开始推进到 100，也可以从 100 开始衰退到 0，亦可从 50 开始进行左右摇摆的抉择。
- 局势当中所有涉及到 Scope 的地方，this/root = situation，并且你可以通过 owner 跳转到 country，target 跳转到目标。
- 局势一般以星球（Planet）与国家（Country）作为目标，当没有设置目标之时，UI 将会自动显示你的国家，但 target = {} 在条目当中将失效。

- 局势的目标 (Target) 可以指定除星球 (Planet) 与国家 (Country) 以外的目标，但这可能会使 triggered_target_modifier 失效，因此需要斟酌目标 (Target) 的选择。
- 局势的目标 (Target) 也可以指向其他国家，这将对模拟复杂的情景环境起到非常重要的作用（以《Overlord》DLC 新增的雇佣兵飞地举例，其依靠局势给予玩家各种随机事件以及奖励）。
- 局势可以最多有两个结局，一个是进度达到 0 (on_fail)，一个是进度走完 (on_progress_complete)。
- 局势进度条被划分为一个或多个阶段，每个阶段都可以有与之相关的不同修正。
- 局势的性能消耗十分大，因此各位需要谨慎处理其中的内容，尽量在编写过程中考虑到优化问题。

3.16.2 属性解析

```
test_situation = {
#以本局势为例，需要的本地化 key 为：
#test_situation: 当局势存在之时显示，可以使用支架命令，如[Target.GetName]等
#test_situation_type: 当局势不存在之时所展示的名称（使用 start_situation 之时展示的
局势名称，此时无法使用支架命令）
#test_situation_desc: 局势描述，可以使用支架命令
#test_situation_monthly_change_tooltip: 描述玩家可以通过怎样的操作影响局势进程。
    desc = {#triggered_desc, 当需要动态变化局势描述之时采用的结构，满足 trigger 的条
件以显示对应的描述
        text = x
        trigger = { }
    }
#可以声明多个以满足不同情况
picture = GFX_evt_alien_nature #声明局势图片
#picture = {    #triggered_picture, 类似事件一样，局势可以通过满足不同 trigger
显示不同图片
#    picture = GFX_evt_alien_nature
#    trigger = {

#    }
#}
```



```

#可以声明多个以满足不同情况
category = positive #控制局势应用在 UI 上的颜色，属性有
positive/negative/neutral 三种。
#通俗来说三种对应局势的：好，坏，有好有坏
complete_icon = GFX_icon      #定义进度条右方的图标
complete_icon_frame = GFX_icon_frame #定义进度条右方图标的外框
fail_icon = GFX_icon          #定义进度条左方的图标
fail_icon_frame = GFX_icon_frame #定义进度条左方图标的外框

on_start = { #当局势被创造之时触发的效果

}

on_progress_complete = { #当完成局势之时触发的效果
    #蠢驴建议最后加上 destroy_situation = this 以结束局势（虽然不加也会自己删）
    destroy_situation = this
}

on_fail = { #当局势归 0 之时触发的效果
    destroy_situation = this
}

on_abort = { } #当局势满足 abort_trigger 之时触发的效果
abort_trigger = { } #当局势满足 trigger 之时将中断局势并触发 on_abort
modifier = { } #给予正在经历局势的国家的修正
triggered_modifier = { #有条件的修正，当满足 potential 之时才会激活修正
    potential = { }
    modifier = { }
}

target_modifier = { } #只有当 target 为 planet 之时该修正才会生效！给予目标
(Target) 的修正
triggered_target_modifier = { #同 triggered_modifier，作用范围为目标 (Target)
    potential = { }
    modifier = { }
}

start_value = 20 #局势将从何值开始进行
progress_direction = bidirectional #局势进度类型
monodirectional/bidirectional (单向/双向 默认为单向)
complete_category = positive #效果为 category，只有局势进度类型为双向
(bidirectional) 之时才生效，只影响右方
fail_category = negative #效果为 category，只有局势进度类型为双向
(bidirectional) 之时才生效，只影响左方

stages = {

```



```

    samplet_stage = {      #本地化 key, 可以定义额外的本地化 samplet_stage_desc 以
显示提示
        icon = GFX_situation_stage_1 #图标
        icon_background = GFX_situation_stage_frame_green #背景
        end = 40 #该阶段结束所需的进度
        on_first_enter = {} #当第一次进入阶段之时所触发的效果
        custom_tooltip = x #显示在阶段的提示当中 (on_first_enter 不会显示, 因为
这将透露修正)
        #以下条目效果同上
        modifier = {}
        triggered_modifier = {
            potential = { }
            modifier = { }
        }
        target_modifier = { }
        triggered_target_modifier = {
            potential = { }
            modifier = { }
        }
    }
}

monthly_progress = { #每个月刷新一次进度
    base = 1 #默认的月进度
    modifier = { #满足何种条件以添加进度, 值支持使用 script_values
        add = 2
        desc = federation_acceptance_reduce_fleet #描述月进度的来源
        owner = {
            has_technology = tech_alloys_1
        }
    }
    modifier = {
        desc = federation_acceptance_reduce_fleet
        owner = {
            has_technology = tech_alloys_2
        }
    }
}

approach = {
    name = approach_a      #本地化 key, 可以定义额外的本地化 approach_a_desc 以显示
提示

```

```

icon = GFX_situation_approach_shrug
icon_background = GFX_situation_approach_bg_red
allow = { }      #满足条件才可以使用该措施
potential = { }   #满足条件才会显示该措施
on_select = { }   #当选择该措施之时触发的效果
default = yes     #如果为 yes，局势开始之时将默认该措施设置为当前采用的措施
#如果不满足 allow 以及 potential，该措施将会失效
#但当该局势通过语句 set_situation_locked = yes 锁定之时，其将仍然生效
#这意味着如果要动态切换措施，需要先用 set_situation_locked = yes 锁定，选择措施之后再解锁即可。
#以下修正只有选择该措施之时才会激活，其他同上
triggered_modifier = {
    potential = { }
    modifier = { }
}
target_modifier = { }
triggered_target_modifier = {
    potential = { }
    modifier = { }
}

resources = {     #措施激活之时的资源开销
    category = situations
    cost = { }
    upkeep = { }
    produces = { }
}
ai_weight = { }   #ai 选择该措施的权重
}
#局势自带一个月检测的 action，其触发的事件类型只能为 situation_event!
on_monthly = { #从技术层面来说，该 action 名为 test_situation，所以不要让你的
situation 名称与预设的 on_action（如 on_monthly_pulse）重名!
#这也意味着局势系统性能消耗很大，请谨慎考虑内容
    events = {
    }
    random_events = {
    }
}
#事件影响 situation 之时可能会导致生成的提示有误，因此在选项当中应该采取 tooltip =
{}来将生成的提示正确显示。
}

```


四、Event Modding

、在进行 Event 编写之前，我们必须要知道 Event 能够实现怎样的功能以及 Event 是如何发挥功效的。

4.1 Event 简介

4.1.1 Event 功能

Event 是一个大型 mod 所必备的内容之一，可以进行很多功能的实现，例如舰队逻辑，自定义 UI 面板，失业人口自动迁移等等，可以说 Event 是丰富 mod 内容的最佳手段，其灵活多变，可以用来实现很多通过 common 无法实现的功能甚至在可以模拟出 common 中某些内容并进行替代实现。

4.1.2 Event 本质

Event 本质是目标对象根据某个条件启动效果并对其他对象或者自身实施效果的行为，其三要素为目标对象，启动条件，执行效果，不论任何事件都离不开这三要素。

4.2 工具准备

4.2.1 编辑器选择

工欲善其事，必先利其器。由于蠢驴的大部分 Mod 文件都是.txt 文件，因此各位在之前的 Mod 制作过程中，应该已经选择了自己熟悉的文本编辑器进行 Mod 制作。但是到了 Event 部分，我个人十分建议采用 VSC(Visual Studio Code)进行 Mod 编写。

官网：<https://code.visualstudio.com/>



当然，其他文本编辑器也是可以的，如下是我推荐的其他几个文本编辑器，不过没有对编写蠢驴语

言有帮助的插件。

Sublime Text3: <http://www.sublimetext.com/3>

NotePad++: <https://notepad-plus-plus.org/> (由于各种各样的原因, 我并不推荐该文本编辑器)

4.2.2 CWTools 安装

在 VSC 中有一个蠢驴全家桶插件, 功能较为完善。该插件支持错误提示, 自动补齐, 生成科技树、事件树, 一键生成本地化 Key 等等一系列功能。在编写 Event 的时候还能够查看每个 Event 的引用情况(包括 Scripted Effect 的内容), 作用域等等功能, 方便我们编写 Event(该插件有点吃内存)

该插件还自带了一个语法高亮, 不过配色有点花哨, 建议配合其他主题组合使用。

```
country_event = {
  id = azur_infinite.0
  is_triggered_only = yes
  hide_window = yes

  trigger = {
    azurlane_default_government_trigger = yes
  }

  immediate = {
    capital_scope = {
      solar_system = {
        every_system_planet = {
          limit = {
            NOT = {
              is_same_value = capital_scope
            }
          }
          surveyed = {
            set_surveyed = no
          }
        }
      }
    }
  }
}
```

```

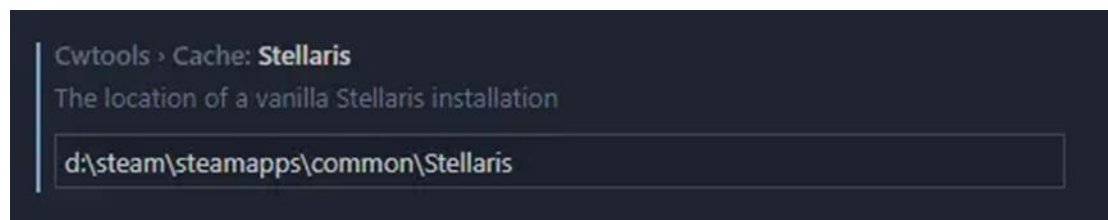
country_event = {
    id = azur_sand_of_time.0
    days = 360
    random = 1800
}
}
}

```

(颜色主题如上)

插件安装使用:

- 拓展中搜索 CWTools, 点击安装
- 右键安装的 CWTolls, 点击拓展设置
- 将群星(Stellaris)的根目录路径复制到 Stellaris 一栏中



- 重启 VSC, 通过 VSC 打开文件夹, 插件将会自动加载。

注意, 此 mod 必须在默认文件夹当中, 也就是路径为 Paradox Interactive\Stellaris\mod 下才能生效, 并且请用文件夹形式打开你 mod 的根目录:



这样就将你所有 mod 内容全部加载入 CWT。

4.2.3 CWTools 使用

<1> 自动补齐

当输入一个字符之时将会进行模糊搜索检测该范围内可用的语句（这个检测仅限于区别 Condition 和 Effect，不包括 Scope 匹配）当我们不断输入字符时，如果默认光标已经移动到我们需要的语句之上时，按下 Tap 键可以进行补齐，如若我们需要切换默认光标位置，按下小键盘上的上下按键可以调整光标位置。如果该语句存在固定格式，则会自动生成代码块（如 if, switch, 各种变量操作语句等），这里以 set_variable 为例（关于变量(Variable)操作我会在之后详细介绍，这里仅仅是为了展示 VSC 生成代码块的一些功能，不必理会有何用途），当你使用 Tap 键入 set_variable 时，将会自动生成标准格式，包括一个“which”以及一个“value”属性。此时你会发现后面两个预设的 x 上都有一层阴影，现在直接输入一个字符，你会发现 x 自动被清除了，留下的是你输入的字符，这里示例为 Test_Variable。

输入完毕之后，如果细心还会发现刚才输入的字符串似乎和之前一样也有阴影，此时再按下 Tap，光标位置已经移动到了 value 后方的 x 上，此时再输入一个数字 4。再次按下 Tap，光标又跳转到了 set_variable 语句最后的花括号外。（某些参数跳转用起来并不舒服，不过既然有这功能就此介绍一下），这种输入方式在许多 IDE 中都存在，差不多是预设参数快速跳转输入，能够方便我们快速跳转输入参数完成语句书写，如若能够熟练掌握，写代码的速度将会提高。

```
set_variable = {
  which = x
  value = x
}
```

```
set_variable = {
  which = Test_Variable
  value = x
}
```

```
set_variable = {
  which = Test_Variable
  value = 4
}
```

```
set_variable = {
  which = Test_Variable
  value = 4
}
```

<2> 查看引用

有时候我们需要知道文件中有哪些地方调用了我们的 Event。此时我们按住 Ctrl 并将鼠标移动到 Event 的
 动到 Event 的
 化，此时点击
 跳转对应文件

```
set_variable = {
  which = x
  value = x
}
```

```
set_variable = {
  which = Test_Variable
  value = x
}
```

左键点击引用

```
azur_expand_
ENERGY =
NUM = 1
NEXT_ENE
}
```

```
set_variable = {
  which = Test_Variable
  value = 4
}
```

```
set_variable = {
  which = Test_Variable
  value = 4
}
```

上方列举的封装效果(Scripted Effect)也可采用上述方法进行定义以及引用跳转

<3> 代码风格

每个人写代码都有自己的风格，比如有些人写语句喜欢一行写完简单语句：

```
set_variable = { which = x value = x }
```


这样的确可以减少行数的变换，不过会造成可读性的降低，在写简单语句之时可以使用，一旦书写较为复杂的逻辑之时，就会使得代码几乎没有可读性，十分难以维护，并且上述写法无法使用 VSC 生成的代码块。如果各位刚刚学习 Event，我更倾向于采用可读性更高的写法，将每个属性以及逻辑门都一一列出来，这样每个属性以及各个语句之间的关系便显现出来了，但这只是一个小的方面，风格还包括喜欢用哪些语句进行书写功能，实现各种功能的方法有很多，在自己的学习过程中能够写出自己的风格也是养成习惯的一种方式(意思是自己写的代码自己得看的下去)。

注意以下代码的缩进格式，虽然 VSC 会进行自动缩进辅助，但是这个功能时好时坏，因此请务必保持良好的缩进习惯!(拿记事本写 Mod 的可以鲨了)这样 Conditions 与 Effects 之间的关系才能一目了然，当然如果已经遇到十分诡异的缩进之时，程序可读性以及美观全部被破坏了，因此我们需要整理我们的代码。在面对大量需要修正的代码之时，我们也许可以一个个手动缩进，但效率低下，此时 VSC 提供了格式化代码功能，在成功加载 CWTools 之后，选中需要格式化的代码，按下 Shift+Alt+F 就可以进行当前文件的代码格式化。

```
planet_event = {
    id = azur_planet_attack.2
    is_triggered_only = yes
    hide_window = yes

    immediate = {
        change_pc = pc_black_hole
        solar_system = {
            every_system_planet = {
                limit = {
                    is_star = no
                }
                every_system_megastructure = {
                    delete_megastructure = this
                }
                if = {
                    limit = {
                        NOR = {
                            is_planet_class = pc_frozen
                            is_planet_class = pc_gas_giant
                        }
                    }
                }
            }
        }
    }
}
```

```

        }
    }
    change_pc = pc_frozen
}
}
every_fleet_in_system = {
    limit = {
        NOT = {
            is_ship_class = shipclass_colonizer
        }
    }
    destroy_fleet = this
}
}
}
}
}

```

注意：CWTools 提供的格式化模式是会清除一切空行，会使得某些部分紧缩成一团造成可读性稍微的下降，除非实在没有办法要处理大规模不规范代码，不建议格式化代码。

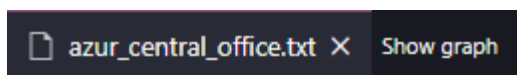
<4> 批量注释

有时候我们调试某个 Event 之时，某些 Effect 我们需要逐个进行排查，而我们又不想删除之前写的代码，因此暂时的注释掉是最好的选择，抑或是某些功能写出来却不想启用，就先注释掉以备后用。注释掉比不进行调用更加节省加载时间并能稍微提高那么一点点性能，前者根本不会被编译，后者还是会进行编译并占用空间。而我们需要注释的地方可能比较的多，那么我们需要进行批量注释，批量注释方法十分简单，框选你需要注释的代码，按下 Ctrl+K+C（按住 Ctrl，然后一次按下 K 和 C），即可完成批量注释，那么对应的需要解除注释，我们可以框选所需代码，按下 Ctrl+K+U 即可解除注释（批量注释原理就是往选择的语句前边都加一个#，因此可以多次添加，但没意义）。以上仅仅是介绍了一些常用方法，其中大部分的快捷键都是编程所经常用到的，常用的还有：复制当前行到下一行 Alt+Shift+Down，快速跳转行末：End，快速跳转行首：Home 等

等。你可以自行设置这些快捷键（如何设置请自行百度），熟练使用这些快捷键将会极大的提升你的代码书写速度，并写出较为美观的代码。

<5> 额外功能

CWT 还提供了其他一些聊胜于无的功能，比如根据前置科技生成科技树，根据事件链(Event 相互触发)生成事件树，当我们当前文件为 event 或者 technology 之时，文件上方标题最右方将会会有一个 Show graph 的按钮，点击之后将会自动生成当前文件所有事件之间的相关性



如下示例：

以下示例来自工坊 id 1617872589 ,Azur Lane Stellaris DLC

```
namespace = azur_shop

country_event = {
    id = azur_shop.0
    is_triggered_only = yes
    title = azur_shop.0.name
    desc = azur_shop.0.desc
    diplomatic = yes
    picture_event_data = {
        portrait = root.ruler
        room = azur_office_room
    }
    trigger = {
        from = {
            is_country_type = azur_research
        }
        azurlane_default_government_trigger = yes
    }
    immediate = {
        set_country_flag = azur_shop
    }
    option = { #研发部
        name = azur_shop.0.a
        country_event = {
            id = azur_shop.1
```

```

    }
}
option = { #誓约
    name = azur_shop.0.bb
    allow = {
        if = {
            limit = {
                hidden_trigger = {
                    has_country_flag = oath_ceremony_begining
                }
            }
            NOT = {
                has_policy_flag = oath_ceremony_close
            }
        }
        else = {
            NOT = {
                has_policy_flag = oath_ceremony_close
            }
            custom_tooltip = {
                fail_text = lack_azur_rings
                check_variable = {
                    which = azurlane_ring
                    value >= 1
                }
            }
        }
    }
}
hidden_effect = {
    country_event = {
        id = azur_oath_ceremony.0
    }
}
}
option = { #偶像
    name = azur_shop.0.cc
    allow = {
        fail_text = {
            text = azur_idol_start_tooltip
            has_global_flag = azur_expand_mu_leader
        }
    }
}
hidden_effect = {

```

```

        country_event = {
            id = azur_shop.5000
        }
    }
}

# option = { #委托
#     name = azur_shop.0.dd
#     trigger = {
#         fail_text = {
#             text = azur_entrust_charge_cooldown
#             NOT = {
#                 has_country_flag = azur_entrust_charge_cooldown
#             }
#         }
#     }
# }
# hidden_effect = {
#     country_event = {
#         id = azur_entrust_charge.0
#     }
# }
# }

option = { #退出
    name = azur_shop.0.00
    hidden_effect = {
        remove_country_flag = azur_shop
    }
    default_hide_option = yes
}

}

country_event = {
    id = azur_shop.1
    is_triggered_only = yes
    title = azur_shop.1.name
    desc = azur_shop.1.desc
    diplomatic = yes
    picture_event_data = {
        portrait = root.ruler
        room = azur_laboratory_room
    }

    option = { #戒指
        name = azur_shop.1.aa
        custom_tooltip = azur_rings_cost
    }
}

```

```

    allow = {
        has_resource = {
            type = energy
            amount >= 10000
        }
    }
    hidden_effect = {
        add_resource = {
            energy = -10000
        }
        change_variable = {
            which = azurlane_ring
            value = 1
        }
        country_event = {
            id = azur_shop.1
        }
    }
}
option = { #返回
    name = azur_shop.1.bb
    hidden_effect = {
        country_event = {
            id = azur_shop.0
        }
    }
    default_hide_option = yes
}
}

country_event = {
    id = azur_shop.1000
    is_triggered_only = yes
    hide_window = yes

    trigger = {
        is_ai = no
        azurlane_default_government_trigger = yes
    }

    immediate = {
        create_country = {
            name = "Name_central_office"
            type = azur_research

```

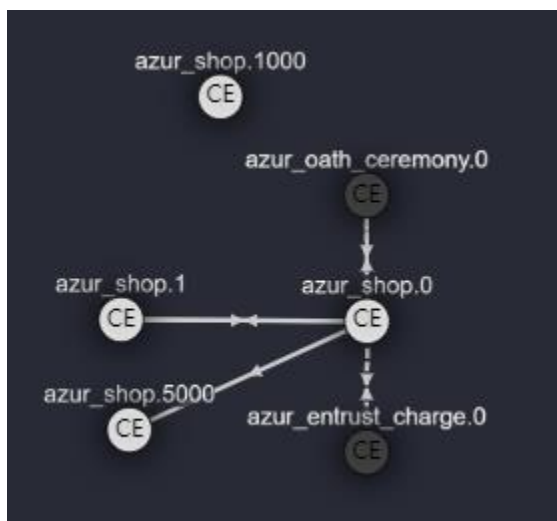
```

        flag = {
            icon = {
                category = "azurlane_flags_sp"
                file = "flag_K_logo.dds"
            }
            background= {
                category="backgrounds"
                file="00_solid.dds"
            }
            colors={
                "black"
                "black"
                "null"
                "null"
            }
        }
    }
    last_created_country = {
        establish_communications_no_message = root
    }
}

country_event = { #引导
    id = azur_shop.5000
    is_triggered_only = yes
    hide_window = yes
    immediate = {
        country_event = {
            id = azur_expand_s.0
        }
    }
}
}

```

将会生成如下表：



在这张表的右上角将会出现如下按钮：

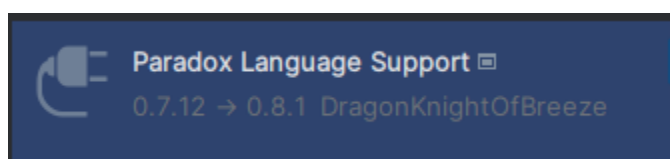
Save graph as image Save graph as json Set graph depth

选择将表以何种形式导出。

科技也可以如此步骤导出，可以迅速生成一条简陋的科技树。

4.2.4 Paradox Language Support

由于对 CWTools 某些地方的不满，群中有一位大佬根据 CWTools 的规则文件并加以完善，移植到了 IDEA 上边，目前可以在插件库当中找到该插件（目前仅对 Stellaris 语法有完整支持，对蠢驴其他游戏的支持并不完整）：



该插件不仅有 CWTools 基础的语法补齐，还有许多独到的地方，如果能够接受使用 IDEA 这种正经 IDE 编写 mod 的话，该插件也是一个十分不错的选择，至少不会像 CWTools 隔山差五抽风无效，目前还支持最新的语法（CWTools 更新速度感人，截止我写下这篇介绍还有很多地方没支持最新的写法），并且由于作者就在 Modder 交流群当中，有什么建议可以直接反馈，因此接更新也会较为贴合各位 Modder 使用。接下来截图介绍一下我认为比较独特的地方，这里采用的示例 mod 为久远之

旅（以下展示仅为冰山一角，如果感兴趣的话可以自行下载体验，目前一直在完善当中）：

<1> 实时预览本地化以及 Scope 解析

```
#德雷克分会
building_KZ_VOY_drake_branch_office : building, capped, corporate 德雷克分会 = {
    base_buildtime = @KZ_VOY_buildtime_normal
    base_cap_amount = 1
    owner_type = corporate

    potential = { this = planet root = planet
        has_branch_office = yes
        branch_office_owner = { KZ_VOY_DCC_available : scripted_trigger = yes }
    }

    allow = { this = planet root = planet
        custom_tooltip = {
            fail_text = requires_not_KZ_VOY_solaria ✖不能在索拉利亚建造
            is_KZ_VOY_solaria : scripted_trigger = no
        }
    }
}

#基础
triggered_country_modifier = { this = country root = country
    potential = { this = planet root = planet
        has_branch_office = yes
        planet.branch_office_owner = { KZ_VOY_DCC_available : scripted_trigger = yes }
    }
    diplo_weight_economy_mult 🌐 来自经济的外交权重 = 0.025
    country_naval_cap_add 海军容量 = 25
}
triggered_planet_modifier = {...}

#是德雷克商会母国
triggered_planet_modifier = { this = planet root = planet
    potential = {
        has_branch_office = yes
        branch_office_owner = { this = country root = planet
            is_same_empire = prev.owner
            KZ_VOY_DCC_available : scripted_trigger = yes
        }
    }
}
```

```

#虫群沃土
ship_event anomaly.6715: event, ship, default, triggered 虫群沃土 = {
  id = anomaly.6715
  title = "anomaly.6715.name" 虫群沃土
  desc = "anomaly.6715.desc" 尽管[nanite_swarm_planet.GetName...]
  picture = GFX_evt_gray_gooed_planet : sprite, normal
  show_sound = event_laboratory_sound
  is_triggered_only = yes

  option = { this = ship root = ship
    name = anomaly.6715.a 殖民船! 起飞!
    event_target:nanite_swarm_planet = { this = ? root = ship
      remove_modifier = nanite_swarm_modifier : static_modifier, planet 纳米虫群
      add_modifier = {
        modifier = tamed_nanite_swarm_modifier : static_modifier, planet 驯化的纳米虫群
        days = -1
      }
    }
  }
  if = {
    limit = {
      OR = {
        owner_species = { this = species root = ship
          OR = {
            has_trait = trait_mechanical : trait, species_trait 机械体
            has_trait = trait_machine_unit : trait, species_trait 机械
            has_trait = trait_pc_habitat_preference : trait, species_trait 居住站偏好
            has_trait = trait_pc_shattered_ring_habitable_preference : trait, species_trait 环形世界偏好
            #修改内容
            has_trait = trait_pc_KZ_VOY_hyper_ecumenopolis_preference : trait, species_trait 超普世城偏好
            has_trait = trait_pc_KZ_VOY_astral_world_preference : trait, species_trait 星体世界偏好
            has_trait = trait_pc_KZ_VOY_astral_world_solaria_preference : trait, species_trait 星体世界偏好
            has_trait = trait_pc_KZ_VOY_luna_preference : trait, species_trait 遗迹月球偏好
            has_trait = trait_pc_KZ_VOY_terran_relic_preference : trait, species_trait 超古代遗迹星球偏好
          }
        }
      }
    }
  }
}

```

<2> 条目图标预览

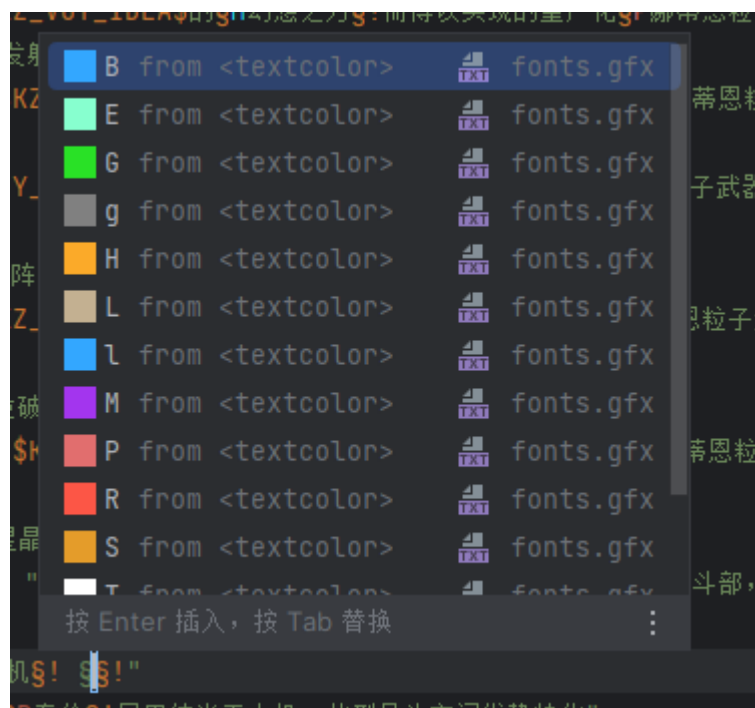
```

3 #超感知进化
4  ap_KZ_VOY_super_perception_evolution : ascension_perk 超感知进化 = {
5     potential = { this = country root = country
6         species = { is_archetype = KZ_VOY_TERRAN_DESCENDANT : species_archetype, not_robotic, use_modifiers }
7         NOT = {
8             has_ascension_perk = ap_KZ_VOY_super_perception_evolution : ascension_perk 超感知进化
9         }
10        is_mechanical_empire : scripted_trigger = no
11    }
12    possible = { this = country root = country
13        custom_tooltip = {
14            fail_text = KZ_VOY_requires_ascension_perks_4 ✖需要首先选择另外四个飞升天赋
15            num_ascension_perks > 3
16        }
17        custom_tooltip = {
18            fail_text = KZ_VOY_PM_gaia_consciousness_not_exist ✖必须首先接触盖亚意识
19            event_target:KZ_VOY_solaria_target = { this = ? root = country
20                has_modifier = KZ_VOY_PM_gaia_consciousness : static_modifier, planet 盖亚意识
21            }
22        }
23        custom_tooltip = {
24            fail_text = requires_tech_KZ_VOY_life_magic_upgrade_01 ✖需要🧙生命魔法术式改良科技
25            has_technology = tech_KZ_VOY_life_magic_upgrade_01 : technology 生命魔法术式改良
26        }
27        custom_tooltip = {
28            fail_text = KZ_VOY_requires_SPE_available_ap_paths ✖必须至少完成基因、义体或灵能中的一条传统树
29            OR = {
30                KZ_VOY_tradition_genetics_completed : scripted_trigger = yes
31                KZ_VOY_tradition_cybernetics_completed : scripted_trigger = yes
32                KZ_VOY_tradition_psionics_completed : scripted_trigger = yes
33            }
34        }
35        custom_tooltip = {
36            fail_text = requires_not_ap_synthetic_evolution ✖不能拥有合成进化飞升天赋。
37            NOT = { has_ascension_perk = ap_synthetic_evolution : ascension_perk 合成进化 }
38        }
39    }
40    on_enabled = { this = country root = country
41        custom_tooltip = enable_KZ_VOY_super_perception_evolution_project 发布特殊项目: 超感知进化...

```

点击即可跳转至对应的图标。

<3> 智能完善本地化



4.3 Event 基础

我们先来看最基础的 Event 格式：

```
namespace = test_event

country_event = {
    id = test_event.0
}
```

没错，这便是一个最基础的 Event，没有任何修饰，那么来解析下各部分用途：

namespace: 如果你学习过编程，那么这个你一定不会陌生，当然，如果没有了解也没关系。通过在文件开头声明 namespace 以开拓一个名称空间用以载入该名称空间下的 Event，多个文件可以写同一个名称空间，但同一文件不能够写多个名称空间，也就是说，一个文件必须有且仅有一个 namespace，namespace 的名称可以使用划线(_)。

country_event: 指定声明的 Event 类型，每个类型都有自己的默认作用域(Scope)，目前有以下几种 Event 类型，关于作用域的问题，之后将会提到。这里我们采取最为普遍的 country_event

来进行示例(country_event 也是应用最多的 Event 类型)，下表给出了所有能够进行声明的 Event 种类以及对应的基础范围(Scope)。

Event Type	Scpoe
event	global
country_event	country
planet_event	planet
fleet_event	fleet
ship_event	ship
pop_event	pop
pop_faction_event	pop_faction

id: 故名思意，给 Event 一个标识，不过需要注意的是，id 必须要是 `[namespace].[int]` 形式（貌似有其他格式，但是这样写一定不会出错）。

以上该事件仅仅是存在而已，并没有任何实际用途，我们稍微的给它增加一点点的修饰，让它成为一个有着标题(Title)，描述(Description)，图片(Picture)，位置(Location)，事件音频(Show Sound) ，选项(Option)的 Event:

```
namespace = test_event

country_event = {
    id = test_event.0
    title = test_event.0.name
    desc = test_event.0.desc
    picture = GFX_evt_analyzing_artifacts
    location = root
    show_sound = select_click

    is_triggered_only = yes
    # mean_time_to_happen = {
    #     months = 3
    #     modifier = {
```

```
#      factor = 2
#      has_civic = civic_agrarian_idyll
#    }
#  }

  option = {
    name = test_event.0.aa
  }
}
```

title: 指定 Event 标题(Title)的本地化 Key

desc: 指定 Event 描述(Description)的本地化 Key

picture: 指定 Event 图片(Picture)的注册 Key

location: 指定事件导航向的位置, 承接基础范围(Scope)或者事件对象(Target), 通过右上角的摄像头按钮进行导航

show_sound = 启动事件之时触发的音频

option: 生成一个选项(Option), 当 Event 不存在 option 之时将会在启动该 Event 之时自动创建一个空 option, option 可以重复设置 (过多会导致屏幕无法全部显示)。

name: 事件选项的本地化 key

is_triggered_only/mean_time_to_happen: 两种不同的触发方式, 在之后会详细讲解。

好了, 你只需要进行本地化完标题(Title), 描述(Description)以及选项(Option), 一个最普通不过的 Event 就写好了, 现在这个 Event 没有任何其他效果, 我们需要给它添加一些效果以及限制, 这便将涉及到了 Event 最核心的三要素: 触发器(Trigger), 条件(Condition), 影响(Effect)。

4.3.1 触发器(Trigger)

顾名思义, 事件需要某个触发器触发才能驱动, 也就是你开车时你得点火车才能发动一样, 蠢驴内置了两种触发模式, 这两种触发模式无法在一个事件种同时存在!

<1> Mean Time To Happen

MTTH，平均发生时间，设置该触发模式之后，Event 将会在一个宏观的时间内按照设定的时间触发，比如下图设定的便是每 50 天触发一次 Event：

```
country_event = {  
    id = test_event.0  
    title = test_event.0.name  
    desc = test_event.0.desc  
    mean_time_to_happen = {  
        days = 50  
        #months = 50  
        #years = 50  
    }  
  
    option = {  
        name = test_event.0.aa  
    }  
    option = {  
        name = test_event.0.bb  
    }  
}
```

同时还支持月数(Months)以及年数(Years)，但注意，这是指宏观表现上来讲的平均发生时间，也就是假设开始年份是 2200 年，胜利年份是 2500 年，你设置的 Event 在这个时间段内的平均发生次数符合 MTTH 的设置，因此这是一种带有随机性质的触发模式，比如你设置了 50 天触发，但是它实际触发时间是未知的，可能过去 20 天左右就触发了，也可能过去 70 天还没触发，这些都是可能的，以下摘自官方 Wiki 对 MTTH 的解释：

A additional condition very often used on event that triggered by regular polling, is the mean_time_to_happen (MTTH). This will delay the execution of the event by a random amount of ingame days. On average the activation will be delayed as given, but the exact number can vary considerably. Counting starts the moment all other trigger

conditions are met and it is not clear how exact the internal implementation of this delay works, however it seems likely the MTTH is regularly polled just like all other triggers.

MTTH 还能像权重(Weight)一样添加修正(Modifier)，用来修正 MTTH 所指定的时间：

```
mean_time_to_happen = {
  years = 100
  modifier = {
    factor = 0.1
    has_civic = civic_agrarian_idyll
  }
}
```

和权重系统相似，years 指定基础发生年份，Modifier 中的 factor 指定乘算修正，当存在公民性 civic_agrarian_idyll 之时将会使得发生年份修正到 10 年。

使用注意：同一个时间写不同表达形式也会有很大影响例如 years = 5 与 days = 1800，在时间观念上来说是一样的，然而却是存在区别的。对于检索频率来说 years 检索次数更少，不过带来的是更大的误差，而采用 days 则是检索触发的频率将十分高，但是误差也会相对小一些，MTTH 过多会导致运行速度明显下降，我不推荐各位使用，应当谨慎。

<2> Is Triggered Only

is_triggered_only 是使用更为广泛的触发模式，在执行性能上明显强于 MTTH 这种触发模式，Event 写入该语句之后将只会受到其他触发器的触发，也就是说，必须由其他的方式来触发该 Event，自身并不能自主开启触发，其他触发器包括：影响(Effect)，行动(Action)。

```
country_event = {
  id = test_event.0
  title = test_event.0.name
  desc = test_event.0.desc
  is_triggered_only = yes

  option = {
    name = test_event.0.a
  }
}
```



```
option = {
    name = test_event.0.bb
}
}
```

影响(Effect)将会在后边详细说明，我们先来看看行动(Action)各位写 Event 时有一种情况各位应该都遇到过：借鉴其他 Mod 或者原文件的 Event 之时，你写的 Event 和别人的明明一模一样，然后发现别人的 Event 能够触发而自己的不行。这就涉及到了 Action，这是写在 common 目录下 on_actions 文件夹中 on_actions 文件里的预设触发器。

这些触发器都是以“on_”为前缀，每一个触发器都会在满足不同情况之时进行一次触发，触发对象就是 events 里面 id 对应的 Event，比如我想要一个事件(*country_event id = test_event.0*)固定每个月触发一遍，我们可以在 Event 里边写上“is_triggered_only = yes”，仿照原文件格式在 common 中新建一个 on_actions 文件夹，并在其中创建一个名称随意（注意不要与原文件重名）的.txt 文件，写入以下内容：

```
on_monthly_pulse_country = {
    events = {
        test_event.0
    }
}
```

这样你的事件就可以每月都进行一次出发了。注意上述操作细节，在 on_actions 里面还有如下预设触发器：

```
on_monthly_pulse = {
    events = {

    }
}
```

该预设触发器与上述预设触发器仅仅差了一个“_country”，但如果我们将上述事件(*country_event id = TestEvent.0*)放入其中，VSC 立刻会报错，如果你仍然要进行测试，结果将是触发失败。这就涉及到了 Event 的默认作用域，在上面我给出了各种 Event Types 的默认作用

域，而同样的，Action 也同样拥有作用域(Scope)，只能够触发相同 Scope 的 Event，例如上述图片的“on_monthly_pulse”是无作用域的，因此其中 Event 的 Event Type 为“event”，而在我们的示例中“on_monthly_pulse_country”的默认作用域是 country，正好对应了我们需要触发的事件(country_event id = TestEvent.0)，因此可以正常触发。通常来说蠢驴会在每一个触发器上方标注 Scope，如若没有标注则是没有作用域，应该采用“event”类型。

Action 还支持随机触发，以下这个例子可以看到在每个月触发 events 的同时还会触发下面的 random_events，有 1/100 的概率触发 test_event.1，99/100 的概率什么也不发生。

```
on_monthly_pulse_country = {
    random_events = {
        99 = 0
        1 = test_event.0
    }
}
```

Action 之所以应用广泛，是由于种类繁多，能够满足一些特定情况触发 Event，比如建筑建造完毕，科技研究完成等等，而且由于不是时刻都在检测，因此效率也是比 MTTH 更高。值得一提的是，尽管蠢驴内置了十分多的 Action，但真正能够用到的还是少数，有相当一部分 Action 原版都没有引用，大部分是由于版本更替而无用的 Action，各位在写 Event 的之时也可以体验一下更多的 Action。

4.3.2 条件(Condition)

有了触发器，我们就可以在一定情况下按照我们想要的方式触发 Event 了，不过这个时候我们发现，我们的 Event 应该需要一些限制，我们不期望每次触发都能够启动 Event，得在我们期望的场合进行触发并启动，那么这就用到了条件(Condition)，Condition 是用来限制我们 Event 启动的一种判断条件，在 Event 里面，我们可以采取 trigger 来装载 Condition，用以判断触发 Event 后是否需要启动它，下图的 Condition 十分简单，仅仅一句“is_ai = no”，用来判定该 Event 的执行者是否是 AI 帝国，如果是 AI 帝国则返回假，如果不是 AI 帝国则返回真，保证 Event 触发后能

够启动的标准是 Condition 返回值为真，Condition 一般情况下不仅仅是一条语句，通常是以一个条件组的形式进行判定，这将涉及到简易逻辑门，可以参考我在灰机 Wiki 上尚未完成的“条件表” (鸽了好久没写了)，也可以查找官方 Wiki。

```
country_event = {
    id = test_event.0
    title = test_event.0.name
    desc = test_event.0.desc
    picture = GFX_evt_analyzing_artifacts
    location = root
    show_sound = select_click

    trigger = {
        is_ai = no
    }

    is_triggered_only = yes
    # mean_time_to_happen = {
    #     months = 3
    #     modifier = {
    #         factor = 2
    #         has_civic = civic_agrarian_idyll
    #     }
    # }

    option = {
        name = test_event.0.a
    }
}
```

灰机 Wiki:

<https://qunxing.huijiwiki.com/wiki/%E6%9D%A1%E4%BB%B6%E8%A1%A8>

官方 Wiki: <https://stellaris.paradoxwikis.com/Conditions>

注意：在官方 Wiki 有这样一段话：

Testing has shown that a simple MTTH will be called with a 50% chance during the period of time specified for each item in that scope e.g. each country for a country event.

It will then check the triggers, and if they are true, the event will be triggered. However, if modifiers to the MTTH are introduced, the game will check the conditions every day for each scope - meaning that these are considerably more expensive in terms of performance.

这段话表明，在采用 MTTH 之时如若加上了 Conditions，那么会使得性能大幅下降，请各位在写 Event 之时能够注意。

4.3.3 影响(Effect)

终于到了 event 的灵魂所在 Effect，虽然 Effect 不是 Event 独有的，但是由于其独到的 Trigger，我们能够写出十分精彩的代码并且实现更多功能。我们现在已经能够触发 Event，并且能够按照我们给出的 Conditions 进行启动，那么现在这个 Event 也仅仅是一个纯文本的 Event，我们期望它能够帮我们做一点事情，比如在事件启动后立刻给国家添加一个修正，其实现代码步骤如下：

先创建一个静态修正(Static Modifier)，其位置在 common 的 static_modifiers 文件夹下，创建一个.txt 文件并写入：

```
test_modifier = {  
    pop_growth_speed = 1  
}
```

普通静态修可以单由一个 Key 加上 Modifiers 组成，其中“test_modifier”代表 Static Modifier 的 Key，“pop_growth_speed = 1”则是 Modifier（其实 Static Modifier 可以指定图标(Icon)，如果是加到星球上的 Static Modifier，还可以为其添加图标框(Icon Form)）。

当我们正确创建好静态修正之后，就可以着手开始向国家添加这个修正了，普通 Event 给予了我们三个可以执行 Effect 的地方：即刻执行(Immediate)，点击执行(Option)，结束执行(After)。

<1> Immediate

在 Event 启动之时立刻执行的 Effect，添加修正的方法很简单，直接采用“add_modifier”（add_modifier 语句能够指定一个参数用来限定修正存在的时间，如果省略则默认是无限时间，见图二）语句并指定修正名称便可，如若 Static Modifier 写入正确的话，VSC 将会有自动提示，如若未正确写入 Static Modifier，VSC 将会报错。由于该 Event 的默认作用域是 county，因此执行完这条语句之后将会给国家添加一个名为 test_modifier 的修正（未本地化的情况下）

```
namespace = test_event

country_event = {
    id = test_event.0
    title = test_event.0.name
    desc = test_event.0.desc
    picture = GFX_evt_analyzing_artifacts
    location = root
    show_sound = select_click

    trigger = {
        is_ai = no
    }

    immediate = {
        add_modifier = {
            modifier = test_modifier
        }
    }

    is_triggered_only = yes

    option = {
        name = test_event.0.a
    }
}
```

可以规定添加修正的时间，以下展示了添加时间为 360 天的修正 Effect:

```
country_event = {
    id = test_event.0
    title = test_event.0.name
    desc = test_event.0.desc
    picture = GFX_evt_analyzing_artifacts
    location = root
    show_sound = select_click

    trigger = {
        is_ai = no
    }

    immediate = {
        add_modifier = {
            modifier = test_modifier
            days = 360
            #months = 12
            #years = 1
        }
    }

    is_triggered_only = yes

    option = {
        name = test_event.0.aa
    }
}
```

<2> Option

有时候你想要的并不是瞬间执行，而是想要能够通过不同选项选触发不同的效果，幸运的是，option 可以执行 Effect，因此我们只需要稍微改动一下就好了，显而易见的是当点击第二个选项的时候才会添加修正。

```
namespace = test_event

country_event = {
    id = test_event.0
    title = test_event.0.name
```

```

desc = test_event.0.desc
picture = GFX_evt_analyzing_artifacts
location = root
show_sound = select_click

trigger = {
    is_ai = no
}

is_triggered_only = yes

option = {
    name = test_event.0.aa
    add_modifier = {
        modifier = test_modifier
    }
}
}

```

当然，这里有个瑕疵，如果你把鼠标放到第二个选项上，会自动显示出一行文本提示你点击后将会发生什么，比如该处将会提示添加一个修正到国家，我们有时候不期望有这样的文本，此时我们可以稍微改动一下，在“add_modifier”外面再套一层“hidden_effect”，这样就能够将 Effect 隐藏了：

```

namespace = test_event

country_event = {
    id = test_event.0
    title = test_event.0.name
    desc = test_event.0.desc
    picture = GFX_evt_analyzing_artifacts
    location = root
    show_sound = select_click

    trigger = {
        is_ai = no
    }

    is_triggered_only = yes

    option = {
        name = test_event.0.aa

```

```

        hidden_effect = {
            add_modifier = {
                modifier = test_modifier
            }
        }
    }
}

```

<3> After

我们有时候并不想在事件启动之时触发 Effect，而是在点击 option 之后进行一些 Effect，after 便可以满足我们，除了执行顺序不同之外，after 与 immediate 执行 Effect 没有什么区别。

```

namespace = test_event

country_event = {
    id = test_event.0
    title = test_event.0.name
    desc = test_event.0.desc
    picture = GFX_evt_analyzing_artifacts
    location = root
    show_sound = select_click

    trigger = {
        is_ai = no
    }

    is_triggered_only = yes

    option = {
        name = test_event.0.a
    }

    after = {
        add_modifier = {
            modifier = test_modifier
        }
    }
}

```

启动游戏，测试 Event 是否能够正常触发，启动，实现效果。

上文 Trigger 部分的 “is_triggered_only” 分支提到了 Effect 是可以作为触发器触发 Event 的，因此我们可以继续改装一下我们的 Event，使之能够在选完某个选项之后触发另一个

Event(country_event id = TestEvent.1):

```
country_event = {
    id = test_event.0
    title = test_event.0.name
    desc = test_event.0.desc
    picture = GFX_evt_analyzing_artifacts
    location = root
    show_sound = select_click

    immediate = {
        add_modifier = {
            modifier = test_modifier
        }
    }

    trigger = {
        is_ai = no
    }

    is_triggered_only = yes

    option = {
        name = test_event.0.aa
    }

    after = {
        country_event = {
            id = test_event.1
            days = 30
            random = 30
        }
    }
}
```

这个触发器看上去是非常简单的，如果去掉 “days” 以及 “random”，和我们一开始最基础的 event 格式是一模一样，意思想必各位都能够猜的到了，“country_event” 是要触发的 Event Type，“id” 则是要触发的 Event 的 id，“days” 指的是延迟多久发生（注意，在这里只

能够使用 days，不能够使用 months 以及 years），“random”则是在基础时间上再增加了一个随机时间，使得触发时间不再固定。

值得一提的是在 Action 中并没有“每三个月触发一次”这样的触发器，因此我们可以通过自己触发自己来达到这个目的：

```
country_event = {
    id = test_event.0
    title = test_event.0.name
    desc = test_event.0.desc
    picture = GFX_evt_analyzing_artifacts
    location = root
    show_sound = select_click

    immediate = {
        add_modifier = {
            modifier = test_modifier
        }
    }

    trigger = {
        is_ai = no
    }

    is_triggered_only = yes

    option = {
        name = test_event.0.aa
    }

    after = {
        country_event = {
            id = test_event.0
            days = 30
        }
    }
}
```

Effect 的数量与 Conditions 数量相当，因此我们应当多看，多用，多学才能够实现更多有意思的功能。

官方 Wiki: <https://stellaris.paradoxwikis.com/Effects>

4.3.4 附加属性

除了上述最常用的属性之外，Event 当中还有一些常用属性可以用来丰富自身，下面列出了大部分常用的基础属性可以帮助调整事件：

```
country_event = {
    id = test_event.0
    #hide_window = yes #是否隐藏界面，隐藏之后文本选项以及
    #is_triggered_only = yes
    is_advisor_event = yes
    title = test_event.0.name
    desc = test_event.0.desc
    location = root
    show_sound = select_click
    diplomatic = yes #是否启用外交界面
    diplomatic_title = TEST #外交界面标题本地化 Key，不写默认为“传入通讯”
    picture_event_data = { #外交界面图片声明，本质为图层的组合
        room = default_room #房间图层，在 gfx\portraits\city_sets 放入，直接使用名称即可
        #可使用基础 Scope(Country)，也可承接 Target
        portrait = ai_crisis_1 #肖像图层，可承接 Target(Leader)
        city_level = root #城市图层，可使用基础 Scope(Planet,Country,Species)，可承接 Target
        graphical_culture = root #城市显示的图像文化，可承接基础 Scope(Country)，可承接 Target
    }
    fire_only_once = yes #只启动一次，通常用于 mtth，事件启动一次之后就被扔掉(写入之后无法被控制台触发)
    mean_time_to_happen = {
        days = 100
        # scaled_modifier = { #依据 calc 给出的参数缩减 mtth 规定的时间
        #     div = 2 #修正 calc 的参数
        #     scope = this #定义 calc 调用的 Scope
        #     calc = planets_in_country #选择需要调用的游戏动态参数
        # } #比如你国家有 10 个球，则被 div 除了 2 之后就是 5，因此规定时间变为 100/5 = 20 天
        modifier = {
            add = -20
            has_civic = civic_agrarian_idyll
        }
    }
}

trigger = {
    is_ai = no
}
```

```

    immediate = {

    }

    option = {
        name = test_event.0.aa
    }
    option = {
        name = test_event.0.bb
        is_dialog_only = yes #是否为对话选项，点了之后不会退出事件，而是显示一行文本在描述中
        response_text = test_event.0.bb.response #配合上述属性，定义回复文本的本地化 Key
        default_hide_option = yes #默认退出选项，如果是按 Esc 退出的事件，默认选择的选项
        #不定义则默认为第一个声明的选项
    }

    after = {

    }
}

```

4.3.5 其他属性

关于事件还有其他可能用到的但不经常用的属性：

```

country_event = {
    id = test_event.0
    is_triggered_only = yes
    title = azur_arc.0.name
    desc = azur_arc.0.desc

    option = {
        name = azur_arc.0.aa
    }
}

```

```

country_event = {
    id = test_event.1
    base = test_event.0
}

```

如上示例，base 可以以其他事件为基准复制其一些基本属性，包括 title, desc, option 以及 is_triggered_only, show_sound 等属性，不过并不复制 immediate, after, trigger 中的内容，包

括下方将要提到的 abort_trigger 及 abort_effect 属性。

其中我们可以重写复制的一些属性，不过仅只有三个条目可以重写，示例如下：

```
country_event = {
    id = test_event.0
    is_triggered_only = yes
    title = azur_arc.0.name
    desc = azur_arc.0.desc
    show_sound = select_click

    option = {
        name = azur_arc.0.aa
    }
    abort_effect = {

    }
}
```

```
country_event = {
    id = test_event.1
    base = test_event.0
    show_sound_clear = yes
    show_sound = select_colony_ship
    desc_clear = yes
    desc = test_event.1.desc
    option_clear = yes
    option = {
        name = test_event.1.aa
    }
}
```

可以重写的条目为 desc, show_sound, option。当我们需要重写某一条目之时我们只需要写上 xx_clear 即可清除复制的基础事件的对应属性,此时即可重写该属性,按照一般的声明写法即可。

有时候我们可能需要防止某些事件一直悬浮在玩家窗口,尤其是要防止玩家挂着事件不点击之时,我们就需要用到事件的另两个属性, abort_effect 与 abort_trigger:

```
country_event = {
    id = test_event.0
    is_triggered_only = yes
```

```

    title = azur_arc.0.name
    desc = azur_arc.0.desc
    immediate = {
        set_country_flag = test_flag
    }
    abort_trigger = {
        has_country_flag = test_flag
    }
    abort_effect = {
        add_resource = {
            minerals = -100
        }
    }
    option = {
        name = azur_arc.0.aa
        add_resource = {
            minerals = -100
        }
    }
}

```

我们直接看事件下半部分即可，abort_trigger 的作用是在满足条件之时终止该事件，对于弹窗事件来说如果在窗口开启期间满足 abort_trigger 的话，在第二天该事件会被关闭，注意，这里必须是需要经过一天进行判定，关闭事件之后将会执行 abort_effect 内的效果。现在我们来看一下上方事件的逻辑：

首先 immediate 先加上了一个 country flag 作为标记，理论上如果 abort_trigger 会立刻被满足而导致事件瞬间关闭，但是由于 abort_trigger 的延迟性质，如果此刻处于暂停阶段的话，该事件仍然会存在并且可以正常点击，不过只需要取消暂停过一天时间，该事件将立刻关闭并执行 abort_effect 的内容。

最后还有一个基本上用不到的属性，major：

```

country_event = {
    id = test_event.0
    is_triggered_only = yes
    title = azur_arc.0.name
    desc = azur_arc.0.desc
}

```

```
major = yes
major_trigger = {

}
option = {
    name = azur_arc.0.aa
}
}
```

major 属性声明为 yes 之后，该事件触发之时将会给予所有满足 major_trigger 的国家发送该事件，起到一个通知的作用，不过真的很少用到。需要注意的是由于所有满足条件的国家都会触发，因此我们需要注意 option 的执行效果是会给予所有国家的，因此需要特别注意。

4.4 作用域(Scope)

终于还是来到了作用域(Scope)，我们如果要写出精彩的功能，就当然不会满足于在一个固定 Scope 进行 Effect 操作，我们需要在一个默认作用域当中去影响其他作用域并实现我们的一些功能，此时我们就需要作用域的转换了，在这一节，你将认识到何为 Scope 以及如何完成 Scope 的转换。

4.4.1 系统作用域(System Scope)

系统作用域包括四个命令：This,From,Root,Prev，不区分大小写，你可以随意写成什么样的形式，可以是 this，也可以是 THIS，当然还可以是 ThIs，不过你需要养成自己的编写习惯并能够同一格式，不然你自己写的文件里各种形式群魔乱舞，美观以及可读性都将会下降（点名批评蠢驴）。

This: This 表示当前作用域，一般来说没有什么用途，不过填写某些作用域型参数之时可能会用到。

Prev: Prev 表示上一个作用域，可以用来回溯以前的作用域，最多可以回溯四层：

PrevPrevPrevPrev。

```
country_event = {
    id = test_event.0
    hide_window = yes
    is_triggered_only = yes

    immediate = {
        random_system = {
```



```

        every_system = {
            limit = {
                distance = {
                    source = prev
                    max_jumps = 5
                }
            }
        }
    }
}

```

如上示例：random_system 随机选取一个 System 范围，将 Scope 转换到 System，在其中执行无承接 Scope 的 Effect（就是指没有规定要在某个 Scope 中才能执行的 Effect，作用域是任意 (Any)）every_system，遍历每个星系并将 Scope 转换到 System，此时我们用 limit 限定其范围，distance 表示距离 source（范围参数，指定一个 Scope）最多 5 跳的范围内，而这个 source 后面的 Prev 也就是代表回溯的上一个 Scope，也就是 random_system 所储存的一个 System（这里需要区分 every 和 random 的，前者是通过范围限定取得一个 Scope 集合，其中存在多个相同 Scope 的元素，后者则是取一个 Scope 元素）。我们可以将更多的 Effect 语句直接写入 every_system 当中（和 limit 并列），此时的 Scope 就已经转换到了随机获取的一个随机星系的 5 跳范围内的所有星系。

Root：表示一个 Event 最初始的默认 Scope。

```

country_event = {
    id = test_event.0
    hide_window = yes
    is_triggered_only = yes

    immediate = {
        every_war = {
            every_war_participant = {
                limit = {
                    NOT = {
                        is_same_value = root
                    }
                }
            }
        }
    }
}

```

```
country_event = {  
    id = test_event.0
```

```
hide_window = yes
is_triggered_only = yes
```

```
immediate = {
    every_owned_planet = {
```

```

        from = {
            }
        }
    }
}

```

如上示例，第二个 Event 中 From 指定的是该 Event 发起者的默认 Scope，也就是第一个 Event 的 Country。使用 From 需要了解该 Event 被谁调用了，此时就需要上文提到的查看引用来找到谁引用了这个 Event 用以确定 From 可能的值。此时你可能会想到如果该事件是被 Action 触发怎么办，此时需要参考蠢驴的 Action 注释，存在 From 之时将会给出注释标明当用该 Action 之时的 From 代表的 Scope，如下展示了采用 Action 触发之时 Event 中的 From 解释：

Action:

```

on_entering_battle = {
    events = {
        test_event.0
    }
}

```

Event:

```

country_event = {
    id = test_event.0
    hide_window = yes
    is_triggered_only = yes

    immediate = {
        from = {

        }
        fromfrom = {

        }
        fromfromfrom = {

        }
    }
}

```

查找蠢驴注释能够知道这个事件中：

This: 进入战斗的舰队的所有者（事件发起者）

From: 进入战斗的第二支舰队的所有者

FromFrom: 舰队（事件发起者的舰队）

FromFromFrom: 第二支舰队

4.4.2 链式作用域(Chaining Scopes)

蠢驴的作用域不仅仅能够用系统作用域进行切换，还支持点操作符进行切换，从而形成一个链式作用域。

```
country_event = {  
    id = test_event.0  
    hide_window = yes  
    is_triggered_only = yes  
  
    immediate = {  
        capital_scope.solar_system.star = {  
  
        }  
    }  
}
```

如上所示，在我们读取链式作用域之时，我们可以将“.”读作“的”，上述链式作用域 capital_scope.solar_system.star， 可以读作：首都星球的星系的主恒星。

注意：系统作用域(System Scopes)也可以参与链式作用域的一部分，比如 owner.prev，但注意，链式作用域仅仅只是指向一个作用域，且[最多嵌套 5 层](#)。

4.4.3 触发器作用域(Triggers and Scopes)、效果作用域(Effect and Scopes)

一切 Trigger 以及 Effect 均存在 Scope，需要符合相同的 Scope 才能正确执行 Effect，上文我们已经了解到了如何转换作用域，那么各位可以自行写一些包含多个作用域的 Event 来测试一下自己的水平。

4.4.4 作用域种类(Scope Types)、作用域列表(List of Types)

每个作用域都是特定类型的对象，类型决定了在何时使用它们，作用域能够被 `is_scope_type = <scope>` 检测，下面是适用于游戏的作用域类型表。

作用域类型表

Scope Type	Scopes of this type
country	owner, controller, space_owner, overlord, subject, last_created_country
sector	sector
galactic_object AKA system	solar_system, star
megastructure	megastructure
ambient_object	ambient_object, last_created_ambient_object
planet	planet, capital_scope, orbit
deposit	deposit
tile	?
archeological_site	archeological_site
army	last_created_army

Scope Type	Scopes of this type
pop	pop, last_created_pop
species	species, owner_species, last_created_species
leader	leader, ruler, last_created_leader
pop_faction	pop_faction
ship	starbase, last_created_ship
fleet	fleet, last_created_fleet
design	design, last_created_design
federation/alliance	federation/alliance
war	-

作用域表

Scope name	Scope type	Can be scoped from <code>xxx = { owner = {...} }</code>	Can scope to <code>owner = { xxx = {...} }</code>
owner	country	fleet, ship, planet, pop, leader, army, megastructure, sector, starbase	capital_scope, ruler

Scope name	Scope type	Can be scoped from <code>xxx = { owner = {...} }</code>	Can scope to <code>owner = { xxx = {...} }</code>
controller	country	fleet, ship, planet, pop, leader, army, megastructure, sector, starbase	capital_scope, ruler
space_owner	country	solar_system	capital_scope, ruler
last_created_country	country	any	capital_scope, ruler
sector	sector	planet	leader, owner
solar_system	galactic_object	planet, star, ship	star, starbase, space_owner
star	galactic_object?	solar_system	solar_system, starbase
megastructure	megastructure	-	planet, owner
ambient_object	ambient_object	-	solar_system
last_created_ambient_object	ambient_object	any	solar_system
planet	planet	pop, deposit, leader, army, archaeological_site, planet (moon)	solar_system, owner, controller

Scope name	Scope type	Can be scoped from <code>xxx = { owner = { ... } }</code>	Can scope to <code>owner = { xxx = { ... } }</code>
capital_scope	planet	country	solar_system, owner, controller
orbit	planet	fleet, ship	solar_system, owner, controller
deposit	deposit	-	planet
archeological_site	archeological_site	planet	planet, leader
army	army	ship	ship, leader, owner, pop
last_created_army	army	any	owner, pop
pop	pop	leader	owner, planet
last_created_pop	pop	any	owner, planet
species	species	pop, leader, country	-
owner_species	species	country	-
last_created_species	species	any	-
leader	leader	fleet, sector, army	species, owner

Scope name	Scope type	Can be scoped from <code>xxx = { owner = { ... } }</code>	Can scope to <code>owner = { xxx = { ... } }</code>
ruler	leader	country	species, owner
last_created_leader	leader	any	species, owner
pop_faction	pop_faction	-	owner
starbase	ship	star, solar_system	orbit, owner
last_created_ship	ship	any	orbit, leader, owner
fleet	fleet	ship	orbit, leader, owner
last_created_fleet	fleet	any	orbit, leader, owner
design	design	-	owner
last_created_design	design	any	orbit, leader, owner
federation/alliance	federation/alliance	country	-
war	war	-	-

参考资料:

<https://stellaris.paradoxwikis.com/Scopes>

<https://stellaris.paradoxwikis.com/Conditions>

4.5 Event 调试

在上一篇我们了解到了如何编写一个简单的可执行 Event，但是相信各位在自己发挥之时总会遇到这样那样的错误，要么是 Event 无法被触发，亦或是触发后并不是你想要的 Event，还可能出现无限弹窗，无法寻找到 Event 甚至崩溃的情况。当出现这些情况并在没有别人的帮助下之时，我们应该自行对自己写的代码反复进行有逻辑的测试，排查错误并加以解决，这个过程便是调试 (Debug)。

由于写 Mod 并非正规编程，我们无法进行像 IDE 一样进行断点调试以及逐行读取之类的一些调试工具，因此如若遇到恶性 Bug（诸如无限弹窗，崩溃之类严重影响游戏运行的问题），我们只能够通过不断修改局部代码，并启动游戏从而达到调试的目的，锁定真正的错误。

虽然正常调试十分的麻烦，但庆幸的是，VSC 给予了我们语法检测这个功能，一般情况来说，只要代码中不存在 VSC 检测的错误 (Errors)，能够避免十分多的语法错误，使得我们可以将注意力集中于对代码的逻辑层面的检测（VSC 的检测功能在某些情况下将会失效，这方面我会在之后进行讲解）。在了解了调试的意义之后，我们就正式开始学习如何调试自己编写的 Mod，这是 Modder 所必须具备的技能。注意：Event 的调试过程相比 Common 之中大部分内容的调试更加困难，需要更多的时间，调试过程将是及其枯燥的，需要做好心理准备。（modder 的游戏时长可不会比正常玩家少啊）

在这一段讲解可能会有一些模糊，因为每个人所犯的错误并不是一样的，因此我只能够粗略的讲解一下一些常见的 Event 错误以及调试过程。当我们一个 Event 并未按照我们的意愿触发之时，我们需要通过控制台进行强制触发（注意：如果 Event 存在语句 `fire_only_once = yes`，那么该 Event 无法通过控制台触发，且每局只会检测触发一次，不论启动与否！因此在这里不建议各位使

用该语句，这将使得调试更加麻烦！），然后查看触发之后的显示情况进行错误诊断（请务必保证 Event id 正确）。

4.5.1 正常触发

如若控制台能够正常触发，则是由于原 Event 触发器失效导致的，尤其是 Action 触发器一定要注意默认 Scope 要与需要触发的 Event 的 Scope 相同，其中还有当然一个因素是 Trigger 中的 Conditions 逻辑错误，这类问题只能够通过不断得进行 Condition 的增删，一步步调试从而达到能够满足自己的需求为止，在这方面没有人能够帮助到你，逻辑问题是这类问题的通病，另一个相近的问题则是正常触发 Event 但 Effect 失效。该问题也基本上是 Effect 语句的逻辑问题，需要自己重新对逻辑进行重构调试，Event 有内外两套执行循序，外层执行顺序是固定的（比如 id，title，immediate 一类），不会随着你写上边还是写下边而改变执行顺序，而内部逻辑

（immediate 内部的 Effect 语句）则是从上到下执行，这里有一个简单的记忆方法：属性语句固定顺序执行，逻辑以及效果语句从上到下依次执行。

4.5.2 无限弹窗

无限弹窗分为两类，一类是正常 Event 的无限弹出，一类是空白弹窗。前者如图：

```
country_event = {
    id = test_event.0
    title = test_event.0.name
    desc = test_event.0.desc

    option = {
        name = test_event.0.aa
    }
}
```

该类很容易排查，没有写上触发模式，此时 VSC 会在无触发模式的 Event 上标记蓝色波浪线用以提示该类错误。后者则是一个恶性 Bug，导致该 Bug 的原因有暂时未确定有多少，常见的便是诸如缺少括号，采用了过时语句等等，缺少括号可以通过 VSC 快速查看，只要缺少或者多出括

号，VSC 会在最后一行进行报错，但具体哪一行缺少或者多出却无从得知（如下图仅仅是最后一行有红线，黄线是由于其他因素），此时我们可以观察到 VSC 的自动提示功能失效，并且之前的已知错误将会一同被清除！如若我们无法迅速找到哪个地方括号出了问题，可以直接在最后放加减括号，修正到该错误提示消失为止。

然后通过格式化代码修正代码结构，不过这会影响原先代码结构的美观，因此记住不要乱删除括号！此时如若排查不是括号问题，那么通常情况下最可能发生的就是不正当使用语句，我们来看这样一个示例：“allow”的效果是满足其中的条件才能够使得 option 可选，不满足则变成灰色，相当于判断 Option 是否可用的 Conditions（如若你不想在不满足条件的时候隐藏该选项，则可以将 allow 换成 trigger），“fail_text”则是表示如若不满足底下的 Conditions（在这里是 NOT = {is_ai = yes}）而显示的一条自定义的文本，文本 Key 为“Test”。如若是这样写的话，那么这个 Event 被执行之后将会无限制的弹出空白窗口，选项仅仅只有一个“OK”，此时如若我们删除掉这个“fail_text”则该 Event 又将变得正常了。

```
option = {  
    name = test_event.0.aa  
    allow = {  
        fail_text = {  
            text = Test  
        }  
        is_ai = yes  
    }  
}
```

此时查阅 Wiki 我们可以看到，该语句应用在 Description 上的

Wiki 说明这要用于 TriggerDescription（一种 Desc 的形式，带条件的 Description 组，满足不同条件能够选定不同的本地化 Key，下面展示写法）上，但是我发现 VSC 貌似并没有 fail_text 的语句提示，并且输入之后还会报错：

```
desc = {  
    text = test  
    trigger = {
```

```

        is_ai = no
    }
}

```

但问题又来了，香(Vanilla)文件似乎并没有在 TriggerDescription 采用这种写法，而是在自定义文本提示(Custom Tool Tip)当中采用了这种写法，此时我们仿照原文件进行书写：

```

option = {
    name = test_event.0.aa
    allow = {
        custom_tooltip = {
            fail_text = test
            is_ai = yes
        }
    }
}

```

此时启动 Event 发现已经能够正常启动并达到我们想要的效果。这个时候我们就有理由相信 VSC 提示的是过时语句，但检测并未过时，同时 Wiki 上的语句解释应当是过时的。通过这个示例我们可以了解到，使用一个 Effect 语句需要优先查找 Wiki，然后再寻找原文件对应语句进行确认是否有效，并从中查找有效属性进行 Effect 语句效果细节扩充，这样就能正确使用 Effect 语句且能够避免语句使用错误的 Bug，但是很多 Effect 语句中的属性并未注明如何使用，因此这需要自己的猜想并进行大量尝试研究，并可以尝试将自己的人实验结果记录下来，即可以方便自己以后使用，也可以帮助后来人节约研究时间(毕竟蠢驴没有语句解析文档，还是太蠢了)。

4.5.3 游戏崩溃

导致游戏崩溃的因素有很多，但基本都能够被 VSC 检测出来，也就是说你能够清除 VSC 上的错误，基本上不会崩溃。当然，VSC 也会误报，有些认为绝对没有出错的地方产生了红线也可以不必理会（这方面只能自己多试），主要出现在变量操作以及认为“不正当”使用语句（比如原文件没有出现过的格式亦或是过时格式）。当崩溃之时，能够从蠢驴遗言中找到一些蛛丝马迹，在祖传文件夹 Paradox Interactive\Stellaris\crashes 中，每次崩溃都会生成一个文件夹，里面有一个 logs 子文件夹，进入之后会有两个文件，error.log 便是我们需要的，进入该文件就能够看到游

戏运行当中的几乎所有错误，其他基本都可以忽略（说不定能从中找出一些隐藏 bug），直接翻到最底下一行错误，这通常是蠢驴最后的遗言，如果该错误已经被证实并不影响游戏，那么很抱歉，暴毙速度太快，还没来得及说出遗言就崩溃了，此时只能够手动慢慢查找错误了。

4.5.4 未寻找到 Event

注意：蠢驴在某次更新之中修复了该问题，现在调用空 event 不会导致崩溃！

这类错误经常是固定的三个方面出错：

a. 未先声明名称空间，此时加上名称空间即可。

b. 文本编码格式错误，采用了 UTF-8 以外的编码格式，编码格式在右下显示，而我们只需要点击编码格式就会出现两个选项，我们先通过 UTF-8 编码格式保存，再点击右下角一次选择用 UTF-8 打开即可（必须先用指定编码格式保存才能再打开！编码调整在右下角）。

c. Event id 出错，使用了不正当的 id 命名方式，未按照指定格式进行 id 命名，或者是 id 的序号过大。尽管在原文件中使用了超过四位数的 id 序号，但是这里建议 id 序号不要超过四位数，也就是不要超过 9999，以免发生不必要的 Bug。

关于蠢驴 Event id 还有一点需要注意的是，.01 与 .1 的 id 是相同的，也就是说这是两个相同的 Event，只会启用先加载的 Event 的效果，写在后边的则直接忽略不进行加载（这里指的前后是），这就涉及到了 Event 的覆写模式了。蠢驴的 Event 覆写机制与其他内容不太相同，其他文件内容大部分都是后加载的同名项目覆盖先加载的项目，但是 Eevnt 一旦加载完毕，不会加载后边的同名项目，而是直接忽视。也就是说同名 Event 无法正常覆写，只能加载一次。因此当需要覆写其他文件中的 Event 之时，请保证自己的文件能够先于需要覆盖的 Event 所在文件之前加载（就是你的文件排序要在被覆写的文件之前）

```
namespace = test_event

country_event = {
    id = test_event.0
```

```

hide_window = yes
is_triggered_only = yes

immediate = {
    log = "测试一"
}
}
country_event = {
    id = test_event.0
    hide_window = yes
    is_triggered_only = yes

    immediate = {
        log = "测试二"
    }
}
}

```

上图的 id 名称事实上是相同的，这和蠢驴内部真正 Event id 的分配模式有关。由于同名，第一个 Event(id = SA_Boot_Deveice.01)加载完之后将不会再次加载第二个 Event(id = SA_Boot_Deveice.1),也就是说不论执行哪个 Event，都是执行第一个 Event 的效果（“log”的效果是在游戏运行的日志(log)上打印后边引号内的内容），因此在执行之后将会在 log 上打印“测试一”。

其实本来一开始应该先讲解 log 的，但是我认为一般来说 log 的意义不大，蠢驴本身游戏都有一堆的错误，而且这种错误只能够告诉你啥地方有错，并不会提示你哪方面错了，而且我们有了 VSC 后大部分情况下避免诸如语法错误，作用域(Scope)错误的问题（关于 VSC 对作用域确定的帮助将会在后面讲解）。当然，编写纯逻辑语句（比如天灾的 AI 逻辑）之时，我们可以通过在 Effect 中采用 log 的形式来提醒自己逻辑的执行情况。当然，也有一些错误只能够通过 log 查看的，所以调试过程中这部分内容也应当结合进去。

最后，如果各位已经完全认为自己的 Event 没有任何问题，但是就是出现这样那样的问题，这时候只能依靠玄学了（此时你可以认为蠢驴读取 Mod 可能出现了一点 Bug），改变该 Event 组的

namespace，也就是换一个名称空间。目前还不能确定名称空间的正确命名方式，目前已知的某些信息是：

- a.可以参杂下划线。
- b.namespace 中存在大写字母将可能导致一些问题，不过概率比较小。
- c.不要在 namespace 中使用“.”，这可能导致 Event id 的识别错误。

关于调试的内容差不多就这些了，接下来还是得靠各位在平时不断的错误当中寻找属于自己的一套解决方案，多查资料以及不断调式是解决 Error 的唯一方法。

4.6 Event 进阶

有了之前的基础知识，各位应该已经能够自己实现一些简单的功能，进阶篇将会对 Event 当中最常用的两类语句进行解析并结合实例进行讲解：标识(Flag)，事件对象(Event Target)。

4.6.1 标识(Flag)

统一通过 `set_<scope>_flag =` 语句对作用域内的对象加上一个标识(Flag)，该标识除非对象消失或者是被对应 Scope 的 `remove_<scope>_flag = <Flag_Key>` 语句清除，否则将一直依附于对象之上。如若我们只期望能够加上短时间内的 Flag 以供我们判断，我们可以通过

`set_timed_<Scope>_flag = {flag = <Flag_Key> <Time> = <int>}` 来设置一个带有时间的 Flag，该语句应该无需多解释，<Scope> 指定范围，<Flag_Key> 指定 Flag 名称，<Time> 指定时间单位 (days, months, years)，<int> 指定时间量（整数）。以下两个例子展示了如何向国家添加 flag。

```
country_event = {
    id = test_event.0
    is_triggered_only = yes
    hide_window = yes

    immediate = {
        set_country_flag = this_is_a_flag
    }
}

country_event = {
    id = test_event.1
    is_triggered_only = yes
    hide_window = yes

    immediate = {
        set_timed_country_flag = {
            flag = this_is_a_flag
            days = 30
        }
    }
}
```

}

既然有添加的手段，那么也有移除的手段，以下展示了如何移除一个 Flag。

```
country_event = {
    id = test_event.1
    is_triggered_only = yes
    hide_window = yes

    immediate = {
        remove_country_flag = this_is_a_flag
    }
}
```

如果移除一个不存在的 flag 将会忽视该语句，但是我仍然建议在移除之前先判断是否存在该 flag，判断 Scope 中是否存在某个 Flag 可以通过 `has_<scope>_flag = <Flag_Key>` 语句进行判定。

```
country_event = {
    id = test_event.1
    is_triggered_only = yes
    hide_window = yes

    immediate = {
        if = {
            limit = {
                has_country_flag = this_is_a_flag
            }
            remove_country_flag = this_is_a_flag
        }
    }
}
```

注意：所有 Flag 操作均需要在对应的 Scope 进行操作！接下来将展示 Flag 的几个实际用途：

示例：仅仅给第一个完成科技“灵能理论”的国家添加标记。

首先我们需要的是“完成科技”，因此寻找 `on_tech_increased` 触发器，填写 Event id

```
on_tech_increased = {
    events = {
        count_pop.0
    }
}
```

last_increased_tech 判断最后一个增加的科技是否为“灵能理论”，trigger 中的 has_global_flag 与 immediate 中的 set_global_flag 一同构成“唯一 Event”，这是最常见的唯一 Event 构建方式（fire_only_once 是在触发 Event 之后便丢弃该 Event，不论其启动与否，而大多数情况下我们期望的是仅仅执行一次 Effect 而非仅仅触发一次）。此时我们已经给这个国家添加了一个 Flag，如若以后我们需要找到第一个完成“灵能理论”的国家，只需要判断这个国家是否拥有“Research_Psionic_Theory”即可寻找到该国家。注意这里设置的 Flag 的 Scope 是 global，也就是全局，该 Scope 可以在任意 Scope 当中进行 set，remove 以及判定是否存在。

```
namespace = count_pop

country_event = {
    id = count_pop.0
    is_triggered_only = yes
    hide_window = yes

    trigger = {
        NOT = {
            has_global_flag = first_research_psionic_theory
        }
        last_increased_tech = tech_psionic_theory
    }

    immediate = {
        set_global_flag = first_research_psionic_theory
        set_country_flag = research_psionic_theory
    }
}
```

然后，设置一个每年触发的全局 Event，检索是否存在第一个完成“灵能理论”的国家（注意要是第一个完成“灵能理论”的国家而非拥有“灵能理论”的国家），如若检索到存在则触发该 Event。

首先我们写上 Scope 是全局的每年检测的 Action 触发器：

```
on_yearly_pulse = {
    events = {
        count_pop.1
```

```
}
}
```

然后是检测任意国家是否存在 Flag，any_country 用以遍历当前所有国家，如若存在任意一个国家满足 has_country_flag，则返回真值。（也可以检测全局 flag）

```
event = {
  id = count_pop.1
  is_triggered_only = yes
  hide_window = yes

  trigger = {
    #has_global_flag = first_research_psionic_theory
    any_country = {
      has_country_flag = research_psionic_theory
    }
  }
}
```

设定上一步 Event(count_pop.1)的效果为：让第一个完成“灵能理论”的国家触发另一个 country_event(count_pop.2)。该 Event(count_pop.2)的效果为：迁移随机一个星球上一半的人口到首都。

先补充完 Event(count_pop.1)，通过 every_country 遍历国家，再通过我们之前设置的 country_flag 筛选国家，然后让筛选出的国家执行接下来的事件：

```
event = {
  id = count_pop.1
  is_triggered_only = yes
  hide_window = yes

  trigger = {
    #has_global_flag = first_research_psionic_theory
    any_country = {
      has_country_flag = research_psionic_theory
    }
  }

  immediate = {
    every_country = {
      limit = {
        has_country_flag = research_psionic_theory
      }
      country_event = {
```

```

        id = count_pop.2
    }
}
}
}

```

然后写出 Event(count_pop.2)的框架：

```

country_event = {
    id = count_pop.2
    is_triggered_only = yes
    hide_window = yes

    immediate = {

    }
}

```

然后几居室碎料机选择一个殖民星球并将 Scope 转到 planet：

```

country_event = {
    id = count_pop.2
    is_triggered_only = yes
    hide_window = yes

    immediate = {
        random_owned_planet = {

        }
    }
}

```

我们完成这个功能的思路是：获取一半的人口，转移到首都。那么如何决定转移人口的数量呢，单纯的在 every_owned_pop 中是无法实现的，即使我们能够通过 limit 筛选一部分 pop，但是仍然无法精确指定一半的人，因此我们需要通过采用 while 循环来进行操作。首先需要明确 while 的用法。循环，顾名思义，循环执行某一段命令，以下示例是通过 count 来限制循环次数，这里为 5，因此将执行 random_owned_pop = {kill_pop = yes}五次，也就是随机杀死星球上五个人口，**注意，这个 count 可以承接变量的值**，这将是关键。

```

country_event = {
    id = count_pop.2
    is_triggered_only = yes

```

```

hide_window = yes

immediate = {
    random_owned_planet = {
        while = {
            count = 5
            random_owned_pop = {
                kill_pop = yes
            }
        }
    }
}
}
}

```

当然，我们也可以通过 limit 来限制循环次数，最多 100 次（曾经测试过两次，一次 100 一次 200，结合其他人的说法，应该是 100 次）。以下这个例子展示了 limit 的用法，需要满足 limit 内的条件才能继续循环，这里写的条件为是星球上人口数量大于 0 结合整体语句效果就是：随机杀死一颗星球上的全部人口（不超过 100，如若超过则杀死 100 人这里有可以优化的地方，但是在这里简单处理，无需考虑 100 人以上的情况）

```

country_event = {
    id = count_pop.2
    is_triggered_only = yes
    hide_window = yes

    immediate = {
        random_owned_planet = {
            while = {
                limit = {
                    count_owned_pop = {
                        count > 0
                    }
                }
                random_owned_pop = {
                    kill_pop = yes
                }
            }
        }
    }
}
}

```

这里我们需要采用第一种 count 的方式来限制循环次数（这里如果要用 limit 完全可以直接采用 every_owned_pop 操作）。我们上边已经提到了 count 的值可以承接变量(Variable)，此时我们需要用到 Flag 以及变量(Variable)的组合了（这里稍微涉及了一些简单变量操作），首先我们将需要统计的范围内所有人口加上 Flag：

```
country_event = {
    id = count_pop.2
    is_triggered_only = yes
    hide_window = yes

    immediate = {
        random_owned_planet = {
            every_owned_pop = {
                set_pop_flag = count_people
            }
        }
    }
}
```

此时我们并不知道我们给多少人口加上了 Flag，我们仅仅只是知道了这些人都有了这个 Flag，现在我们需要通过变量来获取人口数量，首先先写一个 while 循环，因为我们不清楚需要循环多少次，因此采取 limit 限制，限制条件为任意一个人口拥有名为 Count_People 的 Flag，在执行区域我们随机清除一个拥有 Flag 的人口的 Flag，然后再使用 change_variable 使得变量+1（蠢驴变量不需要手动初始化即可直接使用，默认初始化为 0），这样我们就能够每次清除一个 Flag 都使得变量+1，从而得到总人口数量。

```
country_event = {
    id = count_pop.2
    is_triggered_only = yes
    hide_window = yes

    immediate = {
        random_owned_planet = {
            every_owned_pop = {
                set_pop_flag = count_people
            }
        }
    }
    while = {
```



```

        limit = {
            any_owned_pop= {
                has_pop_flag = count_people
            }
        }
        remove_pop_flag = count_people
    }
    change_variable = {
        which = total_pop
        value = 1
    }
}
}
}

```

此时我们需要转移一半的人口，则需要对变量进行除法操作，divide_variable 提供了这一功能，which 填写被除数，value 填写除数，前者必须是一个变量名称，后者即可以是一个变量名称，也可以是一个数字，这里我们需要分半因此填写 2。

```

country_event = {
    id = count_pop.2
    is_triggered_only = yes
    hide_window = yes

    immediate = {
        random_owned_planet = {
            every_owned_pop = {
                set_pop_flag = count_people
            }
        }
        while = {
            limit = {
                any_owned_pop= {
                    has_pop_flag = count_people
                }
            }
            remove_pop_flag = count_people
        }
        change_variable = {
            which = total_pop
            value = 1
        }
    }
    divide_variable = {

```

```

        which = total_pop
        value = 2
    }
}
}

```

现在我们的变量已经是总人口数量的一半了，再创建一个 while 循环，采用 count 进行次数限制，上文已经提过了，count 可承接一个变量的值（自动取整），因此我们只需要把变量名称填写上去就好了，此时需要转移人口，每次转移一个，因此采用 random_owned_people 转移到 pop，再采用 resettle_pop 进行人口转移。pop 选择需要转移的 Scope 为 pop；planet 选择需要转移的 Scope 为 planet。显然这里需要转移的 pop 就是当前随机选中的 pop，因此写上 pop = this，需要转移的 planet 是首都，这里可以直接写上 planet = capital_scope 进行处理（以下示例我将变量操作进行了行数缩减，内容并在了一行，为了能够一张图能够截全，各位自行尝试之时还是建议将其完全展开，增强其可读性）

```

country_event = {
    id = count_pop.2
    is_triggered_only = yes
    title = count_pop.2.name
    desc = count_pop.2.desc

    immediate = {
        random_owned_planet = {
            every_owned_pop = {
                set_pop_flag = count_people
            }
        }
        while = {
            limit = {
                any_owned_pop = {
                    has_pop_flag = count_people
                }
            }
            random_owned_pop = {
                limit = {
                    has_pop_flag = count_people
                }
                remove_pop_flag = count_people
            }
        }
    }
}

```

```
change_variable = {  
    which = total_pop  
    value = 1  
}  
  
}  
  
divide_variable = {  
    which = total_pop  
    value = 2  
}  
  
while = {  
    count = total_pop  
    random_owned_pop = {  
        resettlement_pop = {  
            pop = THIS  
            planet = capital_scope  
        }  
    }  
}  
  
}  
  
}  
  
}  
  
}
```

此时这个功能就完成了。

然后我们需要将变量显示到文本框中出来，这里将应用支架命令(Bracket Commands)。

有时候我们不需要知道我们转移了多少人口，但有时候我们想要知道转移了多少人口，或者我们需要统计人口总数，蠢驴在本地化之时提供了一种显示当前 Scope 内 Variable 的方式：支架命令(Bracket Commands)，支架命令的调用格式为：[.]，其中 Scope 表示需要执行命令的 scope；Commands 表示需要执行何种命令，其有三种类型：

1. 变量显示类：形式为：[<Scope>.<Variable>]，Scope 表示需要显示的变量的 Scope，Variable 表示 Scope 内 variable 的名称。
2. 预设指令类：形式为：[<Scope>.<Pre-Commands>]，Scope 表示需要进行执行 Pre-Commands 的 Scope，Pre-Commands 表示需要对目标 Scope 进行何种命令用以获取

其何种属性。Pre-Commands 可以参考官方 Wiki 的文件检索(Text Retrieval)指令，其中使用最为频繁的是 GetName 指令，作用是获取当前 Scope 的名称。

3. 自定义指令类：形式为：[<Scope>.<Loc-Name>]，Scope 表示需要执行 Loc-Name 的 Scope，Loc-Name 为之定义指令，可在 common/scripted_loc 文件夹下定义。

关于以上内容的 2，3 点将会在之后的章节中进行详细叙述，现在仅仅需要掌握第一点即可，现在我们来改造一下我们上方已经完成的 Event，将最后一个 Event(id = Count_Pop.2)，改造成一个可见 Event，并且在文本当中显示出迁移的人口数量。我们此时需要改变设置 Variable 的 Scope，从之前的直接在 Planet 当中设置 Variable 变成在 root（此处 root 为 Country）中设置 Variable，以方便我们采用支架命令进行显示。

而且我们需要注意的是，之前我们所写的 Event 逻辑其实是有问题的，如果这个 Event 需要不断的启动，我们的变量在使用完之后并为归 0，将会导致下一次进行人口统计的失误，因此我们需要将变量手动置为 0。

```
country_event = {
    id = count_pop.2
    is_triggered_only = yes
    title = count_pop.2.name
    desc = count_pop.2.desc

    immediate = {
        random_owned_planet = {
            every_owned_pop = {
                set_pop_flag = count_people
            }
        }
        while = {
            limit = {
                any_owned_pop = {
                    has_pop_flag = count_people
                }
            }
        }
        random_owned_pop = {
            limit = {
                has_pop_flag = count_people
            }
        }
    }
}
```

```

        }
        remove_pop_flag = count_people
    }
    root = {
        change_variable = {
            which = total_pop
            value = 1
        }
    }
}
root = {
    divide_variable = {
        which = total_pop
        value = 2
    }
    while = {
        count = total_pop
        prev = {
            random_owned_pop = {
                resettle_pop = {
                    pop = THIS
                    planet = capital_scope
                }
            }
        }
    }
}
}
}
}
}

after = {
    set_variable = {
        which = total_pop
        value = 0
    }
}
}
}

```

这里写入 after 和写入 immediate 最后的执行顺序并没有什么区别，仅仅是为了区分一下，使 immediate 的内容仅仅用来实现主要功能，after 进行执行功能完毕后的某些对象的重置处理，使得可读性更高。

以上还要注意的一点是，我每次改变 Variable 之前都嵌套了一个 root，这是因为 Variable 也是有 Scope 的，因此我们需要统一在一个 Scope 中进行操作，在之前的事例当中，我们的 Variable 全部在一个随机的 planet 当中，此时变量是依附于该 Scope，因此可以进行计算，但我们无法取到该 Scope 进行其上的变量显示，因此我们每次操作变量之时都将 Scope 统一转移到 country 当中操作。在下面的 while 循环中我们也切换到了 country 以便取到 Variable 的值，但执行的语句需要在我们随机取到的 planet 之上，因此再嵌套了一个 prev 用以返回到上一个 Scope。

现在来看看本地化（此处我没有写 option，因为没必要）：

```
count_pop.2.name:0 "迁移人口"
count_pop.2.desc:0 "[this.total_pop]"
```

this 代表当前 event 的 this 作用域，这里显然是 country，total_pop 表示将会检索该作用域中同名的 Variable 并将其显示出来，注意这里的作用域一定要是 Variable 的所在 Scope，不然将会显示为空（如果变量为 0 也会显示为空，可以采用 scripted_loc 进行补偿显示）。

4.6.2 事件对象(Event Target)

事件对象分为两种，一种为局部事件对象，由 save_event_target_as = <TargetKey>进行声明，另一种为全局事件对象，由 save_global_event_target_as = <TargetKey>进行声明，该语句作用为存储当前 Scope，以便之后的调用。

1. 局部事件对象

以下示例展示了如何将一个首都星球的 Scope 存储到名为 this_is_capital 的局部事件对象当中：

```
namespace = test_event

country_event = {
    id = test_event.0
    is_triggered_only = yes
    hide_window = yes
```

```

    immediate = {
        capital_scope = {
            save_event_target_as = this_is_capital
        }
    }
}

```

接下来我们可以在该 Event 中任何需要用到首都的地方均可以通过调用该 Target 来代替，现在我们假设需要实现如下功能：

让玩家自行选择为自己某一个领袖添加一个特质

该功能涉及一下问题：

(1)列举所有领袖

(2)添加特质

我们先确定如何列举所有的领袖，蠢驴并没有给出这个接口，因此我们只能通过一个选项列举一个领袖的方式进行穷举，我们无法动态的给予选项数量，而且考虑到我们的界面无法显示过多的选项，我们决定采用八个选项显示八个不同领袖的名称（你也可以自行设定个数），然后通过一个刷新按钮进行名称的刷新以求能够遍历取到我们所需要的领袖。首先，我们进行基础架构：

```

namespace = test_event

country_event = {
    id = test_event.0
    is_triggered_only = yes
    title = test_event.0.name
    desc = test_event.0.desc

    immediate = {

    }

    option = {
        name = test_event.0.aa
    }
    option = {
        name = test_event.0.bb
    }
}

```

```

    option = {
        name = test_event.0.cc
    }
    option = {
        name = test_event.0.dd
    }
    option = {
        name = test_event.0.ee
    }
    option = {
        name = test_event.0.ff
    }
    option = {
        name = test_event.0.gg
    }
    option = {
        name = test_event.0.hh
    }
    option = {
        name = test_event.0.00
    }
    option = {
        name = test_event.0.01
    }
}

```

然后我们需要进行领袖的选择，由于我们无法定向选择某个领袖，因此我们需要用到 random_owned_leader 进行随机选取，每次随机选取我们就存储一个 Target，此时我们需要八个选项，因此我们需要八个不同的 Target 来存储不同的 leader：

```

country_event = {
    id = test_event.0
    is_triggered_only = yes
    title = test_event.0.name
    desc = test_event.0.desc

    immediate = {
        random_owned_leader = {
            save_event_target_as = select_leader_1
        }
        random_owned_leader = {
            save_event_target_as = select_leader_2
        }
    }
}

```



```
    random_owned_leader = {
        save_event_target_as = select_leader_3
    }
    random_owned_leader = {
        save_event_target_as = select_leader_4
    }
    random_owned_leader = {
        save_event_target_as = select_leader_5
    }
    random_owned_leader = {
        save_event_target_as = select_leader_6
    }
    random_owned_leader = {
        save_event_target_as = select_leader_7
    }
    random_owned_leader = {
        save_event_target_as = select_leader_8
    }
}

option = {
    name = test_event.0.aa
}
option = {
    name = test_event.0.bb
}
option = {
    name = test_event.0.cc
}
option = {
    name = test_event.0.dd
}
option = {
    name = test_event.0.ee
}
option = {
    name = test_event.0.ff
}
option = {
    name = test_event.0.gg
}
option = {
    name = test_event.0.hh
}
```

```

    option = {
        name = test_event.0.00
    }
    option = {
        name = test_event.0.01
    }
}

```

我们此时可以想到一个问题，容易重复选取到同一个对象，因此我们需要将已经选取的对象进行筛选，利用我们上边提到的 Flag 的知识，我们可以很容易想到，每选取一个领袖就为该领袖上一个 Flag，筛选没有该 Flag 的领袖存储 Target 即可，不过此时记得在事件完成之后完成清除 flag 以免影响下一次事件触发的判定：

```

country_event = {
    id = test_event.0
    is_triggered_only = yes
    title = test_event.0.name
    desc = test_event.0.desc

    immediate = {
        random_owned_leader = {
            limit = {
                NOT = {
                    has_leader_flag = screening_flag
                }
            }
            save_event_target_as = select_leader_1
            set_leader_flag = screening_flag
        }
        random_owned_leader = {
            limit = {
                NOT = {
                    has_leader_flag = screening_flag
                }
            }
            save_event_target_as = select_leader_2
            set_leader_flag = screening_flag
        }
        random_owned_leader = {
            limit = {
                NOT = {
                    has_leader_flag = screening_flag
                }
            }

```

```
    }  
  }  
  save_event_target_as = select_leader_3  
  set_leader_flag = screening_flag  
}  
random_owned_leader = {  
  limit = {  
    NOT = {  
      has_leader_flag = screening_flag  
    }  
  }  
  save_event_target_as = select_leader_4  
  set_leader_flag = screening_flag  
}  
random_owned_leader = {  
  limit = {  
    NOT = {  
      has_leader_flag = screening_flag  
    }  
  }  
  save_event_target_as = select_leader_5  
  set_leader_flag = screening_flag  
}  
random_owned_leader = {  
  limit = {  
    NOT = {  
      has_leader_flag = screening_flag  
    }  
  }  
  save_event_target_as = select_leader_6  
  set_leader_flag = screening_flag  
}  
random_owned_leader = {  
  limit = {  
    NOT = {  
      has_leader_flag = screening_flag  
    }  
  }  
  save_event_target_as = select_leader_7  
  set_leader_flag = screening_flag  
}  
random_owned_leader = {  
  limit = {  
    NOT = {
```

```
        has_leader_flag = screening_flag
    }
}
save_event_target_as = select_leader_8
set_leader_flag = screening_flag
}
}

option = {
    name = test_event.0.aa
}
option = {
    name = test_event.0.bb
}
option = {
    name = test_event.0.cc
}
option = {
    name = test_event.0.dd
}
option = {
    name = test_event.0.ee
}
option = {
    name = test_event.0.ff
}
option = {
    name = test_event.0.gg
}
option = {
    name = test_event.0.hh
}
option = {
    name = test_event.0.00
}
option = {
    name = test_event.0.01
}

after = {
    every_owned_leader = {
        limit = {
            has_leader_flag = screening_flag
        }
    }
}
```

```

        remove_leader_flag = screening_flag
    }
}
}

```

以此将全部 random_owned_leader 加上判断以及效果，此时我们会考虑可能没有那么多领袖，导致存储了一个空的 Target，因此我们最好再加上一个判断；而且我们也不期望在领袖不足之时弹出过多选项，因此我们也在 option 上加上判断（刷新按钮当然得出现满选项才能使用，因此判断和最后一个承接领袖的 trigger 一样）：

```

country_event = {
    id = test_event.0
    is_triggered_only = yes
    title = test_event.0.name
    desc = test_event.0.desc

    immediate = {
        random_owned_leader = {
            limit = {
                NOT = {
                    has_leader_flag = screening_flag
                }
            }
        }
        if = {
            limit = {
                exists = this
            }
            save_event_target_as = select_leader_1
            set_leader_flag = screening_flag
        }
    }
    random_owned_leader = {
        limit = {
            NOT = {
                has_leader_flag = screening_flag
            }
        }
        if = {
            limit = {
                exists = this
            }
            save_event_target_as = select_leader_2
        }
    }
}

```

```
        set_leader_flag = screening_flag
    }
}
random_owned_leader = {
    limit = {
        NOT = {
            has_leader_flag = screening_flag
        }
    }
    if = {
        limit = {
            exists = this
        }
        save_event_target_as = select_leader_3
        set_leader_flag = screening_flag
    }
}
random_owned_leader = {
    limit = {
        NOT = {
            has_leader_flag = screening_flag
        }
    }
    if = {
        limit = {
            exists = this
        }
        save_event_target_as = select_leader_4
        set_leader_flag = screening_flag
    }
}
random_owned_leader = {
    limit = {
        NOT = {
            has_leader_flag = screening_flag
        }
    }
    if = {
        limit = {
            exists = this
        }
        save_event_target_as = select_leader_5
        set_leader_flag = screening_flag
    }
}
```

```
}
random_owned_leader = {
    limit = {
        NOT = {
            has_leader_flag = screening_flag
        }
    }
    if = {
        limit = {
            exists = this
        }
        save_event_target_as = select_leader_6
        set_leader_flag = screening_flag
    }
}
random_owned_leader = {
    limit = {
        NOT = {
            has_leader_flag = screening_flag
        }
    }
    if = {
        limit = {
            exists = this
        }
        save_event_target_as = select_leader_7
        set_leader_flag = screening_flag
    }
}
random_owned_leader = {
    limit = {
        NOT = {
            has_leader_flag = screening_flag
        }
    }
    if = {
        limit = {
            exists = this
        }
        save_event_target_as = select_leader_8
        set_leader_flag = screening_flag
    }
}
}
```

```
option = {
    name = test_event.0.aa
    trigger = {
        exists = event_target:select_leader_1
    }
}
option = {
    name = test_event.0.bb
    trigger = {
        exists = event_target:select_leader_2
    }
}
option = {
    name = test_event.0.cc
    trigger = {
        exists = event_target:select_leader_3
    }
}
option = {
    name = test_event.0.dd
    trigger = {
        exists = event_target:select_leader_4
    }
}
option = {
    name = test_event.0.ee
    trigger = {
        exists = event_target:select_leader_5
    }
}
option = {
    name = test_event.0.ff
    trigger = {
        exists = event_target:select_leader_6
    }
}
option = {
    name = test_event.0.gg
    trigger = {
        exists = event_target:select_leader_7
    }
}
option = {
```



```

    name = test_event.0.hh
    trigger = {
        exists = event_target:select_leader_8
    }
}
option = {#刷新
    name = test_event.0.00
    trigger = {
        exists = event_target:select_leader_8
    }
}
option = {
    name = test_event.0.01
}

after = {
    every_owned_leader = {
        limit = {
            has_leader_flag = screening_flag
        }
        remove_leader_flag = screening_flag
    }
}
}

```

该判断语句为 `exists = this` 以及 `exists = event_target:<TargetKey>`，后者表示调用之前存储的 Target，语句表示是否存在当前 Scope，虽然存储一个空 Target 会直接删除该 Target，但我期望各位能够有一个良好的习惯，不对空对象进行操作，以免遇到不必要的麻烦。此时我们已经取到了八个不同的领袖，此时我们需要将其依附于不同选项，开始调用这八个 Target 并添加一个特质，在刷新选项添加重启该事件的效果来随机重排选择的领袖：

```

country_event = {
    id = test_event.0
    is_triggered_only = yes
    title = test_event.0.name
    desc = test_event.0.desc

    immediate = {
        random_owned_leader = {
            limit = {
                NOT = {

```

```
        has_leader_flag = screening_flag
    }
}
if = {
    limit = {
        exists = this
    }
    save_event_target_as = select_leader_1
    set_leader_flag = screening_flag
}
}
random_owned_leader = {
    limit = {
        NOT = {
            has_leader_flag = screening_flag
        }
    }
    if = {
        limit = {
            exists = this
        }
        save_event_target_as = select_leader_2
        set_leader_flag = screening_flag
    }
}
random_owned_leader = {
    limit = {
        NOT = {
            has_leader_flag = screening_flag
        }
    }
    if = {
        limit = {
            exists = this
        }
        save_event_target_as = select_leader_3
        set_leader_flag = screening_flag
    }
}
random_owned_leader = {
    limit = {
        NOT = {
            has_leader_flag = screening_flag
        }
    }
}
```

```
    }
    if = {
        limit = {
            exists = this
        }
        save_event_target_as = select_leader_4
        set_leader_flag = screening_flag
    }
}
random_owned_leader = {
    limit = {
        NOT = {
            has_leader_flag = screening_flag
        }
    }
    if = {
        limit = {
            exists = this
        }
        save_event_target_as = select_leader_5
        set_leader_flag = screening_flag
    }
}
random_owned_leader = {
    limit = {
        NOT = {
            has_leader_flag = screening_flag
        }
    }
    if = {
        limit = {
            exists = this
        }
        save_event_target_as = select_leader_6
        set_leader_flag = screening_flag
    }
}
random_owned_leader = {
    limit = {
        NOT = {
            has_leader_flag = screening_flag
        }
    }
    if = {
```

```

        limit = {
            exists = this
        }
        save_event_target_as = select_leader_7
        set_leader_flag = screening_flag
    }
}

random_owned_leader = {
    limit = {
        NOT = {
            has_leader_flag = screening_flag
        }
    }
    if = {
        limit = {
            exists = this
        }
        save_event_target_as = select_leader_8
        set_leader_flag = screening_flag
    }
}

}

option = {
    name = test_event.0.aa
    trigger = {
        exists = event_target:select_leader_1
    }
    hidden_effect = {
        event_target:select_leader_1 = {

        }
    }
}

option = {
    name = test_event.0.bb
    trigger = {
        exists = event_target:select_leader_2
    }
    hidden_effect = {
        event_target:select_leader_2 = {

        }
    }
}

```

```
}
option = {
    name = test_event.0.cc
    trigger = {
        exists = event_target:select_leader_3
    }
    hidden_effect = {
        event_target:select_leader_3 = {

        }
    }
}
option = {
    name = test_event.0.dd
    trigger = {
        exists = event_target:select_leader_4
    }
    hidden_effect = {
        event_target:select_leader_4 = {

        }
    }
}
option = {
    name = test_event.0.ee
    trigger = {
        exists = event_target:select_leader_5
    }
    hidden_effect = {
        event_target:select_leader_5 = {

        }
    }
}
option = {
    name = test_event.0.ff
    trigger = {
        exists = event_target:select_leader_6
    }
    hidden_effect = {
        event_target:select_leader_6 = {

        }
    }
}
```

```

}
option = {
    name = test_event.0.gg
    trigger = {
        exists = event_target:select_leader_7
    }
    hidden_effect = {
        event_target:select_leader_7 = {

        }
    }
}
option = {
    name = test_event.0.hh
    trigger = {
        exists = event_target:select_leader_8
    }
    hidden_effect = {
        event_target:select_leader_8 = {

        }
    }
}
option = {#刷新
    name = test_event.0.00
    trigger = {
        exists = event_target:select_leader_8
    }
    hidden_effect = {
        country_event = {
            id = test_event.0
        }
    }
}
option = {#退出
    name = test_event.0.01
}

after = {
    every_owned_leader = {
        limit = {
            has_leader_flag = screening_flag
        }
        remove_leader_flag = screening_flag
    }
}

```

```

    }
  }
}

```

最后进行本地化，我们需要让玩家知道我们所获取的 Target 到底是谁，因此我们需要显示出 Target 的名称，此时我们需要采用支架命令：

```

test_event.0.aa:0 "[select_leader_1.GetName]"
test_event.0.bb:0 "[select_leader_2.GetName]"
test_event.0.cc:0 "[select_leader_3.GetName]"
test_event.0.dd:0 "[select_leader_4.GetName]"
test_event.0.ee:0 "[select_leader_5.GetName]"
test_event.0.ff:0 "[select_leader_6.GetName]"
test_event.0.gg:0 "[select_leader_7.GetName]"
test_event.0.hh:0 "[select_leader_8.GetName]"
test_event.0.00:0 "刷新"
test_event.0.01:0 "返回"

```

我们可以通过[Target.GetName]获取 Target 的 Name 属性，即显示出其名称。

2. 全局事件对象

全局事件对象的声明以及使用与局部事件对象并无太大区别（声明的语句有些许不同，但使用均是 `event_target:<TargetKey> = {<Effect>}`），但与局部事件对象不同的是，只要这个 Target 被声明，将可以在任何地方进行调用，而不仅仅局限于当前的 Event

注意：

局部事件对象在单个 Event 当中的存在时间为事件开始到事件结束，但是如若该 Event 启动了其他 Event，此时第一个 Event 存储的 Target 将可以继承到下一个 Event：

```

country_event = {
  id = test_event.0
  is_triggered_only = yes
  hide_window = yes

  immediate = {
    save_event_target_as = test_target
    country_event = {
      id = test_event.1
    }
  }
}

```

```

}
country_event = {
    id = test_event.1
    is_triggered_only = yes
    hide_window = yes

    immediate = {
        event_target:test_target = {

        }
    }
}
}

```

此时的 test_target 将会从 Event(id = test.0)继承到下一个 Event(id = event_test.1)，因此第二个 Event(id = event_test.1)中的 event_target:test_target = {} 合法且有效。

全局事件对象声明之后，除非该对象消失或者被手动清除，否则将一直存在，我们需要通过通过一个特定的清除语句进行操作：

```

country_event = {
    id = test_event.0
    is_triggered_only = yes
    hide_window = yes

    immediate = {
        save_global_event_target_as = test_target
    }
}
country_event = {
    id = test_event.1
    is_triggered_only = yes
    hide_window = yes

    immediate = {
        clear_global_event_target = test_target
    }
}
}

```

第一个 Event(id = test.0)进行全局事件对象的声明(test_target)，第二个 Event(id = test.1)进行 test_target 的定向清除(clear_global_event_target = <GlobalTargetKey>)，此时我们需要注意一个相近的语句：

```
clear_global_event_targets = yes
```

该语句的作用是清除当前存储的所有全局事件对象，请慎重使用该语句，很多全局事件对象承载着很多关键 Event 的效果实现，一旦该语句生效，该档可以宣告结束，将会出现一堆不可预知的 bug（指定对象缺失），堪称隐性炸档利器。

4.6.3 脚本效果(Scripted Effect)

在学习事件过程中你肯定见过一种情况，一个你觉得需要写几十行代码的功能，别人一行语句就解决了，然后你尝试也这样去做，然后失败了，这其中涉及到的就是脚本效果(Scripted Effect)。蠢驴给出了减少重复代码编写的方式，即将需要重复使用的代码进行封装到 common/scripted_effects 文件夹下的文件当中，然后外部直接使用封装 Key 实现调用，具体我们看以下示例：

以下示例来自工坊 id 2047260446 ,Arknights Near Future

```
civic_pharmaceutical_enterprises_start_buildings = {
  if = {
    limit = { owner = { is_regular_empire = yes } }

    if = {
      limit = { owner = { has_civic = civic_pharmaceutical_enterprises } }
      remove_building = building_commercial_zone
      add_building = building_ark_hospital
    }
  }
}
```

上述示例展示了如何将一段 Effect 进行封装，只需要在我们需要封装的 Effect 合集外面套一层封装 Key(自定义)即可，封装之后的 Effect 合集也算作 Effect，因此我们可以在任何可以使用 Effect 的地方调用该 Scripted Effect：

以下示例来自工坊 id 2047260446 ,Arknights Near Future

```
planet_event = {
  id = ark_game_start.1
  hide_window = yes
  is_triggered_only = yes

  immediate = {
```

```

    if = {
        limit = { has_planet_flag = planet_terra } # 前提:是泰拉星球
        generate_terra_deposits_and_blockers = yes # 初始地块
        civic_pharmaceutical_enterprises_start_buildings = yes # 初始建筑
        generate_terra_start_districts = yes # 初始区划
    }
}
}

```

如上示例，我们可以很轻易地调用该 Scripted Effect，仅需要<Key> = yes 即调用该 Effect 集合，该段效果与将 Effect 集合直接写入调用区域完全相同，如下示例与上述示例效果完全相同：

```

planet_event = {
    id = ark_game_start.1
    hide_window = yes
    is_triggered_only = yes

    immediate = {
        if = {
            limit = { has_planet_flag = planet_terra } # 前提:是泰拉星球
            generate_terra_deposits_and_blockers = yes # 初始地块
            #civic_pharmaceutical_enterprises_start_buildings = yes # 初始建筑
            if = {
                limit = { owner = { is_regular_empire = yes } }
                if = {
                    limit = { owner = { has_civic = civic_pharmaceutical_enterprises } }
                    remove_building = building_commercial_zone
                    add_building = building_ark_hospital
                }
            }
            generate_terra_start_districts = yes # 初始区划
        }
    }
}
}

```

需要注意的是,Scripted Effect 没有固定的默认 Scope,其 this 作用域和调用之时所在的 Scope 相同，也就是说你在 country 调用 Scripted Effect，那么你所调用的 Scripted Effect 的 this 作用域就是 country。

Scripted Effect 可以有效的减少重复代码的书写，以便在需要改动之时仅需改动我们所封装的 Effect 集合即可达到修改所有调用区域所实现的 Effect，是大型 mod 的必备操作形式。

关于 Scripted Effect 的进阶用法将在之后的章节进行讲解。

4.6.4 脚本条件(Scripted Trigger)

与 Scripted Effect 一样, Scripted Trigger 也是对某一段代码的封装, 不过其承载的是 Condition 而非 Effect, 也就是说 Scripted Trigger 是一个 Condition 的集合封装, 其声明在 common/scripted_triggers 下的文件当中, 其调用以及声明皆与 Scripted Effect 一样, 可以在任何需要承接 Condition 的地方进行调用, 其 this 作用域也遵循与 Scripted Effect 一样的规律

以下示例来自工坊 id 2047260446 ,Arknights Near Future

```
buildings_no_d32_steels = {
  exists = owner
  owner = {
    is_ai = yes
    has_monthly_income = {
      resource = d32_steels
      value < 0
    }
    has_resource = { type = d32_steels amount = 0 }
  }
} # 用于拆除需要 D32 钢但缺少 D32 钢的建筑物
```

调用(使用于 destroy_trigger):

以下示例来自工坊 id 2047260446 ,Arknights Near Future

```
building_military_enterprises = {
  base_buildtime = @b2_time
  base_cap_amount = 1

  category = manufacturing

  potential = {
    exists = owner
    owner = { is_regular_empire = yes }
    NOR = {
      has_modifier = resort_colony
      has_modifier = slave_colony
    }
  }
}

destroy_trigger = {
  exists = owner
```

```
OR = {
    owner = { is_regular_empire = no }
    has_modifier = resort_colony
    has_modifier = slave_colony
    buildings_no_d32_steels = yes
}

}

allow = {
    has_major_upgraded_capital = yes
}

planet_modifier = {
    planet_metallurgists_produces_mult = 0.30
}

triggered_planet_modifier = {
    potential = {
        exists = owner
        owner = { is_gestalt = no }
    }
    modifier = {
        job_foundry_add = 2
    }
}

triggered_planet_modifier = {
    potential = {
        exists = owner
        owner = { is_gestalt = yes }
    }
    modifier = {
        job_alloy_drone_add = 2
    }
}

resources = {
    category = planet_buildings
    cost = {
        minerals = @b2_minerals
        d32_steels = 50
    }
    upkeep = {
        energy = @b2_upkeep
    }
}
```

```

        d32_steels = 1
    }
}

prerequisites = {
    "tech_originiums_mining"
}

triggered_desc = {
    text = job_foundry_effect_desc
}

ai_weight = {
    weight = 0
    modifier = {
        weight = 10
        OR = {
            has_building = building_foundry_1
            has_building = building_foundry_2
            has_building = building_foundry_3
        }
        has_building = building_d32steels_smelter
    }
    modifier = {
        weight = 1
        free_building_slots = 0
    }
}
}

```

关于 Scripted Trigger 的进阶用法将在之后的章节进行讲解。

4.6.5 脚本变量(Scripted Variables)

十分简单的用法，在 common/scripted_variables 下定义，直接看香草文件：

```

# Add global variables here that will be available in all script files.
# @something = 42
# They can be overridden in each normal script file.

@discovery_weight = 3

```

在此定义一个全局常量（说是变量，但完全没法变啊！），采用@<VariableKey> = <int>进行定义一个 Scripted Variable，之后可以在需要填入数字的地方调用其即可，调用方式为

@<VariableKey>, 并可在任何通常的文件中重新赋值(在重新定义的文件当中生效)。

并无太大用途, 在 Technology 这类需要大量调用数值的文件中可以省去部分时间, 这种@声明的变量也可以在其他文件的开始部分进行声明, 不过在其他文件中声明就是局部变量, 只能够在本文当中调用:

```
@tier6cost1 = 30000
@tier6cost2 = 40000
@tier6cost3 = 50000

@tier7cost1 = 100000
@tier7cost2 = 125000
@tier7cost3 = 150000

@tier8cost1 = 250000
@tier8cost2 = 500000
@tier8cost3 = 750000

@tier9cost1 = 1000000
@tier9cost2 = 2000000
@tier9cost3 = 3000000

@tier10cost1 = 5000000
@tier10cost2 = 7500000
@tier10cost3 = 10000000
```

如上示例是在 common/technology 下文件开头中声明的, 因此其作用范围是该文件, 其他文件无法引用其中数值。

4.6.6 脚本值(Script Values)

该条目的更新较为靠后, 因此算是一个补充, 其作用几乎完全替代了上方介绍的 Scripted Variable (这俩不太好翻译, 由于脚本变量 Scripted Variable 使用的十分少, 因此之后的 SV 基本是都代指 Script Value), 其定义在 common/script_value 下方, 是实现动态修正值的必要条目, 接下来解析一下其用法以及高阶应用, 与 Scripted Effect 和 Scripted Trigger 不同, 其高阶应用相较于前两者更有应用价值。

4.6.6.1 Export 语句介绍

在讲解 Script Values 之前我们需要对另一个同样应用广泛的功能进行介绍，因为两者的使用是息息相关的，因此在这里有必要先介绍一下 Effect 当中的 Export 系列语法。

在较为早的 Stellaris 版本当中（2.8 之前），我们是无法直接获取游戏内的各个参数，例如我们无法直接获取国家所拥有的总人口，必须通过其他方式来获取，这样不仅十分麻烦，还对性能有一定影响，在某一次更新之后，蠢驴推出了一系列的语法来让 Modder 可以获取游戏内的各种参数，由于这类语法普遍以 Export 开头，因此我这里统称为 Export 语句（叫什么都无所谓，关键是教程需要一个称呼）。通过这类语句我们可以导出各种我们需要的参数，其语句为：

```
export_modifier_to_variable = {  
    modifier = x  
    variable = x  
}  
export_resource_income_to_variable = {  
    resource = alloys  
    variable = x  
}  
export_resource_stockpile_to_variable = {  
    resource = alloys  
    variable = x  
}  
export_trigger_value_to_variable = {  
    trigger = x  
    variable = x  
}
```

第一个为导出修正的值为变量，第二个为导出资源盈余为变量，第三个为导出资源储备为变量，第四个为导出条件的值为变量，为了方便介绍这四个语句，我们举一个例子来进行说明，该例子目标十分简单，导出关于资源收支的各类参数：

首先我们先来看一下 export_modifier_to_variable, 该语句用于导出当前 Scope 下指定修正语句的值，我们可以进行如下声明：

```
export_modifier_to_variable = {
    modifier = country_base_influence_produces_add
    variable = test_variable
}
```

其中 modifier 为我们指定的需要导出的修正，variable 则是需要将值导入的变量名，整个语句作用就是将 country_base_influence_produces_add 这个修正的值赋予 variable（注意这里相当于 set_variable，因此如果该变量不论之前是否声明或者有值，都相当于 set_variable 了一次），这样我们就得到了一个与值相同的变量，方便我们进行一些“动态”的操作，比方说下面这个例子：

```
export_modifier_to_variable = {
    modifier = country_base_influence_produces_add
    variable = test_variable
}
while = {
    count = test_variable
    capital_scope = {
        create_pop = {
            species = root
        }
    }
}
```

依据 country_base_influence_produces_add 的值来在首都创建同等数量的人口。值得一提的是，这种简单的功能并不一定需要大动干戈的先导出为变量再进行操作，如上的功能可以简化进行实现：

```
while = {
    count = modifier:country_base_influence_produces_add
    capital_scope = {
        create_pop = {
            species = root
        }
    }
}
```

通过语法 modifier:country_base_influence_produces_add 可以直接调用该修正的值作为数字参数，如果不需要进行值的修饰，可以直接采用此类简写，能够有效的减少冗杂的代码。

接下来我们介绍一下 `export_resource_income_to_variable` 与 `export_resource_stockpile_to_variable`，这两个语法并没什么值得多说的，前者导出对应资源的净收支，后者导出资源的存量，而最后的 `export_trigger_value_to_variable` 则需要具体一提了。由于条件（Condition）的语句十分多，因此该语句应用理论上是十分广泛，并且用法相较于上方所示更复杂一些，我们之后会说明这点，这里先从较为简单的地方进行说明。

我们尝试导出一下整个国家的人口，可以通过如下的写法来进行操作：

```
export_trigger_value_to_variable = {
  trigger = num_pops
  variable = test_variable
}
```

`num_pops` 是用于判断人口数量的 trigger，这里的语句也十分好理解，将 `num_pops` 的值赋予 `test_variable`，据此我们写一个较为简单的例子，比如将当前首都人口翻倍：

```
capital_scope = {
  export_trigger_value_to_variable = {
    trigger = num_pops
    variable = test_variable
  }
  while = {
    count = test_variable
    create_pop = {
      species = root
    }
  }
}
```

这里需要注意的细节是对 `scope` 的处理，trigger 的应用是有 `scope` 限制的，而像 `num_pops` 这种语句则可以同时作用于 `country` 与 `planet`，而且判断的值也都不一样，而这个特点也同样作用于 `export` 语句，包括上方的修正导出语句。因此示例将 `export` 语句放在了 `planet`，以便匹配变量及接下来的操作。

如上依照导出修正的示例，我们也可以对这种简单的导出值进行简化：

```
capital_scope = {
```

```

while = {
    count = trigger:num_pops
    create_pop = {
        species = root
    }
}
}

```

与 modifier:<modifier_key>语法相似，简单 trigger 也可以通过 trigger:<trigger_key>进行值的直接调用，以便简写。

接下来我们深入了解一下该语句，这里我们通过资源类型的判断语句举例，首先需要了解一下资源存量的判断语句有哪些，除去上方有专门导出净收支以及存量的语句，我们可以从 trigger 表当中找到资源相关判断的语句：

```

resource_expenses_compare = {
    resource = alloys
    value = x
}
resource_revenue_compare = {
    resource = alloys
    value = x
}
resource_income_to_expenditure_balance_ratio = {
    resource = alloys
    value = x
}
resource_income_compare = {
    resource = alloys
    value = x
}
resource_stockpile_compare = {
    resource = alloys
    value = x
}

```

除去一些不太常用得到的资源判断语句，一般的判断语句如上所示，其中我们可以看到下方两个判断语句 resource_income_compare 与 resource_stockpile_compare，我们先来讲述一下如何用 export_trigger_value_to_variable 实现 export_resource_income_to_variable 以及

export_resource_stockpile_to_variable 的功能：

```
export_trigger_value_to_variable = {
    trigger = resource_income_compare
    parameters = {
        resource = alloys
    }
    variable = test_variable
}
export_trigger_value_to_variable = {
    trigger = resource_stockpile_compare
    parameters = {
        resource = alloys
    }
    variable = test_variable
}
```

我们可以从上方的 trigger 语句当中注意到，各类资源比较的语句都不是简单的一句话就结束了，他们都有自己的字段进行筛选判断，而 export_trigger_value_to_variable 语句也提供了 parameters 属性，使导出复杂条件成为可能，如上示例，我们已知 export_resource_income_to_variable 的属性有 resource 用于选择资源类型，value 用于判断值，而显然 resource 是条件筛选所需的，因此我们需要在 export_trigger_value_to_variable 的 parameters 参数当中添加这个条件来同步需要导出值的条件，虽然 value 也是该条件的属性，但是其涉及的本身就是值的比较，同时 export_trigger_value_to_variable 本身就是需要导出所需的值，因此在这里算作是无关属性，无需对其进行声明。

同样我们可以通过这样的方式对其他几个条件进行值的导出，格式基本一致：

```
export_trigger_value_to_variable = {
    trigger = resource_expenses_compare
    parameters = {
        resource = alloys
    }
    variable = test_variable
}
export_trigger_value_to_variable = {
    trigger = resource_revenue_compare
```

```

    parameters = {
        resource = alloys
    }
    variable = test_variable
}
export_trigger_value_to_variable = {
    trigger = resource_income_to_expenditure_balance_ratio
    parameters = {
        resource = alloys
    }
    variable = test_variable
}

```

第一个为导出该资源的总开销，第二个为导出该资源的总收入，第三个为导出该资源的收支比率。

做一下简单的总结，假设我们合金收入为 80，支出 100，存量为 50 我们通过下方五个资源导出相关的

语句可以得到不同的值，各位读者可以自行判断一下：

```

export_trigger_value_to_variable = {
    trigger = resource_expenses_compare
    parameters = {
        resource = alloys
    }
    variable = test_variable
}
export_trigger_value_to_variable = {
    trigger = resource_revenue_compare
    parameters = {
        resource = alloys
    }
    variable = test_variable
}
export_trigger_value_to_variable = {
    trigger = resource_income_to_expenditure_balance_ratio
    parameters = {
        resource = alloys
    }
    variable = test_variable
}
export_trigger_value_to_variable = {
    trigger = resource_income_compare
    parameters = {

```

```

        resource = alloys
    }
    variable = test_variable
}
export_trigger_value_to_variable = {
    trigger = resource_stockpile_compare
    parameters = {
        resource = alloys
    }
    variable = test_variable
}

```

一、100

二、80

三、0.8

四、-20

五、50

这便是 export_trigger_value_to_variable 的应用方式，如果需要导出更为复杂的条件值，我这里也可以

举一个例子，我们尝试导出下方 trigger 的值（无意义条件，单纯为了例子而写的例子）：

```

calc_true_if = {
    amount = 2
    num_pops > 6
    if = {
        limit = {
            has_technology = tech_akx_worm_1
        }
        has_technology = tech_alloys_2
    }
    else = {
        has_technology = tech_alloys_1
    }
    check_variable = {
        which = test_variable
        value = 6
    }
}

```

导出格式为：

```
export_trigger_value_to_variable = {  
  trigger = calc_true_if  
  parameters = {  
    num_pops > 6  
    if = {  
      limit = {  
        has_technology = tech_akx_worm_1  
      }  
      has_technology = tech_alloys_2  
    }  
    else = {  
      has_technology = tech_alloys_1  
    }  
    check_variable = {  
      which = test_variable  
      value = 6  
    }  
  }  
  variable = test_variable  
}
```

基本规则是，数量比较去除，其他属性都写入 parameters 内部即可。

4.6.6.2 脚本值（Script Values）的简单应用

作为一个较后出现的功能性条目，Script Values 的出现补充了 Stellaris 长期以来匮乏的真正能够“变动”的值，因此其应用范围一般是在各类需要动态变化的修正值当中。我们先拿原版的一个简单例子进行讲解：

在帝国规模大改的版本，法令开销也进行了调整，其开销会随着帝国规模超过 100 而逐步上升，其实现方式并不是内部硬代码操作，而是通过使用 Scripted Values 进行动态变化的，这里我们先来看看原版的一个法令代码：

位置: common/edicts/00_edicts.txt

键值: fortify_the_border

中文本地化: 加强边境防御

```
fortify_the_border = {
  length = @EdictPerpetual
  icon = "GFX_edict_type_policy"

  resources = {
    category = edicts
    upkeep = {
      unity = @Edict1Cost
      multiplier = value:edict_size_effect
    }
  }

  modifier = {
    starbase_upgrade_speed_mult = 0.50
    country_starbase_capacity_add = 2
  }

  ai_weight = {
    weight = 0
    modifier = {
      factor = 0
      NOT = { has_country_flag = has_encountered_other_empire }
    }
  }
}
```

该法令内容十分简单，我们直接来看比较特别的一个点，resources 下方 upkeep 属性的字段 multiplier，一般来说该值应该是一个数字亦或是上方介绍过的 Scripted Variable，而这里采用的则是 value:edict_size_effect，其中 value 代表的就是调用 Script Values 中声明的条目，其声明形式十分像我们上方所介绍过的 modifier:<modifier_key>和 trigger:<trigger_key>，相似的，其冒号后方则是条目的键值，不过不同于前两者，该示例当中其调用的值的位置位于 common/script_values/00_script_values.txt 内的 edict_size_effect:

```
edict_size_effect = {
    base = 0.01
    mult = trigger:empire_size
    add = 1
}
```

相对于 Scripted Variable 的静态值，Script Values 声明的值则可以是随着某些因素而变化，这里我们解析下这段的意义。

base 表示其基准值，这里为 0.01；mult 为将上方计算的值乘以某个数值需要注意的是由于蠢驴的计算为纯线性，从上到下依次执行，因此并不遵循一般的四则运算法则。trigger:empire_size 的格式则在上方讲述 export 语法之时已经介绍过了，表示调用国家规模的值；最后 add 则是在上方计算的值的基础上再加 1，总计算公式为 $[0.01 * \text{帝国规模} + 1]$ 。

4.6.6.3 脚本值（Script Values）属性解释

尽管文档给予了较为详细的属性说明，但为了方便，这里还是将其基本属性全部展示出来以供参考：

```
example_value = {
    base = 10                #设定基础值，默认为 0
    set = 10                 #将值设定为指定数值
    weight = 10              #同 set
    add = 10                 #加算指定数值
    subtract = 10            #减去指定数值
    factor = 10              #乘上指定数值
    mult = 10                #同 factor
    multiply = 10            #同 factor
    #一般情况下推荐使用 multiply，其他两者皆是历史遗留但并未做删除
    #factor, weight 都会覆写 base 值，因此推荐采用 set, mult 或者 multiply 替代
    divide = 10              #除以指定数值
    modulo = 10              #取模
    round_to = 10            #在选定值附近就近取整，e.g. 值 10.7 round_to = 5 将会返回 10
    max = 10                 #限定最大值
    min = 10                 #限定最小值
    pow = 10                 #次方计算，e.g. 值 2 pow = 3 将返回  $2^3 = 8$ 
```



```

round = yes           #就近取整数
ceiling = yes         #向上取整
floor = yes           #向下取整
abs = yes             #取绝对值
square = yes          #取平方值

modifier = {          #通过 modifier 属性可以有条件的进行计算
    add = 10           #必须满足以下条件才进行计算
    owner = {         #只有设置了 max_example_variable 这个变量才执行上述 add
        is_variable_set = max_example_variable
    }
}
multiply = value:some_other_script_value #可以调用其他 script value

complex_trigger_modifier = {    #导出复杂 trigger 的值以供计算使用
    trigger = count_owned_planet #trigger
    trigger_scope = owner       #可以使得你从不同 scope 的 trigger 获取值，
默认为 this
    parameters = {             #同 export_trigger_value_to_variable 语法
的 parameters 使用方法
        limit = { num_pops > 10 }
    }
    mode = add                 #调整导出值之后与基础值的计算方式
    mult = 5                   #将导出的值进行一定运算
}
}

```

总的来说，script_values 的用处十分之大，其可以广泛应用在普通的 common 条目达到动态调整各种之前静态的产出或是维护费，对一些较为复杂的产业系统制作提供了十分便捷的实现方式，不过唯一的问题可能是其带来的性能优化问题，不建议写过多复杂条件判断。

五、Event Modding 进阶

5.1 绪论

这篇教程面向那些较为熟悉群星 mod 制作方法和工具（例如对 common 的大部分文件都有了一定了解），但苦于不熟悉蠢驴事件的结构如何满足自己构想的 modder 们，~~所以完全不知道啥是事件啥是 modifier 啥是 AND OR NOT 的可以出门左转子。~~

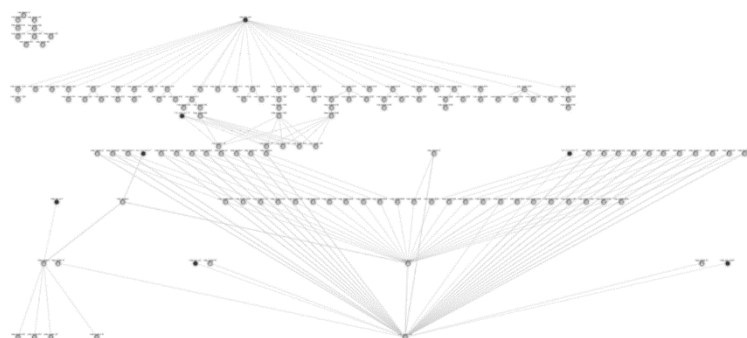
当然如果你满足于修改武器、科技数值，图标和本地化，没有事件写作需求的话，也可以跳过这个教程了，毕竟前面将会是一堆事件代码示例，乱七八糟的效果解析，可能会伤害到不少萌新……但对于有能力在群星 mod 制作中更进一步的同志，我还是十分希望这个教程能提供一些帮助的。



图一「难得有这分才，为何不去磨练精进！」



图二「长度只有 40 行的代码，已经厌倦了吧？」



```
2 > country_event = { ...
1183 }
```

图三 一些简单的事件链，和一些行数较短的事件（）

咳咳，贴这几张图是不是显得我跟赶人走一样.....算了不管了（

我们的进阶教程大部分情况下还是不会碰到*单个*事件上千行的，也不会出现像这个事件链图片一样复杂的设计.....大概。

好啦言归正传，让我们看看绪论要讨论的主要问题：

这个教程究竟是在搞什么？

作为进阶教程，我在这里的主要目的是介绍一些常用的事件写法以及思路，给 modder 们**打通思路**，想明白如何利用蠢驴提供的有限代码达到自己的目的，换句话说，戴着镣铐起舞（笑）。

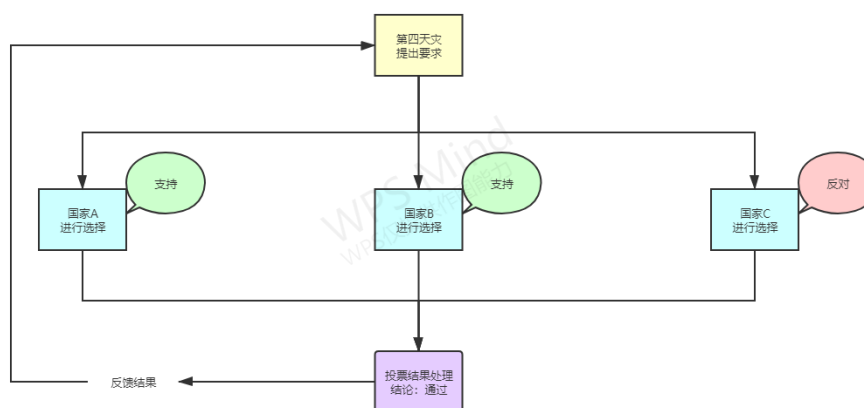
如何理解这个目的呢？我在这里可以举几个简单的事件例子给你们体会下（）

范例 0-1 简单且无聊的投票事件



图四 投票系统，尽管大家好像都喜欢一言堂

举几个小例子，如果你想搞点小弟跟你一起投票开会的排面活，你就需要一系列在各个国家传递的事件，就像这样：



图五 要是能反馈 C 国的名字就好了

很简单的流程图，通过**玩家国家**启动开关，触发 A、B、C 三个国家的事件，A、B、C 三个国家选择后，再将投票数据通过另一个事件反馈到**玩家国家**即可。

我们决定写三个事件：投票触发（以下简称事件α）、投票选择（以下简称事件β）、投票结果总结（以下简称事件γ）

但是这里出现了几个问题：

- 如何在 A、B、C 三个国家的事件β中找到事件α的触发者，以得到文本引用国家名字，以及反馈到正确的目标，使**玩家国家**准时触发事件γ的效果？
- 如何计算投票支持票和反对票数量，以得到法案是否通过的结果？
- 如何在不同的投票结果下同一事件（即事件γ）显示不同文本？

- 如果有投票者是玩家，选择拖延时间，不进行投票（即“断无此疏”），如何强迫其投票？

这样或那样的问题，就会开始出现，无疑对于萌新是一个巨大打击（尽管我敢说没有萌新会尝试写这种东西，写了也不会考虑这些情况）。这个教程的主要目的，就是介绍这些问题的解决思路，帮助进阶 modder 更好的进行事件写作。

如上面这些问题所示，就**技术**层面，这篇教程的主要内容是关于事件的各种小技巧和操作实例。例如 **scope 语句** 的使用详解，UI 的制作，**变量**和 **flag** 的使用，各种**封装效果**的使用方法等。

而事件制作更重要的，则是**思路**。在各个技巧与实例之外，我也会谈谈设计事件的思路方向，如何从自己的能力出发，想出可行的 mod 文案，如何把脑内点子转换为可行的“**蠢驴思路**”，以及如何更快更好的提升 modding 技术。

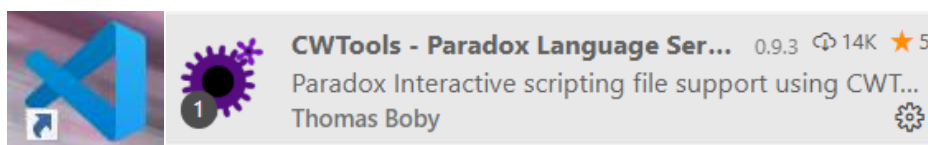
以上一堆废话说完完了，我们就可以开始教程的第一篇了，正式深入 events 这一个 stellaris mod 核心，芜湖~



图六 游戏开始

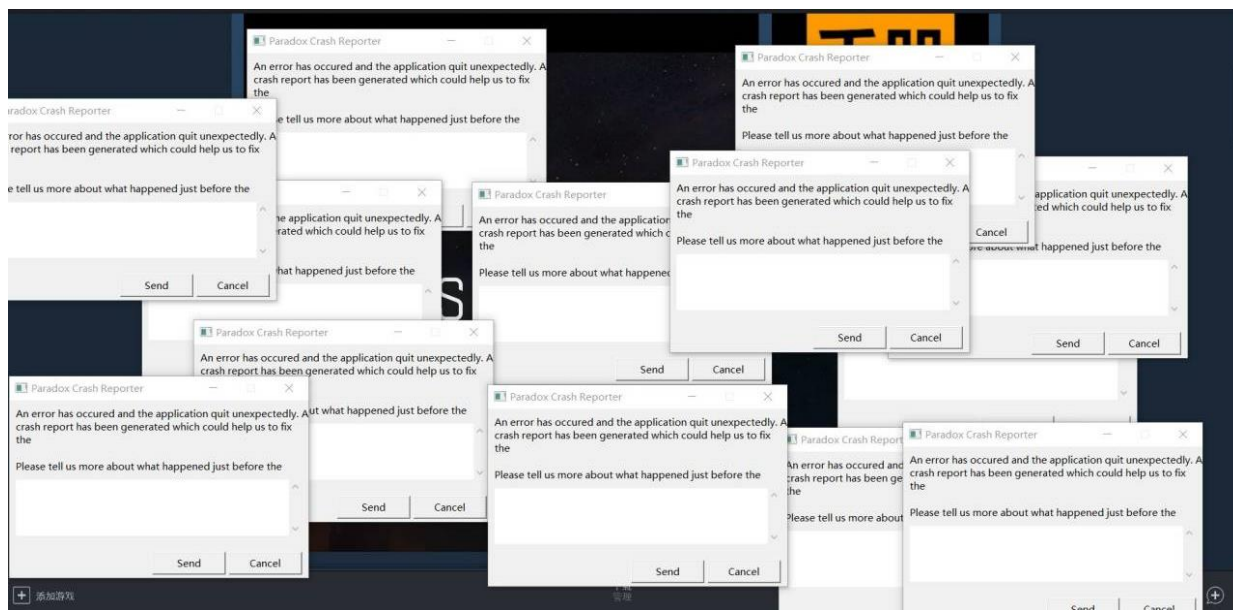
等一下

在上车前，我要先确认一下，各位都掌握了 VSC（也就是 Visual Studio Code）以及插件 cw tools 的用法了吗？



图七 记住这两个货

没有的话.....还不快去看隔壁无言的事件教程（见第四章）（笑），事件编辑要是没有这种工具的协助可是会花式导致游戏暴毙。



图八 体会一下，嗯

顺带来宣传一下 modder 交流群，有具体代码问题都可以来问（）



在这篇进阶教程的第一章，我们将讨论一下蠢驴 event 的基本架构，并回答一个重要的问题：

5.2 认识你与 event 的极限

事件究竟能做什么？

事件，events 文件夹里那些成堆的 txt 文件，究竟能改变这个游戏到何种程度？

这个问题，我曾经以为不会被接触事件不久的萌新遇到，但我发现，如果避开这个问题……就会使很多 modder 在事件写作中产生了错误的判断，做很多无用功，或者在群里引来高血压。

所以在进一步接触事件写作之前，让我们还是摆正心态，了解一下蠢驴游戏的半壁江山（以上）——events 的基本结构，以及它能够为我们带来的东西

5.2.1 我是谁，我在哪 —— scope

scope，也就是范围，相信在无言的教程中大家已经有了一定的接触。但对 scope 这个概念，恐怕各位还是没有有一个比较直观的印象。

scope 的概念是蠢驴语言的一个核心，如果对其没有一个清晰的印象，modder 是无法写出较为复杂的事件的。

那么，如何理解 scope 在蠢驴事件中的作用呢？

大家日常的对话通常包含两个重要部分：主语和谓语。例如萌新 A 写码，就分为萌新 A 这个主语，以及写码这个谓语。蠢驴的绝大多数效果语句也是相同的逻辑。

类比萌新 A 写码这句话，我们不妨看一下下面这个事件。

范例 1-1 天灾？AWSL！

由隔壁舰 R mod 中某联合舰队掀起的天灾热，以相当快的速度席卷了整个 modder 圈，很多小 mod 制作者想要制作这样一个天灾以扩大订阅量。

然后我们的主人公——某个萌新 modder 也想要写一个强力无比的天灾，他想到：“只要把玩家灭国，天灾就无敌了！”最后写下了这个“天灾”。

```

1 country_event = { #如你所见，这是一个*国家*事件，也就是这个事件的“ROOT”scope为国家。
    id = xskipper_teach.0 #一如既往的事件id
    title = "天灾!" #事件标题，这里偷懒了，采用了引号直接写的方法.....萌新不要学。
    desc = "哈哈哈哈哈，我是天灾!" #萌新误学
    is_triggered_only = yes #这个事件只能由触发器触发，例如effect、on_action文件夹中的触发器等
    trigger = { #事件的触发条件，但控制台触发事件时会无视这些条件强行触发。
        2 days_passed > 15 #这是这个事件唯一的condition语句：这个事件只有在距离开局已经15天以上，才能合法触发。
    }
    option = { #这是事件的唯一选项
        name = "是天灾！AWSL!" #AWSL
        3 destroy_country = yes #这是这个事件唯一的effect语句：消灭作为触发者的国家
    }
}

```

图一 范例 1-1 代码雏形

很简单的一个事件：国家 A 触发该事件，国家 A 选择选项，国家 A 暴毙。wdnmd 毫无游戏体验



图二 范例 1-1 不讲武德

相信经历了无言的教程，大家都明白了 trigger、immediate、option 的作用.....不过我还是在这里稍微复习一下：

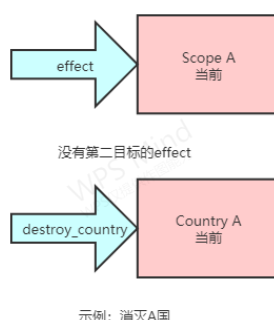
- **id**：事件的名字，调用时就靠它，命名规则就不说了。
- **title&desc**：事件的标题&描述，通常是本地化 key，游戏内显示对应语言对应 key 的本地化文本。
- **is_triggered_only**：仅由触发器触发，若这个属性为 yes，除非有其他的東西（例如另一个事件，或者控制台语句）去触发它，你是不会在游戏中看见它的。
- **trigger**：触发条件，如果 trigger 内的条件不满足，这个事件怎么都不会触发的（控制台除外），内部填充 condition。
- **immediate**：即时效果，在事件触发瞬间就生效的效果，内部填充 effect。
- **option**：事件选项，name 属性为选项的名字（本地化 key），除了部分属性语句，内部填充的也是 effect，与 immediate 的不同在于其在选项被选择后才生效。

在这里先介绍这些，毕竟只是复习（笑），而且这些属性定义永远是实操比看教程更容易懂。

类比萌新 A 写码，我们可以说这个事件执行的主要效果就是国家 A 暴毙。事件代码中的② destroy_country = yes 是一个 effect（接下来会提到），代表着暴毙这个动作。那么，如何确定暴毙的国家是国家 A，为什么这个事件里暴毙的不是国家 B，国家 C？

答案藏在①country_event 中。这代表这个事件是一个**国家事件**，这个事件的根 scope(ROOT) 就是看到这个事件弹窗的，触发这个事件的**国家 A**。在没有经历任何 **scope 跳跃** 时，所有该事件的判断和效果都关于**国家 A** 本身。

②destroy_country = yes 这一语句没有经历 **scope 跳跃**，直接在根 scope (ROOT) 执行，因此暴毙的就是**国家 A** 了。



图三 只有一个目标的 effect

由此说来，**scope** 可以比喻成一种**主语**，它代表着所有效果的目标，有了 **scope**，准确地在 2200 上天的才会是你（而不是住你隔壁的蟑螂兄弟），圆神才能准确地清算你的每家每户（而不是误杀了对面的海星）。

当然这显然不是 **scope** 的全部，我还并没有说明如何进行 **scope 跳跃**，以及很多关于 **scope** 的，别的操作。但作为一个概念理解，绝大部分 **scope** 的相关内容都是由我上边的话延伸。我们可以先暂停在此处，更详细的问题留到下一章进行说明。

5.2.2 老三样——effect、condition 以及 modifier

说到蠢驴的代码，最多的就是这三个东西，在自定义程度较高的代码（不止事件！），几乎都能找到这老三样。

下面我借用**范例 1-1**，简要介绍一下老三样的前两个：**effect** 与 **condition** 的作用。

回到这个“天灾”事件，大家可以看到我做的后两个标记：

②`days_passed > 15`

这是一个标准的 **condition** 语句，判断的是【距离 2200 年 1 月 1 日的日期】，只有当经过日期大于 15 天时，才会返回 **true**。

③`destroy_country = yes`

这是一个标准的 **effect** 语句，执行的效果为【摧毁当前 scope 的国家】（scope 会在之后详细介绍，在该事件中被摧毁的就是事件触发者）

很基础的两条语句，相信大家都能看英文字面意思理解它们（真不能的话你可以机翻）。在这里我可以给出一个 **effect** 与 **condition** 的定义，希望各位不要将两者混淆。

Effect	Condition
执行的效果，例如【摧毁国家】、【增加资源】、【消灭人口】等，它们都是实际执行的效果，改变了这一局游戏的数据。 例如国家 A 暴毙，如果说国家 A 是当前的 scope，那么暴毙就是一个 effect。<code>destroy_country = yes</code> 在这里相当于表动作的谓语。	判断的条件，例如【检查日期】、【检测人口数量】、【判断国家思潮】等，它们是判断的条件，本身不会改变这局游戏的内容。 例如国家 A 有唯物主义，你可以通过代码 <code>is_materialist = yes</code> 进行表达，这句话不会将 A 变成唯物主义，只是进行判断罢了。主语是国家 A，<code>is_materialist = yes</code> 是有唯物主义的 condition，可以说是表状态的谓语。

effect 与 **condition** 都有各自独立的一套代码，该填 **effect** 的地方不要塞 **condition**，反之亦然。倘若你填错了，VSC 往往会警告你（只要你正确使用了插件）：

```
any_owned_planet is unexpected in immediate CW263(CW263)
```

图四 unexpected 就是蠢驴和我都不希望你填的意思

那么如何判断这里应该填 **effect** 还是 **condition**，或者你手上这条代码是哪种呢？

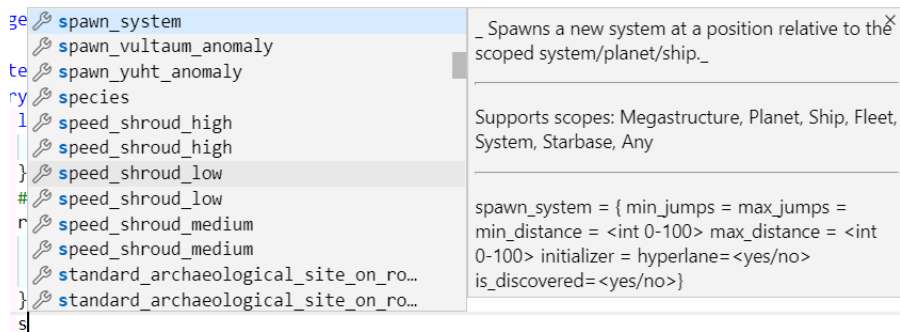
这里提供四种方法：

1. **查表**：你可以在 https://stellaris.paradoxwikis.com/Effects#Scripted_Effects 查询到绝大多数的 **effect** 代码，在 <https://stellaris.paradoxwikis.com/Conditions> 查询到 **condition** 们。只要和已有代码比对，你就能轻松发现这是哪一种。

这是最蠢的办法，正经人谁翻效果表啊（

2. **依靠 VSC 的自动补全和报错功能**：看到图 4 了吗？插件都告诉你不能在这填了，你就别填这类代码了。

一般都用这种法子，自动补全配合一定的英文基础，大部分的代码都能毫不费力的被你找到。



图五 cwtool 真的很方便

3. **找到两者常用的英语词汇**：正如我之前的比喻，蠢驴的 **effect**、**condition** 格式都可以总结为主语和谓语，只是存在表动作和表状态的差异。

那我们不妨像做英语语法题时一样，根据代码的字面含义判断其种类，就想这样：

<1>表动作的单词：**destroy**、**set**、**remove**、**spawn**、**add** 等等——>**effect**

<2>表状态的单词：**has**、**is**、**can** 等等——>**condition**

<3>特殊情况：check、count 这类表示**检查**的动作词语，是 condition。

只要看懂了这些词的大致含义，就能了解这是一个**动作**还是**状态**，进而就能分清代码属于 effect 还是 condition。

4. **找到两者的常见代码**：有些代码的使用频率是远远高于其他的，当你看到一坨代码不知道咋改时，可以通过寻找它们来快速进行判断。

<1>很常见的逻辑判断语句

AND、OR、NOT、NOR、NAND，这些逻辑门的含义相信大家比较熟悉（不熟悉就去百度），它们都是 condition、里边包着的是，和它们并列的也是。

<2>很常见的流程控制语句

if、else、else_if、while、switch.....这些流程控制语句也在蠢驴语中占有一席之地。它们都是 effect、与它们并列的都是，里面包含的（除了 limit）也都是。

在 if、else_if、while 以及很多**遍历型 scope 跳跃语句**中，你都可以在内部看到 limit 语句，这里面需要填 condition（**limit 本身不能单独存在！**），具体含义分情况，例如 if 中的 limit 代表 if 判定的条件，**遍历型 scope 跳跃语句**中的 limit 代表目标必须符合的条件（之后还会提到）。

<3>很常见的**遍历型 scope 跳跃语句**

遍历型 scope 跳跃语句，在之后我会进行进一步说明，但你依然可以靠它们的前缀来辨认种类。

前缀为 every、random 的，例如 every_owned_planet、random_owned_pop 等代码，都是 effect 中的**遍历型 scope 跳跃语句**。

前缀为 any 的，例如 any_owned_planet、any_owned_pop 等代码，都是 condition 中的**遍历型 scope 跳跃语句**。

any 不能出现在 effect 中，反之亦然，不要想当然换着填，没有用的。

有点晕了？看看这个总结表格吧。

代码种类	定义	常见单词	遍历型 scope 跳跃	特殊代码
effect	执行效果	表动作：set、remove、 add 等	前缀为 every 或 random	if、while、switch 等
condition	判断条件	表状态：is、can、has 等 表检查： count、check	前缀为 any	AND、NOT、OR、 NOR、AND
modifier	修正数值	add、mult、reduce	无	无

你可以发现我并没有在前面提到 modifier 们，尽管它出现在标题中。因为你是无法直接在事件代码中使用 modifier 代码的。它们相当于附加在各种东西上的 buff 与 debuff，要想通过事件添加，必须自行创建一组新的静态修正（之后可能提到）。

在这一小节的最后，我再次强调一遍：effect 与 condition 都有各自独立的一套代码，该填 effect 的地方不要塞 condition，该填 condition 的地方不要塞 effect！

5.2.3 如何讲蠢驴语——事件代码构思思路

在接触了 scope、effect 与 condition 之后，让我们再回首看一下我在第一节做出的比喻。

scope 相当于主语，表明要做这个动作的人；effect 与 condition 相当于谓语，表明这个人要做什么动作，或者要满足怎样的条件。将这两部分混合起来，就是一句正常的蠢驴语。

因此我们写作事件时，也要按照蠢驴语的表述方式进行构思。

让我们看看下面这个范例，想一想我们应该怎样构思事件。

范例 1-2 烂大街的自动人口迁移（其一）

尽管创意工坊的自动人口迁移 mod 已经相当多且非常完善了，仍然有很多萌新将其作为事件写作的第一个课题。

.....但这个系统，也并没有他们想象的那么简单。

好，现在让我们想想人口迁移怎么写。

作为第一章的一个范例，我们只要求把失业的人口处理掉。（你也看到了，这是其一，迁移的问题咱们目前可解决不了）

让我们先把要求转化为蠢驴事件的表达方式，我们可以得到：

①人口失业；

②人口被鲨；

我们可以认定这两个表述的 scope 都是 pop（人口），其中①包含 condition “失业”，②包含 effect “消灭人口”。

那么我们写这个事件，就需要首先找到目标人口 scope，再对其进行处理。

那么让我们先完成第一步：

```

country_event = {
    id = xskipper_teach.1
    hide_window = yes    #不显示事件窗口，直接执行效果

    is_triggered_only = yes

    immediate = {
        every_owned_pop = { #遍历型scope跳转语句：跳转scope到—>我们拥有的全部人口
            if = { #如果这个人口失业
                limit = {
                    is_unemployed = yes #判断当前scope（此时指该pop）是否失业
                }
                #处理
            }
        }
    }
}

```

图六 范例 1-2: 找到目标 (1)

我们利用**遍历型 scope 跳转语句**，把当前 **scope** 转换为触发国家拥有的全部人口，并使用 **if** 语句进行判断：如果那个人口失业的话，我们就可以动手了。

当然，我们可以先简化一下代码：

```

country_event = {
    id = xskipper_teach.1
    hide_window = yes    #不显示事件窗口，直接执行效果

    is_triggered_only = yes

    immediate = {
        every_owned_pop = { #遍历型scope跳转语句：跳转scope到—>我们拥有的全部人口
            limit = {
                is_unemployed = yes #判断当前scope（此时指该pop）是否失业
            }
            #处理
        }
    }
}

```

图七 范例 1-2: 找到目标 (2)

这里利用含 **limit** 的**遍历型 scope 跳转语句**来获取失业中的人口，可以让代码量短一些（这些处理之后还会详细解释，总之我们找到了帝国的所有失业人口。）

然后是进行处理，我们现在能够鲨掉这个人口：

```

country_event = {
  id = xskipper_teach.1
  hide_window = yes #不显示事件窗口，直接执行效果

  is_triggered_only = yes

  immediate = {
    every_owned_pop = { #遍历型scope跳转语句：跳转scope到—>我们拥有的全部人口
      limit = {
        is_unemployed = yes #判断当前scope（此时指该pop）是否失业
      }
      kill_pop = yes #杀掉人口的effect
    }
  }
}

```

图八 范例 1-2: 鲨了目标

Kill_pop = yes 能够直接把当前 scope 代表的人口消灭，而写下这个 effect 后，我们第一次的“自动迁移事件”事件写作就结束了。

这个事件能够人道毁灭你的所有失业人口，起码能够让你的岗位界面清爽一些。~~（说好的迁移呢）~~

从这个事件的写作历程可以看出，我们的构思与码字是按照这样的逻辑进行：

1. 把你的需求，翻译成一条条蠢驴语表述的句子。
2. 找到每个句子对应的主语，尝试使用事件寻找 scope。
3. 找到 scope 后，对其进行处理。

这是目前我们由上面已经学习过的内容总结出来的事件写作经验。当你想到了一个想用事件完成的新点子时，就要按照这样的思路把你的需求梳理一下。

实际上，我们在写作事件时，可以把它拆成三种需求：

1. 何时何地，触发该事件？
2. 该事件的目标，包含哪些？
3. 我们要对这些目标进行怎样的处置？

在上面的构思逻辑，我们回答了后两个问题，至于第一个，就涉及到 on_action 等触发器的介

绍，我们之后再展开讲。

5.2.4 你与事件的极限——mod 做不到的事

我们已经讲完了写作一个事件需要的思路，代码分类以及概念，现在就可以开工实现那些只改 `common` 无法实现的梦想了。

不过在此之前，我还是需要在最后谈一谈，如何做出一个合理的构思，最起码，可行的构思。

1. 事件能不能实现这个？

这是你的构思能否实现的最大物理限制，如果蠢驴提供给你的代码无法实现你想要实现的功能，或者你希望调整的东西是硬代码.....

尽快放弃！不要被这种陷阱困住了！（或选择前往瑞典闪击蠢驴，强迫蠢驴放出代码）

尤其是对于萌新，倘若跟一个无法实现的功能死磕，写了一堆无用无效的代码，不论对接受提问的群友，还是对萌新自己都有害无益。如果萌新自己依旧一意孤行的话，也只能是自己受到最大的伤害。

如何判断这个东西能否完成，你可以通过我对需求的分类，进行思考：

1. 何时何地，触发该事件？——>有没有这样的触发机制？
2. 该事件的目标，包含哪些？——>有没有办法找到目标？
3. 我们要对这些目标进行怎样的处置？——>是否存在这样的 effect？

例如你想做一个“创造十年停战期”的事件，我可以告诉你，没有这样的 effect。

没有 effect，你就有极大概率（在你不理解游戏机制时，就是 1000%）写出这个事件，其他的判断要素也是这个道理。

2.我能不能做到这个？

第二个要素，就是 modder 自身的能力。

在不熟悉机制的情况下，尽量不要尝试去处理过于复杂的事件。

这句话是对那些抄代码的同学说的，当你需求的功能很巧的被包含在某个原版，或者其他 mod 的事件时，**抄**是一个很好的主意。

但，很多时候，有些东西不是你单纯抄了改 key 就能解决的。其中涉及到的封装效果，变量处理，特殊属性很可能大大超出了你的处理能力，进而就会出现你一更改改“炸”了，或者他行你不行的情况。

抄代码的技巧我之后可能会再讲，我还是希望各位能够在抄一段代码时知道自己在抄什么的，毕竟，这大概就是这篇教程的存在价值之一。

5.2.5 小结

好啦，负能量发言结束了（）

这一篇教程中，我们了解了事件的几个基本要素，以及基本的写作思路。下面我们会像正经教科书一样进行一些总结。

1. 事件效果描述可以按照**主语+谓语**的格式，**scope** 是主语，**effect** 和 **condition** 是谓语。
2. **effect** 是执行的效果，**condition** 是判断的条件，两者分别有不同的代码，**不能搞混**。
3. 事件写作思路：**触发时机地点**——>**找到目标**——>**处理目标**。

记住这些内容，就可以开始写一些更复杂的事件了。

下一章，让我们更详细地探讨一下 **scope**，回答一下如何找到目标的问题（以及更加靠谱的人口迁移！）。

习题一

(1) 指出以下事件表达句式中的 **scope** 对应的部分。

- ① 领袖年龄大于 45
- ② 舰船属于军事舰船
- ③ 国家有唯物主义
- ④ 国家不是格式塔
- ⑤ 人口公民权利为居留权
- ⑥ 星球大小超过 20
- ⑦ 星系中存在多个宜居星球
- ⑧ 种族拥有灵能特质
- ⑨ 舰队存在领袖
- ⑩ 星球添加建筑：水培农场
- 11 恒星基地添加模块：船员宿舍
- 12 国家去除标记：strange_flag
- 13 联邦减少 300 经验
- 14 舰船变量 ship_variable+1
- 15 领袖被杀死
- 16 星系创建一个新航道

(2) 下面的代码中，哪些是 **effect**，哪些是 **condition**？

- ① set_country_flag
- ② establish_communications

③ create_country

④ has_ethic

⑤ if

⑥ is_moon

⑦ exist

⑧ add_building

⑨ OR

⑩ remove_ship

(3) 试着把下面的需求转换为**事件表达句式**。

① 消灭战力为 1 的舰队

② 为年龄大于 85 的领袖添加尊者特质

③ 所有有种族洁癖国策的国家获得 5000 合金

④ 所有拥有的殖民地获得 5 个人口，倘若此举使得殖民地人口超过 20，摧毁那个殖民地。

⑤ 目标国家失去 700 能量，500 食物。

⑥ 有灵能特质的领袖所有权将被转移到当前国家的宗主国。

(4) 试着找到这些效果对应的代码。

① 获得资源

② 检查资源

③ 检查资源月收入

④ 添加 modifier（修正）

⑤ 获得随机科研选项

- ⑥ 播放声音
- ⑦ 增加信任度
- ⑧ 检查人口数目
- ⑨ 持有文物
- ⑩ 检查海军容量

5.3 如何找到目标 Scope

写作事件时，我们都按照**触发时机地点——>找到目标——>处理目标**的思路进行分析，其中**找到目标**这一部分，往往就是指找到目标对应的 **scope**。这一章我们将聚焦于各种 **scope** 的**跳跃和调用语句**，让各位对 **scope** 的利用有一个比较好的理解，顺利地找到自己的理想对象（笑）。

5.3.1 对象转换——scope 跳跃与调用

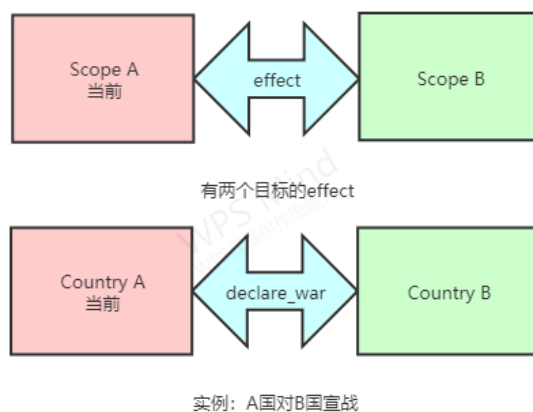
在蠢驴语言的结构中，**scope** 是表达的主语，相信大家已经有所体会了。大多数事件都有它默认的基本 **scope**（根/ROOT scope），例如 **country_event** 的基本 **scope** 便是触发者对应的国家。

但是很多效果，都需要 **scope** 的转换：比如人口的迁移需要寻找到一个国家的两个星球，外交事件需要两个国家之间的互动等等。这个时候就需要我们使用 **scope 跳跃语句**，对 **scope** 进行切换。

scope 跳跃语句，是蠢驴 **effect** 和 **condition** 里一类语句的统称。这类语句都用等号连接着一个{大括号}，在{大括号}内的语句，在下一次 **scope** 跳跃前，就是在**新的 scope** 中，使用了**新的主语**。

另一方面，有些 **effect** 或者 **condition** 拥有**复数**个目标，例如判断**两国**是否在交战状态，或者改善**两国**关系都需要**两个 scope** 的支持。其中一个 **scope** 可以作为**主语**，但**另一个 scope** 呢？只能在这种语句中充当**宾语**的定位。

这种时候我们需要在语句中调用出**另一个 scope**：使用 **scope 调用语句**，这一类语句可以指代一个 **scope**，它们可以是特定的**代码流程词汇**，也可以是你**事先保存的目标**。



图一 有两个目标 scope 的事件

现在让我们从最基础的遍历型开始，深入了解一下这些 **scope 语句**。

5.3.2 遍历，还是遍历——遍历型 scope 跳跃语句

我在上一章就提到了这一类遍历型 scope 跳跃语句，现在让我们就下面的实例来对它们进行探究。

范例 2-1 逃不过的清算

某个圆神信徒在完成了“清算后成功成为河长”的成就后，也兴致冲冲地加入了 mod 制作的行列。第一步就是要加大它生存的难度，搞出各式各样的圆神清算.....

```

country_event = { # 一个瞬间事件
    id = xskiper_teach.2
    hide_window = yes

    is_triggered_only = yes

    immediate = {
        every_owned_pop = { #找到 这个国家的 所有人口
            #以 目标人口 为当前SCOPE执行以下操作
            add_modifier = { #为 当前SCOPE 代表的人口添加目标修正
                modifier = pop_happiness_negative #修正的key
            }
        }
        every_owned_leader = { #找到 这个国家的 所有 领袖
            #以 目标领袖 为当前SCOPE执行以下操作
            set_age = 150 #设定年龄为 150
        }
        every_system_within_border = { #找到 这个国家的 所有 境内星系
            #以 目标星系 为当前SCOPE执行以下操作
            system_star = { #这个星系的 主要恒星
                remove_planet = yes #删除 当前SCOPE 代表的星球
            }
        }
        random_owned_megastructure = { #找到 这个国家的 随机一个 拥有的巨构
            #以 目标巨构 为当前SCOPE执行以下操作
            delete_megastructure = this #摧毁 当前SCOPE 代表的巨构
        }
        every_country = { #找到 所有 国家
            #以 目标国家 为当前SCOPE执行以下操作
            add_trust = { #增加当前SCOPE 代表的国家 对who对应的SCOPE 代表的国家 的信任度
                amount = -50 #增加值
                who = ROOT #目标为 ROOT 代表的国家
            }
        }
        random_owned_planet = { #找到 这个国家的 随机一个 拥有的星球
            #以 目标星球 为当前SCOPE执行以下操作
            solar_system = { #这个星球的 星系
                #以 目标星系 为当前SCOPE执行以下操作
                starbase = { #这个星系的 恒星基地
                    set_starbase_size = starbase_outpost #设定恒星基地的等级为 哨站
                }
            }
        }
    }
}

```

图二 范例 2-1 整体代码

这一片代码看起来好像比较复杂，但其实就是一串并列的效果组合在一起罢了。让我们把代码拆成一个个小的部分，看一看基础的一些 **scope 跳跃语句** 是如何使用的。

首先是**遍历型**。这一类 scope 跳跃语句以 **any**、**every**、**random** 开头，通常是对符合某一标准的群体，或是对这一群体中的随机一个进行处理。

```

every_owned_pop
every_owned_leader
random_owned_megastructure

```

图三 遍历型 scope 跳跃语句的例子

在 **effect** 区域内，这些**遍历型 scope 跳跃语句**以 **every** 或 **random** 打头，分别代表**所有**和**随机一个**。在 **condition** 区域内，这种语句则以 **any** 开头，代表**任何一个**的含义。

这些**遍历型 scope 跳跃语句**可以满足我们写 mod 中查询目标的绝大部分需求，下面由**范例 2-1**，举一些使用该语句进行事件制作的例子。

范例 2-1-1 领袖老化

我们的圆神信徒首先决定从新国家的领袖下手，让被清算国家在之后的每个月，领袖的年龄保持在 150 岁.....虽然没考虑寿命循环的要素，这样一个高龄也极大增加了领袖存活的难度。

首先我们需要用蠢驴语言翻译这个需求，也就是**每个月，该国家的全部领袖，设定年龄为 150**。目前不考虑触发时间的**每个月**，我们需要先找到这些领袖，在触发事件国家为 ROOT 的前提下。

```
every_owned_leader = {
```

图四 废话一堆，这代码会英文的都知道啥意思

这个**遍历型 scope 跳跃语句**将 scope 由 ROOT 对应的触发国家，转换到了 ROOT 所拥有的全部领袖。每个拥有的领袖都会触发大括号内的 effect，也就是：

```
set_age = 150    #设定年龄为 150
```

图五 设定年龄，神奇的是群星到现在都没有增加年龄的 effect

这样你就能够令 ROOT 的所有领袖一夜白头，前提是这些领袖的种族有头发。

范例 2-1-2 拆迁大队

接下来这位仁兄想惩罚一下那些在银河系中到处建造违章建筑的家伙，让那些被戴森球遮挡的恒星重见天日...他计划随机挑选一个触发国家的巨构，将其摧毁。

我们的新任务和**范例 2-1-1** 有什么区别呢？没错，数量。

我们不再需要选择全部目标，而是要**随机挑选一个**，这种时候 every 显然就破坏力太大了，轮到我们的 random 登场：

```
random_owned_megastructure
```

图六 触发国家拥有的随机一个巨构

然后很简单的 effect 系列：

```
delete_megastructure = this #摧毁 当前SCOPE 代表的巨构
```

图七 更加正确的说法是“删除”，毕竟这样的代码不会留下废墟

这两个最简单的 effect 带领我们对 **every** 和 **random** 有了一些了解，接下来，让我们看看 **any** 和 **limit** 的运用。

范例 2-1-3 众叛亲离

为了更进一步地迫害玩家，这个 modder 决定从国家间的关系下手，全部的国家……不，应该把条件设的复杂一点，所有拥有拥有“灵能”特质人口的国家，对触发国家的信任减少，怎样？

好，现在难度上升了，我们剖析一下需求。

- 找到目标 A：所有国家，条件为[拥有 拥有灵能特质的人口]
- 执行 effect，effect 的目标有两个，一个为触发国家，一个是目标 A。

让我们一个个实现，首先是找到目标：既然目标还是所有国家，先写一个这个：

```
every_country = { #找到 所有 国家
```

图八 找到 他们

但我们现在要找的并不是毫无限制，我们需要符合条件的国家，这时候我们就需要使用 **limit** 了：

```

every_country = {    #找到 所有 国家
    limit = {
        #要找到的国家需要满足以下condition
    }
}

```

图八 limit，遍历型的过滤器

通过加入 **limit**，并在 **limit** 内部填入 **condition**，我们可以过滤我们需要选择的目标，只让符合条件的国家执行接下来的 **effect**。

接下来就是确认要符合的条件：[拥有 拥有灵能特质的人口]。我们要判断这个国家拥有人口的情况，就需要在 **condition** 中进行 **scope** 跳跃。我们可以这样：

```

every_country = {    #找到 所有 国家
    limit = {
        #要找到的国家需要满足以下condition
        any_owned_pop = {    #任何一个 拥有的人口
            has_trait = trait_psionic    #持有特质 灵能
        }
    }
}

```

图九 在 condition 中遍历人口

在 **condition** 中的遍历型 **scope** 跳跃语句，往往是以 **any** 开头，这类语句的含义为任意一个，当存在任意一个满足括号内条件的目标，就返回真。

在这里，**any_owned_pop** 就代表着任意一个拥有的人口，括号内的 **scope** 切换至 **pop**，使得我们可以运用 **pop** 对应的 **condition** 语句进行判断。

接下来我们便可以进入 **effect** 的讨论了，减少信任度——按照 **trust** 的自动补全，我们可以轻松地找到这样的语句：

```

add_trust = {
    amount = x
    who = THIS
}

```

图九 增加国家信任度，增加负数就代表减少

很快我们就发现这个语句需要两个目标，其中一个是指令所在的 **scope**，另一个，就是该语句属性中的 **who** 了。

那么 **who** 应该对应的是什么呢？这个语句的效果是：增加[当前 **scope** 国家]对[**who** 对应 **scope** 国家]的信任度，也就是说 **who** 要填入触发国家对应的 **scope**。

那么就应该这样填写：

```
every_country = { #找到 所有 国家
    limit = {
        #要找到的国家需要满足以下condition
        any_owned_pop = { #任何一个 拥有的人口
            has_trait = trait_psionic #持有特质 灵能
        }
    }
    #以 目标国家 为当前SCOPE执行以下操作
    add_trust = { #增加当前SCOPE 代表的国家 对who对应的SCOPE 代表的国家 的信任度
        amount = -50 #增加值
        who = ROOT #目标为 ROOT 代表的国家
    }
}
```

图十 如何指定双目标呢？

.....等一下。

ROOT？这是？

5.3.3 PREVPREVPREV.FROMFROMFROMFROM ——流程 scope 跳跃语句

ROOT 很明显不是一个遍历型 **scope** 跳跃语句。毕竟它不包含 **every** 或者 **random**。很多时候我们会遇到像 **add_trust** 这样，需要指定多个目标 **scope** 的 **effect** 语句，这时候我们除了需要寻找到多个对应的 **scope**，还需要把这些 **scope** 正确地填入到 **effect** 的参数中。

这时候我们有两种选择，一种是 **event_target**，我们之后会提到；另一种便是采用**流程 scope 跳跃语句**。这类语句不像遍历型，它们往往并不能帮助我们找到符合条件的目标，相反，它们的作用是调用那些我们早已找到的目标，减少我们寻找的次数，实现只靠遍历无法实现的功能。

3.1 就在此处——THIS

当前 scope，当前 scope 我说着实在是过于绕口了，在蠢驴的语法中，现在所处的 scope，被称为 THIS。

THIS 指代的是当前所在的 scope，例如范例 2-1-3 中 add_trust 所在位置的 THIS 就是 every_country 找到的国家。

像遍历型一样，你可以通过={}的方法把当前 scope 跳跃到 THIS 所对应的 scope。此举同时对 effect 和 condition 生效。

你可以写这样的代码：

```
THIS = {

```

图十一 我跳走了，但没有完全跳走

但这个代码基本上没有作用，THIS 指代的是当前 scope，从当前 scope 跳到当前 scope 是没什么意义的（）

但 THIS 并非完全没用，有一些语句虽然从字面意义上理解只需要一个目标，它们的目标并不是当前 scope，而需要你进行额外的指定。例如下面这个：

```
destroy_fleet = yes
```

图十二 摧毁舰队，但你不能填 yes

不知道蠢驴为何这样规定，你需要在等号后面确认目标：

```
destroy_fleet = this
```

图十三 摧毁舰队，不用我说此时的 this 必须为舰队类型的 scope

因此理解 THIS 的含义也是相当重要的，接下来我会逐渐用 THIS 取代“当前 scope”。

3.2 追本溯源——ROOT

在第一小节，我们提到大多数事件都有它默认的基本 scope（根/ROOT scope），例如 **country_event** 的基本 scope 便是触发者对应的国家。这个基本 scope 就是我们之前写事件，没有使用 scope 跳跃语句时一直处于的 scope，能取得的，最基本的目标。

目前（截至群星 3.0.3）我们能够使用的事件有这些：

- event （无 ROOT）
- pop_event
- ship_event
- fleet_event
- planet_event
- system_event
- country_event
- observer_event
- starbase_event
- pop_faction_event
- first_contact_event
- espionage_operation_event

很多新的事件种类在 3.0.0 被添加到了群星中，蠢驴也不是只做了一些傻事（）但对于刚进入事件写作的你，我建议优先使用以下的事件种类：

- **country_event** ROOT 为触发的国家，国家即玩家，易于理解。

- **planet_event** ROOT 为触发的**星球**，对于大多数剧情向事件，这样的事件更方便你取到那个星球的相关信息，例如名字。
- **event** 没有 ROOT，主要用于全局设置，或者无可奈何的 on_action 限制。
- **ship_event** ROOT 为触发的舰船，挖坟都是这种事件，同样，剧情方便。

当然这些事件除了基本的 ROOT 不同以外都有着类似的写法，你选择事件种类可以相当随意……除了你在使用 on_action 的时候，会发现每种 on_action 都有着不同的 event 种类限制。

……看起来跑题了。

我们了解到每种事件的触发者，基本 **scope**，也就是 ROOT 之后，便可以看一下 ROOT 作为 **effect** 和 **condition** 在事件中的作用了。

就比方说**范例 2-1-3**，我们见到了这样的代码：

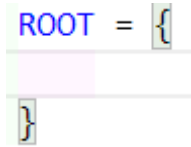
```
every_country = { #找到 所有 国家
    limit = {
        #要找到的国家需要满足以下condition
        any_owned_pop = { #任何一个 拥有的人口
            has_trait = trait_psionic #持有特质 灵能
        }
    }
    #以 目标国家 为当前SCOPE执行以下操作
    add_trust = { #增加当前SCOPE 代表的国家 对who对应的SCOPE 代表的国家 的信任度
        amount = -50 #增加值
        who = ROOT #目标为 ROOT 代表的国家
    }
}
```

图十四 ROOT 是可以随时调用的

蠢驴创造了 ROOT 这个东西，可不只是让我们嘴上说说的。

通过 ROOT，我们可以在事件中随时调用其**基本 scope**，就算你套了 114514 层括号，我也能通过一层 ROOT 识破你的真面目。

ROOT 在事件内的任何地方都代表该事件的**基本 scope**，除了在这类多目标语句内可以作为目标指代，ROOT 还能够这样使用：



图十五 像之前的遍历型一样

像之前的 THIS 一样，你可以通过=}的方法把当前 **scope** 跳跃到 ROOT 所对应的 **scope**。此举同时对 **effect** 和 **condition** 生效。

3.3 回忆上一步——PREV

那么除了 ROOT，我们在之前的事件写作中还找到了很多别的 **scope**，它们不是事件的基本 **scope**，无法通过 ROOT 来查找.....难道必须使用 **event_target** 了吗？大可不必（）

下一个杀器，PREV 在事件中有着更加灵活的运用。

范例 2-1-4 航道重组

是时候让我们接触一些更炫酷的功能了不是吗？我们学会了遍历型，还有着 ROOT 指引方向.....我们的 modder 决定让玩家帝国内的航道发生改变，就想他玩过的更多巨构 mod，其中的彭罗斯炸弹一样。

他的需求是选择触发国家境内的一个随机星系，摧毁三条航道，然后与随机相邻星系创建三条航道。

这个玩意看上去有那么一点高端了，但确实一点也不复杂。

首先我们可以尝试去找到目标：


```

random_system_within_border = { #找到 该国家 随机一个 境内星系
  while = { #循环
    count = 3 #循环次数 3
    random_neighbor_system = { #随机一个相邻星系
    }
  }
  while = { #循环
    count = 3 #循环次数 3
    random_neighbor_system = { #随机一个相邻星系
    }
  }
}

```

图十六 找到目标，相当简单

没错，当你在 VSC 中输入 **random** 或者 **every**、**any** 时，常常会出现惊喜（）很多蠢驴已经用到的**遍历型 scope 跳跃语句**极大简化了我们的写作难度。

while，一个非常常见的循环语句，在 while 里的语句会被重复执行。限制执行的次数有两种方法，一种是 count 语句，如上图，我们可以通过在 while 内写 count=x 来让其内部 effect 仅被执行 x 次；另一种是使用 limit，类似 if，while 只会在 limit 内部条件成立时执行内部的 effect。

```

while = {
  limit = {
    #condition
  }
  #effect
}

```

图十七 使用 limit 的 while

在群星的 mod 制作中，while 的执行次数是有限的，也就是说会出现 limit 仍然满足，while

却不再执行的情况，需要注意。

其实遍历型中的 every 已经可以执行很多循环的效果，所以建议多多思考来优化关于循环的代码，之后会详细介绍。

接下来就要去执行效果了。不过在此之前我们还需要考虑一个问题：**相邻星系就一定存在航道吗？**

显然不是，那么我们就需要增加一个 **limit**。

```
random_system_within_border = { #找到 该国家 随机一个 境内星系
    while = { #循环
        count = 3 #循环次数 3
        random_neighbor_system = { #随机一个相邻星系
            limit = {
                has_hyperlane_to = PREV #拥有到 PREV 的航道
            }
        }
    }
    while = { #循环
        count = 3 #循环次数 3
        random_neighbor_system = { #随机一个相邻星系
            limit = {
                NOT = { #非门
                    has_hyperlane_to = PREV #拥有到 PREV 的航道
                }
            }
        }
    }
}
```

图十八 找到有航道的相邻星系

锵锵，PREV 的初次登场。（是不是还有奇怪的逻辑语句混入了？）

PREV 和 ROOT 都是流程 scope 跳跃语句，和 ROOT 不同，PREV 代表的是，切换至 THIS（当前 scope）之前的 scope。有点绕？我们就该范例给一个详细说明。

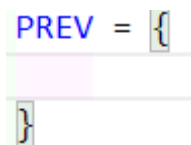
我们是如何找到目标星系的相邻星系的？代码经历了三个 `scope`。

- 首先，我们利用 `random_system_within_border` 将 `scope` 由 `ROOT`，也就是触发国家切换到了随机一个境内的星系。
- 然后我们利用 `random_neighbor_system` 将 `scope` 由目标星系切换至其随机一个相邻星系，也就是 `THIS`。

我们经历了两次 `scope` 转换，三个不同的 `scope`。`PREV` 就是抵达 `THIS`（当前 `scope`）在跳跃之前所处的 `scope`，也就是这里 `random_system_within_border` 所选中的星系。要判断其相邻星系是否与其有航道连接，我们只需使用 `PREV` 便能解决目标寻找的问题。

`x` 个 `PREV` 相连，可以指代 `x` 次跳跃前的 `scope`，例如将范例中的 `PREV` 换为 `PREVPREV` 就会指代 `ROOT` 对应的触发国家。（当然，对于 `has_hyperlane_to`，`PREVPREV` 是国家而不是星系，因此不是一个合法的目标。）

`PREV` 和 `ROOT` 有着类似的特性，也就是说下面这样是可行的：



图十九 不用多说

如果你还是觉得判断 `PREV` 是哪个有一点困难，你可以借助 `VSC` 的帮助：鼠标停留在任意语句上，你可以看到代码介绍的方框中显示着 `PREV` 和 `THIS`，`ROOT` 的 `scope` 种类，辅助你对代码的选择。

```

count = 3 #循环次数 3
Executes enclosed effects on a random system neighboring the scoped system
that meets the limit criteria, bypasses not included

Context Scope
ROOT Country
THIS System
PREV System
PREVPREV Country

1 random_neighbor_system = { #随机一个相邻星系
    limit = {
        NOT = { #非门
            has_hyperlane_to = PREV #拥有到 PREV 的航道
        }
    }
}

```

图二十 很清晰了，不要再问为啥报错了

每个种类的 scope 都有不同的 condition 和 effect，不匹配就会报错，还请多多注意。

让我们回到范例，完成这一段代码：

```

random_system_within_border = { #找到 该国家 随机一个 境内星系
    while = { #循环
        count = 3 #循环次数 3
        random_neighbor_system = { #随机一个相邻星系
            limit = {
                has_hyperlane_to = PREV #拥有到 PREV 的航道
            }
            remove_hyperlane = {
                from = THIS
                to = PREV
            }
        }
    }
}
while = { #循环
    count = 3 #循环次数 3
    random_neighbor_system = { #随机一个相邻星系
        limit = {
            NOT = { #非门

```

```

    has_hyperlane_to = PREV #拥有到 PREV 的航道
  }
}
add_hyperlane = {
  from = THIS
  to = PREV
}
}
}

```

图二十一 增加两个 effect

`add_hyperlane` 和 `remove_hyperlane` 对应着创建/删除超时空航道，如图，你可以发现这两条代码也与 `THIS` 不直接相关，需要你进行指定。

3.3 萌新永远的噩梦——FROM（其一）

终于，我们还是到了最后一个流程 **scope 跳跃语句：FROM**。

FROM 指的到底是什么？为什么他的代码我复制过来就失效了？这样或那样的问题中我们常常见到 FROM 的身影。FROM 就是每个写事件萌新的一大 boss，引起了无数的 bug，因此恐怕这一章我们还没办法做一个完整的介绍。

范例 2-2 来击个掌

还是让我们把那个整天搞破坏的神经病忘掉吧，没人会喜欢那种 mod 的。

来击个掌，如何？

规则很简单，AI 国家触发事件后，随机一个玩家国家会接收到一个消息“AI 和你击掌！”你有两个选择：接受或拒绝，如果接受，什么都不会发生；倘若拒绝……那个发起击掌的 AI 国家就会灭亡。



图二十二 爬!

这个功能包含三个事件，一个测试事件和 AI 方的测试事件放在下面，可以试试解读一下.....很简单的。

```

country_event = {
    id = xskiper_teach.202
    hide_window = yes

    is_triggered_only = yes

    immediate = {
        random_playable_country = {
            country_event = {
                id = xskiper_teach.2020
            }
        }
    }
}

country_event = {
    id = xskiper_teach.2020
    hide_window = yes

    is_triggered_only = yes
    immediate = {
        random_country = {
            limit = {
                is_ai = no
            }
            country_event = {
                id = xskiper_teach.2021
            }
        }
    }
}

```

图二十三 测试事件

想要测试事件，你只需要控制台触发 **xskiper_teach.202** 事件，便能随机抽取一个幸运 ai 与你击掌了。

如图，在 AI 国家触发的事件 **xskiper_teach.2020** 中，选取了一个随机非 ai 国家（就是随机

玩家啦) 触发了一个新的事件，这个事件是这样的：

```
country_event = {
    id = xskiper_teach.2021
    title = "来击个掌"
    desc = "如何? "

    is_triggered_only = yes

    option = {
        name = "好"
    }
    option = {
        name = "爬! "
        FROM = {
            destroy_country = yes
        }
    }
}
```

图二十四 非常简单的事件

这个事件只有三行是我们需要关注的：

```
FROM = {
    destroy_country = yes
}
```

图二十五 非常简单的事件

FROM 在这里首次出现了。

阅读我们的事件要求，我们可以发现，在玩家国家触发事件时，我们需要逆推到上个事件，找到上个事件的 ROOT，从而可以对正确的目标进行灭国。

PREV 只在一个事件内部的 scope 转换有效，这个时候我们就需要 FROM。

在事件 A 触发事件 B 的情况下，事件 B 中的 FROM 代表着事件 A 的 ROOT。

同样，FROM 可以连续叠加，例如事件 C 中的 FROMFROM 在事件 A 触发事件 B，事件 B 触发事件 C 的情况下，代表着事件 A 的 ROOT。

这是 FROM 出现的第一种情况，也是最简单的情况，但正如这个范例，这种 FROM 往往只在触发国家切换转变时不可替代，也就是国家关系，外交相关的那些事件。其他时候，萌新可以采用之后会介绍的 `event_target` 来避免其使用。

或许有萌新会在听了之前的介绍依旧对 FROM 有点理解困难，不妨在游戏内触发事件时输入 `debugtooltip` 的控制台指令，鼠标放到事件的选项后，你可以看到事件的 FROM 和 ROOT。可以辅助你的事件写作。



图二十六 可以看到事件的 FROM, FROMFROM

相信这样的技巧一定能够帮助到各位 modder。

FROM 的功能绝不仅限于此，它对于刚进行 event 制作的 modder 的更大意义可能在于 `on_action` 提供的可调用 scope 上，关于这一点，我在下一篇会进行更详尽的介绍。

3.4 流程 scope 跳跃语句：小结

流程 scope 跳跃语句和遍历型有着很大的不同，它们指代的是一个独立的目标而非一个模糊的群体。因此它们能够成为那些，能够在内部指定额外目标的 effect 或 condition 语句中填写的目标。同时，他们引用着我们之前所使用过的目标，一方面便于我们简化流程，另一方面为我们进行一些更复杂的事件设计做好了铺垫。

他们可以作为独立的 scope 跳跃语句，也能够作为目标填写。其中的 PREV 和 FROM 可以通过堆叠进行跨越多层的引用。熟练运用 ROOT 和 PREV 是之后各位事件写作的基本要求，所以请多练多学。

5.3.4 从前有艘船，船所有者.....——属性 scope 跳跃

从前有艘船，船所有者，所有者有首都，首都有星系，星系有星基.....这样的套娃可以无穷无尽，接下来我们接触的一种 scope 跳跃语句，也是我们今天学习的最后一种 scope 跳跃语句，也就是属性 scope 跳跃语句。

属性 scope 跳跃语句，调用的是与 THIS 相关的其他 scope，例如语句 owner，调用的就是 THIS 的所有者，一个国家。同样的还有语句 ruler，调用的就是 THIS 国家 scope 的统治者。这样的语句和遍历型不同，和流程 scope 跳跃语句同样只指代一个具体的目标。这些语句调用的是 THIS 的属性，取得一个和 THIS 通过某些方式对应的 scope。

其中有一些属性 scope 跳跃语句是直接以 scope 类型来命名，不像 owner 之类浅显易懂，比如这些：

```

pop = {
}
planet = {
}
fleet = {
}

```

图二十七 这些语句的名字就是 scope 种类

在使用这种语句时一定要注意，只有当 **THIS** 对应的目标只有一个时，这样的语句才会正常工作。比方说，对于一个国家，它有无数个 pop，**THIS** 为国家时，使用 pop 来跳跃 scope 显然是错误的；而对于一个 pop，他只在一个星球上，**THIS** 种类为 pop 时使用 planet 来跳跃 scope 就是正确的。

同时这些 scope 语句也能作为目标填写到那些需要填写目标的 effect 或 condition 语句中。这一点和流程 scope 跳跃语句是类似的。

5 小结

终于写完了……要吐了。

我们在本章学习了三种不同的常用 scope 跳跃语句：遍历型 scope 跳跃语句、流程 scope 跳跃语句、属性 scope 跳跃语句。运用这些语句我们可以轻松地找到各种各样的目标。下面的练习题也将会侧重于寻找目标的方法，希望各位对 scope 转换能够在参考习题后有了更深的理解。

来列一个表格吧。

遍历型 scope 跳跃语句	流程 scope 跳跃语句	属性 scope 跳跃语句
搜索模糊的群体，批量	指代独立的个体，寻找	指代独立的个体，来源

处理大量符合条件的目标。	在之前的事件代码中已经找到的目标。	为当前 scope，也就是 THIS 的属性。
--------------	-------------------	-------------------------

接下来让我们回到上一章，也就是**范例 1-2**，现在我们已经有了自动迁移人口的能力了。让我们再解析一下目前需要的目标。

范例 2-3 烂大街的自动人口迁移（其二）

- 找到一个 ROOT（触发国家）拥有的，有空闲岗位的星球。
- 找到一个 ROOT 拥有的，没有工作的 pop。
- 将目标 pop 迁移到目标星球上。

在熟悉掌握了流程 scope 跳跃语句后，我们可以这样填写：

```
country_event = {
    id = xskiper_teach.203
    hide_window = yes #不显示事件窗口，直接执行效果

    is_triggered_only = yes

    immediate = {
        every_owned_pop = { #THIS拥有的 全部人口
            limit = {
                is_unemployed = yes #判断THIS（此时指该pop）是否失业
            }
            ROOT = { #回到 ROOT，即触发国家
                random_owned_planet = { #THIS拥有的 随机一个星球
                    limit = {
                        free_jobs > 0 #有空闲岗位
                    }
                    resettle_pop = {
                        pop = PREVPREV #回到前2个scope，也就是every_owned_pop找到的人口
                        planet = THIS #有空闲岗位的目标星球，就是THIS
                    }
                }
            }
        }
    }
}
```

图二十八 能否理解这些代码？

当然这还不是人口自动迁移的全部，但作为一个基础，它已经有了足够的功能.....我们在（其三）见。

习题二

（1）请找到有这些功能的遍历型 scope 跳跃语句，并尝试指出其适用的 THIS scope，以及指向的新 scope 的类型。

- ① 每一艘拥有的舰船；
- ② 银河中的每一个物种；
- ③ 每一个接壤国家；
- ④ 恒星系内的全部星球；
- ⑤ 轨道内的全部舰队。

（2）请指出下面几段代码中，注释处 THIS 的 scope 类型。

```

planet = {
  solar_system = {
    random_system_megastructure = {
      prev = {
        starbase = {
          root = {
            prevprev = {
              this = {
                prev = {
                  #这里是什么scope?
                }
              }
            }
          }
        }
      }
    }
  }
}
  
```

① }

```

every_owned_leader = {
  fleet = {
    owner = {
      every_pool_leader = {
        species = {
          every_species_pop = {
            prevprev = {
              prevprevprev = {
                #这是什么scope?
              }
            }
          }
        }
      }
    }
  }
  prev = {
  }
}

```

②

(3) 试着找到下面语句所描述的目标

① ROOT 为国家，试着找到首都与 ROOT 首都星系距离小于 20 跳的国家。（首都 capital_scope，查询星系与星球/星系距离 distance）；

② ROOT 为星球，试着找到该星球星系内的军事舰队的上将（星系 solar_system，星系内舰队 every_fleet_in_system，上将可以直接靠 leader 取到）；

③ ROOT 为舰队，试着找到舰队所有者的全部爬行类人口（判断种族小类：is_species_class）。

(4) 试着写出满足下面要求的 effect 语句

① 令所有国家对与其关系低于-50 的国家宣战（判断国家关系使用 opinion）；

② 使我境内的所有舰船归我所有（设定所有权使用 set_owner）；

六、scripted effect 与 trigger 的函数应用

触犯禁忌就要做好被禁忌反噬的准备

6.1 前言

此章节主要是面向想更进一步深究群星触发机制，并希望花费大量时间做出较为复杂的机制或事件效果的 modder 而撰写的。在阅读此章节前，请确保自己有一定编程基础（例如什么是函数，变量，数列）以及对 event 里大部分 effect 能熟练运用（包括清楚了解 scope，effect 与 trigger）。由于笔者对计算机术语不太了解，因此我会用英文特别标注一些具争议的名词。至于这些名词的正确读法就见仁见智吧。

另外，由于这种函数应用涉及到的复杂性以及蠢驴为了让其实现所牺牲的大量性能，大量使用后，**会显著地增加内存消耗或游戏卡顿等情况，使得 MOD 在游戏里的表现十分差强人意。**如非对整套系统完全深入了解，否则 **不建议** 使用本章节内提及的技术内容

6.2 scripted effect 函数化

蠢驴虽然对群星里很多效果语句作了极大限制，并且缺乏游戏内数据的调用函数，但他却打开了一扇窗，能让我们通过一些旁门左道完成一些看似单纯使用 effects 语句无法达成的效果。如果你习惯于翻查英语 wiki 来增进你对群星 modding 的了解，你也许会发现 wiki 里有这一个特别的页面。
https://stellaris.paradoxwikis.com/Dynamic_modding

Parameters [\[edit\]](#)

Scripted effects can have parameters. All parameters are strings and will be dynamically inserted into the scripted effects.

For example, a scripted effect can be like this, so the empire will have their ethic shifted to materialist:

```
# defination
example_scripted_effect = {
    shift_ethic = ethic_$ETHIC$
}

# calling
example_scripted_effect = { ETHIC = materialist }
```

6.2.1 参数传递

如上图所示, scripted effect 是可以传递参数的。这种传递参数的运作方法有点像 C 的宏, 也就是单纯的字串替换。在上述例子中, 你需要首先在 common/scripted_effects 定义一个 scripted effect, 并在想要被替换的内容前后加上一个\$符号, 然后在\$符号内定义这个被替换内容的参数名, 之后回到 event 文件里调用该 scripted effect。但与正常使用 scripted effect 的时候直接在脚本效果名后加上 = yes 不同, 你需要给与参数一个值, 具体写法请看# calling 下方的语句。

在执行的时候, \$符号以及参数名会一起被替换为该参数的输入值然后再执行。

这样就和一般程序的函数 (function) 有异曲同工之妙了, 定义一个参数, 然后调用时给与这个参数的值, 最后在函数内参数的值被替换为执行函数时所定义的值

再清晰一点, 具体执行效果是这样的

首先 scripted effect 的运作原理是, 将它定义里面一大串的程序直接复制粘贴到程序调用的位置

首先定义以下一段 scripted effect:

```
example_scripted_effect = {
    shift_ethic = ethic_materialist
}
```

在 event 调用这段 scripted effect 时需要这样写

```
example_scripted_effect = yes
```


这样这段东西就会替换成: `shift_ethic = ethic_materialist`

但是如果上述的 scripted effect 写成这样:

```
example_scripted_effect = {  
    shift_ethic = ethic_$ETHIC$  
}
```

那么你就需要为 ETHIC 这个参数提供一个值 (\$符号代表定义 ETHIC 是一个参数)

于是你需要将后边原本 `= yes` 的部分的写法换成这样写:

```
example_scripted_effect = { ETHIC = materialist }
```

其中我们定义了 ETHIC 这个参数的值是 materialist, 因此数据在传递后 \$ETHIC\$ 就会被替换成其定义的值, 也就是 materialist, 整个脚本效果复制到 event 后执行的效果是:

```
shift_ethic = ethic_materialist
```

~~和第一种写法的效果其实一模一样~~

6.2.2 Parameter Conditions 参数判断

让我们顺着 wiki 继续看下去，我们会发现他多了使用一个中括号的东西

```
[[homeworld] <<<STUFF HERE>>>]
```

wiki 里的英文的解释意思就是，如果\$homeworld\$参数被输入了一个值，那么中括号里的东西就会被执行，反之就不会。值得注意的是，需要在语句最前面先定义需要判定的参数是哪个，然后用中括号把这个参数括好，最后再用一个中括号把整段需要判断是否执行的代码括好。

依旧是一个例子（请注意在 country 作用域之下调用这段代码）

```
create_fleet_test = {
  create_fleet = {
    [[NAME] name = $NAME$]
    effect = {
      set_owner = PREV
      create_ship = {
        name = random
        design = "NAME_Nomad_Cruiser"
        upgradable = no
      }
    }
  }
}
```

如果你按正常方法调用这个脚本效果，（写成 create_fleet_test = yes）你会发现多了一个以系统随机命名的名字的舰队，舰队里面就只有一艘巡洋舰，并没有什么 bug

而如果按以下方法调用的话

```
create_fleet_test = { NAME = "滑稽" }
```

那么舰队的名字应该会变成了 滑稽

由于 create fleet 方法里 name 这项参数是可以被忽略的，而 scripted effect 里中括号的作用是判断某个参数是否传入。如果参数输入了，那么中括号内的那行代码会传入输入的参数值并执行；反之，

如果调用函数时参数没有输入，那么整行代码都不会被执行。

请注意，**这个判断的参数不需要实际存在于函数中，或使用\$符号宣告这个参数**，也就是说你可以随便写一个用作判断的参数然后随便输入一个值去让它判断是否执行函数中某一些地方的效果。这种中括号的使用方法很适用于一些可忽略某些参数的函数里，这样省去了逐个逐个填写参数的麻烦。与这种参数判断相对应的还有参数预设值

6.2.3 Fallback value 参数预设值

如果你们已经理解上面两节关于 scripted effect 函数的基本用法的话，想要设计很多段需要输入不同参数但内容几乎一样的代码时，那么你们大可以写一个有参数的 scripted effect，然后调用时参数直接写上所需的值，一看就是很高大上的方法。但蠢驴留下的功能远不止这样。在 wiki 中关于中括号用法下方还有一小段：

```
has_global_flag = crisis_stage_${STAGE}|1$
```

这段代码除了我们介绍过的\$符号和参数外，还有一个直线“|”符号和后面跟着的数字

这个“|”符号定义了其后面的值是符号前面的参数的预设值。也就是说当 STAGE 这个参数**没有任何输入值**时，它会自动变成 1，甚至都不需要你写。

同样先来一个小例子：

```
set_variable_test = {  
    set_variable = { which = A value = $VALUE|2$ }  
}
```

如果我直接调用这个脚本效果（也就是 set_variable_test = yes）

那么这个变量就会是 A: 2

但如果我给参数一个值，写成 set_variable_test = { VALUE = 10 }

那么这个变量就会是 A: 10

这些功能能够大大简化代码内容，甚至达到了一些天机工程的效果。~~学到这里的话你就可以写在群里对大部分人装逼了 ()~~

当然还远远不止于此，蠢驴还给了你最后一项法宝

6.2.4 Inline Math 参数计算

有时候或许我们需要对数值进行一些计算，例如每年呈一个指定的数学公式增长一些资源，但以蠢驴的 effect 来说，最多只能做到以月收入为基础增加资源，除此之外就只能自己算然后穷举数值，这样确实非常非常的不方便。或许蠢驴自己写代码的时候也感受到了这点，于是就开发了这段功能。

```
inline_math_example = {
    add_monthly_resource_mult = {
        resource = unity
        value = $COUNT|1$
        min = @\[ $COUNT|1$ * 10 ]
        max = 99999999
    }
}
```

在上面这个例子里，我们再度看到了一个中括号，但它的前面还有两个符号“@\”

（请注意\号的方向，~~不要打错成/号子~~，vscode 里的 cwtool 里是没有任何关于这个章节的内容的判断的，也就是这个章节的东西用了就可能会出红色底线！不会有正确提示的！）

这个@\加中括号的组合是用来计算中括号里面的数学运算的。（目前本人测试只支持[四则运算](#)，开方和三角函数之类的想都不要想了）

假如我们调用 inline_math_example = { COUNT = 5 }，那么这段代码就会给你 5 个月的凝聚力收入，最低为 $5 * 10 = 50$ 点。

假如我们再魔改一下这段函数：

```
inline_math_example = {
    add_monthly_resource_mult = {
        resource = $RESOURCE$
        value = $COUNT|1$
        min = @\[ $COUNT|1$ * 10 ]
        max = 99999999
    }
}
```

其实就和原版的 `add_monthly_resource_mult` 方法几乎没什么分别了，只是少输入了两个参数值而已。所以对于这些并不复杂的函数的魔改就请自己衡量一下把，毕竟蠢驴自己定义的 `effect` 才是效率最高的方法，再此之上的旁门左道都必须得先用蠢驴自己的方法跑一边，然后在跑一遍你的函数，这样效率就会成指数增长了，对于效率厨来说是非常不友好的事情。

而且这种参数计算还有一个坏处，**它在每个脚本效果里只能被使用一次，只有代码最为靠上的一行参数计算生效**。~~（不愧是你，蠢驴；拳头，硬了）~~

```
inline_math_example = {
    add_monthly_resource_mult = {
        resource = unity
        value = @\[ $COUNT|1 + 3$ ]
        min = @\[ $COUNT|1$ * 10 ]
        max = 99999999
    }
}
```

比如这段写了两个参数计算的脚本效果，只有 `value` 那一行的会生效，`min` 那一行的它（大概）会保持原参数不动，而解决方法是——套娃！

6.2.5 recursion 递归

之所以特意开一个小节来解释所谓的套娃是怎么回事，是因为在下一个大章节中会重复用到这个概念。为了让大家有一个好一点概念，我会尽量地解释得更详细，让你们能跟上我的思维。如果你们已经在计算机课或者别的地方学过并熟悉这种概念，那么请跳过这一章。

递归是指函数自己引用自己的情况。例如以下一段数学函数 $f(x) = 3x + 2$ 。如果我让 $x = 2f(x)$

那么先将 $x = 2f(x)$ 代入一次，就会变成 $x = 3(2(f(x)))+2$ ，再次代入 $f(x) = 3x + 2$ 就会变成

$x = 3(2(3x+2)) + 2$ 。黄色底色的部分可以很明显看到原函数的样子，这就是函数自己引用自己，也

就是在套娃。而这样无条件地递归就会导致递归一直下去导致无法停止的情况，也就是俗称的死循环。

所以一般来说我们都要给函数设立一个停止条件，防止这种无限套娃的情况。

值得一提的是，一般大多数程序都会设置有递归次数上限（maximum recursion depth），该数值限制了你能让函数自己引用自己多少次便自动停止的情况，而群星里该上限为 5，这个数值同样也适用于本地化使用 \$ 符号引用其他词条的情况（会导致本地化变成 MAX RECURTION DEPTH 的罪魁祸首）所以请各位 modder 在使用本章节谈及的内容时特别注意这一点。

在上一章节我们已经谈及到使用蠢驴的参数计算 @\[] 的功能的时候只能计算一次的情况。要解决这个问题，本章节所提及的递归就能有效地解决这个困境。先看以下一段函数。

```
# 输入一个值，并输出这个数阶乘的值至 return_value 变量
1 factorial_variable = {
2   # 因为蠢驴没有方法直接判定函数里的输入值，所以我们需要一个变量暂时记录这个数值
3   set_variable = { which = depth value = $VALUE$ }
4   if = {
5     # 在剩余阶乘次数大于 1 (1 没效果) 的时候继续递归，否则不执行任何效果导致函数停止
6     limit = { check_variable = { which = depth value > 1 }}
7     if = {
8       # 在第一次执行函数时先将数值初始化，将 return_value 设定为输入值
9       limit = { check_variable = { which = return_value value <= 0 }}
10      set_variable = { which = return_value value = $VALUE$ }
11    }
12    # 将记录的输入数值-1，然后输出数值乘以这个数值
13    subtract_variable = { which = depth value = 1 }
14    multiply_variable = { which = return_value value = depth }
```

```

# 最后再次递归函数
12     factorial_variable = { VALUE = @\[$VALUE$-1] }
    }
}

```

让我们先手动跑一遍这个程序。假设我这样引用这段效果 `factorial_variable = { VALUE = 3 }`

在一开始 `depth` 变量就会变成 3。执行第 4 行到第一个 `if` 时条件通过 ($3 > 1$)，到第 6 行第二个 `if` 时也通过了（变量初始值为 0，当然如果你已经引用过一次这段效果并且没有将变量归 0 当我没说），所以 `return_value` 就会变成 3。接着到第 10 行 `subtract_variable`，这个效果是将变量减去指定数值，所以 `depth` 减 1 了变成了 2，然后第 11 行 `multiply_variable` 是乘以数值，所以就是将 `return value` 乘以 `depth`，也就是 $3 * 2 = 6$ 。最后第 12 行函数再次被引用，但是 `value` 那里有一个参数计算，参数 `VALUE` 被计算成自己减去 1，所以再次执行函数时，`VALUE` 就会成为 2 而不是 3。如此循环下去，到 1 的时候就停止了，因为任何数乘以 1 都是自己，没有必要计算。结果 `return value` 就会变成 6。

为了让大家直观一点看，我拉了一个表：

	depth	return_value	\$ VALUE \$
第一次引用 - 行 2	3	0	3
第一次引用 - 行 8	3	3	3
第一次引用 - 行 10	2	3	3
第一次引用 - 行 11	2	6	3
第一次引用 - 行 12	2	6	2
第二次引用 - 行 2	2（重新设定了一次）	6	2
第二次引用 - 行 10	1	6	2
第二次引用 - 行 11	1	6	2

第二次引用 - 行 12	1	6	1
第三次引用 - 行 2	1 (重新设定了一次)	6	1
第三次引用 - 行 5	条件不通过，函数结束		

经过这一连串的计算之后，我们就得出了准确的结果，达成成就：在群星做数学运算（1/1）

当然这只是一个简单的例子，并没有任何实用价值，因为实际上当输入值大于 5 的时候这个函数就不管用了，理由是上述提及到的**递归上限**。但希望你能够理解递归和函数间互相引用的概念。

七、GFX 文件与 sprite type

在本篇我们将会讨论 interface 文件下的 GFX 文档以及其里面最重要的组成部分之一：sprite type。实际上这个章节是新加进来的，我决定要将这个章节插进来的原因之一是因为 sprite 构成了群星的可视化界面里的一切元素。小到从编辑事件图，替换飞升、传统、星球图标等等，大至自己创建一个界面，甚至像小伞包那样改造原版界面，sprite type 都是如此的重要且不可替代。因此笔者认为在完全理解如何写一个 sprite 之前，是不可能写好一个界面的。那么，我们就来首先理解什么是 sprite 吧

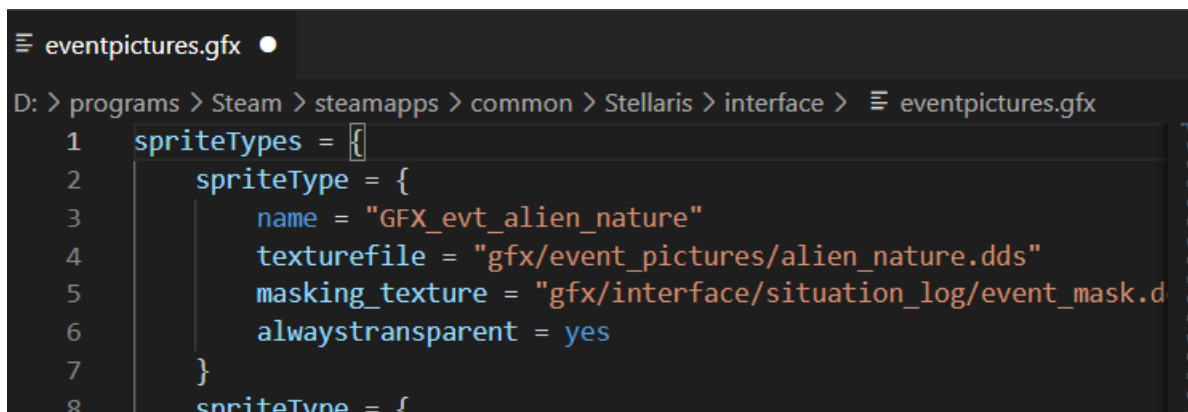
7.1 认识 sprite 与创建 sprite type

相信各位在尝试使用自己的事件图的时候都曾经遇到过这一难题：为什么我把自己弄好的事件图放进去 [gfx/event_pictures](#) 之后，在事件里直接使用 GFX_evt_图的名字 这样会无效呢？明明科技图标建筑图标等等都是可以这样做的？

很明显，脑子里产生疑惑的你已经认识到，事件图所使用的图片结构与科技、建筑图标等的图片结构是不一样的，导致群星无法识别事件图的格式。那么我们该如何正确导入一张事件图呢？这个章节就由这个问题的罪魁祸首——sprite 说起。

如果你去百度什么叫做 sprite，最前面搜出的知乎答案（不用去搜了，笔者已经搜过一遍了）可能会告诉你什么游戏引擎的 sprite 阿，还有一堆 C/C++ 和触发器，精灵等等的定义，但在群星里，这些东西我们统统不需要。在本章节中，我们只需要将 sprite 视作一个简单的**图层**就可以了，没错，就是 ps 里的那个图层。在群星里，sprite 只是一个处理图像的工具，并没有再被赋予什么更高大上的概念，只不过这个概念是从 2D 游戏里一直沿用下来的，所以就叫它做 sprite 了。当然谈论到实际游戏开发的概念，sprite 的定义可以说十分万能，以至于知乎给的那个答案都是争吵过好几遍才得出的较优解，但我们今天把这些概念统统抛诸脑后，只需要认识到 **sprite 只是一个图像处理工具**的用途即可。~~（跑题了跑题了）~~

那么，回到正题，我们该如何正确地创建一个事件图呢？首先，打开蠢驴源文件的 interface/eventpictures.gfx，你会看到一大长串的代码。不过不用担心，我们只需要如下图所示的一个代码块即可（即被 spriteType 的括号包起来的那段代码），其他代码的格式全部都大同小异，是从这个格式复制替换过来的。



在这段代码的结构里，我们首先可以看到 spriteTypes = {} 和 spriteType = {} 两段宣告代码，我们不需要理会它们是什么，只需要知道的是这是在宣告一个 sprite 类型即可，我们需要重点关注的是这个代码块里的 name、texturefile 和 masking_texture 这三个参数的定义。

首先，name 代表了这个 sprite 的名字，也就是引用这个 sprite 时需要填的名字，这个很好理解所以就不着笔墨了；texturefile 即这个 sprite 的底图。前面我说过了 **sprite 在群星的作用就相当**

于 **ps 的图层** 一样，因此 texturefile 就是用来处理这个图层的底图。注意后面是一串路径，因此你可以用来引用**任意一处**的图片档案，在这里我们要处理的是事件图，因此就是 gfx/eventpicture 里的图。最后，masking_texture 是遮罩图片，亦即限制图片显示区域的图层。寻着这个路径，我们可以找到这个图片是长这个样子的：

其中，紫色区域是透明的区域，白色就是不透明的区域。作为一个遮罩图片，蠢驴只会取这个文件的 alpha 值（透明值，不理解这是什么请自己百度），来决定被遮罩的文件（也就是我们的事件图）哪些地方需要显示和哪些不需要（具体来说就是将遮罩文件的 alpha 图层覆盖到原本文件的图层上）。由于事件弹框的界面是被完全限制死的，因此必须要处理成这个样子。图片以及遮罩组合起来之后，事件图就会像我们平常看到的那样是一个不规则的形状。可以看到图片的四个角落以及左右都会凹陷进去一点，这是因为经过了遮罩文件的处理，紫色位置的区域都被处理掉了。



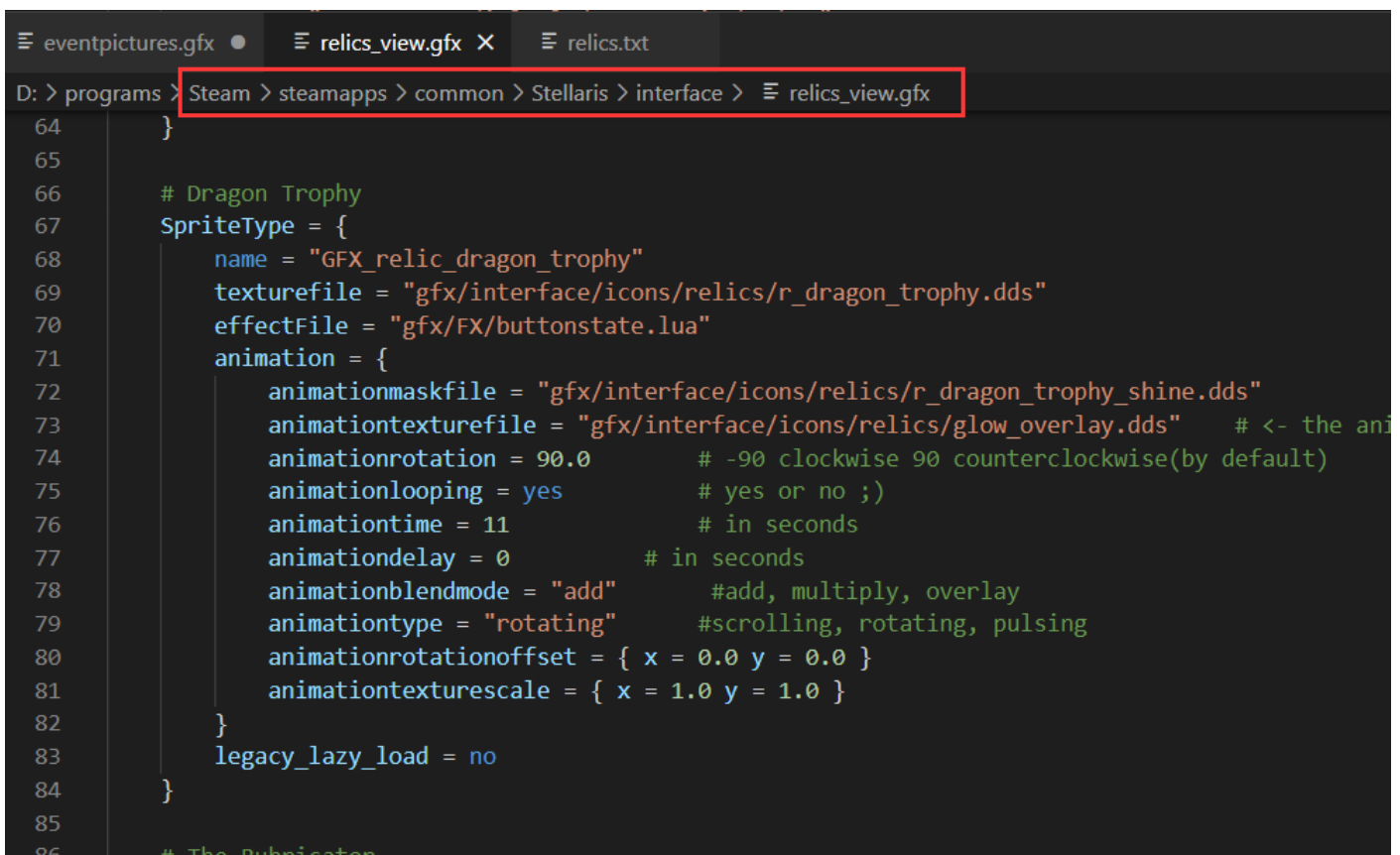
像这样在 gfx 文件处理好之后，我们才能在事件里引用这个 sprite，并将它显示在事件弹框里，而复杂的东西群星就会自动帮你弄好了。

```
▼ ship_event = {  
    id = anomaly.140  
    title = "anomaly.140.name"  
    desc = "anomaly.140.desc"  
    picture = GFX_evt_asteroid_approaching_planet  
    show_sound = event_mystic_reveal  
    location = FROM  
  
    is triggered only = yes
```

至于 gfx 文件，它本身就是一个 txt 文件，只是改了后缀名而已，编码上并没有什么不同……

7.2 sprite type 动画

在学会如何正确导入一个事件图之后的你，开始继续了你的 mod 制作，而某一天你又遇到一个难题了：为什么遗物会有动画？而根据上面代码写的遗物并没有那个闪闪发光的动画？遇事不决先复制蠢驴，让我们来看看蠢驴的代码（路径如红框所示）



```

D: > programs > Steam > steamapps > common > Stellaris > interface > relics_view.gfx
64   }
65
66   # Dragon Trophy
67   SpriteType = {
68     name = "GFX_relic_dragon_trophy"
69     texturefile = "gfx/interface/icons/relics/r_dragon_trophy.dds"
70     effectFile = "gfx/FX/buttonstate.lua"
71     animation = {
72       animationmaskfile = "gfx/interface/icons/relics/r_dragon_trophy_shine.dds"
73       animationtexturefile = "gfx/interface/icons/relics/glow_overlay.dds" # <- the ani
74       animationrotation = 90.0 # -90 clockwise 90 counterclockwise(by default)
75       animationlooping = yes # yes or no ;)
76       animationtime = 11 # in seconds
77       animationdelay = 0 # in seconds
78       animationblendmode = "add" #add, multiply, overlay
79       animationtype = "rotating" #scrolling, rotating, pulsing
80       animationrotationoffset = { x = 0.0 y = 0.0 }
81       animationtexturescale = { x = 1.0 y = 1.0 }
82     }
83     legacy_lazy_load = no
84   }
85
86   # The Rubricator

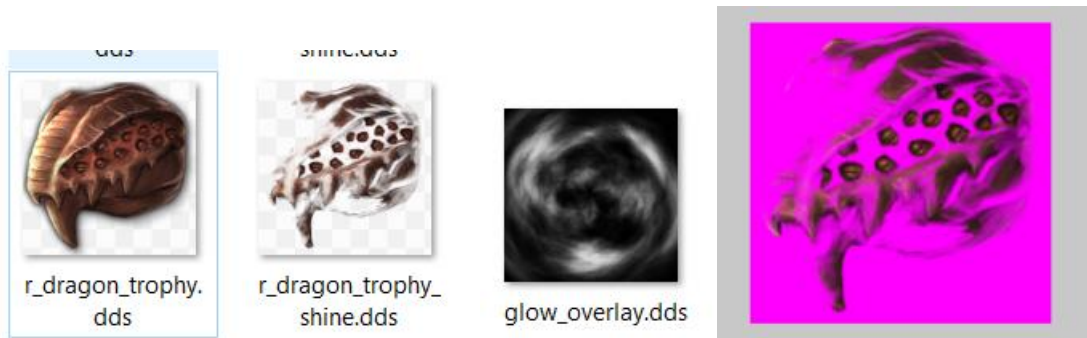
```

嗯？这次的代码明显长了很多？首先我们从第一部分没见过的东西一一讨论吧。

effectFile：这里面的路径引用的是一个.lua 文件，很明显这个文件的内容已经超出本章节讨论的范围了，因此我简单给一个结论给你，它就是一个处理按钮的文件而已，也就是会将一个图片处理成光标放上去会发光，按下去会启动动画效果，sprite 所代表的代码不可用的时候会变成暗色，这样的效果。所以由于其复杂性，最好不要动它。

之后的一长串代码都在 animation 里，也就是我们这个章节最主要的内容：动画

要让一个 sprite 使用动画，就必须要先在 spriteType 里宣告 animation = {}, 然后再填入对应的参数。让我们先来看看这里的 animationmaskfile 和 animationtexturefile 长什么样子：



左 1: texturefile, 左 2: animationmaskfile, 右 2: animationtexturefile

左图很明显就是我们游戏里干掉以太龙后拿到的龙鳞遗物的图片，中间的图是左图经过发光处理后的遮罩图，而右图是动画图。既然中间是遮罩图，我们再来换成第一章的样子看看它实际的 alpha 图层长什么样子：（最右的那个）

可以看见大部分区域都是紫色，这也就是动画不能够显示的区域（会被遮罩裁掉的区域），因为这毕竟是动画，肯定不可能全部覆盖上去，只要让它的阴影部分有闪闪发光的效果就行了

让我们继续解析接下来的代码

```
animationrotation = 90.0           # -90 clockwise 90 counterclockwise(by default)
animationlooping = yes             # yes or no ;)
animationtime = 11                 # in seconds
animationdelay = 0                 # in seconds
animationblendmode = "add"         #add, multiply, overlay
animationtype = "rotating"         #scrolling, rotating, pulsing
```

在 animationrotation 这段代码的右方，蠢驴很贴心地给了注释，-90 度是顺时针，90 度是逆时针，这段我们先跳过。

animationlooping, 这段直接翻译过来意思已经很明显了, 就是动画会不会一直持续。

animationtime, 动画的持续时间 (秒), 也就是这个动画播放一次所需的时间

animationdelay, 循环播放的延迟时间 (秒), 播放完这段动画后, 如果是循环播放则会延迟这个设定的时间后再次播放

animationblendmode, 动画渲染模式, 蠢驴给了三种模式: add, multiply, overlay。效果不明, 最好跟着蠢驴走

animationtype, 动画类型, 蠢驴也给了三种类型: scrolling, rotating, pulsing, 这里用的是 rotating 旋转, scrolling 则是平移, pulsing 是渐入效果 (也就是从透明逐渐显示出来的那种效果)

```
animationrotationoffset = { x = 0.0 y = 0.0 }  
animationtexturescale = { x = 1.0 y = 1.0 }
```

最后两项参数一个是动画偏移坐标, 一个是动画缩放的参数, 一般来说不太用到, 所以就不着笔墨了, 而最后的 legacy_lazy_load = no 笔者也搞不清是啥东西, 不要碰就行。

总结以上的参数, 我们就可以得出这个遗物的动画, 那就是将那个像旋风一样的贴图放在龙鳞上旋转, 显示的位置则是龙鳞的阴影部分, 所以看起来就会有一些闪闪发光的效果。(点下面那个 gif 可以看效果)



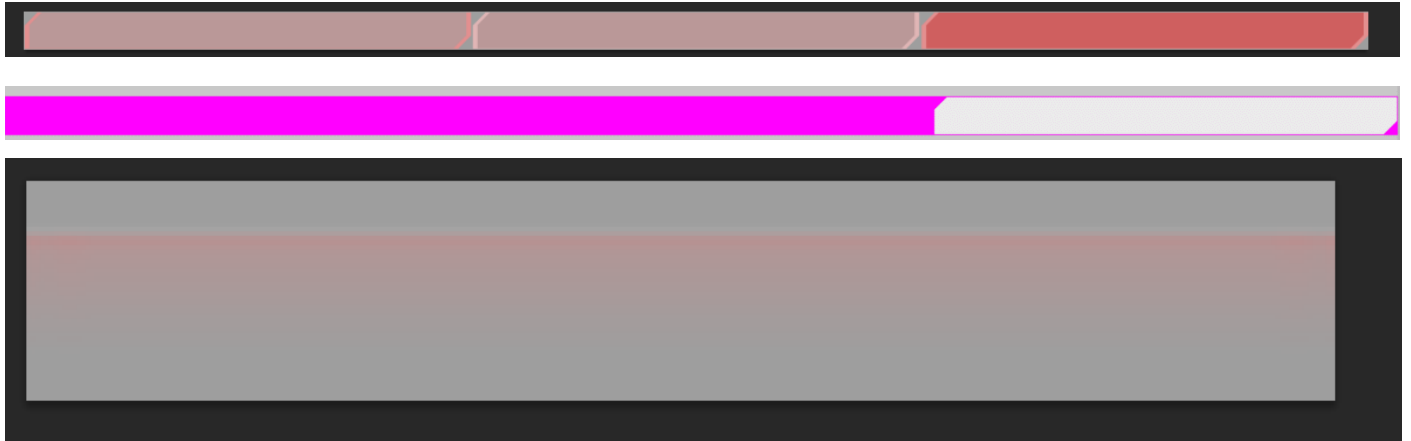
遗物动画.gif

让我们再来看一个 sprite 动画的例子, 来巩固我们刚学到的知识。

```
spriteType = {  
  name = "GFX_wg_gray_button_571_46"  
  texturefile = "gfx/interface/wg_leader_diplo/gray/wg_button_571_46_animated.dds"  
  effectFile = "gfx/FX/buttonstate_onlydisable.lua"  
  noOfFrames = 3  
  animation = {  
    animationmaskfile = "gfx/interface/wg_leader_diplo/wg_button_571_46_animated_mask.dds"  
    animationtexturefile = "gfx/interface/wg_leader_diplo/gray/wg_button_571_46_animated_  
    animationrotation = 180.0  
    animationlooping = yes  
    animationtime = 40.0  
    animationdelay = 0.0  
    animationblendmode = "overlay"      #add, multiply, overlay  
    animationtype = "scrolling"         #scrolling, rotating, pulsing  
    animationrotationoffset = { x = 0.0 y = 0.0 }  
    animationtexturescale = { x = 1.0 y = 1.0 }  
    animationframes = { 1 2 3 }  
  }  
}
```

这是一个很常见的按钮动画（也就是鼠标放上去会有扫描的效果的那种），由于涉及到了自定义的 ui，所以笔者这次干脆放上了自己的舰 Rmod 的按钮代码上去了（~~毕竟是自己写的更好懂~~）

首先请我们的图出场（由于直接用原图会比较难以理解，所以这些图是我用 ps 润色过的）



三张图从上到下分别是 texturefile、animationmaskfile、animationtexturefile

灰色的背景是 ps 加上去的，所以灰色的部分是透明的

那么，作为练习，请自己先看一遍代码和图，根据上一章说叙述的代码，自己判断一下这个 sprite 将会产生什么效果，然后再来看下面的答案解说。

—————7 秒经过，泷泽萝拉达! —————

首先来看一下重要的部分：

```
animationrotation = 180.0
animationlooping = yes
animationtime = 40.0
animationdelay = 0.0
animationblendmode = "overlay"      #add, multiply, overlay
animationtype = "scrolling"         #scrolling, rotating, pulsing
```

从 animationtype 我们已经可以知道，这是个平移的动画，rotation 是 180 度所以是向下滚动（并且第三张图会被垂直反转），animationlooping 是 yes，所以动画会持续播放，animationdelay 是 0，所以动画循环没有延迟，animationtime 是 40，所以整段动画从上平移到下会持续 40 秒，平移的图片对象是 animationtexturefile，也就是第三张图，结合这些讯息，我们就可以想象出这个图片会在按钮上从上到下平移。由于有 animationmaskfile 遮罩文件的限制，所以平移的区域只限制在按钮图片上。最后的效果应该会是这样的：（双击打开 gif）



按钮动画.gif

但是，按钮的这个扫描效果动画，只会在按钮被鼠标碰到的情况下播放，这又是怎么回事呢？

除了这些，这次的 spritetype 还多了其他一些代码，也就是 noOfFrames 和 animationframes，这部分的疑问将涉及到自定义 ui 的内容，因此我会将这些放到下一章节解释。

7.3 sprite type 按钮

除了以上讲述的会在 common 和 event 用到的替换游戏内图片效果动画的 sprite 外，想要创建自己的 ui 就必须还有一件东西需要学会，那就是按钮。通过组合图片以及按钮，我们就可以创建一个最基本的交互界面，也就是在群星里常见的事件弹窗。



拿回上一章节的舰 Rmod 的按钮作为范例，可以看见按钮图片里并不是单独的一个框，而是有三个框从左到右平铺开来了，但进入游戏时它只会显示其中的一个，那么游戏里到底显示的哪一个呢？答案是根据**按钮状态**而定。一般来说，按钮会有三个状态：正常、高亮、按下，当鼠标没有触碰到这个按钮的时候，它会在正常状态，而鼠标触碰到了则会是高亮状态，最后用左键按下了按钮那么就会变成按下的状态。因此，在群星里要创建一个按钮，你必须要提供按钮的三种状态对应的图片来让游戏根据按钮状态切换图片，而提供的方法就是像上面这个图的形式，将按钮的三个状态对应的图片从左到右平铺开来。

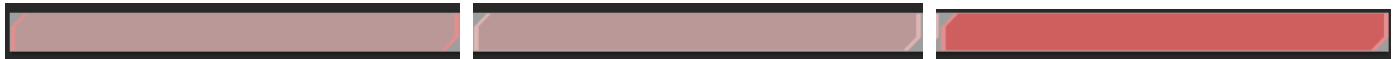
要创建一个 sprite 按钮，只需要如下的代码即可

```
name = "GFX_wg_gray_button_571_46"
texturefile = "gfx/interface/wg_leader_diplo/gray/wg_button_571_46_animated.dds"
effectFile = "gfx/FX/buttonstate_onlydisable.lua"
noOfFrames = 3
```

其中 name, texturefile 和 effectfile 已经在本章节第一节中讨论过，因此不再叙述，让我们把

注意力放在最后一行代码 `noOfFrames = 3` 上：

这个 `noOfFrames` 字面意思是帧数，但实际上在 `gfx` 的定义，它指的是在 `texturefile` 存在的切片数量。简单来说，如果你填了这个参数，那么你的图片就会根据这个数值对 `texturefile` 在水平进行 `n` 等分，像上面代码填了 `noOfFrames = 3`，那么在处理的时候游戏首先会将这个图片在水平切割出三份大小一样的图片，图片将会变成以下的样子：



切割出来的三份图片载入之后，游戏将会根据在游戏中按钮的状态来决定显示哪一个图片。

请注意，提供给 `sprite type` 按钮的图片按照按钮状态从左到右应当是 **正常、按下、高亮** 尽管这有点反常识，但确实如此，请勿将顺序打乱，因为我们是没有办法调整这个顺序的。

讲述到了这里，相信我们已经能够解答在上一章节中遗留下的问题：为什么按钮只会在高亮状态播放按钮？让我们回看一下这个按钮的动画遮罩图片：



可以发现，这个遮罩图只有在属于高亮状态的按钮的区域是白色的，其余两个状态都是紫色，也就是不显示，因此只会在按钮处于高亮状态的时候，动画才会显示，其余状态时读取的遮罩图完全是透明的，因此动画也会变成透明而不会显示。

相信你已经掌握了大部分关于 `sprite type` 的用法了，现在你已经可以尝试着修改组建自己的新 `ui` 了。不过，你还需要阅读下一章节才能够完全清楚如何真正地像小伞包和舰 `r` 那样完全不依靠原版代码自定义一个 `ui` 出来。下一章节将是对一些其他不重要的 `sprite` 的参数和类型的补充。

7.4 其他 `sprite type`

除了定义最基本的 `spriteType` 之外，我们也可以定义一些其他的 `spriteType`，比如舰 `R` 最近研发的一个逐帧动画事件图黑科技为例，它便使用了另外的 `frameAnimatedSpriteType`：

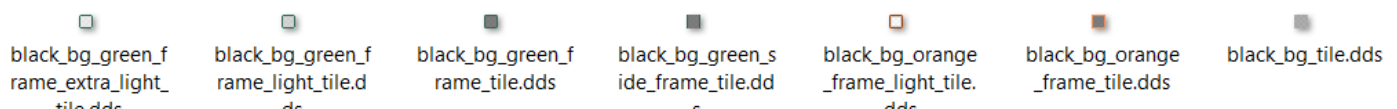
7.4.1 frameAnimatedSpriteType

```
frameAnimatedSpriteType = {
    name = "GFX_evt_wsg_project_s"
    texturefile = "gfx/event_pictures/project_s.dds"
    masking_texture = "gfx/interface/situation_log/event_mask.dds"
    noOfFrames = 30
    transparencecheck = no
    animation_rate_fps = 15
    looping = no
    play_on_show = yes
    alwaystransparent = yes
}
```

这个 sprite 会将图片转化成逐帧图片并且循环播放，其中 noOfFrames 我们已经知道了，是切片数量，由于蠢驴图片是单一的静态 dds 图片，因此要做逐帧动画只能够将动画所有帧全部平铺放到同一图片中，然后使用 noOfFrames 切割成 n 等分来提取动画各帧的图片，而动画是有速率的，因此定义在 animation_rate_fps 中，其中 fps 指的是 frame pre second，因此 15 就是一秒播放 15 帧。最后，looping 很明显是循环播放与否，play_on_show 则是是否动画一出现就会播放，alwaystransparent 设置为 yes 会导致该元件不可点击，即使它是个按钮（即只作为图片的状态）

7.4.2 corneredTileSpriteType

在蠢驴新更新的可以自定义的传统中，我们还能看见这样的 sprite，这个 spritetype 本质上是一种填充背景用的 sprite



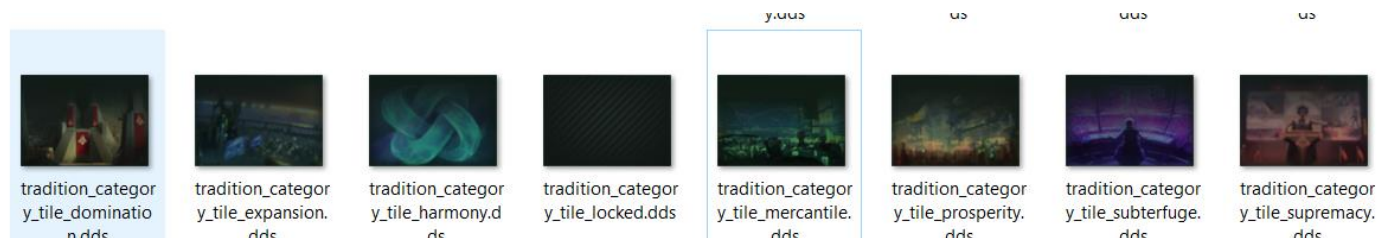
很明显，这是一种将图片直接放大填充所需区域用的 sprite，因此这种 sprite 使用的图片可以弄得十分小，只有 8*8，不过这和我们传统中自定义的传统背景图有一点差别

其中 borderSize 指的是该元素经过拉伸之后的形成的大小（游戏会帮你处理好）

```

corneredTileSpriteType = {
    name = "GFX_tradition_category_tile_tradition_expansion"
    textureFile = "gfx/interface/tiles/tradition_category_tile_expansion.dds"
    borderSize = { x=140 y=140 }
    effectFile = "gfx/FX/buttonstate_onlydisable.lua"
}

```



在蠢驴的传统里，所用的背景图反而是这样的一张完整的图，因此其实充当背景用的图可以大也可以小，只不过使用了这种 sprite 游戏就会将其拉伸至指定大小来填充背景

八、Custom GUI 入门

注意，本篇所讨论的皆为 Event 使用的 Custom GUI，并不涉及对原版 GUI 文件的修改，不过在学习了本篇之后你可以更加轻松的修改原版 GUI 文件以达到你所需要的功能。

首先我们需要认识何为 Custom GUI 以及其到底能做什么。GUI 为 Graphical User Interface 的简写，意思就是“图形用户界面”，Custom GUI 则就是“自定义图形用户界面”。

在制作 mod 过程中可能你会对蠢驴给予的事件面板感到失望，比如你想新写一个面板，面板上有各种自己定义的按钮以实现各种不同的功能，显然原版事件窗口也就一个选项界面可以给予你不同的功能实现，但一串的选项显然并不满足某些富有野心的 modder，我们需要一个更加自由的面板来实现我们的目的，于是蠢驴给予了我们部分自定义 GUI 的可能，至于为什么是部分，在下文你将能体验到蠢驴为何蠢以及学习如何创建自己的第一个自定义 GUI 窗口。

8.1 GUI 文件的构成

首先我们需要稍微认识下 GUI 文件，由于我们所看到的虽然是图形界面，但一切都需要通过底层代码完成架构，因此我们需要认识 GUI 文件是如何构成的。

首先我们随便打开一个原版的 GUI 文件，这里列举了香草的 achievements_view.gui 文件，其文件位置位于 interface/achievements_view.gui。在此我们并不对其游戏表现进行解析，而是单纯从代码角度进行分析，以下是该文件内容展示：

```
guiTypes = {
  containerWindowType = {
    name = "achievement_window"
    position = { x=-550 y=-155 }
    size = { width=512 height=128 }
    orientation = LOWER_RIGHT

    background = {
      name = "background"
      quadTextureSprite = "GFX_tile_outliner_bg"
    }

    instantTextBoxType = {
      name = "achievement_title"
      font = "malgun_goth_24"
      text = "Achievement unlocked!"
      position = { x = 10 y = 0 }
      maxWidth = 450
      maxHeight = 20
      fixedSize = yes
      orientation = "UPPER_LEFT"
      format = Left
    }

    buttonType = {
      name = "achievement_closebutton"
      quadTextureSprite = "GFX_close"
      position = { x = -41 y = 13 }
```

```

        Orientation = "UPPER_RIGHT"
        shortCut = "ESCAPE"
        pdx_tooltip = "CLOSE_TITLE"
        clicksound = "back_click"
    }

    iconType = {
        name = "achievement_icon"
        position = { x = 18 y = 34 }
        spriteType = "GFX_planet_background_frame_blocker"
    }

    instantTextBoxType = {
        name = "achievement_header"
        font = "cg_16b"
        position = { x = 128 y = 36 }
        text = "The Industrial Re-Revolution"
        format = Left
    }

    instantTextBoxType = {
        name = "achievement_text"
        font = "cg_16b"
        position = { x = 128 y = 58 }
        text = "Earn a mineral income each month above 1000"
        format = Left
    }

    # Invisible button over the main area of the view so that we can click
    on anything to activate more info
    containerWindowType = {
        name = "mainwindow_overlay"
        position = { x=0 y=0 }
        size = { width=460 height=128 }

        background = {
            name = "mainwindow_overlay_bg"
            quadTextureSprite = "gfx_transparency_button"
            #alwaysTransparent = yes
        }
    }
}

```



```

containerWindowType = {
    name = "achievements_view"
    position = { x=0 y=0 }
    size = { width=100% height=100% }
}
}

```

首先在创立任何一个 GUI 文件之后，都需要声明一个 guiTypes，该条目下所有子元素（即 guiTypes 下方第一层所有并列的元素）皆是一个独立 UI，其中包括类型控件以及窗口，下方先列举几个接下来将会使用到的概念：

控件是指能够依附于一个窗口以实现某些目的的修饰性容器

窗口是指能够独立显示出来并通过承接各类控件将其显示于界面之上给予控件各类交互机会的主要容器类型。

锚地指的是位置标定点，即物理上常用的概念参考系，蠢驴的 UI 定位锚点默认为容器的左上角。

GUI 无非就是一层层的容器相互嵌套而形成的，而每个容器又有其各自的属性，通过调整容器嵌套以及其属性，便能见到各种各样的 UI，而这也是我们编写 Custom GUI 的关键所在。

8.2 GUI 模板

上文提到过，Custom GUI 并非十分自由，蠢驴限制了其模板文件只能为外交窗口，并且我们只能新增元素，不能够删除已有的元素，否则将会崩溃！不过好在我们可以通过其他一些手段来达到我们的目的，在此我先给出在 Wiki 上的一段已经基本被清空的 GUI 模板（由于 word 的问题，排版比较丑陋，建议各位到 [Interface modding - Stellaris Wiki \(paradoxwikis.com\)](https://paradoxwikis.com/wiki/Interface_modding_-_Stellaris_Wiki) 最下方拿到该模板）：

```

guiTypes = {

    ### FACTION OVERVIEW GUI ###

    containerWindowType = {
        name = "irm_faction_overview"
        orientation = center
    }
}

```



```

    origo = center
    moveable = yes
    size = { width = 660 height = 300 }

    background = { name = "background" quadTextureSprite = "GFX_tile_outline
r_bg" }

    iconType = { name = "hex_bg" spriteType = "GFX_hex_bg" position = { x =
-10 y = -14 } alwaysTransparent = yes }
    iconType = { name = "empire_header_line" quadTextureSprite = "GFX_line_
Long" position = { x = 6 y = 22 } alwaysTransparent = yes }

    buttonType = {
        name = "close"
        quadTextureSprite = "GFX_close"
        position = { x = -42 y = 12 }
        Orientation = "UPPER_RIGHT"
        shortcut = "ESCAPE"
        clicksound = "back_click"
    }

    # Custom header
    instantTextBoxType = {
        name = "irm_some_header"
        font = "malgun_goth_24"
        text = "ui_faction_header" # header text
        position = { x = 20 y = 5 }
        maxWidth = 543
        maxHeight = 22
        fixedSize = yes
        alwaysTransparent = yes
    }

    ### ALL OF THIS IS HIDDEN OR DISPLACED ! ###

    buttonType = { name = "focus_button" position = { x = -1180 y = -
1112 } spriteType = "GFX_fleetview_focus" } # hidden
    instantTextBoxType = { name = "heading" font = "malgun_goth_24" positio
n = { x = -1120 y = -115 } } # hidden
    buttonType = { name = "alien_message_background" size = { x = 0 y = 0 }
position = { x = -1110 y = -
11430 } spriteType = "GFX_tiles_dark_area_cut_8" } # hidden

```

```

        buttonType = { name = "confirm_button" quadTextureSprite = "GFX_standar
d_button_142_34_button" } # hidden
        containerWindowType = {
            name = "portrait_background"
            position = { x = -1117 y = -1145 }
            size = { width = 0 height = 0 } # hide portrait window
            iconType = { name = "event_picture" spriteType = "GFX_diplomacy_por
trait_frame" } # hidden
            iconType = { name = "portrait" spriteType = "GFX_portrait_character
" } # hidden
        }
        containerWindowType = {
            name = "portrait"
            position = { x = -1117 y = -1145 }
            size = { width = 0 height = 0 } # hide portrait window
            iconType = { name = "portrait" spriteType = "GFX_portrait_gamesetup
_mask" } # hidden
            iconType = { name = "black_frame" spriteType = "GFX_diplomacy_portr
ait_shadow_frame" } # hidden
            iconType = { name = "stripes_1" spriteType = "GFX_diplomacy_stripes
_2" } # hidden
            iconType = { name = "city_frame" spriteType = "GFX_diplomacy_portra
it_frame" } # hidden
        }
        iconType = { name = "empire_info_bg" spriteType = "GFX_diplomacy_dark_f
ade_bg" } # hidden
        instantTextBoxType={ name = "empire_name" font = "malgun_goth_24" } # h
idden
        instantTextBoxType={ name = "empire_government_type" font = "cg_16b" }
# hidden
        instantTextBoxType={ name = "empire_personality_type" font = "cg_16b" }
# hidden
        OverlappingElementsBoxType = { name = "empire_ethics_icons" position =
{ x = -1145 y = -11138 } } # hidden
        iconType = { name = "empire_flag" spriteType = "GFX_empire_flag_128" po
sition = { x = -1125 y = -11235 } } # hidden
        containerWindowType = {
            name = "leader_details"
            position = { x = -1125 y = -1175 }
            containerWindowType = { name = "empire_traits_box" } # hidden
            instantTextBoxType = { name = "empire_traits_label" font = "cg_16b"
} # hidden

```

```

        overlappingElementsBoxType = { name = "leader_traits" } # hidden
        instantTextBoxType = { name = "leader_name" font = "cg_16b" } # hid
den
        instantTextBoxType = { name = "leader_species" font = "cg_16b" } #
hidden
    }
    containerWindowType = {
        name = "opinion_window"
        position = { x = -1127 y = -11110 }
        size = { width = 94 height = 30 }
        iconType = { name = "their_opinion_icon" spriteType = "GFX_diplomac
y_opinion" } # hidden
        instantTextBoxType = { name = "their_opinion" font = "cg_16b" }
    }

    ### EVENT DIPLO ###

    containerWindowType = {
        name = "EVENT_DIPLO"
        position = { x = 2 y = 50 }
        #size = { width=563 height=220 }
        moveable = no

        instantTextBoxType={
            name = "action_title"
            font = "malgun_goth_24"
            position = { x = 20 y = 0 }
            maxWidth = 200
            alwaysTransparent = yes
        }
        instantTextBoxType={
            name = "action_desc"
            font = "malgun_goth_24"
            position = { x = 490 y = 0 }
            maxWidth = 500
            maxHeight = 180
            Orientation = "UPPER_LEFT"
            format = center
            alwaysTransparent = yes
            text_color_code = "H"
        }
    }

```

```

        listboxType = { name = "option_list" position = { x = -11500 y = -
1130 } } # hidden option list
    }
    # Description Section
    instantTextBoxType = {
        name = "alien_message"
        font = "cg_16b"
        position = { x = 20 y = 100 }
        maxWidth = 465
        maxHeight = 220
        format = Left
        scrollbarType = "standardtext_slider"
    }
}

```

我们来一步步解析该模板（某些我认为不重要的容器将不会被解释）：

containerWindowType 为声明该容器的类型，这是 Custom GUI 限定使用的容器类型，也是我们将会用到最多的容器类型。

-name 是表示其唯一性，类似于事件的 id，在需要调用该容器之时代表该容器的值。

-orientation 定义了该容器的锚点位置，此位置为相对父元素锚点。

-origin 定义了当使用 orientation 属性之时锚点的参考点。

-background 定义了该容器的背景图片，quadTextureSprite 则是调用在.gfx 文件下注册的图片。

-iconType 其作用为显示图片，图片调用定义与 background 相同，需要注意的是，由于整个容器皆为蠢驴的外交事件框改变而来，因此某些元素将会调用事件传递过来的参数，如旗帜，肖像，背景等等，尽管我们有时候并不需要他们，但是我们仍然不能够直接删除这些接口或者修改器 name，擅自修改原容器控件的 name 或者删除原有控件将会大概率导致崩溃！我们可以从这些 name 当中猜测该控件所链接的事件参数并加以利用（假如用得到的话）。

-buttonType 一个可点击的按钮控件，不过此类控件所链接的是硬代码，不能够更改其 name，

不过我们可以调整期调用的图片以及位置大小——将某个不可删除的控件 `size` 调整为 0 亦或是将 `position` 调整为一个极其离谱的值都是一个假装删除控件的方法，在此模板内例子比比皆是。

-`instanTextType` 一个可以显示文本的控件，其文本本地化定义在 `text` 属性当中，`font` 属性规定了其使用的字号，不过由于并不能采用支架命令，其存在除了蠢驴用来链接内部文本之外毫无作用（一般情况下直接把文字做成图丢 `iconType` 还能给文件加点艺术效果，这个文本框实在不懂在不能用支架命令还有啥调用的价值，也就是蠢驴内部链接在用了）

-`position` 表示相对于父元素锚点的位置，如若没有父元素则相对于屏幕锚点进行位置调整。锚地指的是位置标定点，即物理上常用的概念参考系，蠢驴的 UI 定位锚点默认为容器的左上角。

-`size` 定义了该容器的大小，其属性有 `width`, `height`，分别标定宽与高，还有另一种直角坐标系写法，将 `height` 用 `y` 代替，`width` 用 `x` 代替，两者作用相同。

差不多关键元素都讲完了，剩下的控件内容看看下表以及 wiki 界面就差不多了，常用的属性基本就上方内几个，常用的容器类型也就只有 `iconType`、`containerWindowType` 以及我们接下来要讲的最重要的东西，`effectButtonType`，由于 `buttonType` 这个控件仅仅是蠢驴内部链接使用，因此对我们来说是无效控件，蠢驴还是给予了我们一线希望，那就是这个 `effectButtonType` 控件，该控件是唯一一个能够链接 Custom GUI 与事件的控件，因此我们接下来先详细介绍下该控件：

```
effectButtonType = {
    name = star_pledge_main_window_faction_1
    quadTextureSprite = "GFX_star_pledge_main_window_faction_1"
    size = { x = 200 y = 200 }
    position = {x = 50 y = 50 }
    effect = test_effect
}
```

该控件的声明也是十分简单，图片，位置，大小皆与 `iconType` 保持一致，最主要的是其 `effect` 属性，这是链接该控件被点击之时所产生的效果，其定义文件为 `common/button_effects` 下：

```
test_effect = {
    allow = {
```

```

    }
    potential = {

    }
    effect = {

    }
}

```

-allow 定义该按钮不满足条件之时变灰

-potential 定义该按钮不满足条件之时直接消失（不显示）

-effect 定义该按钮点击之后所执行的 effect

好了，现在我们已经了解了 GUI 模板以，接下来就是该处理如何调用 Custom GUI 了，我们只能够通过事件来调出 Custom GUI，因此我们需要一个事件作为载体，这里给出了一个例子：

```

country_event = {
    id = test.1
    is_triggered_only = yes
    diplomatic = yes
    custom_gui = star_pledge_select_window
}

```

该事件需要采用调用 custom_gui 调用我们定义的 Custom GUI，值为我们定义的首个 containerWindowType 的 name，并且由于是外交界面更改而来，因此必须声明 diplomatic = yes，这样既可，然后示我们隐藏了什么元素以及我们需要什么元素来决定该事件是否需要添加 title 及其他属性。

至此，所有 GUI 理论方面的讲解便结束了，下一篇将会以一个实例来讲解如何制作自己的第一个 Custom GUI，当然，仅仅有前两章的内容也可以尝试自己实现一下，以下是 wiki 的控件参考表，你也可以通过查阅 [Interface modding - Stellaris Wiki \(paradoxwikis.com\)](http://paradoxwikis.com/interface-modding-stellaris-wiki) 获取更多帮助。

Element	Type	Modability	Description	Attributes
containerWindowType	Container Element	Attributes and Content	The main container element when structuring the GUI.	name, position, size, moveable, clipping, orientation, origo, background, alwaysTransparent
buttonType	Action Element	Attributes Only	This element is in most cases tied to a specific hardcoded action, but can also be used to display a graphical element, like a fake background.	name, position, orientation, buttonFont, buttonText, appendText, oversound, clicksound, shortcut, format, vertical_alignment, pdx_tooltip, alwaysTransparent, web_link, rotation
effectButtonType	Action Element	Attributes and Effect	Similar to <code>buttonType</code> , just that it's manipulable with the <code>effect</code> attribute, which are events called from <code>/common/button_effects/</code> and can run <code>custom_gui</code> . <code>pdx_tooltip</code> doesn't work for <code>effectButtonType</code> , so use <code>custom_tooltip</code> when coding <code>button effect</code> instead.	name, position, orientation, buttonFont, text, appendText, oversound, clicksound, shortcut, format, vertical_alignment, effect, alwaysTransparent
iconType	Graphical Element	Attributes and Content	This element whose sole purpose is to display graphics; images, icons etc.	name, position, orientation, spriteType, quadTextureSprite, frame, scale, pdx_tooltip, alwaysTransparent, rotation, centerPosition
instantTextBoxType	Text Element	Attributes and Content	This element is for displaying text, which can either be directly inserted or using localisation references.	name, position, orientation, font, text,

Element	Type	Modability	Description	Attributes
				appendText, maxWidth, maxHeight, fixedSize, format, vertical_alignment, text_color_code, pdx_tooltip, alwaysTransparent
scrollbarType	Scrollbar Element	Attributes and Content	This element is used to define the behavior of scrollbars for listBoxType and smoothListboxType, but also for sliders as seen in the settings window. Can be used both horizontally and vertically.	
extendedScrollbarType	Scrollbar Element	Attributes and Content	This element is used to define the behavior of scrollbars for containerWindowType. Can be used both horizontally and vertically.	
spinnerType	Carousel Element	Attributes and Content	This element is used to define the behavior of a clickable carousel, like seen in the settings window.	
guiButtonType	Action Element	Attributes Only	Same as buttonType, but specifically for scrollbars and spinnerType elements.	
positionType	Position Element	Attributes Only	Used to define positions for certain elements. These links are hardcoded.	
listboxType	List Element	Attributes Only	A scrollable list element that jumps between sub elements when scrolling.	
smoothListboxType	List Element	Attributes Only	A scrollable list element that smoothly scrolls.	name, position, size, orientation, alwaysTransparent, borderSize, spacing, scrollbarType, offset,

Element	Type	Modability	Description	Attributes
				defaultSelection, priority, autohide_scrollbar
OverlappingElementsBoxType	Horizontal List Element	Attributes Only	This element displays its content horizontally and will start overlapping them if width limit has been reached.	
gridBoxType	Grid Element	Attributes Only	This element displays its content in a grid layout, depending on horizontal and vertical limits.	name, position, size, orientation, alwaysTransparent, slotsize, format, add_horizontal, max_slots_horizontal, max_slots_vertical
checkboxboxType	Action Element	Attributes Only	This element is a yes/no graphical action box.	
editBoxType	Action Text Element	Attributes Only	This element is an editable text box, as seen when modifying names of objects in the game.	
dropDownBoxType	Dropdown Element	Attributes Only	Container element for <code>expandButton</code> and <code>expandedWindow</code> elements.	
expandButton	Action Element	Attributes Only	Similar to <code>buttonType</code> , just that it's linked to a toggleable <code>expandedWindow</code> element.	
expandedWindow	Toggleable Container Element	Attributes and Content	Element that is toggleable with <code>expandButton</code> .	

Element	Type	Modability	Description	Attributes
windowType	Floating Container Element (?)	Attributes Only		

九、代码覆盖规则

注意，本篇所讨论的内容都会对游戏本体及其他 mod 造成严重影响，请确保你已经能够完成基础的 common 和 event 文件编写，再尝试对代码进行覆盖。

Mod 是必须建立在游戏本体的基础上才能运行的，我们编写的代码如何生效，为什么会生效，这些都由游戏本体进行了规定和解释。但是，当你不断尝试完成新的内容时，你会发现，原版是有极限的。为此，你必须超越原版，必须修改原版的文件。所以，了解并掌握代码的覆盖规则，将会帮助你更好的建立新的秩序，并且与大多数 mod 兼容。

9.1 基础知识

在正式学习覆盖规则之前，我们首先需要了解群星是如何载入 mod 文件的。

mod 文件的载入顺序与文件名的 Unicode 编码排列顺序有关，你可以形象地理解为，在载入 mod 时，群星会将所有的 mod 文件与游戏本体的文件排成一个队列，Unicode 编码靠前的文件会优先读取（此时，mod 文件甚至会优先于原版文件读取），如 00_buildings.txt 将优先于 09_buildings.txt 读取，ABCD_jobs.txt 将优先于 ZBCD_jobs.txt 读取。Unicode 编码的顺序你可以通过百度查询的方式获得，当然你也可以记住一个简单的规则，数字优先于字母，大写字母优先

于小写字母。如果存在两个 Unicode 编码完全相同的文件，那么 mod 的排列顺序将变得至关重要。启动器中，排序在下方，即优先度低的 mod 的同名文件，将覆盖排序在上方的同名文件。

9.2 排序规则

类似于图片扩展、加载背景扩展、文本扩展、音乐扩展等 mod，通常情况下不需要考虑排序问题，只有那些存在依赖关系，或者修改了原版文件的 mod 之间才必须进行排序。

一般，Modder 们会告诉你他们的 mod 依赖的前置以及排序的位置（虽然大多数 modder 都会说，排在后面），如果某个 mod 存在前置 mod 依赖，那么请务必把前置 mod 放在上面，否则可能会导致游戏内 bug，甚至会导致游戏崩溃。对于大型种族 mod 来说，通常情况下并没有十全十美的排序方式，除非这些大型种族 mod 之间存在兼容，否则必然会失去一部分 mod 的内容（经典案例：看不到某个 mod 的资源）。因此，你现在最想要玩哪个种族 mod，就把该 mod 排序在下方。

在遵从上述规则的情况下，我们给出一个简单的 mod 排序规则（从上到下）：

- 1) UI 类、原版支持和扩展类（UI 大修、伞的优化等）
- 2) 美化类（萌化星河、更多舰船模型、物种图片扩展、更多星球等）
- 3) 修改内容单一、独立性较好的 mod（如 more events mod、整活传统、远古故事集等）
- 4) 大型物种 mod（战舰少女 R、少女前线、碧蓝航线等）

PS：这部分 mod 会互相覆盖一部分内容，如 mod 资源，想玩谁就把谁放下面

- 5) 大型物种 mod 之间的兼容补丁
- 6) 大型功能扩展类、原版重构类（产业革命等）
- 7) 6) 类与其他 mod 的兼容补丁
- 8) 汉化类、文本类、加载背景扩展类、音乐类

其实 2)，3)，8) 并没有严格的排序要求，但是为了整洁和美观，请使用这种格式来排序。

9.3 覆盖规则

不同文件夹下的内容遵从不同的覆盖规则，其主要有以下几点：

1) key 覆盖（关键词覆盖）

顾名思义，就是命名覆盖。拥有相同命名的代码，后读入的会覆盖先读入的，与 mod 排列顺序无关，仅与 mod 文件名称的 Unicode 编码排列顺序有关（如果存在同名文件，排序靠后的 mod 会覆盖掉排序靠前的 mod），大多数代码也都是这种覆盖规则。通常情况下，这部分覆盖规则会导致较为严重的兼容问题。

2) 文件名覆盖

部分代码（如 pop_jobs）无法通过 key 进行覆盖，因此，想要修改这部分代码，必须通过同名文件的方式进行覆盖。此时，mod 的排序方式至关重要，后读入的文件会彻底覆盖掉先读入的文件，使先读入的文件完全失效。如果排序出现问题，极有可能出现游戏 bug。

3) 只读一次

部分代码的 key（如 events）在游戏加载时只会读取一次，后序无论遇到多少同名 key 值，都会直接丢弃。因此，这部分代码通常需要放在一个文件名的 Unicode 编码靠前的文件中，保证优先读取。

4) 特殊覆盖规则

本地化 (localisation) 具有特殊的覆盖规则，通常情况下，想要覆盖具有相同 key 的本地化时，需要在 localisation 文件夹下创建一个 replace 的文件夹，此文件夹下的本地化文件将在所有本地化文件加载完成后读取，并且覆盖掉所有具有相同 key 的本地化。当然，也可以在不创建 replace 文件夹的情况下覆盖本地化，但是其覆盖规则并不明确，既不是 key 覆盖，也不是只读一次，很可能导致意外的效果。因此，想要覆盖本地化，请务必创建 replace 文件夹。

5) 合并规则

触发器 (on_actions) 并不会互相覆盖, 具有相同 key 值的 on_actions 会将其下的所有事件代码合并。

9.4 各文件的覆盖规则列表

以下列表翻译自群星英文百科关于文件覆盖的列表, 并不一定准确, 仅供参考。

覆盖规则大多按照 ASCII / UNICODE 编码顺序排列, 详见上一章, 若覆盖规则为**后读覆盖**, 则编码顺序最后的文件内容会生效, 若**只读一次**, 则编码顺序靠前的文件内容会生效, 并省略编码顺序靠后的文件的所有内容。其他按情况可能会生成重复内容或必须整个文件覆盖 (或只能同名文件直接覆盖, 该情况下按照 mod 载入顺序覆盖)

9.4.1 common 文件夹

档案类型 (上级文件夹名)	覆盖规则	备注
Agendas	后读覆盖	
AI Budget		
Ambient Objects		
Anomalies	后读覆盖	
Armies	后读覆盖	
Artifact Actions	后读覆盖	
Ascension Perks	后读覆盖	
Asteroid Belts		
Attitudes	后读覆盖	
Bombardment Stances	后读覆盖	
Buildings	后读覆盖	局部覆盖该文件会导致自动生成的相关修正失效
Button Effects		
Bypass	后读覆盖	
Casus Belli	后读覆盖	
Colony Types	后读覆盖	
Colors		
Component Sets	只读一次	
Component Tags		
Component Templates	只读一次	
Country Customization		

Country Types	后读覆盖	
Crisis Levels	后读覆盖	
Crisis Objectives	后读覆盖	
Decisions	后读覆盖	
Defines	后读覆盖	必须写入上一级的代码块 E.g.: NGameplay = { POLICY_YEARS = 10 }
Deposit Categories		
Deposits	后读覆盖	
Diplomatic Phrases		
Diplomatic Economy	后读覆盖	
Diplomatic Actions	后读覆盖	不能只覆盖局部的文件内容，必须复制整个文件覆盖
Districts	后读覆盖	局部覆盖该文件会导致自动生成的相关修正失效
Economic Categories		
Edicts	后读覆盖	
Ethics	后读覆盖	可能会导致选择思潮的图像损毁 警告： 如果存在多个思潮文档，可能会导致一些关联思潮的自动生成的代码或文件随机被损毁，建议整文件全部覆盖
Event Chains	只读一次	
Fallen Empires		
Game Rules	后读覆盖	
Global Ship Designs	只读一次	
Governments	后读覆盖	
├── Authorities	后读覆盖	不能只覆盖局部的文件内容，必须复制整个文件覆盖
└── Civics	后读覆盖	
Graphical Culture		
Leader Classes	后读覆盖	
Mandates	后读覆盖	
Map Modes	后读覆盖	
Megastructures	后读覆盖	
Name Lists		
Notification Modifiers		
Observation Station Missions		
On Actions	自动整合，不可局部覆盖	读取顺序是从先到后
Opinion Modifiers	内容重复/后读覆盖	add_opinion_modifier 遵循后读覆盖原则

Personalities	后读覆盖	
Planet Classes	后读覆盖	内容可能会重复出现；如果不整文件覆盖会损毁自动生成的宜居度修正
Planet Modifiers	后读覆盖	
Policies	后读覆盖	3.X 后可使用 NAME = §局部覆盖
Pop Categories	后读覆盖	
Pop Faction Types	后读覆盖	
Pop Jobs	不可局部覆盖	不能只覆盖局部的文件内容，必须复制整个文件覆盖（最好同名覆盖），局部覆盖该文件会自动生成的相关修正失效（如“职业+xxx”，“每 xxx 个人口增加 xxx 个职业”）
Precursor Civilizations		
Random Names		
Relics	不可覆盖	
Resolutions	后读覆盖	
Scripted Effects	后读覆盖	
Scripted Localisation		
Scripted Triggers	后读覆盖	
Scripted Variables	只读一次	
Section Templates	只读一次	警告： 不能只覆盖局部的文件内容，必须复制整个文件 同名覆盖 已使用该文件撰写的舰船部件的舰船/恒星基地覆盖后会自动被移除
Sector Focuses	后读覆盖	
Sector Types		
Ship Behaviors	只读一次	
Ship Sizes	后读覆盖	不能只覆盖局部的文件内容，必须复制整个文件覆盖
Solar System Initializers	只读一次	
Special Projects	只读一次	
Species Archetypes	后读覆盖	局部覆盖该文件会导致自动生成的相关修正失效
Species Classes	后读覆盖	
Species Names		
Species Rights	后读覆盖	效果提示文案(effect tooltip)独立于内容效果
Star Classes	只读一次	
Starbase Buildings	后读覆盖	
Starbase Levels	后读覆盖	
Starbase Modules	后读覆盖	不能只覆盖局部的文件内容，必须复制整个文件覆盖
Starbase Types	后读覆盖	
Start Screen Messages	只读一次	每个位置(?)的第一个有效词条将会生效，此后无视其他词条

Static Modifiers	只读一次	
Strategic Resources	只读一次	
Subjects	后读覆盖	其功能受限于 diplomatic_actions 及 game_rules
System Types	后读覆盖	
Technology	内容重复 / 后读覆盖	不能只覆盖局部的文件内容，必须复制整个文件覆盖
——Tiers	后读覆盖	
Terraform (Links)	内容重复	
Trade Conversions	后读覆盖	Trade Policy (?)
Tradition Categories	后读覆盖	
Traditions	后读覆盖	效果提示文案(effect tooltip)独立于内容效果
Traits	内容重复/不可局部覆盖	不能只覆盖局部的文件内容，必须复制整个文件覆盖
War Goals	后读覆盖	

9.4.2 Localisation 本地化文件夹

见 9.3 (4)

9.4.3 Events 文件夹

文件内容只读一次，因此编码顺序靠前的文件里的事件将会生效，并省略编码顺序靠后的文件的所有事件

9.4.2 Interface 文件夹

覆盖规则应该为后读覆盖

实例展示

由于在制作某些具体内容之时，各个文件之间具有较强的关联性质，并不好采用条目说明的方式进行展示，因此在此给予一些实际例子以供参考学习。

一、新建一个物种并向其添加专属特质

如果对 mod 熟悉的话，你应该能看到工坊有十分多增加或修改了物种肖像类型的 mod，小有单纯修改肖像的萌化银河，大则有类似舰 R 之类的 mod，虽然内容上也许天差地别，但其最为基础的物种系统的肖像都是基于同一个准则进行导入的，本篇将介绍如何将你的素材作为物种肖像类别导入到群星并新建一个全新的物种应用它。

1. 素材制作

首先拿到你扣好的图，一般来说较为常见的正常素材图应该是 png 或者 tga 格式，亦或是其他带有透明图层格式，因为一般来说物种肖像是带背景的，不过如果确实带着的话，也没有问题，不过会影响你导入之后的美观，这里建议还是先剔除掉肖像背景。

由于蠢驴采用的素材图基本上都是 dds，因此我们原则上也需要将我们的素材转换为 dds 格式。这里用到的图像处理工具是经典的 PS——当然，你完全可以找某个 DDS 转换网站干这种事情，这样你就只需要将图片裁剪为指定形状即可。这里推荐一个转换网址：

Convertio 文件转换

如果我们确定采用 PS 处理，不幸的是 PS 并不能够编辑 dds 文件，这里我们需要到英伟达官方网站去下载该插件，地址如下：

插件下载地址

进去之后我们应该会看到这样的情况：

DOWNLOADS

Standalone Application

Operating System	Windows 10, 11 (64-bit only)
Requirements	NVIDIA CUDA-capable GPU

[Download](#)[Release Notes](#)

我们下载右边的即可，下载之前可能会提示你要注册一个英伟达账号，这里你直接注册完登录就可以下载了，如果嫌麻烦的话也可以加 modder 群到群文件拿，注意，这里的插件适用 PS 版本比较高，如果版本过低可能需要老版本插件才能够使用，这里我并没有找到网上曾经有老版本插件下载的地方，因此如果需要的话也可以加入 modder 群拿插件（目前没把插件单独拉出来放云盘，之后有时间再拉出来，这样就不用加群了）。

总之这玩意下载之后会让你安装，安装完毕之后确认一下 PS 的插件位置有没有 NVIDIA 开头的文件夹，如果没有则是没装上，重新安装即可。装上插件后我们就可以打开以及将文件保存为 dds 格式了，接下来我们开始素材编辑。

如果你的素材符合无背景标准，那么其实你已经将近完成素材编辑了，这时你只需要将素材裁剪为指定尺寸即可。由于我们需要能让蠢驴的各个框装得下，因此肖像的宽度没有严格要求，但是高度必须保证在一个合理范围之内，我们先参考以下图像，确认一下我们肖像的大概位置：



游戏表现：

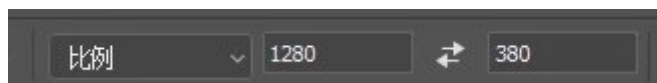


在有了上图印象之后我们就可以开始进行裁剪操作了，虽然在宽度上无严格要求，但为了保证能够在一个合理的范围之内进行位置参考，因此这里我的建议是采用 1280*380 的图片作为物种肖像素

材，以下示例演示了如何将原素材裁剪为指定的大小（由于本人 PS 技术十分有限，该部分仅供参考，如果直接处理完毕的可以直接跳过），以下示例素材大小为 2033*2184，大概图像如下：



显然这是不符合要求的，我们先选中裁剪工具，将比例设置为 1280*380：



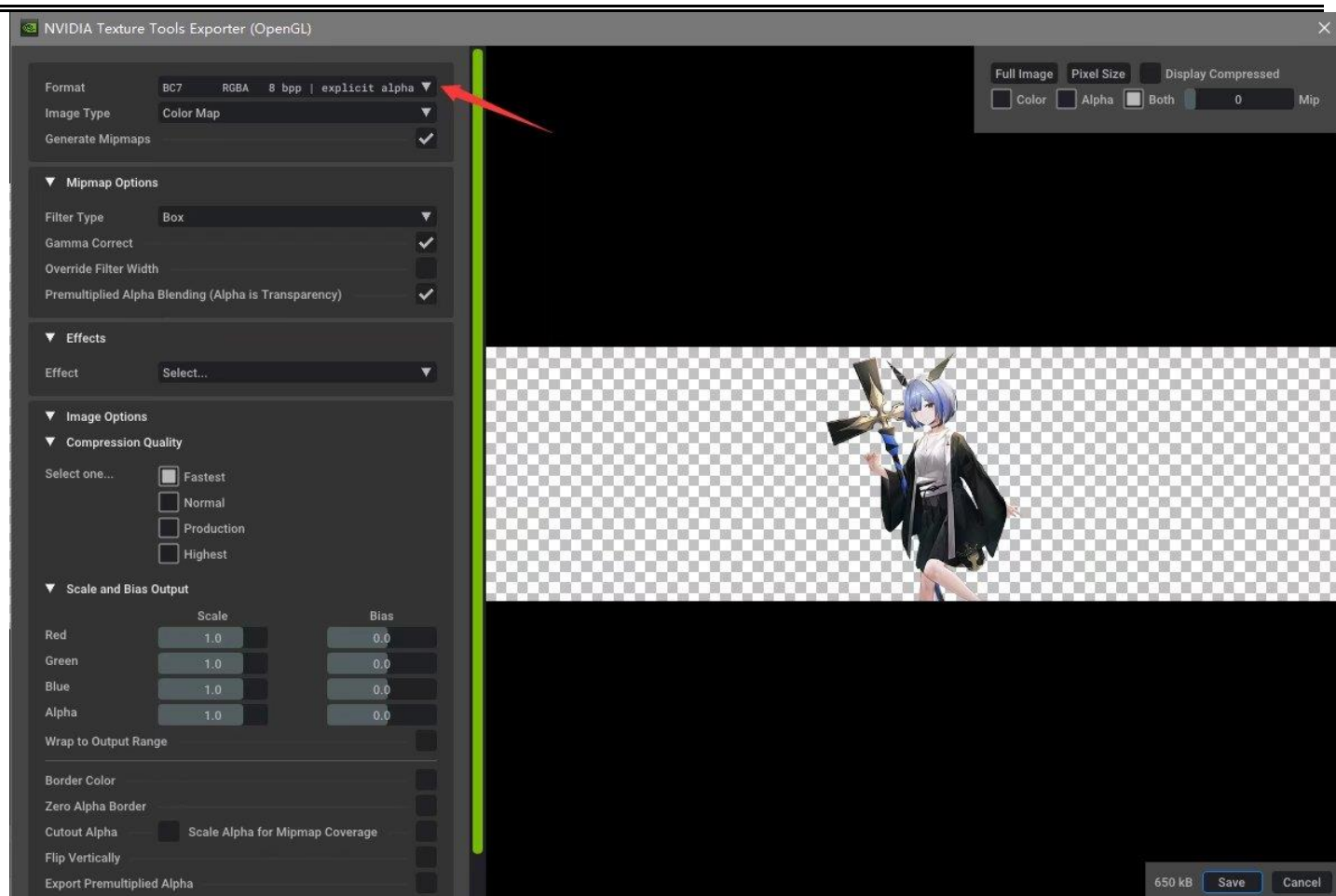
然后开始裁剪，由于该比例与我们原图严重不符，因此我们肯定需要拓宽整个图像以覆盖到我们所需的高度：



然后将图像大小再缩放为 1280*360:

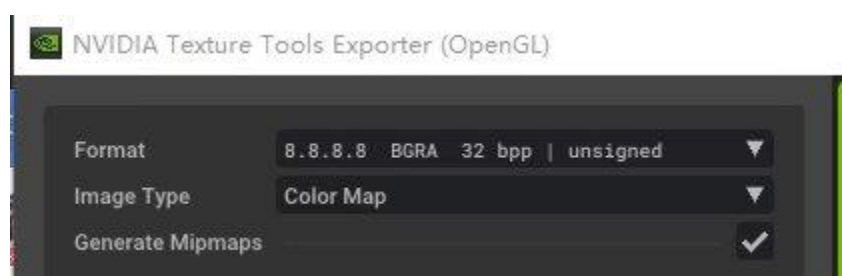


确认即可得到一张尺寸符合要求的图像了。不过现在我们仍然需要做出一些调整，目前我示例当中的肖像位置在图像正中间，我们应当以该位置作为一个调整点，先保存一次肖像并进入游戏查看，再做出微调。保存不用多说，选择另存为选择 dds 文件即可，这时候会弹出一个如下图类似的窗口：



点击红色箭头所示位置，选择以下格式保存：

8.8.8.8 BGRA 32 bpp | unsigned



选择完之后点右下角的 Save 即可保存。

2. 素材导入

我们现在就要将已经处理好的素材导入进游戏，首先我们一路创建文件夹到

gfx/models/portraits/Test，这个 Test 文件夹可以随意英文命名，作为你注册素材的存放地区，

我们将素材丢进去之后就可以开始注册导入了。

接下来我们来到 gfx/portraits/portraits 文件夹来注册我们放进去的素材，在这里新建一个.txt 文件，名称随意，写入以下内容：

```
1 portraits = {  
2   Mine_Gascony = {textureFile = "gfx/models/portraits/Test/Gascony.dds"}  
3 }
```

portraits 表示肖像注册；Mine_Gascony 为一个标识 Key，之后我们需要采用该肖像之时只需要调用这个值即可；textureFile 后面的路径则指向刚才我们丢进去的素材。

3.应用

注册完成的肖像可以用在两个地方，一个是最传统的当做物种肖像，另一个则是当做图片素材作为各种参数使用，例如事件当中 picture event date 的 portrait 调用以及 create_leader 等操作进行调用。这里我就介绍一下最基础的物种创建以及肖像组调用：

首先，我们确定目标，新建一个物种，并赋予其肖像。

物种的创建这里我们就简单说一说，在 common 下方的 species_classes 条目当中，我们结合以下示例来看：

```
test_species = {  
  archetype = BIOLOGICAL  
  randomized = no  
  gender = no  
  
  portrait_modding = no  
  
  leader_age_min = 15  
  leader_age_max = 25  
  
  portraits = {  
  }  
  non_randomized_portraits = {  
  }  
  graphical_culture = mammalian_01  
  move_pop_sound_effect = "moving_pop_confirmation"  
  trait = "trait_azurlanegirls"  
}
```

注：后面推出的物种教程将会详细讲解这部分，因此这里如果没法理解请直接先抄上去

开头 test_species 为 key，不必赘述；

archetype 则是该物种类别归为何种物种大类，这里的物种大类可以另外定义，在 common 下方的 species_archetypes 当中，具体指的是该物种归为机械，还是机器人还是生物之类的，这里我们简单处理写生物大类；

randomized 表示该物种是否随机出现，这里规定不随机出现；gender 规定是否有性别，这里规定为无；

portrait_modding 表示加上特质之后不进行肖像变换，类似于灵飞两个阶段会变肖像，这里可以规定不变（虽然该变还是变）；

下面两个属性规定该物种随机出现领袖的年龄，都是字面意思；

portraits 和 non_randomized_portraits 都是表示调用的肖像组，这里暂且先不填，因为我们还没有注册过肖像组；

graphical_culture 声明该物种的图形文化，具体应该是关联模型和城市背景用什么，这里找个你觉得顺眼的物种抄就行了；

move_pop_sound_effect 是挪动该物种发出的声音，这是 sound 文件定义，这里也不多说，直接用原版；trait 则是表示该物种的必要特质，这里绑定了碧蓝航线 mod 里边的 KAN-SEN，如果是测试的话，为了方便也可以不填。

至此，该物种就已经创立完毕，我们就差个肖像组了，打开我们之前注册肖像的文件，我们继续在其下方注册一个肖像组，其具体写法如下：


```

portraits = {
  Mine_Gascony = {textureFile = "gfx/models/portraits/Test/Gascony.dds"}
}

portrait_groups = {
  "test_portrait_groups" = {
    default = Mine_Gascony
    game_setup = {
      add = {
        portraits = {
          Mine_Gascony
        }
      }
    }
    species = {
      add = {
        portraits = {
          Mine_Gascony
        }
      }
    }
    leader = {
      add = {
        portraits = {
          Mine_Gascony
        }
      }
    }
    pop = {
      add = {
        portraits = {
          Mine_Gascony
        }
      }
    }
    ruler = {
      add = {
        portraits = {
          Mine_Gascony
        }
      }
    }
  }
}

```

上方 portraits 我们之前介绍过这里就不多说了，下方的 portrait_groups 则是重中之重。首先声明注册组 portrait_groups，其下方属性我们一一讲解：

第一个是 test_portrait_groups，这是我们注册的肖像组的 Key，也是之后我们需要填写到 species_class 中 portraits 里面的标识；default 表示在展示该物种的时候开始第一张展现的肖像；game_setup，species，leader，pop，ruler 这几个同阶属性分别表示在开局，物种，领袖，人口，统治者生成之时，随机出现的领袖所能够调用的肖像；add 表示添加肖像；portraits 下方填写我们上面 portraits 注册的肖像 key 即可，如果有多个可以有如下写法：

```

portrait_groups = {
  "Azur_Lane_k_de" = {
    default = K_Dragon001
    game_setup = {
      add = {
        portraits = {
          K_Dragon001
          K_Dragon002
          K_Dragon003
          K_Dragon004
          K_Dragon005
          K_Dragon006
          K_Dragon007
          K_Dragon008
          K_Dragon009
          K_Dragon010
          K_Dragon011
          K_Dragon012
        }
      }
    }
  }
}
species = {

```

注意的是，不同条目可以填不同的肖像，所以你可以做出哪些肖像可以在开始选，哪些只能出现在领袖之类的限制。

注册完毕之后回去补充 species_class 的属性：

```

test_species = {
  archetype = BIOLOGICAL
  randomized = no
  gender = no

  portrait_modding = no

  leader_age_min = 15
  leader_age_max = 25

  portraits = {
    "test_portrait_groups"
  }
  non_randomized_portraits = {
    "test_portrait_groups"
  }
  graphical_culture = mammalian_01
  move_pop_sound_effect = "moving_pop_confirmation"
  trait = "trait_azurlanegirls"
}

```

显然这里也可以填多个属性，调用多个肖像组。

接下来打开游戏测试便可：



如果一切没失误的话就是这样的，一个可选物种加上一个肖像展示，这个肖像展示就是你导入的肖像组 default 展示的肖像，该物种调用了多少肖像组就有多少个肖像框选择。

4.新建物种大类(Species Archetypes)

首先我们需要确定我们的物种是否隶属于某一个大的生物类别，比如石质、死灵、机械、生物这些大的范围，但在这里我们不考虑采用原版的，物种大类的规定在 common/species_archetypes 下方，我们新加文件并写入以下内容：

```
1 EXAMPLE_ARC = {
2   species_trait_points = 3
3   species_max_traits = 20
4 }
```

species_trait_points 表示该物种大类下物种可用的特质点数。

species_max_traits 表示该物种大类下物种最多能拥有的特质数量。

如果期望该物种大类下的物种能够选择其他物种大类的特质，我们可以添加下方所示属性：

```
1 EXAMPLE_ARC = {  
2     species_trait_points = 3  
3     species_max_traits = 20  
4     inherit_traits_from = { BIOLOGICAL }  
5 }
```

inherit_traits_from 表示该物种大类将会继承括号中所示其他物种大类的特质。这样我们就能够使用分配在 BIOLOGICAL 下方的所有特质。

5.新建物种类型 (Species Classes)

由于蠢驴在 3.2 更新之时修正了部分属性，导致教程部分有一些属性发生了变动，因此在这里我直接列出了物种类型所有属性：

```

1  ExampleClass = {
2      archetype = EXAMPLE_ARC
3      randomized = no
4      gender = no
5      leader_age_min = 15
6      leader_age_max = 25
7      possible = {
8          always = yes
9      }
10     portrait_modding = no
11     portraits = {
12         "human"
13         "humanoid_02"
14         "humanoid_03"
15         "humanoid_04"
16         "humanoid_05"
17     }
18     custom_portraits = {
19         randomized = { host_has_dlc = "Humanoids Species Pack" }
20         playable = { local_has_dlc = "Humanoids Species Pack" }
21         portraits = {
22             "humanoid_hp_01"
23             "humanoid_hp_02"
24             "humanoid_hp_06"
25             "humanoid_hp_07"
26             "humanoid_hp_08"
27             "humanoid_hp_09"
28             "humanoid_hp_10"
29             "humanoid_hp_11"
30             "humanoid_hp_12"
31             "humanoid_hp_13"
32         }
33     }
34     non_randomized_portraits = {
35         "human"
36     }
37     graphical_culture = humanoid_01
38     move_pop_sound_effect = "moving_pop_confirmation"
39 }

```

我们来逐条分析每个属性：

archetype：其调用的物种大类。

randomized：其是否能够随机生成。

gender：其是否存在性别。

leader_age_min/max：自动生成的该物种领袖的年龄最小值/最大值。

possible：能够选择该物种的条件。

portrait_modding：是否在增减特质的时候刷新肖像（失效）

potraits：表示其调用的肖像组

custom_portraits: 自定义肖像组使用，其中 randomized 表示可随机性，满足条件即可随机出现；playable 表示可选择性，满足条件即可选择；portraits 表示自定义的肖像组也就是上方条件控制显示的肖像组。这里我均采用的原版条件以及肖像组（没错，你只需要删掉原版物种肖像中的几个判断 dlc 的条件就可以正常使用该肖像，完全不需要 dlc）。

non_randomized_portraits: 表示不进行随机出现的肖像组。

graphical_culture: 表示该物种调用的图形文化，这个定义较为麻烦，在

common/graphical_culture 下声明，一般是控制舰船模型，殖民地窗口视图面板等一些属性，这里就不赘述了（因为不涉及自己的船模以及外观的话随便写个你喜欢的就行了）

move_pop_sound_effect: 该物种人口在移动时候的声音（强制迁移）。

6. 专属特质

由于蠢驴在 3.2 更新之时修正了部分属性，导致教程部分有一些属性发生了变动，因此在这里我直接列出了物种类型所有属性，由于全部属性过多，这里分批截属性：

```
1 ExampleTraits = {
2     cost = 2
3     icon = "gfx/interface/icons/traits/trait_azurlanegirls.dds"
4
5     leader_age_min = 20
6     leader_age_max = 50
7     potential_crossbreeding_chance = 0
8     initial = yes
9     infertile = no
10    randomized = no
11    sapient = no
12    forced_happiness = no
13    immortal_leaders = no
14    advanced_trait = no
```

cost: 定义该特质消耗的特质点数

icon: 定义该特质图标调用路径

leader_age_min/max: 应用该特质的物种自然产生的领袖年龄将会在两个值之间随机，原版蜂群使用该属性来让生成的领袖更年轻，而石质生物使用该属性来让生成的领袖更年长。

potential_crossbreeding_chance: 自然生成概率(需满足所有限制条件)。

initial: 定义该特质是否开局可选

infertile: 定义该特质物种是否能够自然繁殖（自然繁殖：能够通过第一个人口增长槽位增长）。

randomized: 定义该特质是否能够随机出现

sapient: 定义该特质物种是否为未开化物种。

forced_happiness: 定义该特质物种是否无视幸福度。

immortal_leaders: 定义该特质物种所生成的领袖是否不朽。

advanced_trait: 定义该特质是否为高级特质。如果是，则将会在完成“基因重组”之后才可选择，这将覆盖 initial 属性，使该无法开局选择。

```

15     species_possible_remove = {
16         always = no
17     }
18     species_potential_add = {
19         always = no
20     }
21     species_possible_add = {
22         always = no
23     }
24     species_potential_remove = {
25         always = no
26     }
27     species_possible_remove = {
28         always = no
29     }
30     species_possible_merge_add = {
31         always = no
32     }
33     species_possible_merge_remove = {
34         always = no
35     }
36     opposites = {
37         "trait_fleeting"
38     }
39     species_class = {
40         ExampleClass
41     }
42     allowed_archetypes = {
43         EXAMPLE_ARC
44     }

```

species_possible/potential_(merge_)add/remove: 六个属性十分相似，功能也差不多，都是一组条件判断，共同判断该特质在物种修饰界面的可选性，这里放到一起说：

采用复杂 condition 判断，因此存在 Scope 操作；

Scope: this=speciesfrom=country;

注意，使用 from 之前一定要判断是否存在，例如：

```
species_potential_add = {  
  exists = from  
  from = {  
    has_country_flag = harvested_radiotrophic_dna  
  }  
}
```

带有 possible 的属性在该特质不满足条件的情况下将变灰；

带有 potential 的属性在该特质不满足条件的情况下将不显示；

带有 merge 的属性为控制物种杂交过程而非基因修饰；

带有 add 的属性在不满足的情况下将无法增加该特质；

带有 remove 的属性在不满足的情况下无法移除该特质；

依据上方规则进行组合六个条件，默认均为 always=yes，酌情进行增添，没必要全写上（貌似其中还有些无效的，万恶的蠢驴）。

opposites: 定义该特质的相斥特质，也就是不能一起选。

species_class: 定义该特质需要何种物种类别才可选择。

allowed_archetypes: 定义该特质需要何种物种大类才可选择。


```

46     modifier = {
47         army_damage_mult = 0.5
48         pop_government_ethic_attraction = 0.25
49         pop_housing_usage_mult = -0.1
50         species_empire_size_mult = -0.1
51     }
52     declining_modifier = {
53     }
54     assembling_modifier = {
55     }
56     growing_modifier = {
57     }
58     triggered_pop_modifier = {
59     }
60     triggered_planet_growth_habitability_modifier = {
61         potential = {
62             always = yes
63         }
64         planet_jobs_produces_mult = 0.05
65     }
66     triggered_planet_growth_habitability_modifier = {
67         potential = {
68             exists = planet
69             planet = { is_planet_class = pc_ocean }
70         }
71         pop_growth_speed = 1.00
72     }
73     #custom_tooltip = <localisation key>
74     #custom_tooltip_with_modifiers = <localisation key>
75     slave_cost = {
76         energy = 1500
77     }
78     ai_weight = {
79         weight = 1
80     }
81 }

```

modifier: 定义拥有该特质人口所拥有的修正;

Scope: this = country;

declining/assembling/growing_modifier: 定义拥有该特质人口

所拥有减少/组装/自然增长的修正, 与人口增长或者组装之类的修正丢这里面去;

Scope: this = planet;

triggered_pop_modifier: 定义拥有该特质人口所拥有的修正

, 需要满足条件才能生效;

Scope: this=pop;

triggered_planet_growth_habitability_modifier: 定义拥有该特质人口所拥有的修正, 需要满足条件才能生效;

Scope: this=planet;

custom_tooltip: 定义该特质修正文本, 将采用自定义本地化替代自动生成的修正文本。

custom_tooltip_with_modifiers: 定义该特质修正文本, 将在自动生成的修正文本后方显示自定义本地化。

slave_cost: 定义拥有该特质的物种人口在奴隶市场的价格变化。

ai_weight: ai 在基因修饰之时选择该特质的权重。

以上特质无需全部写上, 本篇仅做参考示例, 因此展开了所有属性, 实际应用各取所需即可。

回到正题, 如果我们想要创建我们自己物种的专属特质, 则只需要注重 species_class 以及 allow_archetypes 两个属性即可。

如果我们期望该特质是某一个物种所必要的, 则可以在 species_class 进行如下声明:

```

1  ExampleClass = {
2      archetype = EXAMPLE_ARC
3      trait = ExampleTraits
4      randomized = no
5      gender = no
6      leader_age_min = 15
7      leader_age_max = 25
8      possible = {
9          always = yes
10     }
11     portrait_modding = no
12     portraits = [
13         "human"
14         "humanoid_02"
15         "humanoid_03"
16         "humanoid_04"
17         "humanoid_05"
18     ]
19     custom_portraits = {
20         randomized = { host_has_dlc = "Humanoids Species Pack" }
21         playable = { local_has_dlc = "Humanoids Species Pack" }
22         portraits = {
23             "humanoid_hp_01"
24             "humanoid_hp_02"
25             "humanoid_hp_06"
26             "humanoid_hp_07"
27             "humanoid_hp_08"
28             "humanoid_hp_09"
29             "humanoid_hp_10"
30             "humanoid_hp_11"
31             "humanoid_hp_12"
32             "humanoid_hp_13"
33         }
34     }
35     non_randomized_portraits = {
36         "human"
37     }
38
39     graphical_culture = humanoid_01
40     move_pop_sound_effect = "moving_pop_confirmation"
41 }

```

这样选择该物种的时候就会自动选取该特质。

如果按照教程来写好这三个地方，现在你就可以看到下方这样的情况，物种选择界面：



特质选择界面：



至于本地化则不在此赘述，至此，一个完整的物种大类+物种类别+专属特质的建立已经完毕，其他则是看你所需要的补充更更多的特质来丰富你自己的物种。

二、向游戏导入字库并应用在 Custom GUI

首先我们需要一个字库文件，具体怎么获取应该都知道，蠢驴原版的字库都是.ttf 文件，因此我们这里也采用.ttf 格式的字库，在获取到正常的字库文件之后你打开应该是如下差不多的格式：



确定一下有多少可用的 size 格式，到时候在注册字体文件的时候可以起到参考作用。

然后我们需要新建文件夹到和 common 同级的 fonts 文件夹下新建一个.asset 文件，文件名随意：



内容如下：


```

1 font = {
2     name = "regular_script"
3
4     fontstyle = {
5         style = regular
6         file = "gfx/fonts/regular_script.ttf"
7     }
8 }

```

这样我们就已经将我们的字体文件导入到了群星当中，但如果我们希望能够调用的话，还需要再进行一次注册。

到和 common 同级的 interface 文件夹下方新建一个.gfx 文件，文件名随意，写上以下内容：

```

1 bitmapfonts = {
2     bitmapfont = {
3         name = "infinite_regular_script_font_24"
4         path = "gfx/fonts/malgun_goth_24"
5         color = 0xffffffff
6         cursor_offset = { -4 -3 }
7         selection_offset = { 0 1 }
8     }
9     bitmapfont_override = {
10        name = "infinite_regular_script_font_24"
11        ttf_font = "regular_script"
12        ttf_size = "24"
13        languages = {"l_simp_chinese" }
14        vertical_offset = "5"
15    }
16 }

```

我们逐步拆解以下语句，手电 bitmapfonts 表示我们需要注册的类为 bitmapfont 合集，再嵌套一层表示为声明单个具体注册对象，name 为我们注册该字体的名称，这也是之后我们调用该字体需要用到的 key；path 为源 ttf 文件所在位置，应该与上方声明 font 之时的 file 的路径保持一致，但需要注意的是这里的路径并没有带上文件名后缀.ttf；color 保持默认即可，该属性可以通过本地化调整，并没有什么作用；下方为光标偏移以及字体选择偏移，如果不清楚其作用，如图默认即可。

下方 `bitmapfont_override` 属性则表示需要重写上方注册的 `bitmapfont` 字体的某些属性并链接到我们之前注册过的 `font`。`name` 属性与上方 `bimapfont` 声明的属性保持一致；`ttf_font` 后面需要跟上我们之前注册过的 `font` 的 `key`，即 `regular_script`；`ttf_size` 后面跟上的数字则为我们注册的字体大小，依据我们之前在 `ttf` 文件中的大概预览可以得知我们想要的大小；`languages` 表示我们需要注册到哪种语言，注意，该选项可以多次声明，如需添加其他语言则直接在后方用引号括起来：

```
languages = { "l_russian" "l_polish" }
```

我们这里仅针对中文即可；`vertical_offset` 为垂直偏移程度，这里是调整字体错位问题的属性，如果没有出现错位则可以忽视。

接下来是把字体应用到 GUI，`effectButtonType` 是 Custom GUI 绕不开的一个实用控件，这里采用一个简单示例展示如何测试字体：

```
effectButtonType = {  
    name = hoi_system_iron_ore  
    position = { x = 20 y = 100 }  
    size = { x = 500 y = 50 }  
    quadTextureSprite = GFX_tile_outliner_bg  
    effect = hoi_button_effect  
    text = hoi_system_iron_ore_text  
    format = LEFT  
    font = infinite_regular_script_font_24  
}
```

`font` 直接调用注册的字体 `key` 即可开始测试。